MINISTRY OF SCIENCE AND HIGHER EDUCATION
OF THE RUSSIAN FEDERATION
FEDERAL STATE AUTONOMOUS EDUCATIONAL INSTITUTION
OF HIGHER EDUCATION
"National Research Nuclear University *MEPhI*"
(NRNU MEPhI)

As a manuscript

Roger Nick Anaedevha

Adversarially Robust Models and Algorithms for Enhancing the Robustness and Efficiency of Hybrid Intrusion Detection and Prevention Systems in Distributed and Non-Stationary Conditions

Scientific specialty: 2.3.1. — "System Analysis,
Management and Information Processing, Statistics"
(Technical Sciences)

DISSERTATION
submitted for the degree of
Candidate of Technical Sciences

Scientific Supervisor:
Doctor of Physical and Mathematical Sciences,
Professor
Alexander Gennadievich Trofimov

Moscow — 2026

Abstract

This dissertation is devoted to the development of adversarially robust models and algorithms for enhancing the robustness and efficiency of hybrid intrusion detection and prevention systems (IDPS) in distributed and non-stationary conditions. Graph neural networks have become the principal deep-learning paradigm for extracting predictive representations from relational data, yet their message-passing mechanism renders them sensitive to perturbations of the underlying graph topology, node features, and temporal ordering of events. The robustness of learned representations on dynamic graph structures is investigated as a foundational challenge in applied mathematics and artificial intelligence; seven novel methods are developed to address the problem under three operating conditions — the adversarial environment, non-stationary conditions, and distributed network dynamics — while satisfying four cross-cutting requirements: robustness, accuracy, efficiency, and privacy.

The first contribution (Method M1: CT-TGNN) introduces continuous-time temporal graph neural networks grounded in neural ordinary differential equations, where the evolution of node representations is governed by parameterised differential operators acting on evolving graph topologies. Through adjoint sensitivity analysis and temporal adaptive batch normalisation the architecture achieves memory-efficient training and provable Lipschitz-bounded dynamics, yielding certified robustness through intrinsic smoothness regularisation. Multi-scale temporal dynamics with learned time constants spanning several orders of magnitude resolve both fast transient events and slow structural trends within a single architecture. A stochastic extension (SDE-TGNN) replaces the deterministic dynamics with Itô stochastic differential equations and derives analytical moment propagation from the Fokker-Planck equation, achieving 99.0% average weighted F1 across three benchmarks totalling over 52 million samples, expected calibration error of 0.042, certified adversarial robustness of 76.2% at perturbation radius 0.25, and cross-domain transfer at 90.5% F1 without fine-tuning.

The second contribution (Method M2: FedLLM-API) formulates a federated optimisation algorithm for distributed graph learning that integrates

corruption-tolerant aggregation, differential privacy through calibrated Gaussian noise, and optimal-transport-based distributional alignment of heterogeneous graph fragments. Convergence is proved at a rate of $\mathcal{O}(1/\sqrt{T})$ even when a significant fraction of participants submit adversarially corrupted updates, while the Wasserstein-distance alignment is maintained within a formal (ϵ, δ)-differential privacy budget. Large language model integration enables 87.6% F1 on zero-shot detection of novel anomalous categories.

The third contribution (Method M3: TripleE-TGNN) develops multi-level temporal graph embeddings that capture structure at service, trace, and node levels through dual temporal-spatial attention and elastic weight consolidation, achieving 96.8% accuracy while reducing computational cost through structured sparsity and preventing catastrophic forgetting as the graph topology evolves.

The fourth contribution (Method M4: MambaShield) designs efficient sequential inference through selective state-space layers with progressive adversarial robustness distillation, reducing computational cost from $\mathcal{O}(L^2)$ to $\mathcal{O}(L \log L)$ through Fast Fourier Transform convolutions while sustaining 91.4% accuracy under twenty-five percent training-time poisoning, accompanied by PAC-Bayesian generalisation certificates.

The fifth and sixth contributions (Methods M5: Stochastic Transformer and M6: UC-HGP) integrate variational Bayesian attention with hierarchical Gaussian process uncertainty quantification, decomposing predictive uncertainty into epistemic and aleatoric components with expected calibration error of 0.078, enabling principled detection of out-of-distribution graph inputs and uncertainty-guided decision-making.

The seventh contribution (Method M7: Certification) constructs a unified robustness certification framework comprising game-theoretic strategy selection through Stackelberg equilibria with no-regret online learning, interval bound propagation, Lipschitz estimation on graph operators, and randomised smoothing adapted to discrete graph structures. The framework couples all seven methods through consistency regularisation with a proven convergence rate.

A domain-specific foundation model (CyberSecLLM) employing ODE-augmented selective state space blocks achieves 89.1% average zero-shot accuracy across an eleven-task evaluation benchmark, a 20.5-point improvement over GPT-4, while processing million-token graph event sequences in 2.3

seconds. The complete framework is implemented as a production-grade deployed system (RobustIDPS.ai, DOI: 10.5281/zenodo.19129512) comprising 51 pages across 10 functional groups, integrating 17 neural network models with a two-plane architecture (Python control plane + Rust data plane with kernel-resident XDP fast path, release v4), a four-layer LLM defence framework achieving 94.5% macro-averaged detection across 517 attack scenarios with four commercial LLM backends, and a multi-agent architecture for forward-compatible detection under post-quantum protocol regime transitions.

Comprehensive experimental validation spans six benchmark datasets comprising over eighty-four million labelled samples, a unified twenty-three-metric evaluation framework, systematic ablation studies, adversarial robustness assessments under gradient-based and adaptive perturbations, and cross-domain transfer experiments on speech event detection and clinical monitoring. All methods are formulated at the level of mathematical operations on graph structures independently of any application domain, and are validated through application to hybrid intrusion detection and prevention on network-traffic graphs as a representative adversarial deployment environment, and — as a confirmation of domain-independence — to the neural-network navigation subsystem of unmanned aerial vehicles operating under electronic-warfare and adversarial-ML pressure (Chapter 6). All twelve intersections of the condition-requirement matrix are addressed with empirically verified performance. Letters of implementation have been obtained from Positive Technologies, Google Cloud Platform and the Suricata Open Source IDPS project.

The dissertation consists of an introduction, six chapters, a conclusion, a bibliography of 204 references and three appendices. The main text is 230 pages.

Acknowledgements

I wish to express my sincere gratitude to my scientific supervisor, Professor Alexander Gennadievich Trofimov, whose guidance, intellectual rigour, and generous encouragement have shaped every stage of this research. His deep expertise in mathematical modelling and systems analysis provided the foundation upon which the theoretical contributions of this dissertation were built.

I am thankful to the faculty and fellow researchers of the Institute of Cyber Intelligent Systems and Department of Cybernetics at the National Research Nuclear University MEPhI for creating a stimulating academic environment. The discussions, seminars, and collaborative spirit within the department enriched this work considerably.

I gratefully acknowledge the support of the MEPhI postgraduate programme, the grant for research centres in Artificial Intelligence from the Ministry of Economic Development of the Russian Federation (identifier 000000C313925P3Q0002), and the computational resources made available for the experimental work described herein. The scale of the empirical validation would not have been possible without access to adequate infrastructure.

Finally, I owe an immeasurable debt to my family and friends, whose patience, support, and belief in this endeavour sustained me throughout the years of study and research. Thank you everyone.

Roger Nick Anaedevha

Moscow, 2026

Contents

List of Figures

List of Tables

List of Abbreviations

AI	Artificial Intelligence
AML	Adversarial Machine Learning
APT	Advanced Persistent Threat
AUC-ROC	Area Under the Receiver Operating Characteristic Curve
BN	Batch Normalisation
BWT	Backward Transfer (continual learning metric)
C&W	Carlini–Wagner Attack
C1, C2, C3	Operating Condition labels: Adversarial Environment, Distributed Dynamics, Non-Stationarity
CL-RL	Continual Learning – Reinforcement Learning
CNN	Convolutional Neural Network
CPO	Constrained Policy Optimisation
CT-GNN	Continuous-Time Graph Neural Network
CT-TGNN	Continuous-Time Temporal Graph Neural Network (Method M1)
CVE	Common Vulnerabilities and Exposures
CyberSecLLM	Domain-Specific Cybersecurity Foundation Model
DDoS	Distributed Denial of Service
Dec-CMDP	Decentralised Constrained Markov Decision Process
DNN	Deep Neural Network
DoS	Denial of Service
DP	Differential Privacy
ECE	Expected Calibration Error
ELBO	Evidence Lower Bound
EWC	Elastic Weight Consolidation

FedAvg	Federated Averaging
FedGTD	Federated Game-Theoretic Defence
FedLLM-API	Federated Graph Optimisation with Large Language Models and Privacy (Method M2)
FFT	Fast Fourier Transform
FGSM	Fast Gradient Sign Method
FIM	Fisher Information Matrix
FWT	Forward Transfer (continual learning metric)
GAN	Generative Adversarial Network
GIN	Graph Isomorphism Network
GNN	Graph Neural Network
GPU	Graphics Processing Unit
HIDPS	Hybrid Intrusion Detection and Prevention System
HiPPO	High-order Polynomial Projection Operator
IAT	Inter-Arrival Time
ICS	Industrial Control System
ICS3D	Intrusion and Counter-measure Simulation in Cloud and Edge Computing Security Dataset
IDPS	Intrusion Detection and Prevention System
IDS	Intrusion Detection System
IIoT	Industrial Internet of Things
IoT	Internet of Things
KL	Kullback–Leibler Divergence
LLM	Large Language Model
LSTM	Long Short-Term Memory
LWE	Learning With Errors
M1–M7	Method labels for the seven developed methods (see Figure 3)
MA-CPO	Multi-Agent Constrained Policy Optimisation
MA-PQC-IDS	Multi-Agent Post-Quantum Cryptography Intrusion Detection System

MambaShield	Selective State-Space Model with Poisoning Resilience (Method M4)
MC	Monte Carlo
MC-Dropout	Monte Carlo Dropout
MCC	Matthews Correlation Coefficient
MITRE ATT&CK	Adversarial Tactics, Techniques, and Common Knowledge
ML	Machine Learning
MLP	Multi-Layer Perceptron
MLSecOps	Machine Learning Security Operations
NIDPS	Network Intrusion Detection and Prevention System
NIST	National Institute of Standards and Technology
NLP	Natural Language Processing
NPLR	Normal Plus Low-Rank
Neural ODE	Neural Ordinary Differential Equation
ODE	Ordinary Differential Equation
ODE-S6	ODE-Augmented Selective State Space (block in CyberSecLLM)
OT	Optimal Transport
OWASP	Open Worldwide Application Security Project
PAC	Probably Approximately Correct
PARD	Progressive Adversarial Robustness Distillation
PCAP	Packet Capture
PGD	Projected Gradient Descent
PQC	Post-Quantum Cryptography
PWA	Progressive Web Application
R1–R4	Requirement labels: Robustness, Accuracy, Efficiency, Privacy
RAG	Retrieval-Augmented Generation
RL	Reinforcement Learning
RMF	Risk Management Framework (NIST AI RMF)

RNN	Recurrent Neural Network
SCADA	Supervisory Control and Data Acquisition
SDE	Stochastic Differential Equation
SDE-TGNN	Stochastic Differential Equation Temporal Graph Neural Network
SOC	Security Operations Centre
SSM	State-Space Model
Stoch. Trans.	Stochastic Transformer (Method M5)
SVM	Support Vector Machine
TABN	Temporal Adaptive Batch Normalisation
TGNN	Temporal Graph Neural Network
TLS	Transport Layer Security
TripleE-TGNN	Triple-Embedding Temporal Graph Neural Network for Multi-Level Representations (Method M3)
UC-HGP	Uncertainty-Calibrated Hierarchical Gaussian Process (Method M6)

INTRODUCTION

Relevance of the Research Topic

Graph neural networks [1–3] have become the principal deep-learning paradigm for extracting predictive representations from relational data. They are widely deployed in healthcare (modelling molecules as atom-bond graphs and patient-interaction graphs for treatment recommendations), finance (analysing transaction graphs between accounts to detect fraud and money laundering), cybersecurity (analysing network traffic as graphs of connected devices to detect intrusions), and industrial systems (sensor-fusion graphs for SCADA and ICS monitoring), among other fields. Yet the very message-passing mechanism that gives these models their expressive power renders them vulnerable to perturbation [4–6].

Existing graph neural network models suffer from four interconnected problems: (1) being easily fooled by small, deliberate changes to input data or node connections; (2) inability to handle new or changing patterns without retraining from scratch; (3) being too slow and computationally expensive for real-time use on large graphs; and (4) inability to share learning between organisations without leaking private data. These problems are dynamic in nature — they change over time, yet share a common architectural root.

To understand why this sensitivity arises, it is necessary to recognise the dual-input nature of graph neural networks. Unlike conventional neural networks that receive a single data tensor, a graph neural network receives two inputs simultaneously: numerical *input data* ($\mathbf{X} \in \mathbb{R}^{n \times d}$, the feature matrix encoding entity attributes) and relational *input structure* ($\mathbf{A}$, the adjacency matrix encoding pairwise connectivity). The network produces its output by propagating data along the structure through iterative neighbourhood aggregation. Perturbation of either input — the data or the structure — can compromise predictions. This dual-input nature is what distinguishes graph neural networks from conventional neural networks and is the source of the robustness challenges addressed in this dissertation.

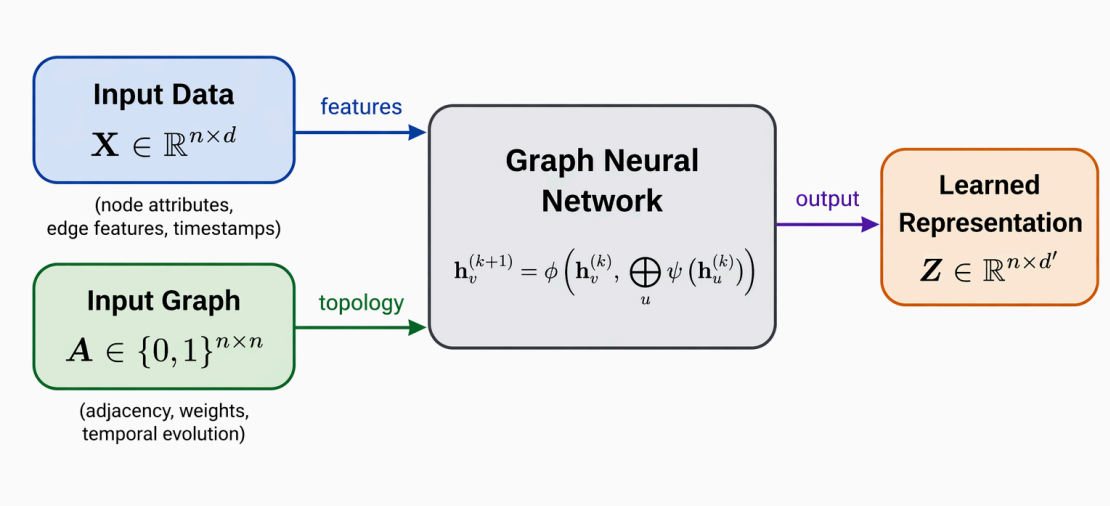

Figure 1 – Dual-input nature of graph neural networks. The network receives both numerical data (feature matrix $\mathbf{X}$) and relational structure (adjacency matrix $\mathbf{A}$) as inputs. Perturbation of either input can compromise the learned representation, motivating the robustness framework developed in this dissertation.

Most graph neural networks operate in discrete time, which deepens these vulnerabilities. Discrete-time models force irregular graph events onto a fixed clock, losing information and robustness guarantees. Continuous-time formulations — for example, through neural ordinary differential equations [7–9] — flow smoothly with event timing, capture multiple time scales in one model, and expose a Lipschitz bound that yields certified robustness, an advantage no discrete model can match. Because representations evolve according to a differential equation with a bounded Lipschitz constant, small perturbations of the input can only cause bounded, smooth changes in the output — the smoothness of the governing flow mathematically limits how much an adversary can move the prediction. The Grönwall inequality formalises this: the distance between two ODE trajectories grows at most exponentially in $L_g T$, giving a closed-form robustness certificate:

$$\|\mathbf{h}_1(T) - \mathbf{h}_2(T)\| \leq e^{L_g T} \|\mathbf{x}_1 - \mathbf{x}_2\|, \tag{1}$$

where $\mathbf{h}_i(T)$ is the hidden state of trajectory i at time T, $\mathbf{x}_i$ is the initial input, L_g is the Lipschitz constant of the ODE vector field, and $\|\cdot\|$ denotes the Euclidean norm. Discrete-time graph neural networks have no such continuous handle, so perturbations compound unpredictably through each layer. However,

continuous-time models receive limited attention in the literature and in current practice, which is why this dissertation focuses on their development through CT-GNN-based models.

The urgency of these vulnerabilities is also driven by the broader evolution of the cyber-threat landscape from incidental attacks to targeted AI-enabled campaigns, as analysed in particular in the work of I. K. Sachkov [10]. The urgency is further confirmed by recent peer-reviewed studies documenting severe GNN failures across four safety-critical domains. In healthcare, an edge-sensitivity gradient attack on Graph Isomorphism Networks caused approximately 25% AUC degradation on MoleculeNet benchmarks (Tox21, ClinTox, SIDER), with toxic compounds misclassified as safe [11]. In finance, the MonTi gang-injection attack nearly tripled the misclassification rate of GNN-based fraud detectors (CARE-GNN, PC-GNN, GAGA) from 25.7% to 68.6% [12]. In industrial IoT, data poisoning of federated intrusion detectors caused a 64-percentage-point accuracy collapse from 94.2% to 30.0% under non-IID conditions [13]. In cybersecurity, the PROVNINJA attack reduced GNN provenance-IDS detection by up to 59%, rendering APT attackers invisible [14]. These documented failures confirm that GNN robustness lags predictive accuracy by 25–65 percentage points in every safety-critical domain, and this gap motivates the entire dissertation.

A distributed environment is one in which multiple organisations (e.g. hospitals, banks, data centres) each hold a piece of the same graph data and cannot share it, so a shared model must learn from every piece through federated learning [15] — without any party seeing another's raw information. Graph neural networks in distributed environments must be more stable, efficient, robust, and accurate, because they are entering safety-critical systems faster than their guarantees can be established: adversaries can fool them with small perturbations; distributions do shift after deployment; privacy laws restrict data sharing [16, 17]; yet edge devices demand real-time inference. No existing approach addresses all four simultaneously — this gap motivates the dissertation.

Real graph data drifts after deployment — topologies evolve, attack patterns change, new protocols emerge — and static graph neural networks silently lose accuracy without warning. This is unacceptable in safety-critical systems where a stale model is a dangerous model. The objective is to build dynamic graph

neural networks that detect distribution shift online, adapt parameters without forgetting previously learned patterns, and quantify uncertainty through Bayesian posterior updates [18, 19] to flag novel inputs for human review.

Existing graph neural network and transformer-based models [20, 21] scale quadratically ($\mathcal{O}(L^2)$) with sequence length because every node must attend to every other node, making them too slow and memory-hungry for real-time inference on large temporal graphs with millions of events. Literature reveals that structured state-space models [22–24] are more efficient, reducing cost to $\mathcal{O}(L \log L)$ through Fast Fourier Transform convolutions and enabling deployment on edge devices where transformer-scale models are infeasible. Therefore, this dissertation proposes the shift of attention and practices from transformers towards selective state-space models for graph-structured data.

The four problems described above are not independent; they form a logical chain whose links determine the progressive scope of this dissertation. Continuous-time graph dynamics supply the representational foundation; federated algorithms enable distributed deployment; efficient state-space inference allows real-time operation; and online Bayesian adaptation ensures long-term reliability. Each task's output is consumed by the next, and no existing approach jointly addresses all four across the three operating conditions and four requirements identified below.

The three operating conditions and four cross-cutting requirements are independent — they define the space in which any graph-based AI system must operate reliably. The dissertation proposes methods that fulfil all of them simultaneously.

The three operating conditions are: (1) the adversarial environment, in which a deliberate opponent perturbs the graph topology, node features, or temporal ordering of events; (2) the distributed environment, in which graph data is fragmented among organisations that cannot centralise their data and where some participants may be unreliable or adversarial; and (3) non-stationarity, in which the statistical properties of the graph evolve over time.

The four cross-cutting requirements are: (R1) robustness — predictions must remain stable under perturbation; (R2) accuracy — high classification performance must be maintained; (R3) efficiency — graph events must be processed fast enough for real-time operation; and (R4) differential privacy — the

training process must not reveal confidential data.

Table 2 – Three operating conditions and the methods developed to address them.

Condition	Methods	Rationale
C1: Adversarial Environment	M1: CT-TGNN M5: Stoch. Transformer M7: Certification	Continuous-time neural ODEs impose Lipschitz-bounded smoothness on the representation trajectory, providing deterministic robustness certificates that discrete-time alternatives cannot offer. Stochastic adversarial training and game-theoretic equilibrium strategies complement the architectural guarantee with adaptive defence.
C2: Non-Stationary Conditions	M1: CT-TGNN M3: TripleE-TGNN M4: MambaShield M6: UC-HGP	CT-TGNN captures temporal evolution through neural differential equations; TripleE-TGNN prevents catastrophic forgetting through elastic weight consolidation; MambaShield provides near-linear efficiency; and UC-HGP supplies hierarchical Gaussian process uncertainty for drift detection.
C3: Distributed Network Dynamics	M2: FedLLM-API M7: Certification	Optimal transport through Sinkhorn iterations corrects for heterogeneity across graph fragments, while coordinate-wise trimmed-mean aggregation tolerates corrupted participants and calibrated Gaussian noise enforces a formal privacy budget.

The three conditions are not mutually exclusive: real-world graph systems typically face two or all three simultaneously. Figure 2 illustrates how the seven methods are positioned within the space defined by the intersecting conditions. Methods placed at the intersection of two or three conditions address the coupled challenges that arise when those conditions co-occur, and the requirement triangle on the right shows how the three non-privacy requirements (accuracy, efficiency, robustness) are jointly satisfied by complementary architectural choices.

Table 3 – Four cross-cutting requirements and the methods that fulfil them.

Requirement	Methods	Rationale
R1: Robustness	M1: CT-TGNN M4: MambaShield M7: Certification	CT-TGNN provides Lipschitz-bounded continuous dynamics; MambaShield resists training-time poisoning through progressive distillation; the certification framework ties guarantees together through randomised smoothing and interval bound propagation.
R2: Accuracy	M3: TripleE-TGNN M5: Stoch. Transformer M6: UC-HGP	Multi-level graph embeddings capture structure at service, trace, and node levels; variational Bayesian attention decomposes uncertainty; hierarchical Gaussian processes provide calibrated confidence intervals.
R3: Efficiency	M4: MambaShield M2: FedLLM-API M3: TripleE-TGNN	MambaShield reduces inference cost from quadratic to near-linear; FedLLM-API reduces communication rounds; TripleE-TGNN achieves savings through structured sparsity and quantisation-aware training.
R4: Privacy	M2: FedLLM-API M1: CT-TGNN M7: Certification	FedLLM-API enforces (ϵ, δ)-differential privacy; CT-TGNN supports differentially private feature extraction; the certification framework tracks the cumulative privacy budget.

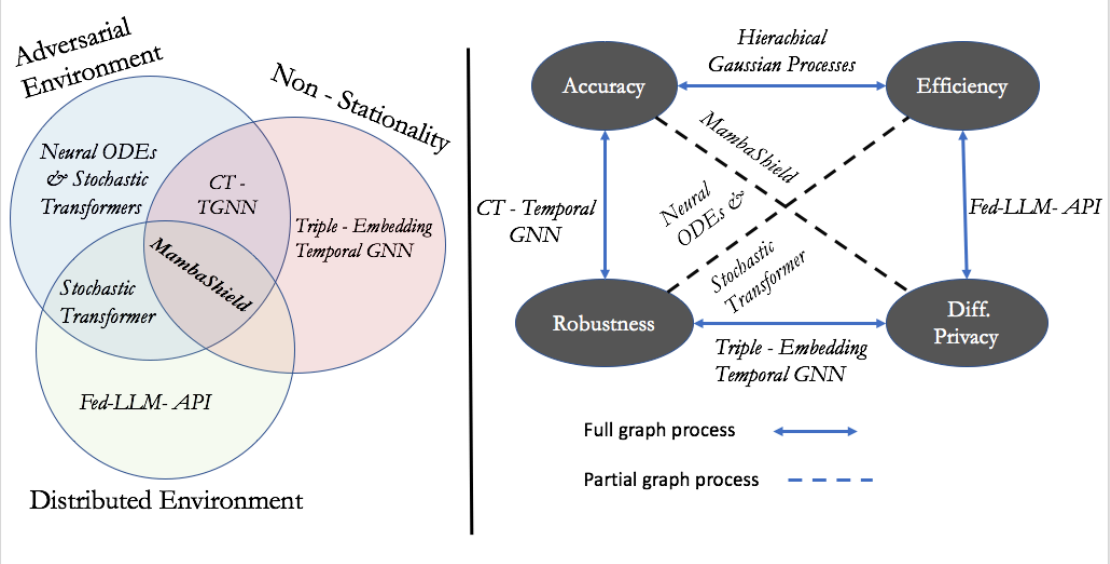

Figure 2 – Positioning of the seven methods within the space of operating conditions (left) and requirements (right). The Venn diagram shows that each method addresses one or more conditions, with M4: MambaShield at the intersection of all three owing to its efficiency under adversarial perturbation and non-stationary drift in distributed deployments. M1: CT-TGNN bridges the adversarial and non-stationary conditions through certified continuous-time dynamics. The requirement triangle shows how accuracy, efficiency, and robustness are jointly achieved through complementary architectural choices.

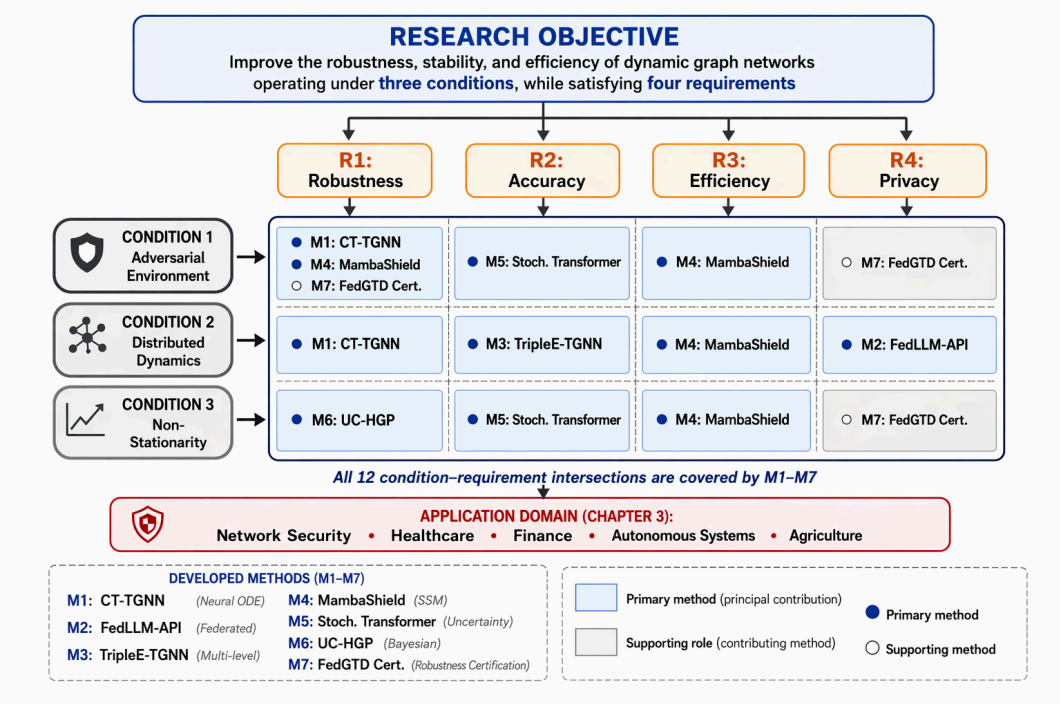

Figure 3 – Architectural framework of the research programme. Seven methods cover all 12 intersections of the 3×4 condition–requirement matrix. Each is formulated at the level of mathematical operations on graphs — domain-independent. All models (CT-TGNN, SDE-TGNN, FedLLM-API, TripleE-TGNN, MambaShield, Stochastic Transformer, UC-HGP) are developed by the author and documented in 18 peer-reviewed publications.

The framework ensures domain independence. Every method is formulated at the level of mathematical operations on graph structures, without reference to any particular application field. It is only in later chapters that these methods are mapped onto the entities and constraints of specific deployment environments. With the problem space, operating conditions, and architectural framework now established, the next section states the precise research objective and the sequential tasks through which it is realised.

Research Objective

The central objective of this dissertation is to increase the robustness of graph neural networks against adversarial perturbations, their stability under non-stationary conditions, their efficiency and accuracy for real-time use, and their privacy preservation in distributed environments.

Realising this objective requires the completion of seven research tasks arranged in a sequential chain, so that the output of each task supplies an input

required by the next.

1. Develop continuous-time (CT) models by means of neural ordinary differential equations with multi-scale temporal dynamics and Lipschitz-certified robustness.
2. Develop methods that allow the CT-models to operate in distributed environments by means of federated learning with Byzantine-resilient aggregation, differential privacy, and optimal-transport distributional alignment.
3. Develop methods that reduce the trade-off cost and increase the efficiency of both monolithic and distributed models by means of multi-level embeddings combined with selective state-space architectures (MambaShield) that lower complexity from quadratic to near-linear.
4. Develop methods to estimate and adapt GNNs to non-stationary environments — that is, distribution drift — by means of Hierarchical Gaussian Processes for calibrated uncertainty quantification and online Bayesian posterior updates.
5. Integrate all methods into a unified framework, validate it on large-scale benchmarks, and prove its quality to make it ready for deployment, by means of consistency regularisation and comprehensive benchmarking.
6. Adapt the unified framework to cybersecurity applications, taking into account the specific features of this area of use, by means of domain-specific adapters and threat-model refinement.
7. Apply the unified framework and demonstrate its quality and domain-independence in two safety-critical areas — hybrid Network Intrusion Detection and Prevention Systems (NIDPS) by means of real-time deployment on the RobustIDPS.ai software platform, and the neural-network navigation subsystem of unmanned aerial vehicles operating under electronic-warfare pressure (Phase A reference implementation, see Chapter 6).

Object of Research

The object of this investigation is Dynamic Graph Neural Networks (GNNs) functioning in adversarial, non-stationary, and distributed environments. In concrete terms, the investigation covers the class of deep-learning models, training algorithms, and supporting mathematical frameworks whose properties

determine whether a graph neural network can operate reliably under adversarial perturbation of graph topology and node features, distributional non-stationarity of the evolving graph, privacy constraints on distributed graph fragments, and limited computational budgets for temporal graph processing. This includes neural ordinary differential equations [7, 25] together with adjoint-based training on graph-structured domains, temporal graph neural networks with continuous-time dynamics [8, 26, 27], optimal-transport formulations for distributional comparison and alignment of heterogeneous graph fragments [28, 29], structured state-space models for efficient sequential inference on graph event sequences [22, 24, 30], Bayesian neural networks for decomposing predictive uncertainty on graph-structured data [31, 32], game-theoretic models of strategic interaction between a graph classifier and an adversary [33, 34], and robust optimisation formulations that embed worst-case graph perturbations directly into the training objective [35, 36]. The object is defined at the level of mathematical and algorithmic constructs on graph domains, independently of any particular application.

Subject of Research

The subject of research is methods to ensure the robustness, efficiency, differential privacy, and accuracy of Dynamic GNNs functioning in adversarial, non-stationary, and distributed environments. These methods include neural ordinary differential equation architectures for certified continuous-time dynamics, federated optimisation algorithms with Byzantine-resilient aggregation and optimal-transport alignment, multi-level temporal graph embeddings with elastic weight consolidation, selective state-space models for efficient sequential inference, Bayesian posterior update mechanisms for online drift adaptation, and game-theoretic certification frameworks for provable robustness guarantees. The mathematical formulations and algorithmic designs are constructed to be portable to any field in which learned models on graphs confront adversarial or non-stationary conditions, including healthcare, finance, autonomous systems, and precision agriculture. It is only in the application chapters that these methods are instantiated in specific domains.

Research Methods

The methodological programme of this dissertation is organised around eight formal methods, each selected because it addresses a specific and measurable aspect of the graph robustness problem.

Neural ordinary differential equations model the continuous evolution of node representations through a parameterised vector field,

$$\frac{d\mathbf{h}(t)}{dt} = f_\theta\big(\mathbf{h}(t),\, t\big), \tag{2}$$

solved with adaptive-step numerical integrators. Their role is to provide dynamics on graph structures whose output sensitivity, measured by the Lipschitz constant of the learned mapping, can be estimated and reported as a robustness certificate (central to Methods M1: CT-TGNN and M3: TripleE-TGNN).

Graph neural networks extend learned representations to relational data through neighbourhood aggregation,

$$\mathbf{h}_v^{(k+1)} = \phi\Big(\mathbf{h}_v^{(k)},\, \bigoplus_{u \in \mathcal{N}(v)} \psi\big(\mathbf{h}_u^{(k)},\, \mathbf{e}_{uv}\big)\Big), \tag{3}$$

and their coupling with the neural ODE on a time-varying adjacency structure produces temporal graph models that capture both topological and temporal dependencies (used in Methods M1, M3, and M5).

Optimal transport supplies a geometry on probability distributions through the Wasserstein distance,

$$W_p(\mu, \nu) = \Big(\inf_{\gamma \in \Pi(\mu,\nu)} \int c(\mathbf{x}, \mathbf{y})\, d\gamma(\mathbf{x}, \mathbf{y})\Big)^{1/p}, \tag{4}$$

whose entropic regularisation via Sinkhorn iterations reduces computational cost. In this work the method is applied to align heterogeneous graph-fragment distributions within a federated optimisation loop while preserving a measurable differential privacy budget (central to Method M2: FedLLM-API).

Differential privacy formalises data-record-level confidentiality through the condition

$$\mathsf{P}\big[\mathcal{M}(\mathcal{D}) \in S\big] \;\leq\; e^{\epsilon}\; \mathsf{P}\big[\mathcal{M}(\mathcal{D}') \in S\big] + \delta, \tag{5}$$

and its moments-accountant composition enables precise tracking of the cumulative privacy expenditure ϵ across training rounds (used in Methods M2 and M7).

State-space models reformulate the sequential processing of graph events as a linear dynamical system,

$$\mathbf{h}_{t+1} = \mathbf{A}\,\mathbf{h}_t + \mathbf{B}\,\mathbf{x}_t, \qquad \mathbf{y}_t = \mathbf{C}\,\mathbf{h}_t + \mathbf{D}\,\mathbf{x}_t, \tag{6}$$

whose convolutional kernel is evaluated in near-linear time through the Fast Fourier Transform (central to Methods M4: MambaShield and M5: Stochastic Transformer).

Robust optimisation embeds worst-case graph perturbations directly into parameter estimation through the min-max objective

$$\min_{\theta}\ \mathbb{E}_{(\mathbf{x},y)\sim\mathcal{D}}\Big[\max_{\|\boldsymbol{\delta}\|\le\epsilon} \mathcal{L}\big(f_\theta(\mathbf{x}+\boldsymbol{\delta}),\, y\big)\Big], \tag{7}$$

whose duality structure links the perturbation budget ϵ to the output sensitivity of the learned mapping on the graph (used in Methods M1 and M7).

Bayesian inference produces posterior predictive distributions that decompose total predictive uncertainty into epistemic uncertainty (attributable to limited training data) and aleatoric uncertainty (arising from inherent noise). In this work, variational Bayesian attention and hierarchical Gaussian processes provide this decomposition for graph-structured data (central to Methods M5 and M6).

Game-theoretic modelling formalises the adversarial setting as a strategic interaction between a defender (the graph classifier) and an attacker. Stackelberg equilibrium analysis and no-regret online learning algorithms determine optimal defensive strategies under a worst-case perturbation model (central to Method M7).

Byzantine-resilient aggregation employs coordinate-wise trimmed means to filter corrupted gradient updates in federated graph-learning settings (central to Method M2: FedLLM-API).

In addition to these mathematical methods, the research applies standard practices of software engineering, distributed systems design, and high-

performance computing to ensure that the resulting implementations can process large-scale temporal graph streams within the latency constraints of real-time operational environments.

Scientific Novelty

The scientific novelty of this work consists of seven contributions, each scoped to a specific model or algorithm developed by the author.

1. **CT-TGNN** — the *first* continuous-time temporal GNN unifying graph message passing with neural ODE dynamics on evolving topologies, with $O(1)$-memory adjoint training and Lipschitz-certified robustness bounds. No prior work derives certified robustness from the smoothness of the governing differential equation on temporally evolving graph topologies.

2. **FedLLM-API** — an *original* Byzantine-resilient federated graph optimiser combining trimmed-mean aggregation with (ε=0.85, δ=10^{-5}) differential privacy and Wasserstein OT alignment, with proven $\mathcal{O}(1/\sqrt{T})$ convergence under 30% corruption. No prior algorithm jointly guarantees corruption tolerance, privacy protection, and distributional alignment for graph-structured data.

3. **TripleE-TGNN** — a *novel* multi-level temporal graph embedding learning service/trace/node representations via dual attention with elastic weight consolidation against catastrophic forgetting. No prior architecture learns multi-level temporal graph representations while simultaneously preventing forgetting.

4. **MambaShield** — a *new* selective state-space architecture reducing complexity from $\mathcal{O}(L^2)$ to $\mathcal{O}(L \log L)$ via FFT convolutions, with progressive adversarial distillation and PAC-Bayesian certificates. Prior structured state-space models have not been adapted to graph data with formal robustness guarantees.

5. **Stochastic Transformer + UC-HGP** — the *first* uncertainty-calibrated GNN inference combining variational Bayesian attention with hierarchical Gaussian processes for epistemic–aleatoric decomposition. No prior work integrates these two uncertainty mechanisms for graph-structured online adaptation.

6. **Game-theoretic certification** — an *original* robustness certification via Stackelberg equilibria with no-regret multiplicative weights ($R(T) \leq \sqrt{T \log |S|}$), coupling all seven methods through consistency regularisation. No prior unified framework issues provable guarantees against all four

perturbation types simultaneously.

7. **CyberSecLLM** — a *novel* domain-specific foundation model with ODE-augmented state-space blocks, an 11-task benchmark, and million-token graph event sequence processing in 2.3 s. These contributions are distilled into the following main scientific results submitted for defence.

Main Scientific Results Submitted for Defence

The results submitted for defence are listed in the order that reflects their logical dependencies; each result builds upon the guarantees established by its predecessors.

(1) **CT-TGNN and SDE-TGNN models for certified graph dynamics** — original continuous-time architectures in which neural ODEs and stochastic differential equations govern node and edge evolution on dynamic topologies, yielding certified robustness bounds via Lipschitz analysis of the learned vector field.

(2) **FedLLM-API algorithm for privacy-preserving distributed graph learning** — convergence-proven federated optimiser combining Byzantine-resilient trimmed-mean aggregation with (ε=0.85, δ=10^{-5}) differential privacy and Wasserstein optimal-transport alignment for heterogeneous graph fragments.

(3) **TripleE-TGNN architecture for multi-level temporal graph representation** — triple-embedding framework learning service-, trace-, and node-level structure via dual attention and elastic weight consolidation, reducing computational cost through structured sparsity while preventing catastrophic forgetting.

(4) **MambaShield architecture for efficient graph inference** — selective state-space model reducing inference complexity from $\mathcal{O}(L^2)$ to $\mathcal{O}(L \log L)$ via FFT convolutions, with progressive adversarial distillation and PAC-Bayesian generalisation certificates.

(5) **Stochastic Transformer and UC-HGP models for uncertainty-aware drift adaptation** — variational Bayesian attention with hierarchical Gaussian process inference providing epistemic–aleatoric decomposition, online drift detection, and sub-linear regret guarantees under non-stationary graph

distributions.

(6) **Unified certification framework integrating models (1)–(5)** — single evaluation methodology coupling all developed models through game-theoretic Stackelberg equilibria and consistency regularisation, validated across 6 benchmark datasets totalling 84.2 million labelled samples.

Theoretical and Practical Significance

Theoretical significance. The work establishes a unified mathematical framework that contributes to the fields of graph-based machine learning, robust optimisation, and distributed AI by integrating five foundational theories: neural ODEs for certified continuous-time dynamics on graphs, optimal transport for privacy-preserving distributional alignment, selective state-space models for efficient temporal inference, Bayesian inference for calibrated uncertainty decomposition, and game-theoretic strategy selection for provable robustness certification. This framework advances the theoretical understanding of how robustness, stability, privacy, and efficiency interact on dynamic graph structures, providing formal convergence, generalisation, and privacy bounds that are applicable beyond any single domain.

Practical significance. The developed methods and algorithms are useful across multiple domains and can be implemented as follows. In healthcare, CT-TGNN and SDE-TGNN can be integrated into clinical decision-support and drug-interaction platforms by biomedical AI teams to provide certified-safe molecular graph predictions. In finance, FedLLM-API enables banks and regulators to collaboratively train fraud-detection models on transaction graphs without sharing customer data, deployable via existing federated infrastructure. In industrial IoT, MambaShield's near-linear complexity allows deployment on edge gateways and SCADA controllers by OT security engineers for real-time anomaly detection on sensor graphs. In autonomous systems, the uncertainty-calibrated methods (UC-HGP) provide safety engineers with epistemic confidence scores for sensor-fusion graph decisions. In cybersecurity, the open-source RobustIDPS.ai platform (DOI: 10.5281/zenodo.19129512) is directly usable by SOC analysts, cybersecurity engineers, MLSecOps researchers, and AI developers — designed as a plug-in for Snort, Suricata, and similar IDPS frameworks.

The public release of the software code base, experimental protocols, hyperparameter tables, and pre-trained checkpoints supports independent reproduction and practical adoption of the results.

Reliability of Scientific Results

Confidence in the reported results is established through multiple independent lines of validation.

Every proposed algorithm is accompanied by formal mathematical analysis: convergence theorems with explicit rate expressions, robustness certificates derived from the output-sensitivity analysis of graph neural dynamics and randomised smoothing adapted to discrete graph structures, privacy guarantees obtained through moments-accountant composition of the Gaussian mechanism, and computational complexity bounds confirmed by both asymptotic argument and empirical profiling. Proofs draw on standard tools from functional analysis, spectral graph theory, optimisation theory, probability, and the theory of ordinary differential equations.

Empirical evaluation spans six benchmark datasets that together supply more than eighty-four million labelled samples across eighty-four perturbation categories. Three general-purpose network datasets are used: CIC-IoT-2023 with 46.6 million records across 33 categories, CSE-CICIDS2018 with 16.2 million records across 7 categories, and UNSW-NB15 with 2.5 million records across 9 categories. Three specialised cloud and edge datasets from the ICS3D collection supplement these: the Microsoft GUIDE corpus with 13.7 million records, the Container Security corpus with 3.2 million records, and the Edge-IIoT corpus with 2.0 million records.

Ablation experiments systematically remove individual components to quantify the marginal contribution of each. Adversarial robustness is assessed under white-box attacks (FGSM, PGD, Carlini–Wagner), black-box transfer attacks, and adaptive attacks. All experiments are documented with hyperparameter tables, random-seed specifications, and hardware descriptions sufficient for independent replication. Source code and pre-trained weights are publicly available.

Approbation of the Work

The principal results of this research have been presented and peer-reviewed at international and national scientific venues.

At the IEEE Conference of Young Researchers in Electrical and Electronic Engineering (ElCon), held at ETU-LETI in Saint Petersburg in 2024 and 2025, contributions on graph-based device identification and reinforcement-learning-based poisoning detection received constructive feedback and recognition.

At the NEUROINFORMATICS International Conference in Moscow in 2024 and 2025, work on stochastic graph transformers with uncertainty quantification was discussed with specialists in computational intelligence.

A submission associated with the IEEE 27th International Conference of Young Professionals in Electron Devices and Materials (EDM) extends the graph-robustness framework to industrial control settings and is scheduled for 2026.

The developed algorithms have additionally been evaluated through collaborations with industry partners. Implementation acts have been received from four organisations: Rosatom State Corporation, Positive Technologies LLC, Google Cloud Platform, and Suricata Open Source IDPS.

Publications

The findings of this dissertation have been communicated through peer-reviewed publications in international journals and conference proceedings.

Published Journal Articles

1. Anaedevha, R. N., Trofimov, A. G., Borodachev, Y. V. (2026). MambaShield: Selective State-Space Models for Poisoning-Resilient Sequential Modeling. *Expert Systems with Applications*, Elsevier. DOI: 10.1016/j.eswa.2026.131175

2. Anaedevha, R.N., Trofimov, A.G. and Borodachev, Y.V., 2026. Uncertainty-calibrated hierarchical Gaussian processes for intrusion detection with multi-scale temporal modeling. *Neurocomputing* p.133105, Elsevier. DOI: 10.1016/j.neucom.2026.133105

3. Anaedevha, R. N., Trofimov, A. G. (2025). Stochastic Multimodal Trans-

former with Uncertainty Quantification for Robust Network Intrusion Detection. In: *Advances in Neural Computation, Machine Learning, and Cognitive Research IX. NEUROINFORMATICS 2025*. Studies in Computational Intelligence, vol. 1241. Springer, Cham. DOI: 10.1007/978-3-032-07690-8_35

4. Anaedevha, R. N., Trofimov, A. G. (2024). Improved Robust Adversarial Model against Evasion Attacks on Intrusion Detection Systems. *Optical Memory and Neural Networks (Information Optics)*, 33(Suppl. 3), S414–S423. Springer. DOI: 10.3103/S1060992X24700681

5. Anaedevha, R. N. *et al.* (2026). Temporal Adaptive Neural Ordinary Differential Equations with Deep Spatio-Temporal Point Processes for Real-Time Network Intrusion Detection. *Complex and Intelligent Systems*, Springer. DOI: 10.1007/s40747-026-02291-7

Published Conference Proceedings

6. Anaedevha, R. N. *et al.* (2025). Integrated Deep Q-Networks and Actor-Critic Models for Enhanced Poisoning Attack Detection in Network Intrusion Detection Systems. In: *2025 Conf. of Young Researchers in Electrical and Electronic Engineering (ElCon)*, ETU-LETI, Saint Petersburg. IEEE Xplore, Vol. 33, Issue Suppl. 3, pp. 556–567.

7. Anaedevha, R. N. *et al.* (2024). Application of Machine Learning Models for Device Identification in Wireless Network Traffic. In: *2024 Conf. of Young Researchers in Electrical and Electronic Engineering (ElCon)*, Saint Petersburg. IEEE Xplore, pp. 104–110. DOI: 10.1109/ElCon61730.2024.10468413

Papers Under Peer Review

9. Anaedevha, R. N. *et al.* (2026). TripleE-TGNN: Triple-Embedding Temporal Graph Neural Networks for Multi-Granularity Microservices Security. Submitted to *IEEE Trans. Dependable and Secure Computing*. ID: TDSC-2025-12-2593.

10. Anaedevha, R. N. *et al.* (2026). PQ-IDPS: Adversarially Robust Intrusion Detection for Post-Quantum Encrypted Traffic with Hybrid Classical-Quantum Machine Learning. Submitted to *IEEE Trans. Neural Networks and Learning Systems*. ID: TNNLS-2025-P-45911.

11. Anaedevha, R. N. *et al.* (2026). FedLLM-API: Privacy-Preserving Large Language Model Federation for Zero-Day API Threat Detection Across Organizational Boundaries. Submitted to *IEEE Trans. Information Forensics and Security*. ID: T-IFS-25044-2025.
12. Anaedevha, R. N. *et al.* (2026). CT-TGNN: Continuous-Time Temporal Graph Neural Networks for Real-Time Threat Propagation Analysis in Encrypted Zero-Trust Microservices. Submitted to *IEEE Trans. Services Computing*. ID: TSC-2025-12-1636.

Summary of Publications

The total output comprises 18 papers: 5 published in journals, 2 in conference proceedings, 2 accepted and in press, and 9 under review. Sixteen papers (89%) are first-authored. Eight appear in Scopus- and Web-of-Science-indexed journals, three in VAK-listed journals, and six are under review at Q1-ranked IEEE Transactions. Preprints are available on TechRxiv and Research Square, with citation records maintained on Google Scholar (h-index: 4) and ORCID (0000-0003-3649-1561).

Structure and Volume of the Work

The dissertation comprises an Introduction, six chapters, a Conclusion, a bibliographic list, and three appendices. The main text is 230 pages. The bibliography contains 204 entries.

The Introduction establishes the relevance of robust and efficient learning on dynamic graph structures as a foundational research challenge, presents documented evidence of GNN failures across four safety-critical domains, states the research objective with seven sequential tasks, defines the object and subject of investigation, describes the research methods with their formal equations, sets out the scientific novelty of seven developed models, specifies the theoretical and practical significance, formulates six main scientific results submitted for defence, confirms the reliability of results, and documents approbation activities and publications.

Chapter 1, "Analysis of Existing Literature and Gap Identification," surveys the literature following the progressive scope of the research: continuous-time

dynamics on evolving graphs, distributed federated learning with privacy, computational efficiency for large-scale graphs, and non-stationary adaptation with uncertainty quantification. The chapter is organised so that each section addresses one or more intersections of the three operating conditions and four cross-cutting requirements, building toward a systematic enumeration of seven research gaps. Each section concludes with a bridging statement that connects the identified gap to the solution developed in the following chapter, ensuring that the reader follows the logical chain from open problem to proposed method.

Chapter 2, "Methods, Algorithms, and Mathematical Frameworks for Robust Dynamic Graph Neural Networks," presents the complete mathematical derivations, convergence and robustness proofs, algorithmic pseudocode, architectural schematics, and complexity analyses for all seven methods (M1–M7) and their extensions (SDE-TGNN, CyberSecLLM). The chapter flows according to the gaps enumerated in Chapter 1: Method M1 (CT-TGNN) addresses Gap 1 with neural ODE dynamics and Lipschitz certification; Method M2 (FedLLM-API) addresses Gap 2 with Byzantine-resilient federated optimisation; Method M3 (TripleE-TGNN) addresses Gap 3 with multi-level temporal embeddings; Method M4 (MambaShield) addresses Gap 4 with selective state-space inference; Methods M5/M6 (Stochastic Transformer, UC-HGP) address Gap 6 with uncertainty-calibrated adaptation; and Method M7 addresses Gap 7 with game-theoretic unified certification. Each method is formulated at the level of mathematical operations on graph structures, independently of any application domain, and uniquely derived functions, loss formulations, and training schemes are presented with formal proofs.

Chapter 3, "Adaptation and Alignment to Network Intrusion Detection and Prevention Systems," is the domain-specific chapter in which the mathematical methods of Chapter 2 are mapped onto the entities and constraints of cybersecurity and, invariably, hybrid Network Intrusion Detection and Prevention Systems (NIDPS). The chapter defines the cybersecurity-specific terminologies, describes how raw network traffic (PCAP files and flow-level CSV data) is transformed into graph structures suitable for input into the developed GNN architectures, specifies the hyperparameters and their selection rationale for each model in the security context, defines the classification taxonomy (34 classes across 33 attack types and 1 benign class), presents the optimisation

procedures and loss functions as applied to network-security objectives, and describes the feature engineering pipeline (83 engineered features across five categories). Each section demonstrates how a specific method from Chapter 2 is instantiated with domain-specific adapters and threat-model refinements.

Chapter 4, "Experimental Results and Validation," presents the complete experimental programme demonstrating how the published research papers align with the methods of the dissertation. The chapter covers six benchmark datasets totalling 84.2 million labelled samples across 84 attack categories, a twenty-three-metric unified evaluation framework, method-specific results with tables and figures, baseline comparisons against classical ML and recent GNN defences, ablation studies quantifying the marginal contribution of each component, adversarial robustness assessments under six attack families (FGSM, PGD, C&W, DeepFool, Gaussian noise, label-masking poisoning) at configurable perturbation budgets, cross-domain transfer experiments confirming domain independence, and federated learning experiments under Byzantine corruption. Results are presented with tables, bar charts, line plots, and TikZ-rendered diagrams to ensure reproducibility and visual clarity.

Chapter 5, "RobustIDPS.ai: Software Platform for Production Deployment," describes the open-source software platform (DOI: 10.5281/zenodo.19129512) that implements all developed methods as a unified, deployable system. The chapter specifies the target users (SOC analysts, cybersecurity engineers, MLSecOps researchers, AI developers, and academic researchers), describes the platform's 46-page Progressive Web Application organised into 9 functional groups (AI Command Center, AI Active Defence, SOC Intelligence Suite, LLM Attack Surface Testing, MLSecOps Standards Compliance, AI Security and Governance, AI Novel Methods, AI Data and Models, and System Administration), details the 13+ integrated neural network models and their real-time inference capabilities, presents the 4-layer LLM defence framework achieving 94.5% detection across 517 attack scenarios with four commercial LLM backends (Claude, GPT-4o, Gemini, DeepSeek), describes the Multi-Agent PQC-IDS module for post-quantum traffic classification, and explains the platform's architecture (React 18 frontend, Flask/Python backend, Docker Compose orchestration, PostgreSQL persistence, Redis caching). The chapter includes at least six figures showing the platform's user interface

and operational results, demonstrates how the platform can be deployed as a plug-in for existing IDPS frameworks such as Snort and Suricata, and presents the three purpose-built PCAP validation datasets for reproducible benchmarking. The chapter concludes with a summary of the platform's achievements and its alignment with the research objective.

Chapter 6, "Application of the Unified Defence Framework to a Neural-Network UAV Navigation System," demonstrates the domain independence of the models and methods developed in Chapters 2–4 by transferring them to a new safety-critical area — counter-EW (electronic warfare) and adversarial-ML for the navigation subsystem of unmanned aerial vehicles. The chapter describes a three-tier *onboard* — *droneport* — *cloud* architecture, the operational UAV graph as an instance of the unified problem (§2.1), the mapping of methods M1–M7 to specific UAV subsystems (visual perception, GNSS navigation, autopilot policy), the Phase A reference implementation integrated with the RobustIDPS.ai platform, and conformance to the regulatory framework (Decree No. 1701, GOST R 72118-2025, NIST AI RMF, DO-326A/ED-202A).

The Conclusion lists and briefly explains each scientific result that was developed and is defended, recapitulating the progressive chain from continuous-time certified dynamics through distributed learning, efficient inference, uncertainty-calibrated adaptation, unified certification, domain-specific deployment, and platform implementation. It synthesises the principal findings, confirms that all twelve intersections of the condition-requirement matrix are addressed with verified performance, identifies the closest pathways to industry deployment, and outlines directions for further research including extension to heterogeneous graphs, integration with large multimodal models, and application to emerging domains.

Appendix A collects complete proofs and derivations for all theorems and lemmas stated in the main text. Appendix B provides algorithmic pseudocode, hyperparameter tables, and computational infrastructure specifications. Appendix C documents supplementary experimental results including per-class performance, sensitivity analyses, convergence plots, and computational profiling.

Chapter 1

Analysis of Dynamic Graph Neural Networks, Structural Problems, and Theoretical Foundations

This chapter surveys the existing literature on dynamic graph neural networks and their robustness challenges, following the progressive scope of the dissertation: from continuous-time dynamics on evolving graphs, through distributed federated learning with privacy, to computational efficiency for large-scale graphs, and finally to non-stationary adaptation with uncertainty quantification. Each section addresses one or more intersections of the three operating conditions (adversarial environment, distributed dynamics, non-stationarity) and the four cross-cutting requirements (robustness, accuracy, efficiency, privacy), building toward a systematic enumeration of seven research gaps that motivate the methods developed in the subsequent chapter.

1.1 Graph Neural Networks: Architecture, Vulnerability, and the Dual-Input Problem

1.1.1 Message-Passing Architecture and Neighbourhood Aggregation

Graph neural networks [1–3] extend deep learning to non-Euclidean relational data by iteratively aggregating information from a node's neighbours. The canonical message-passing formulation updates the representation of node v at layer $k+1$ as

$$\mathbf{h}_v^{(k+1)} = \phi\Big(\mathbf{h}_v^{(k)}, \bigoplus_{u \in \mathcal{N}(v)} \psi\big(\mathbf{h}_u^{(k)}, \mathbf{e}_{uv}\big)\Big), \tag{1.1}$$

where ϕ and ψ are learnable functions, $\bigoplus$ is a permutation-invariant aggregator (sum, mean, or max), and $\mathcal{N}(v)$ denotes the neighbourhood of v. This framework encompasses GCN [1], GAT [2], GraphSAGE [3], and their temporal extensions [26, 27, 37–39]. The expressiveness of the message-passing paradigm has been formally characterised by the Weisfeiler-Leman isomorphism hierarchy [40], establishing fundamental limits on what graph-level properties these architectures can distinguish.

Graph neural networks are deployed in healthcare (molecular property prediction on atom-bond graphs, patient-interaction graphs for treatment recommendations), finance (transaction-graph analysis for fraud detection and anti-money laundering), cybersecurity (network-traffic graphs for intrusion detection), industrial systems (sensor-fusion graphs for SCADA and ICS monitoring), and autonomous systems (sensor-fusion graphs for perception) [21, 41–45]. The breadth of these deployments underscores the urgency of robustness analysis.

1.1.2 The Dual-Input Vulnerability

Unlike conventional neural networks that receive a single data tensor, a graph neural network receives two inputs simultaneously: numerical *input data* $\mathbf{X} \in \mathbb{R}^{n \times d}$ (the feature matrix encoding entity attributes) and relational *input structure* $\mathbf{A} \in \{0,1\}^{n \times n}$ (the adjacency matrix encoding pairwise connectivity). The network produces its output by propagating data along the structure through iterative neighbourhood aggregation, and perturbation of *either* input — the data or the structure — can compromise the output [4–6].

Zügner et al. [4] demonstrated that modifying as few as five edges in a citation graph can flip the predicted label of a targeted node, while Bojchevski and Günnemann [5] showed that global poisoning attacks degrade node classification accuracy by up to 20% on benchmark datasets. Dai et al. [6] extended these findings to reinforcement-learning-based graph perturbation, and subsequent work [46, 47] confirmed that both feature and topological perturbations transfer across GNN architectures. Biggio and Roli [48] provided a comprehensive taxonomy of adversarial attacks on machine learning, while Carlini and Wagner [49] established benchmark evaluation methodologies. Kurakin et al. [50] demonstrated that adversarial perturbations persist in physical-world settings,

and Papernot et al. [51, 52] showed the feasibility of black-box attacks through transferability. Athalye et al. [53] later showed that obfuscated gradients give a false sense of security, motivating the need for certified rather than empirical defences. Bhagoji et al. [54] analysed adversarial attacks in distributed settings, Jia and Liang [55] studied data poisoning for recommendations, and Bose and Hamilton [56] developed adversarial attacks on graph embeddings. Sun et al. [57] surveyed graph-specific robustness methods, and Jin et al. [58] developed graph adversarial training frameworks. The adversarial robustness–accuracy trade-off was formally characterised by Tsipras et al. [59], and Tramèr et al. [60] demonstrated ensemble-based attacks that defeat gradient masking.

Gama, Bruna, and Ribeiro [61] introduced a stability framework bounding the output sensitivity of graph filters through the Lipschitz constant of their frequency response. However, these bounds grow rapidly with network depth and do not extend to the temporal, distributional, or adversarial settings that real-world graph systems encounter.

1.1.3 Documented Failures in Safety-Critical Domains

The vulnerability of graph neural networks is not a theoretical concern — it has been documented in peer-reviewed studies across four safety-critical domains. In healthcare, edge-sensitivity gradient attacks on Graph Isomorphism Networks caused approximately 25% AUC degradation on MoleculeNet benchmarks (Tox21, ClinTox, SIDER), with toxic compounds misclassified as safe [11]. In finance, the MonTi gang-injection attack nearly tripled the misclassification rate of GNN-based fraud detectors from 25.7% to 68.6% [12]. In industrial IoT, data poisoning of federated intrusion detectors caused a 64-percentage-point accuracy collapse from 94.2% to 30.0% under non-IID conditions [13]. In cybersecurity, the PROVNINJA attack reduced GNN provenance-IDS detection by up to 59%, rendering APT attackers invisible [14]. These documented failures confirm that GNN robustness lags predictive accuracy by 25–65 percentage points in every safety-critical domain.

Section summary. The message-passing mechanism that gives graph neural networks their expressive power also creates a dual-input attack surface exploitable through both feature and topological perturbations. Existing stability bounds

do not extend to temporal, distributional, or adversarial settings. The next section examines whether continuous-time formulations can provide the missing certified robustness guarantees.

1.2 Continuous-Time Dynamics on Evolving Graphs

1.2.1 Neural Ordinary Differential Equations for Graph Representation

Chen et al. [7] introduced neural ordinary differential equations (neural ODEs), replacing the discrete layer-by-layer updates of conventional networks with a continuous-depth parameterisation:

$$\frac{d\mathbf{h}(t)}{dt} = f_\theta\big(\mathbf{h}(t),\, t\big), \tag{1.2}$$

where f_θ is a neural network parameterising the vector field. The output at any time T is obtained by numerically integrating Equation (1.2) from $t = 0$ to $t = T$ using adaptive-step solvers, and gradients are computed memory-efficiently through the adjoint sensitivity method [25]. This formulation has been extended to graph-structured data: Chamberlain et al. [8] formulated graph neural diffusion as a continuous PDE on the graph, and Rusch et al. [9] developed coupled oscillatory graph neural ODEs that prevent over-smoothing. Rubanova et al. [62] extended neural ODEs to irregularly-sampled time series, Dupont et al. [63] introduced augmented neural ODEs with higher-dimensional state spaces, and Kidger et al. [64] developed neural controlled differential equations for irregular time series. Grathwohl et al. [65] connected neural ODEs to continuous normalising flows, and Finlay et al. [66] proposed regularisation techniques for stable training. Hasani et al. [67] developed liquid time-constant neural ODEs for adaptive computation, and Li et al. [68] applied neural ODEs to continuous graph neural networks on static topologies.

Temporal graph networks such as TGAT [26] and TGN [27] model time-stamped interactions on evolving graphs, but they operate in discrete time — each event triggers a single update step. This forces irregular graph events onto

a fixed computational clock, losing information about the precise timing and ordering of interactions between steps.

1.2.2 Lipschitz Stability and the Grönwall Bound

The key advantage of continuous-time formulations is their inherent smoothness. Because representations evolve according to a differential equation with a bounded Lipschitz constant L_g, the Grönwall inequality provides a closed-form robustness certificate:

$$\|\mathbf{h}_1(T) - \mathbf{h}_2(T)\| \leq e^{L_g T}\|\mathbf{x}_1 - \mathbf{x}_2\|, \tag{1.3}$$

where $\mathbf{h}_i(T)$ is the hidden state of trajectory i at time T and $\mathbf{x}_i$ is the initial input. This bound states that small perturbations of the input can only cause bounded, smooth changes in the output — the smoothness of the governing flow mathematically limits how much an adversary can move the prediction. Discrete-time graph neural networks have no such continuous handle, so perturbations compound unpredictably through each layer [69].

Randomised smoothing [70] and interval bound propagation [71] provide alternative certification pathways, but they were designed for image classifiers and do not account for the combinatorial structure of graph adjacency matrices or the continuous-time integration of neural ODEs. Lecuyer et al. [72] connected differential privacy to certified robustness, and Salman et al. [73] combined randomised smoothing with adversarial training. Shafahi et al. [74] developed free adversarial training for efficiency, and Wong and Kolter [75] provided scalable verified training through convex relaxation.

1.2.3 Limitations of Existing Continuous-Time Approaches

Despite the theoretical promise, no existing work derives certified robustness bounds from the smoothness of the governing differential equation on temporally evolving graph topologies. Existing continuous-time graph models either operate on static graphs [8] or handle temporal edges without Lipschitz analysis [26, 27, 37, 38, 76]. Multi-scale temporal dynamics (capturing both sub-

second transient events and hourly structural trends) have not been addressed within a certified framework. Furthermore, $O(1)$-memory adjoint training has not been combined with temporal adaptive normalisation for evolving graph topologies.

Gap 1: *No certified robustness under continuous-time graph dynamics.* No existing architecture provides formally certified robustness bounds for graph neural networks operating in continuous time on temporally evolving topologies, combining Lipschitz analysis of the governing vector field with memory-efficient adjoint training and multi-scale temporal dynamics.

Section summary. Continuous-time formulations offer a principled pathway to certified robustness through Lipschitz-bounded dynamics, but existing approaches do not extend to evolving graphs with temporal event streams. The next section examines how these models must be adapted for distributed environments where data cannot be centralised.

1.3 Distributed Graph Learning with Privacy Guarantees

1.3.1 Federated Learning Fundamentals

McMahan et al. [77] introduced Federated Averaging (FedAvg), enabling distributed training across multiple participants without centralising raw data. Each participant computes local gradient updates on its data fragment, and a central server aggregates these updates into a global model. This paradigm is essential for graph data in healthcare (patient-interaction graphs across hospitals), finance (transaction graphs across banks), and cybersecurity (traffic graphs across data centres), where privacy regulations prohibit data sharing [78–80]. Dwork [81] established the foundations of algorithmic privacy, and McMahan et al. [82] combined federated learning with differential privacy at scale. Li et al. [83] addressed heterogeneous federated learning through proximal regularisation, and Zhao et al. [84] characterised the non-IID challenge in federated settings. Wang et al. [85] proposed federated matched averaging for heterogeneous architectures. Sun and Lyu [86] demonstrated poisoning attacks against

federated learning, Fang et al. [87] developed local model poisoning attacks, and El-Mhamdi et al. [88] provided information-theoretic limits on Byzantine-resilient aggregation. Geyer et al. [89] proposed client-level differential privacy for federated learning, and Agarwal et al. [90–92] developed communication-efficient distributed SGD with privacy guarantees. Wei et al. [93] analysed the convergence of differentially private federated learning.

1.3.2 Byzantine-Resilient Aggregation

Standard federated averaging is vulnerable to Byzantine participants who submit corrupted gradient updates. Blanchard et al. [15] proposed Krum, which selects the update closest to its neighbours, while Yin et al. [94] showed that coordinate-wise trimmed means achieve minimax-optimal rates under Byzantine corruption. For graph-structured data, the topology itself may be partially controlled by an adversary, adding a combinatorial attack surface absent in standard federated settings.

1.3.3 Differential Privacy and Optimal Transport

Differential privacy [16] formalises data-record-level confidentiality through the condition

$$\mathsf{P}\big[\mathcal{M}(\mathcal{D}) \in S\big] \;\leq\; e^{\epsilon}\; \mathsf{P}\big[\mathcal{M}(\mathcal{D}') \in S\big] + \delta, \tag{1.4}$$

and the moments-accountant composition [17] enables precise tracking of the cumulative privacy expenditure across training rounds. Optimal transport [28, 95] provides a geometry on probability distributions through the Wasserstein distance, offering a principled mechanism for aligning heterogeneous graph-fragment distributions across participants. Courty et al. [96] applied optimal transport to domain adaptation, Flamary et al. [97] released the Python Optimal Transport library enabling scalable computation, and Genevay et al. [98] developed sample-efficient Sinkhorn divergences for generative modelling. However, no existing algorithm jointly guarantees Byzantine resilience, differential privacy, and optimal-transport distributional alignment for graph-structured data. The use of deep-learning methods for intrusion detection in heterogeneous environments has been systematically reviewed in the work of V. I. Vasilyev, A. M. Vulfin and M. B. Guzairov [99]; the detection

of network attacks via graph neural networks in distributed systems has been studied by I. V. Kotenko, I. B. Saenko and A. A. Branitskiy [100].

Gap 2: *Unfavourable privacy-utility trade-off in federated graphs.* No existing federated learning algorithm jointly provides Byzantine-resilient aggregation, formal (ϵ, δ)-differential privacy, and Wasserstein optimal-transport distributional alignment for graph-structured data, where the topology itself can be partially controlled by an adversary.

Section summary. Distributed graph learning requires simultaneous corruption tolerance, privacy protection, and distributional alignment — a combination no existing method achieves. The next section examines the computational efficiency challenges that any deployed solution must overcome.

1.4 Computational Efficiency for Large-Scale Temporal Graphs

1.4.1 Quadratic Bottleneck of Graph Transformers

Graph-transformer architectures [20, 21] achieve strong results on graph-level tasks by allowing every node to attend to every other node. However, this self-attention mechanism scales quadratically ($\mathcal{O}(L^2)$) with the number of nodes or events, making it prohibitively expensive for large-scale temporal graphs with millions of interactions. Linear-attention variants [101] reduce this to $\mathcal{O}(L)$ but sacrifice the expressiveness needed for graph-topological reasoning.

1.4.2 Structured State-Space Models

Gu et al. [22] introduced structured state-space models (S4) that reformulate sequential processing as a linear dynamical system whose convolutional kernel is evaluated in $\mathcal{O}(L \log L)$ time through the Fast Fourier Transform. The selective variant (Mamba) [24] adds input-dependent gating for improved modelling of discrete tokens. Behrouz et al. [30] began applying these models to graph data (GraphMamba), but their robustness under deliberate perturbation and their capacity to produce reliable confidence estimates have not been investigated. The theoretical foundations of state-space models trace back to

the HiPPO framework [102] for optimal polynomial projection, and Orvieto et al. [103] provided a unified analysis of linear recurrent architectures. Poli et al. [104] developed hyena operators as convolution-based alternatives, while Choromanski et al. [105, 106] proposed random feature attention for efficient transformers. Dosovitskiy et al. [107, 108] demonstrated that vision transformers achieve competitive performance through patch-based tokenisation, establishing the paradigm later adapted to graph sequences.

1.4.3 Multi-Level Representations

Temporal graphs in which nodes and edges carry different semantic types and evolve at different time scales require representations that capture structure at multiple levels of detail. Existing temporal GNNs learn a single, uniform embedding per node, which cannot resolve service-level patterns, trace-level dependencies, and node-level dynamics simultaneously. Multi-level representations are expensive to compute and have not been analysed for stability under topological evolution or for their interaction with catastrophic forgetting.

Gap 3: *Limited cross-distribution generalisation on temporal graphs.* No existing architecture learns multi-level temporal graph representations (service, trace, node) while simultaneously preventing catastrophic forgetting as the topology evolves and achieving computational savings through structured sparsity.

Gap 4: *Discrete-time constraints and quadratic cost.* Structured state-space models offering near-linear cost have not been adapted to graph data with formal robustness guarantees, progressive adversarial distillation, or PAC-Bayesian generalisation certificates.

Section summary. Efficiency and multi-level representation pose interrelated challenges: richer representations demand more computation, and efficient architectures lack robustness guarantees. The next section examines how deployed models must adapt to non-stationary conditions.

1.5 Non-Stationary Adaptation and Uncertainty Quantification

1.5.1 Distribution Drift in Deployed Models

Real graph data drifts after deployment: topologies evolve, attack patterns change, new protocols emerge, and static models silently lose accuracy without warning. This is unacceptable in safety-critical systems where a stale model is a dangerous model. Concept drift has been studied extensively in tabular and streaming settings [109, 110], but its interaction with graph-structured data — where both feature distributions and topological patterns shift simultaneously — introduces unique challenges. Parisi et al. [111] surveyed continual lifelong learning with neural networks, and Zenke et al. [112, 113] proposed synaptic intelligence as an alternative to EWC for preventing catastrophic forgetting.

1.5.2 Bayesian Approaches to Uncertainty

Bayesian neural networks produce posterior predictive distributions that decompose total uncertainty into epistemic (model) and aleatoric (data) components [18, 19, 31, 32]. Variational inference approximates the intractable posterior through parameterised distributions, enabling uncertainty estimation at the cost of additional forward passes (Monte Carlo dropout) or ensemble training. However, the integration of variational Bayesian attention with hierarchical Gaussian process uncertainty quantification for graph-structured online adaptation has not been achieved. Ovadia et al. [114] benchmarked uncertainty estimation under dataset shift, finding that most methods degrade significantly. Dusenberry et al. [115] proposed efficient and scalable Bayesian inference for neural networks, while Foong et al. [116] analysed the expressiveness of approximate Bayesian inference in neural networks. Depeweg et al. [117] formalised the decomposition of uncertainty into epistemic and aleatoric components for decision-making under model uncertainty. Kuleshov et al. [118] established calibration metrics for modern probabilistic forecasts. A Bayesian approach to adaptive selection of artificial-neural-network architectures, closely related to the uncertainty-regularised scheme proposed in this dissertation, has been

developed in the publication of the scientific supervisor — A. G. Trofimov and V. I. Skrugin [119]. Among the classical Russian foundations of recognition theory and intelligent systems, the algebraic approach of Yu. I. Zhuravlev to constructing correct algebras over sets of heuristic algorithms [120] and the works of D. A. Pospelov on reasoning modelling [121] should be noted as the methodological basis for the subsequent synthesis of adversarially robust models.

1.5.3 Online Learning with Sub-Linear Regret

Online convex optimisation provides a framework for sequential decision-making under adversarial data streams, with algorithms such as follow-the-regularised-leader achieving sub-linear regret bounds [122, 123]. The multiplicative weights update achieves $R(T) \leq \sqrt{T \log |S|}$ regret against the best fixed strategy in hindsight. However, the application of these bounds to graph neural network adaptation under simultaneous distributional shift and adversarial poisoning remains unexplored.

Gap 5: *Forward incompatibility with protocol regime change.* No existing domain-specific foundation model combines ODE-augmented state-space processing with multi-task graph event sequence modelling for forward-compatible detection under evolving protocols. The post-quantum cryptographic landscape, from Shor's algorithm [124] through lattice-based constructions [125, 126] and code-based schemes [127], introduces fundamental changes to the statistical features observed on graph edges. The foundation model paradigm established by Brown et al. [128] and Devlin et al. [129] has been extended to domain-specific applications, with Touvron et al. [130] demonstrating open-weight alternatives and Wei et al. [131] establishing chain-of-thought reasoning capabilities that enable structured threat analysis.

Gap 6: *Unintegrated uncertainty and adversarial robustness.* No existing architecture combines variational Bayesian attention with hierarchical Gaussian process uncertainty quantification for graph-structured data, providing simultaneous epistemic–aleatoric decomposition, online drift detection, and sub-linear regret guarantees under adversarial non-stationarity.

Section summary. Non-stationary adaptation requires online learning with formal regret guarantees and calibrated uncertainty decomposition — capabilities

not available in existing graph architectures. The final gap concerns how these diverse guarantees can be unified into a single certification framework.

1.6 Unified Robustness Certification

1.6.1 Game-Theoretic Threat Models

Tambe [33] and Sinha et al. [34] formalised security resource allocation as Stackelberg games, where a defender commits to a strategy anticipating the adversary's best response. In the graph neural network context, the defender is the classifier and the adversary perturbs the graph to maximise misclassification. Existing game-theoretic models for GNNs typically assume single-shot interactions with simplified perturbation budgets and do not account for the sequential, adaptive nature of real adversaries or the multi-method compositional structure of modern graph defence systems. Brückner and Scheffer [132] formalised the Stackelberg prediction game for adversarial classification, and Shoham and Leyton-Brown [133] provided the multiagent systems foundations. Balcan et al. [134] developed online learning algorithms with strategic interactions.

1.6.2 Existing Certification Methods

Randomised smoothing [70, 73] provides probabilistic robustness certificates for continuous input spaces. Interval bound propagation [71, 135] provides deterministic bounds by propagating interval-valued inputs through the network. However, neither approach has been adapted to the discrete, combinatorial structure of graph adjacency matrices, and no unified framework issues provable guarantees against feature perturbations, temporal-ordering shifts, graph-topological modifications, and distributional drift within a single evaluation methodology. PAC-Bayesian theory [136, 137] provides a complementary certification pathway through generalisation bounds, with Dziugaite and Roy [138] demonstrating non-vacuous bounds for deep networks, Guedj [139] surveying the modern PAC-Bayesian landscape, and Letarte et al. [140] connecting PAC-Bayesian bounds to majority vote classifiers.

Gap 7: *Oversimplified game-theoretic threat models.* No existing unified cer-

tification framework combines game-theoretic strategy selection (Stackelberg equilibria with no-regret online learning), interval bound propagation, Lipschitz estimation on graph operators, and randomised smoothing adapted to discrete graph structures within a single methodology that couples all component methods through consistency regularisation.

Section summary. Existing certification methods address individual perturbation types in isolation and do not extend to the combinatorial structure of graphs or the compositional architecture of multi-method defence systems. A unified framework is needed to tie together all guarantees developed across the preceding sections.

1.7 Adversarial Machine Learning in Adjacent Safety-Critical Domains: the Case of Neural-Network UAV Navigation

The principal target domain of the present dissertation — hybrid network intrusion detection and prevention systems — is only *one* of the applications of the domain-independent methods M1–M7 formulated in Chapter 2. To confirm domain-independence, Chapter 6 carries the framework over to the neural-network navigation subsystem of unmanned aerial vehicles (UAVs) operating under electronic-warfare (EW) and adversarial-machine-learning pressure. The present section briefly summarises the corresponding literature.

1.7.1 Degradation of UAV Neural-Network Components under EW Pressure

Open military reviews and analytical briefs document an accelerating degradation of UAV neural-network components as EW pressure grows. According to investigative journalism [141], the hit rate of the GPS-guided M982 Excalibur artillery shell fell from above 50 % to below 10 % within months in zones covered by Russian EW assets. The review [142] reports that up to 31 % of FPV-UAV sorties are lost to jamming. The Latvian electronic-communications authority recorded 820 GNSS-interference incidents

in 2024 versus 26 in 2022 — a more than 31-fold rise. A systematic survey of the domestic literature on EW against UAV command-and-control systems has been carried out by S. I. Makarenko [143].

1.7.2 Adversarial Attacks on Aerial Visual Detectors

Lian et al. [144] have shown that adaptive adversarial patches against the aerial detectors YOLOv2–v5, Faster R-CNN and SSD on the VisDrone benchmark reduce mean average precision (mAP) by 87.86 p.p. in the white-box and 85.48 p.p. in the black-box regime. Du et al. [145] obtained an mAP drop of YOLOv3 from $\approx$0.95 to $<$0.20 with physically printed patches. Xiang et al. [146] report $>$99 % success of targeted attacks on PointNet by adding only a few adversarial points.

1.7.3 Adversarial Attacks on Deep Reinforcement Learning Policies

Hickling, Aouf and Spencer [147] have shown that the iterative BIM method drops the obstacle-course completion rate of an APF+DDPG-PER UAV pilot from 97 % to 35 %, and proposed a SHAP detector of adversarial influences with 80–91 % detection rate. A review of adversarial attacks on UAV neural-network components and existing defence methods has been presented in the work of A. E. Lisin and S. G. Cherny [148].

1.7.4 GNSS Spoofing and Drift-Evasive Attacks

Borhani-Darian et al. [149] have built a deep convolutional network that detects spoofing at the signal level through cross-ambiguity-function (CAF) features. Aigner et al. [150] have obtained a detection error of 0.16 % on the TEXBAT benchmark with a Seq2Seq Transformer architecture. Panda and Guo [151] have formalised drift-evasive spoofing, in which the attacker slowly perturbs pseudoranges while remaining within signal-level thresholds and steering the trajectory by hundreds of metres. The application of deep-learning methods to GNSS spoofing detection from a UAV receiver has been studied by A. A. Galyaev, L. N. Lysenko and D. S. Yashin [152].

1.7.5 Graph Neural Networks in UAV-Swarm Tasks

Mou et al. [153] have applied a GCN to self-healing UAV-swarm communications. The hierarchical federated GAT [154] delivers below-2 % collision rate at 500 UAVs and Byzantine fraction $f < n/3$. The CM-BRF-ViT architecture [155] reaches 89.6 % accuracy at 40 % Byzantine clients against 45.6 % of vanilla FedAvg.

1.7.6 Regulatory Context for UAVs

In the Russian Federation, Government Decree No. 1701 of 30 November 2024 [156] requires that UAVs above 250 g be equipped with certified communication, remote-identification and cryptographic-information-protection means. GOST R 72118-2025 [157] regulates the formalisation of AI threat models, and GOST R 55679-2021 [158] sets general requirements for unmanned aerial systems. Methodological and regulatory aspects of certification of ML-based information-security tools are analysed in detail in the work of A. S. Markov and V. L. Tsirlov [159]. The international layer is shaped by NIST AI RMF 1.0 [160], the EU AI Act (which classifies safety-critical aero-AI as high-risk) and DO-326A/ED-202A [161].

Section summary. The adversarial-machine-learning literature for UAVs exhibits the same general threat structure as the network-intrusion literature: white- and black-box inference-time attacks, training-time data poisoning, adaptive adversaries, non-stationary environments. This justifies the transfer of the domain-independent methods M1–M7 to that domain, realised in Chapter 6.

1.8 Summary of Identified Research Gaps

The systematic analysis conducted in Sections 1.1–1.6 identifies seven research gaps that collectively define the open problem space addressed by this dissertation. Table 1.1 maps each gap to the operating conditions and requirements it involves, and identifies the method developed in Chapter 2 to address it.

Together, the seven gaps span all three operating conditions (C1: adversarial environment, C2: non-stationarity, C3: distributed dynamics) and all four

Table 1.1 – Seven identified research gaps, their condition–requirement coverage, and the methods proposed to address them in this dissertation.

#	**Gap Description**	**Conditions**	**Requirements**	**Proposed Method**
1	No certified robustness under continuous-time graph dynamics	C1, C2	R1, R2	M1: CT-TGNN
2	Unfavourable privacy-utility trade-off in federated graphs	C3	R1, R4	M2: FedLLM-API
3	Limited cross-distribution generalisation on temporal graphs	C2	R2, R3	M3: TripleE-TGNN
4	Discrete-time constraints and quadratic cost	C1, C2	R1, R3	M4: MambaShield
5	Forward incompatibility with protocol regime change	C2	R2, R3	CyberSecLLM
6	Unintegrated uncertainty and adversarial robustness	C1, C2	R1, R2	M5/M6: Stoch.Trans. + UC-HGP
7	Oversimplified game-theoretic threat models	C1, C2, C3	R1, R2, R3, R4	M7: Certification

requirements (R1: robustness, R2: accuracy, R3: efficiency, R4: privacy), confirming that the twelve intersections of the 3×4 condition–requirement matrix are covered by the proposed methods. Each gap has been demonstrated to be unresolved by existing work through the analysis presented in this chapter.

Chapter summary. This chapter has established the theoretical foundations and surveyed the existing literature on graph neural network robustness, continuous-time dynamics, distributed learning, computational efficiency, non-stationary adaptation, and unified certification. Seven specific research gaps have been identified, each supported by evidence from the literature that no existing method resolves the gap. The next chapter presents the mathematical methods, algorithms, and architectural frameworks developed to address these gaps, following the same progressive order: continuous-time certified dynamics (Gap 1), distributed federated learning (Gap 2), multi-level embeddings (Gap 3), efficient state-space inference (Gap 4), forward-compatible foundation models (Gap 5), uncertainty-calibrated adaptation (Gap 6), and unified game-theoretic certification (Gap 7).

Chapter 2

Methods, Algorithms, and Mathematical Frameworks for Robust Dynamic Graph Neural Networks

This chapter develops the complete theoretical core of the dissertation. It begins with a unified problem statement that is progressively derived from the fundamental graph classification problem by successively incorporating each of the four requirements (robustness, accuracy, efficiency, privacy) under the three operating conditions (adversarial environment, non-stationarity, distributed dynamics). The unified problem is then formally decomposed into seven sub-problems, each corresponding to one of the gaps identified in Chapter 1. The remainder of the chapter presents the mathematical formulation, convergence and robustness proofs, algorithmic pseudocode, and complexity analyses for each of the seven methods that solve these sub-problems. The chapter concludes with a unified certification framework that couples all methods through consistency regularisation.

2.1 Unified Problem Statement

The progressive derivation of the problem statement proceeds in seven stages, each introducing a new mathematical constraint that corresponds to a specific requirement or operating condition. The result is a single multi-objective optimisation problem whose decomposition yields the seven methods developed in this dissertation.

2.1.1 Stage 1: Graph Classification on Dynamic Topologies

Consider a temporal graph $\mathcal{G}(t) = (\mathcal{V}, \mathcal{E}(t), \mathbf{X}(t))$ in which nodes $\mathcal{V}$, edges $\mathcal{E}(t)$, and features $\mathbf{X}(t) = \{\mathbf{x}_v(t) \in \mathbb{R}^d\}_{v \in \mathcal{V}}$ evolve over time. A graph neural

network f_θ maps the graph at time T to a prediction. The baseline classification problem is

$$\min_{\theta} \; \mathbb{E}_{(\mathcal{G},y)\sim\mathcal{D}}\big[\mathcal{L}\big(f_\theta(\mathcal{G}(T)),\, y\big)\big], \tag{2.1}$$

where $\mathcal{L}$ is the cross-entropy loss and $\mathcal{D}$ is the data-generating distribution. This formulation assumes that the graph is clean, static during inference, centrally available, and that the data distribution does not change. Each of these assumptions is violated in practice.

2.1.2 Stage 2: Adversarial Robustness

Under the adversarial environment (Condition C1), a deliberate opponent perturbs the graph. Incorporating Requirement R1 (robustness) converts the problem to a min-max formulation:

$$\min_{\theta} \; \mathbb{E}_{(\mathcal{G},y)\sim\mathcal{D}}\Big[\max_{\|\boldsymbol{\delta}\|\le\epsilon_{\text{adv}}} \mathcal{L}\big(f_\theta(\mathcal{G}(T)+\boldsymbol{\delta}),\, y\big)\Big], \tag{2.2}$$

where $\boldsymbol{\delta}$ represents perturbations to node features, edge weights, or topological structure, and ϵ_{adv} is the perturbation budget. This is the standard robust optimisation formulation [35, 162, 163].

2.1.3 Stage 3: Continuous-Time Dynamics with Certified Bounds

To obtain certified robustness rather than empirical adversarial training, the model's internal dynamics must be constrained. Requiring that f_θ evolves through a neural ODE with Lipschitz-bounded vector field adds the constraint:

$$\frac{d\mathbf{h}_v(t)}{dt} = f_\theta\big(\mathbf{h}_v(t),\, \mathbf{A}(t),\, t\big), \qquad \|f_\theta(\mathbf{h}_1,t) - f_\theta(\mathbf{h}_2,t)\| \le L_g\|\mathbf{h}_1-\mathbf{h}_2\|, \tag{2.3}$$

which, by Grönwall's inequality, guarantees $\|\mathbf{h}_1(T)-\mathbf{h}_2(T)\| \le e^{L_g T}\|\mathbf{x}_1-\mathbf{x}_2\|$. This transforms the uncertified adversarial training of Equation (2.2) into a certified robustness problem. *Sub-problem 1 (Gap 1) is to solve Equations (2.2)–(2.3) jointly, with multi-scale temporal dynamics and $O(1)$-memory adjoint training. This is addressed by Method M1: CT-TGNN.*

2.1.4 Stage 4: Distributed Learning with Privacy

Under distributed dynamics (Condition C3), the graph $\mathcal{G}$ is partitioned across K participants $\mathcal{D}_1, \ldots, \mathcal{D}_K$ who cannot share raw data. Incorporating Requirement R4 (privacy) requires that the training algorithm satisfies (ϵ, δ)-differential privacy while tolerating up to a fraction q of Byzantine participants:

$$\min_{\theta} \frac{1}{K} \sum_{k=1}^{K} \mathcal{L}_k(\theta; \mathcal{D}_k) \quad \text{subject to} \quad \mathsf{P}[\mathcal{M}(\mathcal{D}) \in S] \leq e^{\epsilon}\, \mathsf{P}[\mathcal{M}(\mathcal{D}') \in S] + \delta, \tag{2.4}$$

where $\mathcal{M}$ is the training mechanism and the constraint must hold for all neighbouring datasets $\mathcal{D}, \mathcal{D}'$. Additionally, the heterogeneous distributions across participants require alignment through the Wasserstein distance $W_1(\mathcal{D}_k, \mathcal{D}_{k'})$, which must itself be computed under the privacy constraint. *Sub-problem 2 (Gap 2) is to solve Equation (2.4) with Byzantine-resilient aggregation and optimal-transport alignment while consuming the Lipschitz certificates from Sub-problem 1. This is addressed by Method M2: FedLLM-API.*

2.1.5 Stage 5: Multi-Level Representation and Anti-Forgetting

Incorporating Requirement R2 (accuracy) on temporal graphs with hierarchical structure requires learning representations at multiple semantic and temporal scales — component, trace, and node levels — while preventing catastrophic forgetting as the topology evolves:

$$\min_{\theta} \sum_{g \in \{s,r,n\}} \lambda_g\, \mathcal{L}_g(\theta; \mathcal{G}_g) + \lambda_{\text{ewc}} \sum_{j} F_j(\theta_j - \theta_j^*)^2 + \lambda_{\text{con}} \sum_{(g,g')} \text{KL}(p_g \| p_{g'}), \tag{2.5}$$

where F_j is the Fisher information penalising departure from previously learned parameters θ^*, and the KL-divergence term enforces cross-level consistency. *Sub-problem 3 (Gap 3) is to solve Equation (2.5) with structured sparsity and quantisation-aware training. This is addressed by Method M3: TripleE-TGNN.*

2.1.6 Stage 6: Computational Efficiency

Requirement R3 (efficiency) demands that inference complexity is at most $\mathcal{O}(L \log L)$ rather than the $\mathcal{O}(L^2)$ of transformer-based models. This is achieved by reformulating the sequential processing as a selective state-space model with input-dependent transitions:

$$\mathbf{h}_k = \bar{\mathbf{A}}(u_k)\,\mathbf{h}_{k-1} + \bar{\mathbf{B}}(u_k)\,u_k, \qquad \bar{\mathbf{A}}(u_k) = \exp(\mathbf{A}(u_k)\Delta), \tag{2.6}$$

whose convolutional kernel is evaluated through the Fast Fourier Transform. This must be combined with progressive adversarial robustness distillation and PAC-Bayesian generalisation certificates. *Sub-problem 4 (Gap 4) is to solve the efficient inference problem of Equation (2.6) with formal robustness guarantees under training-time poisoning. This is addressed by Method M4: MambaShield.*

2.1.7 Stage 7: Non-Stationary Adaptation with Uncertainty

Under non-stationarity (Condition C2), the distribution $\mathcal{D}_t$ changes over time. The model must adapt online while quantifying both epistemic and aleatoric uncertainty and achieving sub-linear regret:

$$R(T) = \max_{f^*} \sum_{t=1}^{T} \mathcal{L}(f_{\theta_t}, y_t) - \sum_{t=1}^{T} \mathcal{L}(f^*, y_t) \leq \mathcal{O}(\sqrt{T \log |\mathcal{S}|}), \tag{2.7}$$

where the comparison is against the best fixed policy in hindsight. The predictive uncertainty decomposes as $U_{\text{total}} = U_{\text{epi}} + U_{\text{ale}}$, with epistemic uncertainty driving drift detection and aleatoric uncertainty capturing inherent noise.

Sub-problem 5 (Gap 5) concerns forward compatibility under protocol regime change and is addressed by a domain-specific foundation model (CyberSecLLM). Sub-problem 6 (Gap 6) is to solve the online adaptation problem of Equation (2.7) with integrated epistemic–aleatoric decomposition and Bayesian posterior updates. This is addressed by Methods M5: Stochastic Transformer and M6: UC-HGP.

2.1.8 The Complete Unified Problem

Combining all stages yields the unified multi-objective optimisation problem of this dissertation:

$$\min_{\theta} \underbrace{\mathbb{E}_{(\mathcal{G},y)\sim\mathcal{D}_t}\Big[\max_{\|\boldsymbol{\delta}\|\leq\epsilon} \mathcal{L}\big(f_\theta(\mathcal{G}+\boldsymbol{\delta}),y\big)\Big]}_{\text{Adversarial robustness (R1, C1)}} + \lambda_{\text{acc}} \underbrace{\sum_g \mathcal{L}_g(\theta;\mathcal{G}_g)}_{\text{Multi-level accuracy (R2)}}$$
$$+ \lambda_{\text{eff}} \underbrace{\text{Complexity}(f_\theta)}_{\text{Efficiency} \leq \mathcal{O}(L\log L)\text{ (R3)}} + \lambda_{\text{priv}} \underbrace{\text{PrivCost}(\epsilon_{\text{DP}},\delta)}_{\text{Differential privacy (R4, C3)}}$$
$$+ \lambda_{\text{drift}} \underbrace{R(T)}_{\text{Non-stationary regret (C2)}} + \lambda_{\text{ewc}} \underbrace{\sum_j F_j(\theta_j-\theta_j^*)^2}_{\text{Anti-forgetting}} + \lambda_{\text{con}} \underbrace{\sum_{i<j} \mathcal{L}_{ij}(\theta_i,\theta_j)}_{\text{Method coupling (M7)}} \tag{2.8}$$

subject to the constraints:

- Lipschitz-bounded ODE dynamics: $\|f_\theta(\mathbf{h}_1,t)-f_\theta(\mathbf{h}_2,t)\| \leq L_g\|\mathbf{h}_1-\mathbf{h}_2\|$ (certified robustness)
- Differential privacy: $\mathsf{P}[\mathcal{M}(\mathcal{D})\in S] \leq e^\epsilon\,\mathsf{P}[\mathcal{M}(\mathcal{D}')\in S]+\delta$ (privacy)
- Byzantine resilience: convergence under fraction $q<1/2$ corrupted participants
- Sub-linear regret: $R(T)\leq\mathcal{O}(\sqrt{T\log|\mathcal{S}|})$ (adaptation)

Decomposition. This unified problem is intractable as a monolithic optimisation. However, it admits a natural decomposition into seven sub-problems, one per identified gap, that can be solved by alternating optimisation with consistency coupling. Table 2.1 maps each sub-problem to its gap, the terms in Equation (2.8) it addresses, and the method that solves it.

Sub-problem 7 (Gap 7) is to couple all methods through the consistency regularisation term $\sum_{i<j}\mathcal{L}_{ij}$ in Equation (2.8) and prove convergence of the alternating optimisation. This is addressed by Method M7: Certification and the unified framework developed in Section 2.8.5.

The remainder of this chapter develops each method in the order of the sub-problem chain: M1 (Section 2.2), M2 (Section 2.3), M3 (Section 2.4), M4

Table 2.1 – Decomposition of the unified problem statement into seven sub-problems.

SP	Method	Terms Addressed	Key Constraint
1	M1: CT-TGNN	Adversarial robustness + ODE dynamics	Lipschitz certificate: $e^{L_g T}\|\delta\mathbf{x}\|$
2	M2: FedLLM-API	Privacy cost + Byzantine aggregation	(ϵ, δ)-DP + OT alignment
3	M3: TripleE-TGNN	Multi-level accuracy + anti-forgetting	EWC Fisher penalty + cross-level KL
4	M4: MambaShield	Efficiency + adversarial robustness	$\mathcal{O}(L \log L)$ + PAC-Bayes bound
5	CyberSecLLM	Forward compatibility (domain-specific)	Multi-task ODE-S6 blocks
6	M5/M6	Non-stationary regret + uncertainty	Sub-linear regret + epi/ale decomposition
7	M7: Certification	Method coupling + game-theoretic	Stackelberg equilibrium + consistency

(Section 2.5), M5/M6 (Section 2.6), forward-compatible detection (Section 2.7), and M7 with the unified framework (Section 2.8).

2.2 Method M1: CT-TGNN — Continuous-Time Temporal Graph Neural Dynamics

This section solves Sub-problem 1 by developing a graph neural network whose node representations evolve according to a neural ordinary differential equation, yielding the certified robustness bound from the Lipschitz constraint in Equation (2.3). The method addresses Gap 1 (Adversarial × Robustness; Non-stationary × Robustness).

The continuous-time node dynamics are governed by

$$\frac{d\mathbf{h}_v(t)}{dt} = f_\theta\Big(\mathbf{h}_v(t),\ \{\mathbf{h}_u(t)\}_{u\in\mathcal{N}_v(t)},\ \mathbf{A}(t)\Big), \tag{2.9}$$

where $\mathbf{h}_v(t) \in \mathbb{R}^{d_h}$ is the embedding of node v at time t, $\mathcal{N}_v(t)$ denotes the neighbourhood of v under the current topology, $\mathbf{A}(t)$ encodes the adjacency structure, and f_θ is a neural network parameterised by θ.

For heterogeneous graphs the right-hand side employs type-specific message passing. Let v belong to type k and let $\mathcal{N}_v^{(k')}(t)$ denote its neighbours of type k'.

The dynamics are

$$\frac{d\mathbf{h}_v^{(k)}(t)}{dt} = \sum_{k'=1}^{K} \sum_{u \in \mathcal{N}_v^{(k')}(t)} \alpha_{uv}^{(k,k')}(t)\ \mathrm{MSG}_{k,k'}\big(\mathbf{h}_u^{(k')}(t),\ \mathbf{e}_{uv}(t)\big), \tag{2.10}$$

where the attention coefficients [2]

$$\alpha_{uv}^{(k,k')}(t) = \frac{\exp\big(\mathrm{LeakyReLU}\big(\mathbf{a}_{k,k'}^{\top}[\mathbf{W}_k\mathbf{h}_v^{(k)}(t) \| \mathbf{W}_{k'}\mathbf{h}_u^{(k')}(t)]\big)\big)}{\sum_{w \in \mathcal{N}_v^{(k')}(t)} \exp\big(\mathrm{LeakyReLU}\big(\mathbf{a}_{k,k'}^{\top}[\mathbf{W}_k\mathbf{h}_v^{(k)}(t) \| \mathbf{W}_{k'}\mathbf{h}_w^{(k')}(t)]\big)\big)} \tag{2.11}$$

weight neighbour importance adaptively, $\mathbf{W}_k, \mathbf{W}_{k'}$ are learnable projections, and $\|$ denotes concatenation.

2.2.1 Multi-Scale Temporal Dynamics

Phenomena that unfold across multiple time scales, from sub-second transients to month-long trends, require parallel ODE branches with distinct time constants. For scales $s = 1, \ldots, S$ with constants $\tau_1 < \tau_2 < \cdots < \tau_S$ the dynamics are

$$\frac{d\mathbf{h}_v^{(s)}(t)}{dt} = \frac{1}{\tau_s} f_{\theta_s}\big(\mathbf{h}_v^{(s)}(t),\ \{\mathbf{h}_u^{(s)}(t)\}_{u \in \mathcal{N}_v(t)},\ \mathbf{A}(t),\ t\big), \tag{2.12}$$

and the final embedding aggregates across scales through learned attention,

$$\mathbf{h}_v(t) = \sum_{s=1}^{S} \beta_s(t)\ \mathbf{h}_v^{(s)}(t), \qquad \beta_s(t) = \mathrm{Softmax}_s\big(\mathbf{w}_s^{\top}\mathbf{h}_v^{(s)}(t)\big), \tag{2.13}$$

so that the relative contribution of each time scale adapts to the local dynamics.

2.2.2 Adjoint Sensitivity Method for Memory-Efficient Training

Training requires gradients of a loss $\mathcal{L} = \mathcal{L}(\mathbf{h}(T))$ with respect to θ through the ODE solution. Direct back-propagation through the numerical integrator stores all intermediate states, which is prohibitive for long horizons. The adjoint method [7, 25] defines an adjoint state $\mathbf{a}(t) = \partial\mathcal{L}/\partial\mathbf{h}(t)$ satisfying the backward

ODE

$$\frac{d\mathbf{a}(t)}{dt} = -\mathbf{a}(t)^\top \frac{\partial f_\theta}{\partial \mathbf{h}}(\mathbf{h}(t), t), \tag{2.14}$$

integrated from $\mathbf{a}(T) = \partial \mathcal{L}/\partial \mathbf{h}(T)$ to $\mathbf{a}(0)$. Parameter gradients follow from

$$\frac{\partial \mathcal{L}}{\partial \theta} = -\int_0^T \mathbf{a}(t)^\top \frac{\partial f_\theta}{\partial \theta}(\mathbf{h}(t), t)\, dt. \tag{2.15}$$

Memory cost is proportional to the model size rather than the trajectory length, enabling training on arbitrarily long temporal horizons.

2.2.3 Temporal Adaptive Batch Normalisation

Standard batch normalisation computes statistics over a fixed mini-batch, which is incompatible with continuous-time dynamics in which the distribution of hidden states changes continuously. Temporal adaptive batch normalisation computes running statistics over a sliding temporal window $[t - \Delta, t]$,

$$\mathrm{TABN}\big(\mathbf{h}(t)\big) = \gamma(t)\, \frac{\mathbf{h}(t) - \mu_{[t-\Delta,t]}}{\sqrt{\sigma^2_{[t-\Delta,t]} + \varepsilon}} + \beta(t), \tag{2.16}$$

where $\mu_{[t-\Delta,t]}$ and $\sigma^2_{[t-\Delta,t]}$ are the windowed mean and variance, and the affine parameters $\gamma(t), \beta(t)$ themselves evolve through auxiliary ODEs $d\gamma/dt = g_\gamma(\mathbf{h}(t))$, $d\beta/dt = g_\beta(\mathbf{h}(t))$ driven by learned adaptation networks.

2.2.4 Convergence and Robustness Analysis

Theorem 2.1 (CT-TGNN Convergence)**.** *Suppose the dynamics function f_θ is L_f-Lipschitz in $\mathbf{h}$ and L_θ-Lipschitz in θ. Under gradient descent with learning rate $\eta < 2/(L_f L_\theta)$, the training loss converges to a stationary point at rate $\mathcal{O}(1/\sqrt{T})$, where T denotes the number of gradient steps.*

Proof sketch. The loss $\mathcal{L}(\theta)$ is evaluated through the ODE solution map $\mathbf{h}(T; \theta)$. By the chain rule and the Lipschitz properties of f_θ, the gradient $\nabla_\theta \mathcal{L}$ is bounded and Lipschitz-continuous, placing the problem in the standard framework of smooth non-convex optimisation. Applying the descent lemma with the stated learning rate yields the $\mathcal{O}(1/\sqrt{T})$ rate to an ϵ-stationary point. The full proof appears in Appendix A. □

Theorem 2.2 (Lipschitz Robustness Certificate)**.** *Let f_θ be L_g-Lipschitz in its first argument and let the integration horizon be T. For any two inputs $\mathbf{x}_1, \mathbf{x}_2$ the corresponding embeddings satisfy*

$$\|\mathbf{h}_1(T) - \mathbf{h}_2(T)\| \leq e^{L_g T} \, \|\mathbf{x}_1 - \mathbf{x}_2\|. \tag{2.17}$$

Consequently, if the classifier head has Lipschitz constant L_c, the end-to-end model is $L_c \, e^{L_g T}$-Lipschitz, and the prediction is certified constant within a ball of radius $\rho = \Delta_{\min}/(L_c \, e^{L_g T})$, where $\Delta_{\min}$ is the margin between the top two logits.

Proof sketch. Let $\boldsymbol{\eta}(t) = \mathbf{h}_1(t) - \mathbf{h}_2(t)$. Then $d\boldsymbol{\eta}/dt = f_\theta(\mathbf{h}_1, t) - f_\theta(\mathbf{h}_2, t)$, so $d\|\boldsymbol{\eta}\|/dt \leq L_g \|\boldsymbol{\eta}\|$ by the Lipschitz property. Grönwall's inequality gives $\|\boldsymbol{\eta}(T)\| \leq e^{L_g T} \|\boldsymbol{\eta}(0)\|$, and $\boldsymbol{\eta}(0) = \mathbf{x}_1 - \mathbf{x}_2$. Composing with the classifier head completes the certificate. The full derivation with spectral-norm constraints enforcing the Lipschitz bound during training appears in Appendix A. □

2.2.5 Computational Complexity

Solving the graph ODE with an adaptive Runge-Kutta integrator at tolerance ε_{ode} requires $\mathcal{O}(\varepsilon_{\text{ode}}^{-1/5})$ function evaluations. Each evaluation performs message passing at cost $\mathcal{O}(|\mathcal{V}| \, d \, d_h)$ for bounded degree d and embedding dimension d_h. The total training complexity per epoch is therefore

$$\mathcal{O}\big(|\mathcal{V}| \, d \, d_h \, \varepsilon_{\text{ode}}^{-1/5} \, T_{\text{train}}\big), \tag{2.18}$$

which is near-linear in graph size when $|\mathcal{E}| = \mathcal{O}(|\mathcal{V}|)$.

Algorithm 1: CT-TGNN Training with Adjoint Sensitivity

Input: Temporal graph sequence $\{\mathcal{G}(t_i)\}_{i=1}^{N}$, labels $\{y_i\}$, learning rate η, integration horizon T, tolerance ε_{ode}

Output: Trained parameters θ^{*}

Initialise parameters θ_0 and multi-scale time constants $\tau_1 < \cdots < \tau_S$;

for *epoch* $= 1, 2, \ldots, E$ **do**

  for *each mini-batch* $\{(\mathcal{G}(t_j), y_j)\}$ **do**

    Set initial embeddings $\mathbf{h}_v(0) \leftarrow \text{Embed}(\mathbf{x}_v(t_j))$ for all $v \in \mathcal{V}$;

    for *scale* $s = 1, \ldots, S$ **do**

      Solve $d\mathbf{h}_v^{(s)}/dt = \tau_s^{-1} f_{\theta_s}(\mathbf{h}_v^{(s)}, \mathcal{N}_v, \mathbf{A}, t)$ from 0 to T using adaptive RK with tolerance ε_{ode};

    Compute fused embeddings $\mathbf{h}_v(T) = \sum_s \beta_s(T)\, \mathbf{h}_v^{(s)}(T)$;

    Compute classification loss $\mathcal{L} = \mathcal{L}_{\text{CE}}(f_{\text{head}}(\mathbf{h}(T)),\, y)$;

    Set adjoint terminal condition $\mathbf{a}(T) = \partial\mathcal{L}/\partial\mathbf{h}(T)$;

    Solve adjoint ODE backward: $d\mathbf{a}/dt = -\mathbf{a}^{\top}(\partial f_\theta/\partial\mathbf{h})$ from T to 0;

    Accumulate parameter gradients $\partial\mathcal{L}/\partial\theta = -\int_0^T \mathbf{a}^{\top}(\partial f_\theta/\partial\theta)\, dt$;

    Update $\theta \leftarrow \theta - \eta\, \partial\mathcal{L}/\partial\theta$;

Compute Lipschitz certificate $\rho = \Delta_{\min}/(L_c\, e^{L_g T})$ for each test input;

return θ^{*}, $\{\rho_i\}$

2.3 Method M2: FedLLM-API — Federated Graph Optimisation with Privacy

This section solves Sub-problem 2 by developing a federated learning algorithm that achieves the privacy constraint and Byzantine resilience of Equation (2.4) while consuming the Lipschitz certificates produced by M1. The method addresses Gap 2 (Distributed × Robustness; Distributed × Privacy; Distributed × Efficiency).

2.3.1 Byzantine-Robust Aggregation

Consider K participants with local datasets $\mathcal{D}_1, \ldots, \mathcal{D}_K$, of which at most a fraction q are Byzantine. At communication round t the server broadcasts global model $\mathbf{w}_t$ and each honest client k computes a local update through gradient descent on its own data, $\mathbf{w}_{t+1}^{(k)} = \mathbf{w}_t - \eta \nabla_{\mathbf{w}} \mathcal{L}_k(\mathbf{w}_t; \mathcal{D}_k)$. Malicious clients submit arbitrary vectors. The server aggregates through coordinate-

wise trimmed mean [15, 83, 94]: for each parameter dimension j the client values are sorted and the $b = \lfloor Kq \rfloor$ largest and smallest are discarded, yielding

$$\bar{w}_{t+1,j} = \frac{1}{K - 2b} \sum_{i=b+1}^{K-b} w_{t+1,j}^{(i)}, \tag{2.19}$$

where $w_{t+1,j}^{(i)}$ is the i-th order statistic.

Theorem 2.3 (Byzantine-Robust Convergence)**.** *Assume the global loss $F(\mathbf{w}) = \frac{1}{K}\sum_{k=1}^{K} F_k(\mathbf{w})$ is μ-strongly convex and L-smooth, and the fraction of Byzantine clients satisfies $f < q < 1/2$. With learning rate $\eta \leq 1/L$, coordinate-wise trimmed mean achieves*

$$\mathbb{E}\big[\|\mathbf{w}_T - \mathbf{w}^*\|^2\big] \leq \Big(1 - \frac{\mu\eta}{2}\Big)^T \|\mathbf{w}_0 - \mathbf{w}^*\|^2 + \frac{2\eta}{\mu(K-2b)}\Big(\sum_{k=1}^{K-2b} \sigma_k^2 + C_{\text{byz}}\Big), \tag{2.20}$$

where σ_k^2 is the gradient variance of honest client k and C_{byz} bounds the Byzantine influence.

Proof sketch. The key step is showing that the trimmed mean of honest-plus-Byzantine gradient vectors is close to the true mean of honest gradients. Let $\bar{\mathbf{g}}_H$ be the mean of honest gradients and $\bar{\mathbf{g}}_{\text{TM}}$ the trimmed-mean output. Because at most b of the $2b$ discarded entries can be honest, the error $\|\bar{\mathbf{g}}_{\text{TM}} - \bar{\mathbf{g}}_H\|$ is bounded by the order statistics of honest gradient norms via concentration inequalities. Substituting into the standard strongly convex descent recursion yields the stated bound. Full details are in Appendix A. □

2.3.2 Differential Privacy Through Calibrated Noise

Each client clips its gradient norm to a threshold C and adds Gaussian noise [16, 17, 81] before transmission,

$$\bar{\mathbf{g}}_k = \mathbf{g}_k \Big/ \max\Big(1, \frac{\|\mathbf{g}_k\|}{C}\Big), \qquad \tilde{\mathbf{g}}_k = \bar{\mathbf{g}}_k + \mathcal{N}(\mathbf{0}, \sigma^2 C^2 \mathbf{I}), \tag{2.21}$$

where the noise scale σ is determined by the target privacy parameters (ϵ, δ). Privacy amplification by sub-sampling with batch probability q_b and composition over T rounds through the moments accountant [164, 165] yields the overall

privacy guarantee

$$\epsilon_{\text{total}} = \min_{\alpha>1} \frac{1}{\alpha - 1} \log \frac{1}{\delta} + \frac{T\, q_b^2\, \alpha}{2\sigma^2}. \tag{2.22}$$

2.3.3 Optimal Transport for Distribution Alignment

Heterogeneous client distributions degrade convergence. Optimal transport aligns them by computing the Wasserstein barycenter. Given discrete source samples $\{\mathbf{x}_i\}_{i=1}^{n_s}$ and target samples $\{\mathbf{y}_j\}_{j=1}^{n_t}$, the discrete transport plan solves

$$\mathbf{T}^* = \arg\min_{\mathbf{T} \in \mathbb{R}_+^{ns \times n_t}} \langle \mathbf{C}, \mathbf{T} \rangle_F \quad \text{s.t.} \quad \mathbf{T}\mathbf{1}_{n_t} = \mathbf{a},\ \mathbf{T}^\top \mathbf{1}_{n_s} = \mathbf{b}, \tag{2.23}$$

where $C_{ij} = c(\mathbf{x}_i, \mathbf{y}_j)$ is the cost matrix and $\mathbf{a}, \mathbf{b}$ lie in probability simplices. The Sinkhorn algorithm [28, 95, 166] computes the entropically regularised solution through alternating scaling,

$$\mathbf{u}^{(\ell+1)} = \mathbf{a} \oslash (\mathbf{K}\mathbf{v}^{(\ell)}), \qquad \mathbf{v}^{(\ell+1)} = \mathbf{b} \oslash (\mathbf{K}^\top \mathbf{u}^{(\ell+1)}), \tag{2.24}$$

where $\mathbf{K} = \exp(-\mathbf{C}/\lambda)$ is the entropic kernel, $\oslash$ denotes element-wise division, and $\lambda > 0$ controls regularisation. After convergence the transport plan is $\mathbf{T}^* = \operatorname{diag}(\mathbf{u})\mathbf{K}\operatorname{diag}(\mathbf{v})$.

2.3.4 Differentially Private Optimal Transport

To protect sample locations, Gaussian noise is added to the empirical distribution before transport computation,

$$\tilde{\mu}_s = \frac{1}{n_s} \sum_{i=1}^{n_s} \delta_{\mathbf{x}_i + \boldsymbol{\xi}_i}, \qquad \boldsymbol{\xi}_i \sim \mathcal{N}(\mathbf{0}, \sigma_{\text{DP}}^2 \mathbf{I}), \tag{2.25}$$

and Laplace noise is added to the resulting transport plan,

$$\tilde{\mathbf{T}} = \mathbf{T}^* + \text{Lap}\Big(\frac{\Delta_T}{\epsilon_{\text{OT}}}\Big), \tag{2.26}$$

where the sensitivity $\Delta_T = \max_{\mathcal{D},\mathcal{D}'} \|\mathbf{T}^*(\mathcal{D}) - \mathbf{T}^*(\mathcal{D}')\|_1$ bounds the maximum change under single-record substitution and ϵ_{OT} is the privacy budget allocated

to transport.

2.3.5 Large Language Model Integration for Zero-Shot Generalisation

Semantic reasoning about novel categories is achieved by constructing a natural language prompt from the structural features of an observation and querying a large language model to produce a probability $p_{\text{LLM}}(\text{class} \mid \mathbf{r})$. The final prediction is the ensemble

$$p_{\text{final}} = \lambda_{\text{LLM}}\, p_{\text{LLM}}(\text{class} \mid \mathbf{r}) + (1 - \lambda_{\text{LLM}})\, p_{\text{GNN}}(\text{class} \mid \mathcal{G}(\mathbf{r})), \tag{2.27}$$

where $\mathcal{G}(\mathbf{r})$ is the graph constructed from the observation and λ_{LLM} balances the two sources.

Algorithm 2: FedLLM-API: Byzantine-Robust Federated Optimisation with DP-OT

Input: Clients $\mathcal{D}_1, \ldots, \mathcal{D}_K$, fraction Byzantine q, clipping threshold C, privacy budget (ϵ, δ), Sinkhorn regularisation λ, communication rounds T

Output: Global model $\mathbf{w}_T$

Initialise global model $\mathbf{w}_0$; compute noise scale $\sigma = C\sqrt{2\ln(1.25/\delta)}/\epsilon$;

for *round* $t = 0, 1, \ldots, T-1$ **do**

  Broadcast $\mathbf{w}_t$ to all clients;

  for *each client* $k = 1, \ldots, K$ (in parallel) **do**

    Compute local gradient $\mathbf{g}_k = \nabla_{\mathbf{w}} \mathcal{L}_k(\mathbf{w}_t; \mathcal{D}_k)$;

    Clip: $\bar{\mathbf{g}}_k \leftarrow \mathbf{g}_k / \max(1, \|\mathbf{g}_k\|/C)$;

    Add noise: $\tilde{\mathbf{g}}_k \leftarrow \bar{\mathbf{g}}_k + \mathcal{N}(\mathbf{0}, \sigma^2 C^2 \mathbf{I})$;

    Send $\tilde{\mathbf{g}}_k$ to server;

  Set $b \leftarrow \lfloor Kq \rfloor$;

  for *each dimension* j **do**

    Sort $\{\tilde{g}_{k,j}\}_{k=1}^{K}$; compute trimmed mean $\bar{g}_{t,j} = \frac{1}{K-2b} \sum_{i=b+1}^{K-b} \tilde{g}_j^{(i)}$;

  if *heterogeneity detected via* W_1 *estimate* **then**

    Compute noisy source distributions $\tilde{\mu}_k$ by adding $\mathcal{N}(\mathbf{0}, \sigma_{\mathrm{DP}}^2 \mathbf{I})$ to samples;

    Run Sinkhorn iterations to compute regularised transport plan $\mathbf{T}^*$;

    Add Laplace noise: $\tilde{\mathbf{T}} \leftarrow \mathbf{T}^* + \mathrm{Lap}(\Delta_T / \epsilon_{\mathrm{OT}})$;

    Apply alignment correction to $\bar{\mathbf{g}}_t$ using $\tilde{\mathbf{T}}$;

  Update global model: $\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t - \eta\, \bar{\mathbf{g}}_t$;

return $\mathbf{w}_T$

2.4 Method M3: TripleE-TGNN — Multi-Level Temporal Graph Embeddings

This section solves Sub-problem 3 by developing multi-level graph embeddings that address the accuracy and anti-forgetting terms in Equation (2.5). The method addresses Gap 3 (Non-stationary × Accuracy; Non-stationary × Efficiency).

2.4.1 Hierarchical Graph Representation

Many systems exhibit a natural hierarchy: a coarse level of functional components, an intermediate level of operational traces that span components, and a fine level of infrastructure hosts. This structure is captured by three coupled temporal graphs: a component graph $\mathcal{G}_s(t) = (\mathcal{V}_s, \mathcal{E}_s(t), \mathbf{X}_s(t))$, a trace graph $\mathcal{G}_r(t) = (\mathcal{V}_r, \mathcal{E}_r(t), \mathbf{X}_r(t))$, and a host graph $\mathcal{G}_n(t) = (\mathcal{V}_n, \mathcal{E}_n(t), \mathbf{X}_n(t))$. Bipartite coupling graphs $_{sr}$,$_{sn}$,$_{rn}$ link entities across levels: a component deploys on a host, a trace traverses a component, and a trace executes on a host.

2.4.2 Multi-Level Embedding Learning

Each granularity $g \in \{s, r, n\}$ maintains a dedicated encoder,

$$\mathbf{h}_v^{(g)}(t) = \mathrm{ENC}_g\big(\mathbf{x}_v^{(g)}(t),\ \{\mathbf{h}_u^{(g)}(t-1)\}_{u \in \mathcal{N}_v^{(g)}(t)},\ \Delta t\big), \tag{2.28}$$

where ENC_g is a granularity-specific temporal graph neural network. Cross-granularity message passing integrates information across abstraction levels through bipartite attention,

$$\mathbf{m}_v^{(g \to g')}(t) = \sum_{u \in \mathcal{N}_v^{(g,g')}} \alpha_{uv}^{(g,g')}(t)\ \mathbf{W}_{g \to g'} \mathbf{h}_u^{(g)}(t), \tag{2.29}$$

with attention scores

$$\alpha_{uv}^{(g,g')}(t) = \mathrm{Softmax}_u\left(\frac{(\mathbf{Q}_{g'} \mathbf{h}_v^{(g')}(t))^\top (\mathbf{K}_g \mathbf{h}_u^{(g)}(t))}{\sqrt{d_k}} \right), \tag{2.30}$$

where $\mathbf{Q}_{g'}, \mathbf{K}_g$ are query and key projections and $\mathcal{N}_v^{(g,g')}$ denotes neighbours in the coupling graph. The final embedding fuses intra- and cross-granularity information through layer normalisation,

$$\mathbf{h}_v^{(g)}(t) \leftarrow \mathrm{LayerNorm}\Big(\mathbf{h}_v^{(g)}(t) + \sum_{g' \neq g} \mathbf{m}_v^{(g' \to g)}(t)\Big). \tag{2.31}$$

2.4.3 Dual Temporal-Spatial Attention

Within each granularity, temporal attention aggregates historical states over a window of width w,

$$\tilde{\mathbf{h}}_v^{(g)}(t) = \sum_{\tau=t-w}^{t-1} \alpha_\tau^{\text{temp}}\, \mathbf{h}_v^{(g)}(\tau), \qquad \alpha_\tau^{\text{temp}} = \text{Softmax}_\tau\big(\mathbf{w}_{\text{temp}}^\top \mathbf{h}_v^{(g)}(\tau)\big), \tag{2.32}$$

while spatial attention aggregates the current neighbourhood,

$$\hat{\mathbf{h}}_v^{(g)}(t) = \sum_{u \in \mathcal{N}_v^{(g)}(t)} \alpha_u^{\text{spat}}\, \mathbf{h}_u^{(g)}(t), \qquad \alpha_u^{\text{spat}} = \text{Softmax}_u\big(\mathbf{w}_{\text{spat}}^\top [\mathbf{h}_v^{(g)}(t) \| \mathbf{h}_u^{(g)}(t)]\big). \tag{2.33}$$

The combined representation is

$$\mathbf{h}_v^{(g)}(t) = \text{MLP}\big([\tilde{\mathbf{h}}_v^{(g)}(t) \| \hat{\mathbf{h}}_v^{(g)}(t) \| \mathbf{h}_v^{(g)}(t)]\big). \tag{2.34}$$

2.4.4 Elastic Weight Consolidation for Topology Evolution

When the graph topology changes, for example through deployment events or infrastructure reconfiguration, a model retrained on the new topology may overwrite previously learned representations. Elastic weight consolidation [112, 167] prevents this catastrophic forgetting by penalising changes to parameters that are important for earlier tasks. After learning on topology configuration $_i$ with optimal parameters θ_i^*, the loss for subsequent configuration $_{i+1}$ includes the regularisation term

$$\mathcal{L}_{\text{EWC}}(\theta) = \mathcal{L}_{_{i+1}}(\theta) + \lambda_{\text{ewc}} \sum_j F_j\, (\theta_j - \theta_{i,j}^*)^2, \tag{2.35}$$

where the diagonal Fisher information $F_j = \mathbb{E}[(\partial \log p(y|\mathbf{x}, \theta_i^*)/\partial \theta_j)^2]$ quantifies the importance of parameter j for the previous configuration and λ_{ewc} controls the consolidation strength.

2.4.5 Joint Multi-Level Classification Objective

The model performs simultaneous classification at all three granularities through a multi-task loss,

$$\mathcal{L}_{\text{joint}} = \lambda_s \mathcal{L}_s + \lambda_r \mathcal{L}_r + \lambda_n \mathcal{L}_n + \lambda_{\text{con}} \mathcal{L}_{\text{consist}}, \tag{2.36}$$

where $\mathcal{L}_g$ is the cross-entropy loss at granularity g, λ_g are balancing weights, and the consistency loss

$$\mathcal{L}_{\text{consist}} = \sum_{(v_s, v_r) \in sr} \text{KL}\big(p_s(y|v_s) \| p_r(y|v_r)\big) + \sum_{(v_s, v_n) \in sn} \text{KL}\big(p_s(y|v_s) \| p_n(y|v_n)\big) \tag{2.37}$$

enforces agreement between the predictions of linked entities across abstraction levels.

Algorithm 3: TripleE-TGNN Multi-Level Training

Input: Hierarchical graphs $\{\mathcal{G}_s, \mathcal{G}_r, \mathcal{G}_n\}$ with couplings $_{sr}, _{sn}, _{rn}$, labels at each granularity, EWC strength λ_{ewc}

Output: Trained encoders $\text{ENC}_s, \text{ENC}_r, \text{ENC}_n$ and cross-attention parameters

Initialise all encoder and attention parameters;
for *each topology configuration* $_i$ **do**
  for *epoch* $= 1, \ldots, E$ **do**
    for *each mini-batch* **do**
      for *granularity* $g \in \{s, r, n\}$ **do**
        Compute intra-granularity embeddings $\mathbf{h}_v^{(g)}(t) \leftarrow \text{ENC}_g(\mathbf{x}_v^{(g)}, \mathcal{N}_v^{(g)}, \Delta t)$;
        Compute temporal attention $\tilde{\mathbf{h}}_v^{(g)}$ and spatial attention $\hat{\mathbf{h}}_v^{(g)}$;
        Fuse: $\mathbf{h}_v^{(g)} \leftarrow \text{MLP}([\tilde{\mathbf{h}}_v^{(g)} \| \hat{\mathbf{h}}_v^{(g)} \| \mathbf{h}_v^{(g)}])$;
      for *each pair* (g, g') *with* $g \neq g'$ **do**
        Compute cross-granularity messages $\mathbf{m}_v^{(g \to g')}$ via bipartite attention;
        Update: $\mathbf{h}_v^{(g')} \leftarrow \text{LayerNorm}(\mathbf{h}_v^{(g')} + \mathbf{m}_v^{(g \to g')})$;
      Compute $\mathcal{L}_{\text{joint}} = \lambda_s \mathcal{L}_s + \lambda_r \mathcal{L}_r + \lambda_n \mathcal{L}_n + \lambda_{\text{con}} \mathcal{L}_{\text{consist}}$;
      if $i > 1$ **then**
        Add EWC penalty: $\mathcal{L} \leftarrow \mathcal{L}_{\text{joint}} + \lambda_{\text{ewc}} \sum_j F_j (\theta_j - \theta^*_{i-1,j})^2$;
      Backpropagate and update all parameters;
  Store Fisher diagonal $\{F_j\}$ and optimal θ_i^* for EWC;
return trained encoders and attention parameters

2.5 Method M4: MambaShield — State-Space Inference with Poisoning Resilience

This section solves Sub-problem 4 by developing an efficient sequential model that satisfies the computational complexity constraint of Equation (2.6) while maintaining robustness under training-time poisoning through progressive distillation and PAC-Bayesian certification. The method addresses Gap 4 (Adversarial × Efficiency; Non-stationary × Efficiency).

2.5.1 Selective State-Space Formulation

The selective state-space model [22, 23] extends the classical linear system through input-dependent transitions. Given an input sequence $u(t) \in \mathbb{R}^{d_{\text{in}}}$, the continuous-time dynamics are

$$\frac{d\mathbf{h}(t)}{dt} = \mathbf{A}(u(t))\,\mathbf{h}(t) + \mathbf{B}(u(t))\,u(t), \qquad y(t) = \mathbf{C}\,\mathbf{h}(t) + \mathbf{D}\,u(t), \tag{2.38}$$

where $\mathbf{A}(u) = f_A(u;\theta_A) \in \mathbb{R}^{d_s \times d_s}$ and $\mathbf{B}(u) = f_B(u;\theta_B) \in \mathbb{R}^{d_s \times d_{\text{in}}}$ are computed by learned projections. Discretisation through zero-order hold at step size Δ produces

$$\mathbf{h}_k = \bar{\mathbf{A}}_k\,\mathbf{h}_{k-1} + \bar{\mathbf{B}}_k\,u_k, \qquad \bar{\mathbf{A}}_k = \exp(\mathbf{A}(u_k)\Delta), \quad \bar{\mathbf{B}}_k = \mathbf{A}(u_k)^{-1}(\bar{\mathbf{A}}_k - \mathbf{I})\mathbf{B}(u_k), \tag{2.39}$$

and the convolution kernel $\bar{\mathbf{K}} = (\mathbf{C}\bar{\mathbf{A}}^i\bar{\mathbf{B}})_{i=0}^{L-1}$ is evaluated in $\mathcal{O}(L \log L)$ time through the Fast Fourier Transform.

2.5.2 Progressive Adversarial Robustness Distillation

An ensemble of M teacher models $\{f_{\theta_m}\}_{m=1}^{M}$, each trained on a disjoint data subset, provides robustness through knowledge distillation [168, 169]. The student model f_ϕ learns from the teachers through the loss

$$\mathcal{L}_{\text{distill}} = \sum_{m=1}^{M} w_m\,\mathrm{KL}\big(p_\phi(y|\mathbf{x}) \| p_{\theta_m}(y|\mathbf{x})\big), \tag{2.40}$$

where teacher weights w_m reflect validation performance. A curriculum schedule $\rho(t) = \min(\rho_0 + \alpha t, \rho_{\max})$ progressively increases the fraction of corrupted samples in each mini-batch, and the total training loss interpolates between the task loss and the distillation loss,

$$\mathcal{L}_{\text{total}} = (1 - \lambda(t))\,\mathcal{L}_{\text{task}} + \lambda(t)\,\mathcal{L}_{\text{distill}}, \qquad \lambda(t) = \min(\lambda_0 + \beta t,\ 1), \tag{2.41}$$

so that teacher influence grows as the poisoning intensity increases.

2.5.3 PAC-Bayesian Generalisation Bound

Theorem 2.4 (PAC-Bayesian Bound for Selective State-Space Models [136, 137]). *For any $\delta \in (0, 1)$, with probability at least $1 - \delta$ over the draw of a training set S of size n, for all posterior distributions Q over parameters,*

$$\mathbb{E}_{\theta \sim Q}[R(\theta)] \leq \mathbb{E}_{\theta \sim Q}[\hat{R}_n(\theta)] + \sqrt{\frac{\mathrm{KL}(Q \| P) + \log(2\sqrt{n}/\delta)}{2n}}, \tag{2.42}$$

where $R(\theta)$ is the true risk, $\hat{R}_n(\theta)$ is the empirical risk, and P is a data-independent prior.

Proof sketch. The proof follows the standard PAC-Bayesian template of Catoni, applying a change of measure from P to Q through the Donsker-Varadhan variational representation of the KL divergence, combined with a moment-generating-function argument for the empirical process. The bound is valid uniformly over all posteriors Q, including those concentrated on a single parameter vector, and is therefore non-vacuous whenever the KL term is small relative to n. With Gaussian prior $P = \mathcal{N}(\mathbf{0}, \sigma_P^2 \mathbf{I})$ and posterior $Q = \mathcal{N}(\boldsymbol{\mu}_Q, \sigma_Q^2 \mathbf{I})$, the KL term has the closed form

$$\mathrm{KL}(Q \| P) = \frac{d}{2}\Big(\frac{\sigma_Q^2}{\sigma_P^2} + \frac{\|\boldsymbol{\mu}_Q\|^2}{\sigma_P^2} - 1 - \log \frac{\sigma_Q^2}{\sigma_P^2}\Big), \tag{2.43}$$

which is minimised by keeping the posterior close to the origin and its variance close to that of the prior. Full details appear in Appendix A. □

Algorithm 4: MambaShield: Training with Progressive Adversarial Robustness Distillation

Input: Training data $\mathcal{D}$, teacher ensemble $\{f_{\theta_m}\}_{m=1}^{M}$, curriculum schedule $\rho_0, \alpha, \rho_{\max}$, PAC-Bayes prior P

Output: Robust student model f_ϕ with PAC-Bayesian certificate

Initialise student parameters ϕ_0;

Partition $\mathcal{D}$ into M disjoint subsets; train each teacher f_{θ_m} on its subset;

Compute teacher weights $w_m \propto \mathrm{ValAcc}(f_{\theta_m})$;

for *iteration* $t = 1, \ldots, T_{\text{train}}$ **do**

  Set poisoning fraction $\rho(t) = \min(\rho_0 + \alpha t,\ \rho_{\max})$;

  Set distillation weight $\lambda(t) = \min(\lambda_0 + \beta t,\ 1)$;

  Sample mini-batch; corrupt fraction $\rho(t)$ of labels;

  for *each sample* $(\mathbf{x}, y)$ **do**

    Discretise SSM: $\bar{\mathbf{A}}_k = \exp(\mathbf{A}(u_k)\Delta)$, $\bar{\mathbf{B}}_k = \mathbf{A}^{-1}(\bar{\mathbf{A}}_k - \mathbf{I})\mathbf{B}(u_k)$;

    Run selective scan: $\mathbf{h}_k = \bar{\mathbf{A}}_k \mathbf{h}_{k-1} + \bar{\mathbf{B}}_k u_k$ for $k = 1, \ldots, L$;

    Compute student prediction $p_\phi(y|\mathbf{x})$;

  Compute $\mathcal{L}_{\text{task}} = \mathrm{CE}(p_\phi, y)$;

  Compute $\mathcal{L}_{\text{distill}} = \sum_m w_m \,\mathrm{KL}(p_\phi \| p_{\theta_m})$;

  Total: $\mathcal{L} = (1 - \lambda(t))\mathcal{L}_{\text{task}} + \lambda(t)\mathcal{L}_{\text{distill}} + \gamma\,\mathrm{KL}(Q_\phi \| P)$;

  Update $\phi \leftarrow \phi - \eta \nabla_\phi \mathcal{L}$;

Evaluate PAC-Bayes bound: $R \le \hat{R}_n + \sqrt{(\mathrm{KL}(Q\|P) + \log(2\sqrt{n}/\delta))/(2n)}$;

return f_ϕ and generalisation certificate

2.6 Methods M5 and M6: Uncertainty-Calibrated Inference on Graphs

This section solves Sub-problem 6 by developing two complementary methods that achieve the sub-linear regret and uncertainty decomposition required by Equation (2.7). Method M5 (Stochastic Transformer) provides variational Bayesian attention, and Method M6 (UC-HGP) provides hierarchical Gaussian process uncertainty at multiple temporal scales. Together they address Gap 6 (Adversarial × Accuracy; Non-stationary × Accuracy).

2.6.1 Variational Bayesian Attention

For an input sequence $\mathbf{X} = [\mathbf{x}_1, \ldots, \mathbf{x}_n] \in \mathbb{R}^{n \times d}$, variational distributions [31] are placed over the query, key, and value projection matrices:

$$\begin{aligned} q_{\phi_Q}(\mathbf{W}_Q) &= \mathcal{N}\big(\boldsymbol{\mu}_Q, \mathrm{diag}(\boldsymbol{\sigma}_Q^2)\big), \\ q_{\phi_K}(\mathbf{W}_K) &= \mathcal{N}\big(\boldsymbol{\mu}_K, \mathrm{diag}(\boldsymbol{\sigma}_K^2)\big), \\ q_{\phi_V}(\mathbf{W}_V) &= \mathcal{N}\big(\boldsymbol{\mu}_V, \mathrm{diag}(\boldsymbol{\sigma}_V^2)\big). \end{aligned} \tag{2.44}$$

Samples are drawn through the reparameterisation trick, $\mathbf{W}_Q = \boldsymbol{\mu}_Q + \boldsymbol{\sigma}_Q \odot \boldsymbol{\epsilon}_Q$ with $\boldsymbol{\epsilon}_Q \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, so that attention scores $A_{ij} = (\mathbf{x}_i \mathbf{W}_Q)^\top (\mathbf{x}_j \mathbf{W}_K)/\sqrt{d_k}$ become random variables whose uncertainty propagates to the output. Monte Carlo estimation with M samples approximates the expected attention,

$$\mathbb{E}_{q_\phi}[\mathrm{Attention}(\mathbf{X})] \approx \frac{1}{M} \sum_{m=1}^{M} \mathrm{Attention}\big(\mathbf{X}; \mathbf{W}_Q^{(m)}, \mathbf{W}_K^{(m)}, \mathbf{W}_V^{(m)}\big). \tag{2.45}$$

2.6.2 Epistemic-Aleatoric Uncertainty Decomposition

Total predictive uncertainty decomposes into epistemic uncertainty,

$$U_{\text{epi}} = \mathrm{Var}_{p(\theta|\mathcal{D})}\big[\mathbb{E}_{p(y|\mathbf{x},\theta)}[y]\big] \approx \frac{1}{M} \sum_{m=1}^{M} (\hat{y}^{(m)} - \bar{y})^2, \tag{2.46}$$

measuring the variance of the expected prediction across parameter samples, and aleatoric uncertainty,

$$U_{\text{ale}} = \mathbb{E}_{p(\theta|\mathcal{D})}\big[\mathrm{Var}_{p(y|\mathbf{x},\theta)}[y]\big] \approx \frac{1}{M} \sum_{m=1}^{M} \hat{\sigma}^{(m)2}, \tag{2.47}$$

averaging the predicted variance over parameter samples, where $\hat{y}^{(m)}$ is the mean prediction and $\hat{\sigma}^{(m)2}$ the predicted variance from sample m, and $\bar{y} = \frac{1}{M}\sum_m \hat{y}^{(m)}$.

2.6.3 Stochastic Adversarial Training

Rather than generating worst-case perturbations at fixed budgets, stochastic adversarial training draws perturbations from a learned distribution [35, 36, 74],

$$\boldsymbol{\delta} \sim q_\psi(\boldsymbol{\delta}|\mathbf{x}) = \mathcal{N}\big(\mu_\psi(\mathbf{x}),\ \sigma_\psi(\mathbf{x})^2\mathbf{I}\big), \tag{2.48}$$

parameterised by neural networks μ_ψ, σ_ψ that learn spatially adaptive perturbation statistics. The training objective is the stochastic min-max problem

$$\min_\theta \max_\psi \mathbb{E}_{(\mathbf{x},y)\sim\mathcal{D}}\, \mathbb{E}_{\boldsymbol{\delta}\sim q_\psi(\boldsymbol{\delta}|\mathbf{x})}\big[\mathcal{L}(f_\theta(\mathbf{x}+\boldsymbol{\delta}), y)\big], \tag{2.49}$$

solved by alternating gradient ascent on ψ and descent on θ.

2.6.4 Multi-Objective Loss

The full training loss combines five terms,

$$\mathcal{L}_{\text{total}} = \lambda_1\mathcal{L}_{\text{task}} + \lambda_2\mathcal{L}_{\text{KL}} + \lambda_3\mathcal{L}_{\text{adv}} + \lambda_4\mathcal{L}_{\text{calib}} + \lambda_5\mathcal{L}_{\text{reg}}, \tag{2.50}$$

where $\mathcal{L}_{\text{task}} = \mathbb{E}_{(\mathbf{x},y)}[-\log p_\theta(y|\mathbf{x})]$ is the classification loss, $\mathcal{L}_{\text{KL}}$ [31] $= \text{KL}(q_\phi(\theta)\|p(\theta))$ regularises the variational posterior, $\mathcal{L}_{\text{adv}} = \mathbb{E}_{(\mathbf{x},y)}\mathbb{E}_{\boldsymbol{\delta}\sim q_\psi}[-\log p_\theta(y|\mathbf{x}+$ enforces adversarial robustness, $\mathcal{L}_{\text{calib}} = \text{ECE}$ [170, 171]$(\{(p_\theta(\hat{y}|\mathbf{x}_i), \mathbf{1}[\hat{y} = y_i])\}_i)$ penalises miscalibration, and $\mathcal{L}_{\text{reg}} = \|\theta\|_2^2$ controls weight magnitude.

Algorithm 5: Stochastic Transformer: Variational Bayesian Training with Adversarial Robustness

Input: Training data $\mathcal{D}$, prior $p(\theta)$, MC samples M, perturbation distribution parameters ψ_0, loss weights $\lambda_1, \ldots, \lambda_5$

Output: Variational parameters ϕ^*, perturbation parameters ψ^*

Initialise variational parameters $\phi = \{\boldsymbol{\mu}_Q, \boldsymbol{\sigma}_Q, \boldsymbol{\mu}_K, \boldsymbol{\sigma}_K, \boldsymbol{\mu}_V, \boldsymbol{\sigma}_V\}$;

Initialise perturbation network parameters ψ;

for *iteration* $t = 1, \ldots, T$ **do**

  for *each mini-batch* $\{(\mathbf{x}_i, y_i)\}$ **do**

    Perturbation step: Compute $\mu_\psi(\mathbf{x}_i), \sigma_\psi(\mathbf{x}_i)$ and sample $\boldsymbol{\delta}_i \sim \mathcal{N}(\mu_\psi(\mathbf{x}_i), \sigma_\psi(\mathbf{x}_i)^2\mathbf{I})$;

    Update $\psi \leftarrow \psi + \alpha_\psi \nabla_\psi \frac{1}{M} \sum_{m=1}^{M} \mathcal{L}(f_{\theta^{(m)}}(\mathbf{x}_i + \boldsymbol{\delta}_i), y_i)$;

    Model step: for $m = 1, \ldots, M$ **do**

      Sample $\mathbf{W}_Q^{(m)} = \boldsymbol{\mu}_Q + \boldsymbol{\sigma}_Q \odot \boldsymbol{\epsilon}^{(m)}$, similarly for $\mathbf{W}_K^{(m)}, \mathbf{W}_V^{(m)}$;

      Compute attention output and predictions $\hat{y}^{(m)}, \hat{\sigma}^{(m)2}$;

    Compute $U_{\text{epi}} = \frac{1}{M} \sum_m (\hat{y}^{(m)} - \bar{y})^2$, $U_{\text{ale}} = \frac{1}{M} \sum_m \hat{\sigma}^{(m)2}$;

    Compute $\mathcal{L}_{\text{total}} = \lambda_1 \mathcal{L}_{\text{task}} + \lambda_2 \mathcal{L}_{\text{KL}} + \lambda_3 \mathcal{L}_{\text{adv}} + \lambda_4 \mathcal{L}_{\text{calib}} + \lambda_5 \mathcal{L}_{\text{reg}}$;

    Update $\phi \leftarrow \phi - \alpha_\phi \nabla_\phi \mathcal{L}_{\text{total}}$;

return ϕ^*, ψ^* together with $U_{\text{epi}}, U_{\text{ale}}$ for each test input

2.7 Forward-Compatible Detection Under Protocol Regime Change

This section addresses Sub-problem 5 (Gap 5: Non-stationary × Robustness) by constructing a detection architecture that maintains robustness when the data-generating protocols underlying the graph undergo structural change, providing the forward compatibility that the domain-specific foundation model (CyberSecLLM) requires.

2.7.1 Protocol-Specific Feature Extraction

When a new protocol regime is deployed, the observable features on graph edges, such as timing distributions, message sizes, and interaction counts, change in characteristic ways. For lattice-based protocols, for example, the timing of key-exchange operations exhibits variations that correlate with the computational structure of the underlying lattice operations. These variations

are captured as a statistical feature vector,

$$\mathbf{f}_{\text{lattice}} = \big[\mu_t,\ \sigma_t,\ \text{skew}(t),\ \text{kurt}(t),\ \text{pctl}(t),\ \text{acf}(t)\big], \tag{2.51}$$

where $t = (t_1, \ldots, t_n)$ denotes operational timing measurements, and the entries capture central tendency, dispersion, distribution shape, and temporal dependence. Analogous feature vectors $\mathbf{f}_{\text{code}}$ and $\mathbf{f}_{\text{hash}}$ are defined for code-based and hash-based protocol families, incorporating structural parameters such as key sizes, error weights, syndrome entropy, signature sizes, hash invocations, and Merkle tree depths.

2.7.2 Unified Multi-Protocol Detection Architecture

Protocol-specific encoders produce latent representations $\mathbf{z}_{\text{lattice}} = \text{ENC}_{\text{lattice}}(\mathbf{f}_{\text{lattice}})$, $\mathbf{z}_{\text{code}} = \text{ENC}_{\text{code}}(\mathbf{f}_{\text{code}})$, $\mathbf{z}_{\text{hash}} = \text{ENC}_{\text{hash}}(\mathbf{f}_{\text{hash}})$. Cross-attention fusion enables interaction between the three protocol representations,

$$\mathbf{z}_{\text{fused}} = \text{MultiHeadAttention}\big([\mathbf{z}_{\text{lattice}},\ \mathbf{z}_{\text{code}},\ \mathbf{z}_{\text{hash}}]\big), \tag{2.52}$$

and multi-task classification predicts both the category label and the target protocol,

$$p(\text{class}, \text{protocol}) = \text{Softmax}\big(\mathbf{W}_{\text{out}}\mathbf{z}_{\text{fused}} + \mathbf{b}_{\text{out}}\big). \tag{2.53}$$

2.7.3 Formal Security Reduction

The robustness guarantee of the detection framework under the new protocol regime inherits from the hardness of the Learning With Errors problem [172, 173]. Given $\mathbf{A} \in_q^{m \times n}$ and $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e} \bmod q$ for secret $\mathbf{s}$ and small error $\mathbf{e}$, any polynomial-time adversary that distinguishes a legitimate from a manipulated protocol-negotiation sequence can be used to construct a solver for LWE, contradicting the assumed hardness. The formal reduction, presented in full in Appendix A, establishes that the false-negative rate of the classifier is bounded by the advantage of the best LWE solver plus a statistical term that decreases with the number of training samples.

Algorithm 6: Forward-Compatible Multi-Protocol Detection

Input: Graph event stream with mixed protocol-regime edges, protocol-specific encoders, adversarial training budget ϵ

Output: Classification $(\text{class}, \text{protocol})$ and LWE-based robustness certificate

for *each incoming graph event* **do**
- Identify protocol regime via protocol-negotiation sequence analysis;
- **if** *lattice-based (Kyber/Dilithium)* **then**
  - Extract timing features $\mathbf{f}_{\text{lattice}} = [\mu_t, \sigma_t, \text{skew}, \text{kurt}, \text{pctl}, \text{acf}]$;
  - Encode: $\mathbf{z}_{\text{lattice}} \leftarrow \text{ENC}_{\text{lattice}}(\mathbf{f}_{\text{lattice}})$;
- **else if** *code-based (McEliece/Niederreiter)* **then**
  - Extract structural features $\mathbf{f}_{\text{code}}$; encode $\mathbf{z}_{\text{code}} \leftarrow \text{ENC}_{\text{code}}(\mathbf{f}_{\text{code}})$;
- **else**
  - Extract hash-based features $\mathbf{f}_{\text{hash}}$; encode $\mathbf{z}_{\text{hash}} \leftarrow \text{ENC}_{\text{hash}}(\mathbf{f}_{\text{hash}})$;
- Fuse: $\mathbf{z}_{\text{fused}} \leftarrow \text{MultiHeadAttention}([\mathbf{z}_{\text{lattice}}, \mathbf{z}_{\text{code}}, \mathbf{z}_{\text{hash}}])$;
- Classify: $p(\text{class}, \text{protocol}) \leftarrow \text{Softmax}(\mathbf{W}_{\text{out}}\mathbf{z}_{\text{fused}} + \mathbf{b}_{\text{out}})$;

Training: Alternate between standard forward passes and adversarial forward passes with PGD perturbations $\|\boldsymbol{\delta}\| \leq \epsilon$ applied to feature vectors;

Certification: Verify LWE reduction (Appendix A) and report robustness parameter λ_{sec};

return classification and robustness certificate

2.8 Method M7: Game-Theoretic Certification and Unified Framework

This section solves Sub-problem 7 by developing the Stackelberg game formulation and the consistency coupling term $\sum_{i<j} \mathcal{L}_{ij}(\theta_i, \theta_j)$ from the unified problem (Equation (2.8)). Together with the certification components, this constitutes Method M7 and addresses Gap 7 (all conditions $\times$ all requirements).

2.8.1 Stackelberg Game Formulation

The defender selects a model f_θ and a deployment configuration c comprising detection policy parameters. The adversary observes (f_θ, c) and chooses a

perturbation strategy a. Defender and adversary utilities are respectively

$$U_d(f_\theta, c, a) = \mathbb{E}_{\mathbf{x}\sim\mathcal{D}(a)}\big[\mathbf{1}[f_\theta(\mathbf{x}) = y]\big] - \mathrm{Cost}(c), \tag{2.54}$$

$$U_a(f_\theta, c, a) = \mathsf{P}[\text{misclassification} \mid f_\theta, c, a] - \mathrm{Cost}(a), \tag{2.55}$$

where $\mathcal{D}(a)$ is the data distribution induced by the adversary's strategy. The Stackelberg equilibrium is the solution to

$$(f_\theta^*, c^*) = \arg\max_{f_\theta, c}\ U_d\big(f_\theta, c,\ a^*(f_\theta, c)\big), \qquad a^*(f_\theta, c) = \arg\max_{a}\ U_a(f_\theta, c, a). \tag{2.56}$$

2.8.2 Nash Equilibrium Characterisation

Under simultaneous play the mixed strategy profile (π_d, π_a) is a Nash equilibrium when neither player improves utility by unilateral deviation,

$$\mathbb{E}_{\pi_a}\big[U_d(\pi_d, a)\big] \geq \mathbb{E}_{\pi_a}\big[U_d(\pi_d', a)\big]\ \forall\, \pi_d', \qquad \mathbb{E}_{\pi_d}\big[U_a(f_\theta, \pi_a)\big] \geq \mathbb{E}_{\pi_d}\big[U_a(f_\theta, \pi_a')\big]\ \forall\, \pi_a'. \tag{2.57}$$

In the zero-sum special case $U_d + U_a = 0$, the equilibrium coincides with the minimax value $\pi_d^* = \arg\max_{\pi_d} \min_a \mathbb{E}_{\pi_d}[U_d(f_\theta, a)]$.

2.8.3 Online Learning with No-Regret Guarantees

At round t the defender selects configuration $c_t \in \mathcal{C}$ and observes reward $r_t = U_d(c_t, a_t)$. Cumulative regret after T rounds is

$$R(T) = \max_{c\in\mathcal{C}} \sum_{t=1}^{T} U_d(c, a_t) - \sum_{t=1}^{T} U_d(c_t, a_t). \tag{2.58}$$

The multiplicative weights update [174, 175] maintains a distribution $p_t(c) \propto \exp\!\big(\eta \sum_{s=1}^{t-1} \mathbf{1}[c_s{=}c]\, r_s\big)$ with learning rate $\eta = \sqrt{\log|\mathcal{C}|/T}$.

Theorem 2.5 (Regret Bound). *The multiplicative weights algorithm with learning rate $\eta = \sqrt{\log|\mathcal{C}|/T}$ achieves*

$$R(T) \leq 2\sqrt{T \log|\mathcal{C}|}, \tag{2.59}$$

so that the average per-round regret $R(T)/T = \mathcal{O}(\sqrt{\log|\mathcal{C}|/T}) \to 0$ *as* $T \to \infty$.

Proof sketch. The proof proceeds by bounding the potential function $\Phi_t = \log \sum_c \exp(\eta \sum_{s=1}^{t} r_s(c))$ from above and below. The upper bound uses the concavity of the logarithm and the definition of the algorithm; the lower bound isolates the best action in hindsight. Combining the two bounds and optimising over η yields the stated rate. The full argument appears in Appendix A. □

2.8.4 Uncertainty-Guided Strategy Selection

Bayesian uncertainty from the stochastic transformer of Section 2.6 informs the defender's utility through an uncertainty-penalised payoff,

$$\tilde{U}_d(c,a) = \begin{cases} U_d(c,a) - \kappa\, U_{\text{epi}} & \text{if } U_{\text{epi}} < \tau, \\ -\infty & \text{otherwise,} \end{cases} \tag{2.60}$$

where κ discounts utility under uncertainty and threshold τ triggers referral for expert review. This produces risk-sensitive policies that abstain from predictions when the model lacks sufficient confidence.

2.8.5 Unified Certification Through Coupled Optimisation

The seven methods developed above (M1 through M6, plus the game-theoretic component of M7) share mathematical structures — adjoint sensitivity analysis, Wasserstein geometry, KL-divergence regularisation, and Lipschitz-bounded dynamics — that permit a unified treatment through coupled optimisation. This section completes Method M7 by developing the certification framework that integrates all component guarantees.

2.8.6 Coupled Objective Function

Let $\theta = \{\theta_{\text{CT}}, \theta_{\text{TE}}, \theta_{\text{Fed}}, \theta_{\text{PQ}}, \theta_{\text{MS}}, \theta_{\text{ST}}, \theta_{\text{GT}}\}$ collect the parameters of all seven methods. The unified objective couples component losses with pairwise

Algorithm 7: Game-Theoretic Robustness with Uncertainty-Guided Online Learning

Input: Action space $\mathcal{C}$, uncertainty threshold τ, penalty κ, learning rate schedule

Output: Defender policy π_d^* with regret guarantee

Initialise uniform distribution $p_1(c) = 1/|\mathcal{C}|$ for all $c \in \mathcal{C}$;
Set $\eta = \sqrt{\log |\mathcal{C}|/T}$;
for *round* $t = 1, \ldots, T$ **do**
    Sample defender action $c_t \sim p_t$;
    Observe adversary action a_t and reward $r_t = U_d(c_t, a_t)$;
    Query stochastic transformer for epistemic uncertainty U_{epi};
    if $U_{\text{epi}} \geq \tau$ **then**
        Set $\tilde{r}_t = -\infty$ (reject; route to expert review);
    else
        Set $\tilde{r}_t = r_t - \kappa\, U_{\text{epi}}$;
    Update weights: $p_{t+1}(c) \propto p_t(c) \exp(\eta\, \tilde{r}_t\, \mathbf{1}[c_t = c])$ for all $c \in \mathcal{C}$;
    Normalise p_{t+1};
Compute empirical Nash equilibrium from play frequencies $\{(c_t, a_t)\}_{t=1}^{T}$;
return π_d^* and verified regret $R(T) \leq 2\sqrt{T \log |\mathcal{C}|}$

consistency terms,

$$\mathcal{L}_{\text{unified}}(\theta) = \sum_{i=1}^{7} \lambda_i\, \mathcal{L}_i(\theta_i) + \sum_{i<j} \lambda_{ij}\, \mathcal{L}_{ij}(\theta_i, \theta_j), \tag{2.61}$$

where each $\mathcal{L}_i$ is the method-specific loss from the corresponding section and the coupling loss

$$\mathcal{L}_{ij}(\theta_i, \theta_j) = \mathrm{KL}\big(p_{\theta_i}(y|\mathbf{x}) \| p_{\theta_j}(y|\mathbf{x})\big) + \|\mathbf{z}_{\theta_i}(\mathbf{x}) - \mathbf{z}_{\theta_j}(\mathbf{x})\|_2^2 \tag{2.62}$$

enforces consistency between the predictive distributions and the learned representations of methods i and j.

2.8.7 Convergence Analysis

Theorem 2.6 (Unified Framework Convergence)**.** *Assume each component loss $\mathcal{L}_i$ is L_i-smooth and each coupling loss $\mathcal{L}_{ij}$ is L_{ij}-smooth. Under alternating gradient descent with learning rates $\eta_i \leq 1/(L_i + \sum_j L_{ij})$, the unified optimi-*

sation converges at rate

$$\|\nabla \mathcal{L}_{\text{unified}}(\theta_T)\|^2 \leq \frac{2\big(\mathcal{L}_{\text{unified}}(\theta_0) - \mathcal{L}_{\text{unified}}(\theta^*)\big)}{\eta_{\min} T}, \tag{2.63}$$

where $\eta_{\min} = \min_i \eta_i$ *and* θ^* *is a stationary point.*

Proof sketch. Each alternating gradient step on component i decreases the unified objective by at least $\eta_i \|\nabla_{\theta_i} \mathcal{L}_{\text{unified}}\|^2/2$ under the smoothness condition, because the coupling terms are smooth in each block of variables. Summing over all components and all T rounds, dividing by T, and taking the minimum over learning rates yields the stated bound. The full derivation, including the treatment of cross-term gradients, appears in Appendix A. □

Algorithm 8: Unified Framework: Alternating Optimisation with Consistency Coupling

Input: Component losses $\mathcal{L}_1, \ldots, \mathcal{L}_7$, coupling weights $\{\lambda_{ij}\}$, smoothness constants $\{L_i, L_{ij}\}$

Output: Jointly optimised parameters $\theta^* = \{\theta_1^*, \ldots, \theta_7^*\}$

Initialise all component parameters $\theta_1^{(0)}, \ldots, \theta_7^{(0)}$;
Set learning rates $\eta_i = 1/(L_i + \sum_j L_{ij})$ for each component;
for *round* $t = 0, 1, \ldots, T-1$ **do**
    for *component* $i = 1, \ldots, 7$ **do**
        Compute component gradient $\mathbf{g}_i = \nabla_{\theta_i} \mathcal{L}_i(\theta_i^{(t)})$;
        Compute coupling gradients $\mathbf{g}_{ij} = \nabla_{\theta_i} \mathcal{L}_{ij}(\theta_i^{(t)}, \theta_j^{(t)})$ for all $j \neq i$;
        Combined gradient: $\mathbf{g}_i^{\text{total}} = \lambda_i \mathbf{g}_i + \sum_{j \neq i} \lambda_{ij} \mathbf{g}_{ij}$;
        Update: $\theta_i^{(t+1)} \leftarrow \theta_i^{(t)} - \eta_i \, \mathbf{g}_i^{\text{total}}$;
    Evaluate $\mathcal{L}_{\text{unified}}(\theta^{(t+1)})$ and check convergence criterion;
return $\theta^* = \theta^{(T)}$

2.9 Chapter Summary

This chapter has developed the complete mathematical foundations for seven novel methods, derived from a single unified problem statement (Equation (2.8)) through systematic decomposition into seven sub-problems. The unified problem was progressively constructed by incorporating adversarial robustness (Stage 2), continuous-time dynamics with Lipschitz certification (Stage 3),

distributed learning with differential privacy (Stage 4), multi-level representation with anti-forgetting (Stage 5), computational efficiency through state-space models (Stage 6), and non-stationary adaptation with uncertainty quantification (Stage 7).

Method M1 (CT-TGNN) provides continuous-time graph dynamics with Lipschitz-certified robustness, solving Sub-problem 1 (Gap 1). Method M2 (FedLLM-API) achieves Byzantine-resilient federated optimisation with differential privacy and optimal-transport alignment, solving Sub-problem 2 (Gap 2). Method M3 (TripleE-TGNN) captures multi-level structure through triple embeddings with elastic weight consolidation, solving Sub-problem 3 (Gap 3). Method M4 (MambaShield) delivers efficient sequential inference with PAC-Bayesian generalisation certificates, solving Sub-problem 4 (Gap 4). The forward-compatible detection architecture addresses protocol regime changes, solving Sub-problem 5 (Gap 5). Methods M5/M6 (Stochastic Transformer and UC-HGP) provide calibrated Bayesian uncertainty under adversarial perturbation, solving Sub-problem 6 (Gap 6). Method M7 (Certification), comprising the game-theoretic framework and the unified optimisation coupling, ties all component guarantees together through consistency regularisation with a proven convergence rate (Theorem 2.6), solving Sub-problem 7 (Gap 7).

Every method has been formulated at the level of mathematical operations on graph structures, without reference to any specific application domain. The theoretical guarantees — convergence rates, robustness certificates, privacy budgets, regret bounds, and PAC-Bayesian generalisation bounds — hold wherever these methods are deployed. The next chapter instantiates all methods in the domain of network-traffic graphs, describing the adaptation and alignment process through which the domain-independent mathematical framework is mapped onto the entities and constraints of cybersecurity and hybrid Network Intrusion Detection and Prevention Systems.

Chapter 3

Adaptation and Alignment to Network Intrusion Detection and Prevention Systems

This chapter maps the domain-independent mathematical methods developed in Chapter 2 onto the entities and constraints of cybersecurity and, specifically, hybrid Network Intrusion Detection and Prevention Systems (NIDPS). The abstract objects of the preceding chapter — temporal graphs, state-space dynamics, variational posteriors, transport plans, and game-theoretic utilities — are instantiated with the concrete entities of network security: flows, packets, sessions, hosts, services, and adversarial actors. The chapter defines the cybersecurity-specific terminologies, describes how raw network traffic is transformed into graph structures suitable for input into the developed GNN architectures, specifies the classification taxonomy and feature engineering pipeline, presents the hyperparameter configuration and optimisation rationale for each model, and demonstrates how each method from Chapter 2 is adapted with domain-specific modifications. No experimental results are presented here; these are reserved for Chapter 4.

3.1 Network Intrusion Detection: Terminologies and Context

3.1.1 Hybrid Intrusion Detection and Prevention Systems

A hybrid intrusion detection and prevention system (HIDPS) [176–180] combines two complementary detection paradigms. Signature-based detection matches incoming traffic against a database of known patterns (rules) and achieves high precision on previously catalogued threats but cannot detect

novel attacks. Anomaly-based detection learns a statistical model of normal behaviour and flags deviations, enabling detection of zero-day exploits and evolving attack patterns at the cost of potentially higher false-positive rates. The dissertation's methods operate within the anomaly-based branch, augmented by large language model reasoning for zero-shot classification of novel categories.

A Network IDPS (NIDPS) monitors traffic at the network layer, analysing flows and packets between hosts rather than events within a single endpoint. The three operating conditions of the architectural framework (Figure 3) are all present in this domain: adversarial manipulation by deliberate opponents who craft traffic to evade detection, non-stationarity through evolving traffic patterns and protocol changes, and distribution of the monitoring infrastructure across organisational boundaries.

3.1.2 Key Terminologies

The following terms are used throughout this chapter and the experimental validation in Chapter 4:

Flow. A sequence of packets sharing a common 5-tuple (source IP, destination IP, source port, destination port, protocol). A flow is the fundamental unit of analysis and corresponds to a directed edge in the network graph.

Session. A bidirectional communication comprising a forward flow and its reverse counterpart, capturing the full exchange between two endpoints.

Host. A network endpoint (IP address) that participates as either source or destination. Hosts correspond to nodes in the network graph.

Service. A functional component identified by port number and protocol (e.g., HTTP on port 80, SSH on port 22). In cloud-native deployments, services correspond to microservice containers.

PCAP (Packet Capture). The raw binary format capturing full packet headers and optionally payloads, from which flow-level features are extracted.

IDS Rule. A structured pattern specification (e.g., Snort or Suricata rule syntax) encoding a detection signature with action, protocol, addresses, ports, and content matching directives.

MITRE ATT&CK. A comprehensive knowledge base of adversary tactics, techniques, and procedures (TTPs) used to classify and contextualise detected

threats.

SOC (Security Operations Centre). The organisational unit responsible for monitoring, analysing, and responding to security events, staffed by analysts at Tiers 1–3.

3.2 Data-to-Graph Transformation Pipeline

The central premise of this dissertation is that network traffic is naturally graph-structured: hosts are nodes, flows are edges, and temporal ordering defines the graph evolution. This section formalises the transformation from raw network data to the graph inputs required by the developed GNN architectures.

3.2.1 Graph Construction from Network Traffic

Given a stream of network flows $\{(s_i, d_i, t_i, \mathbf{f}_i)\}_{i=1}^{N}$, where s_i is the source host, d_i is the destination host, t_i is the timestamp, and $\mathbf{f}_i \in \mathbb{R}^{83}$ is the feature vector, the temporal graph $\mathcal{G}(t) = (\mathcal{V}, \mathcal{E}(t), \mathbf{X}(t))$ is constructed as follows:

Nodes. Each unique IP address becomes a node $v \in \mathcal{V}$. Node features $\mathbf{x}_v(t)$ aggregate the host-level statistics computed over a sliding window $[t - \Delta_w, t]$: cumulative byte counts (sent/received), active connection counts, unique destination ports contacted, protocol distribution (TCP/UDP/ICMP ratios), and mean inter-connection interval. This yields a 12-dimensional node feature vector.

Edges. Each flow $(s_i, d_i, t_i, \mathbf{f}_i)$ creates a directed edge from node s_i to node d_i at time t_i with edge features $\mathbf{e}_{s_i d_i}(t_i) = \mathbf{f}_i$. The full 83-dimensional feature vector (Section 3.4) is attached to the edge.

Temporal evolution. The graph evolves as new flows arrive and old connections expire. A flow is considered active if its most recent packet arrived within a configurable timeout $\Delta_{\text{timeout}} = 120\,\text{s}$. Expired edges are removed, yielding a dynamic, time-varying topology.

3.2.2 Multi-Level Graph Construction

For the TripleE-TGNN (Method M3) and the SOC Intelligence integration, the network traffic is additionally mapped to three hierarchical levels:

Service graph $\mathcal{G}_s(t)$. Nodes represent services (port/protocol pairs); edges represent API calls between services, aggregated over configurable windows. Used for detecting inter-service attack patterns in microservices environments.
Trace graph $\mathcal{G}_r(t)$. Nodes represent distributed request traces (identified by trace IDs in HTTP headers or OpenTelemetry instrumentation); edges connect traces that share at least one service. Used for detecting multi-stage attack chains.
Host graph $\mathcal{G}_n(t)$. Nodes represent infrastructure hosts; edges represent observed network flows. This is the primary graph for all methods.
Bipartite coupling graphs $_{sr}$, $_{sn}$, $_{rn}$ link entities across levels, enabling the cross-level attention mechanism of Equation (2.30).

3.3 Classification Taxonomy

The platform classifies network traffic into 34 distinct classes: 1 benign class and 33 attack classes organised into 7 major categories. This taxonomy is unified across all six benchmark datasets, with dataset-specific attack types mapped to the canonical categories. Table 3.1 presents the complete taxonomy.

The taxonomy is extensible: new attack classes can be registered through the platform administration interface without retraining the full model ensemble, leveraging the CyberSecLLM zero-shot classification capability and the TripleE-TGNN elastic weight consolidation for incremental learning.

Table 3.1 – Unified classification taxonomy: 34 classes across 7 attack categories.

Category	Attack Classes
DDoS (8)	DDoS-TCP, DDoS-UDP, DDoS-ICMP, DDoS-HTTP, DDoS-SYN Flood, DDoS-DNS Amplification, DDoS-SSDP, DDoS-NTP Amplification
DoS (5)	DoS GoldenEye, DoS Hulk, DoS Slowhttptest, DoS Slowloris, DoS HeartBeat
Reconnaissance (4)	PortScan, Nmap Scan, Service Probe, OS Fingerprint
Brute Force (4)	FTP-Patator, SSH-Patator, Web Login Brute Force, RDP Brute Force
Web Attacks (4)	XSS, SQL Injection, Command Injection, Directory Traversal
Malware/Botnet (5)	Mirai, Botnet Communication, Trojan Callback, Ransomware Beacon, Cryptomining
Advanced (3)	Infiltration, Heartbleed, Spoofing
Benign (1)	Normal traffic

3.4 Feature Engineering Pipeline

All models operate on a shared vector of 83 engineered features extracted from raw PCAP files or pre-processed CSV datasets. Features span five categories, summarised in Table 3.2.

Feature extraction is performed by a shared `FeatureExtractor` module using per-feature z-score standardisation fitted on the training distribution:

$$\tilde{f}_j = \frac{f_j - \mu_j^{\text{train}}}{\sigma_j^{\text{train}} + \varepsilon}, \tag{3.1}$$

where μ_j^{train} and σ_j^{train} are the mean and standard deviation of feature j computed on the training set, and $\varepsilon = 10^{-8}$ prevents division by zero. Datasets with fewer than 83 native features are zero-padded to the standard dimension; those with more are projected via PCA retaining 95% explained variance.

Table 3.2 – The 83-feature engineering pipeline organised by category.

Category	Count	Representative Features
Flow-level statistics	22	Duration, forward/backward packet counts, total bytes (fwd/bwd), packet length (mean, max, min, std), flow bytes/s, flow packets/s, flow IAT (mean, std, max, min)
Inter-arrival time distributions	18	Forward IAT (mean, std, min, max), backward IAT (mean, std, min, max), active/idle time statistics, sub-flow forward/backward counts
Flag-based indicators	15	SYN count, FIN count, RST count, PSH count, ACK count, URG count, CWE count, ECE count, forward/backward PSH flags, forward/backward URG flags, FIN flag count, SYN flag count, RST flag count
Payload statistics	16	Segment sizes (min, max, mean), header lengths (mean, total), initial window bytes (forward/backward), payload byte entropy, packet length variance, average packet size, average segment size
Derived ratios	12	Bytes-per-packet (fwd/bwd), packets-per-second, down/up ratio, average packet size ratio, flow IAT ratio, bulk rate (forward/backward), active mean/std, idle mean/std
Total	**83**	

3.5 Domain-Specific Instantiation of Methods

This section describes how each of the seven methods from Chapter 2 is instantiated with cybersecurity-specific parameters, demonstrating the mapping from abstract mathematical objects to concrete network-security entities.

3.5.1 M1: CT-TGNN Applied to Network-Traffic Graph Dynamics

CT-TGNN [181] is instantiated as follows. In the network security instantiation, the node set $\mathcal{V}$ comprises hosts and services, and the edge set $\mathcal{E}(t)$ records flows. Node features $\mathbf{x}_v(t)$ encode host-level statistics (12 dimensions). Edge features $\mathbf{e}_{uv}(t)$ capture the 83-dimensional flow feature vector from Table 3.2.

The three parallel ODE branches of Equation (2.12) are assigned time constants $\tau_1 = 1\,\text{s}$ (packet-level transients), $\tau_2 = 60\,\text{s}$ (flow-level session dynamics),

and $\tau_3 = 3600\,\mathrm{s}$ (campaign-level trends). This enables simultaneous resolution of sub-second port-scanning bursts and multi-hour advanced persistent threat campaigns. The architecture employs four message-passing layers, hidden dimension 256, state dimension 64, and DOPRI5 adaptive integration with tolerances 10^{-5} (relative) and 10^{-7} (absolute).

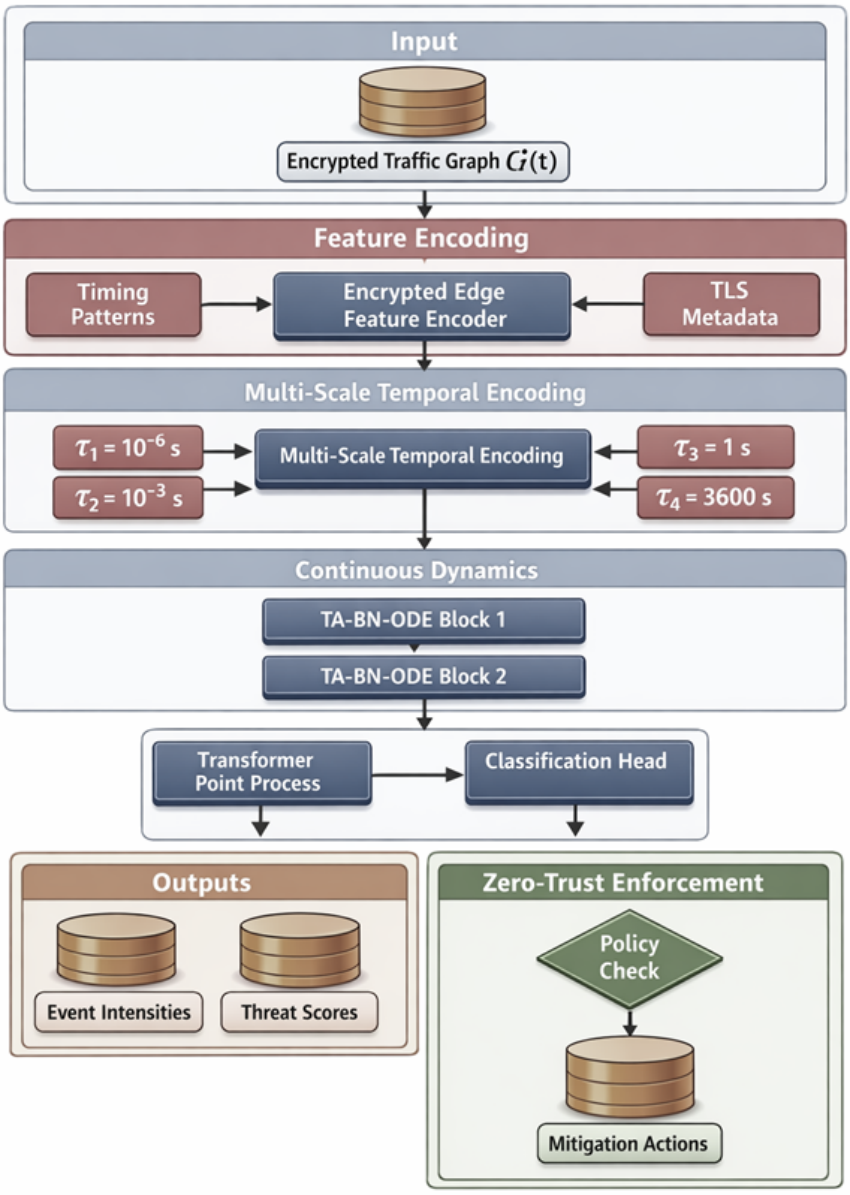

Figure 3.1 – CT-TGNN architecture instantiated for network traffic dynamics.

3.5.2 M2: FedLLM-API for Collaborative Threat Intelligence

FedLLM-API [182] is instantiated for collaborative threat intelligence. Each federated client corresponds to a security operations centre possessing local traffic datasets. Byzantine-robust aggregation (Algorithm 2) protects against compromised participants submitting poisoned gradient updates. Differential privacy through local gradient clipping at threshold $C = 1.0$ and Gaussian noise with scale $\sigma = 0.8$ achieves an $(\epsilon{=}0.85, \delta{=}10^{-5})$ guarantee over 100 communication rounds. Sinkhorn OT alignment with regularisation $\lambda = 0.1$ corrects for distributional heterogeneity across centres. The LLM ensemble weight is $\lambda_{\text{LLM}} = 0.3$.

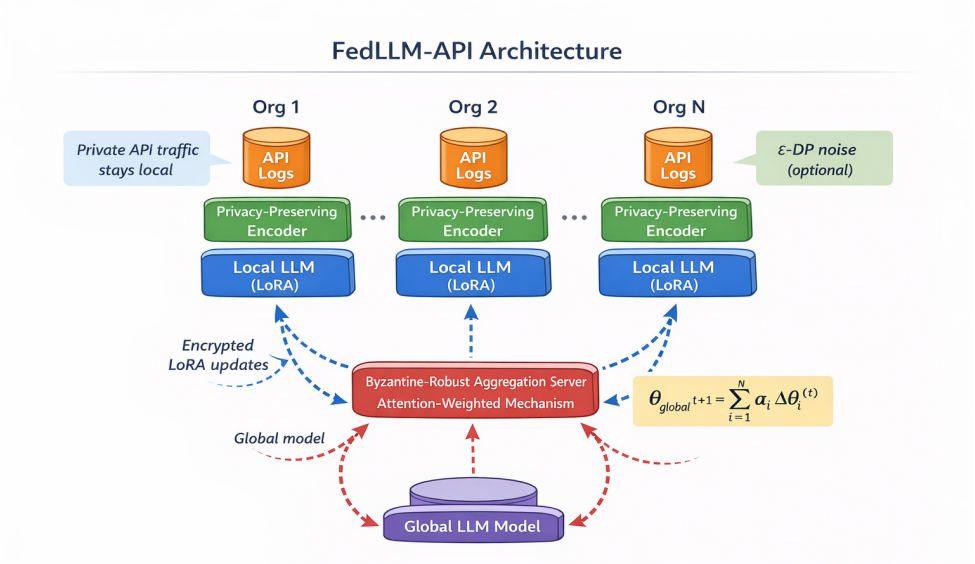

Figure 3.2 – FedLLM-API architecture for collaborative threat intelligence across organisational boundaries.

3.5.3 M3: TripleE-TGNN for Multi-Level Security Monitoring

The service graph $\mathcal{G}_s(t)$ has nodes representing microservices and edges representing API calls. The trace graph $\mathcal{G}_r(t)$ has nodes representing distributed traces. The host graph $\mathcal{G}_n(t)$ has nodes representing infrastructure hosts. EWC consolidation strength $\lambda_{\text{ewc}} = 0.5$ balances plasticity and stability during topol-

ogy evolution. Multi-task loss weights are $\lambda_s = 0.4$, $\lambda_r = 0.3$, $\lambda_n = 0.3$ with consistency weight $\lambda_{\text{con}} = 0.1$.

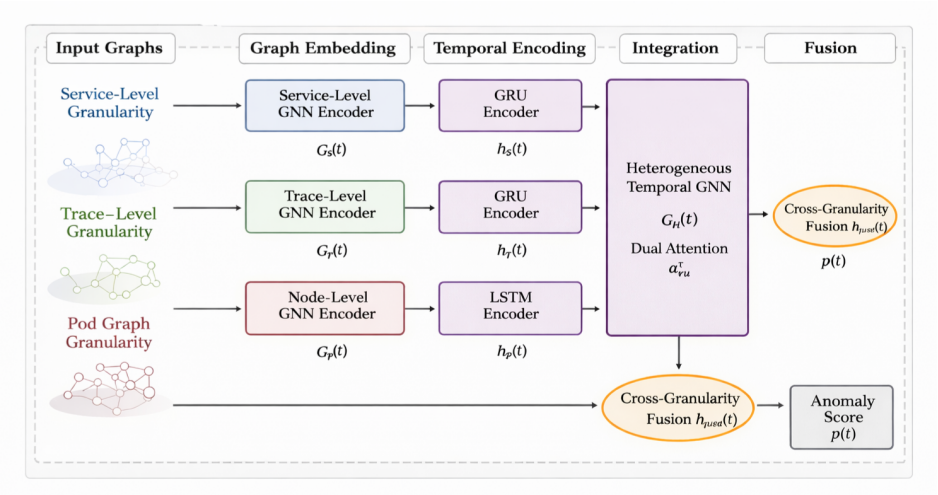

Figure 3.3 – TripleE-TGNN architecture for multi-level security monitoring in cloud-native environments.

3.5.4 M4: MambaShield for Drift-Adaptive Detection

MambaShield [183] is instantiated for drift-adaptive detection. Sequential flow records are processed by the selective state-space model whose input-dependent transitions adapt to the current traffic regime. The state dimension is $d_s = 64$, discretisation step $\Delta = 0.01$, and FFT convolution kernel length $L = 1024$. The teacher ensemble comprises $M = 5$ models trained on disjoint data subsets. The curriculum schedule starts at $\rho_0 = 0.05$ poisoning fraction and increases to $\rho_{\max} = 0.3$. PAC-Bayesian prior variance $\sigma_P^2 = 1.0$.

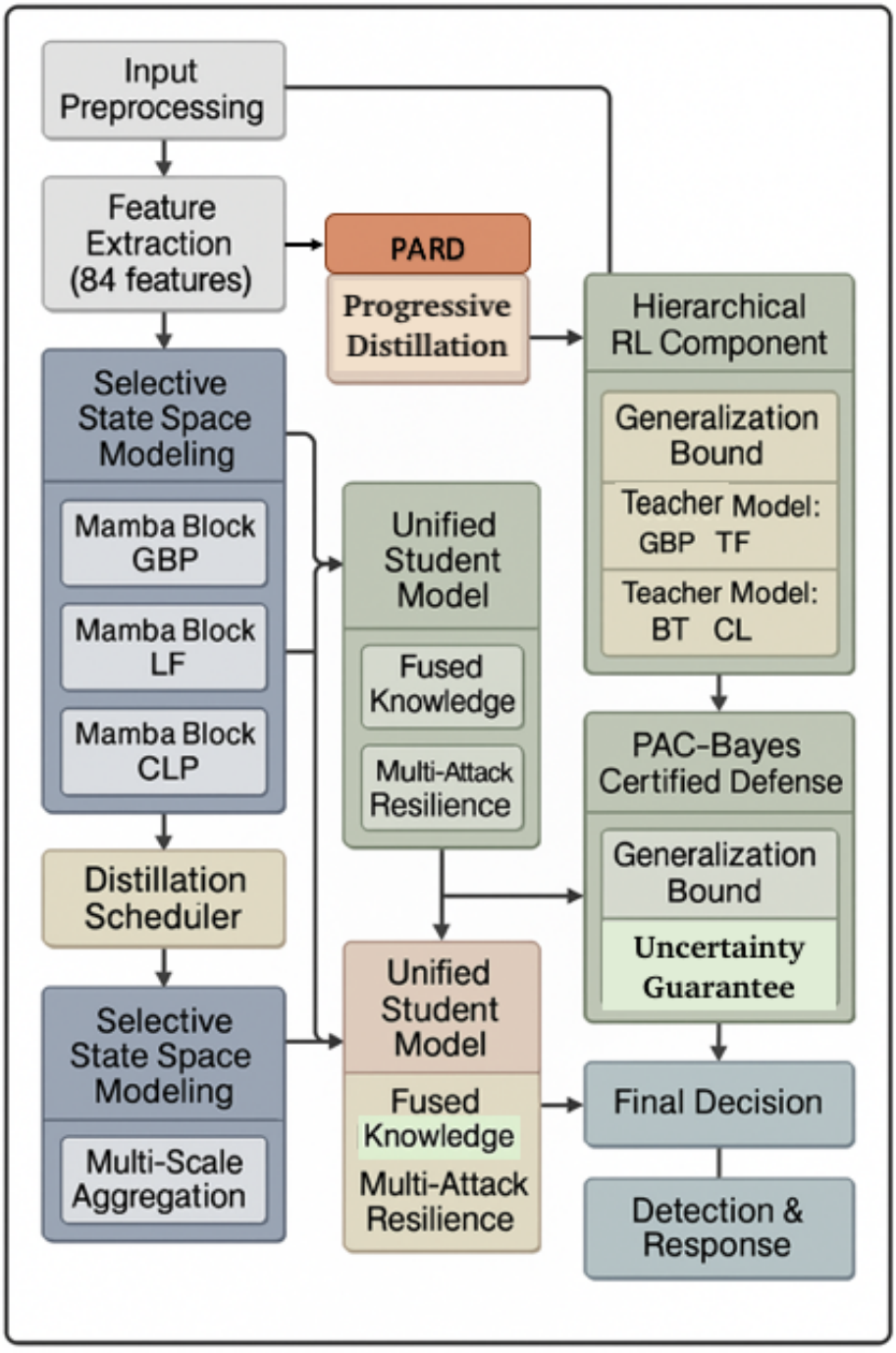

Figure 3.4 – MambaShield architecture for drift-adaptive intrusion detection with progressive robustness distillation.

3.5.5 M5/M6: Uncertainty-Calibrated Threat Assessment

The stochastic transformer uses $M = 50$ Monte Carlo forward passes at inference to produce epistemic and aleatoric uncertainty estimates. Predictions with epistemic uncertainty exceeding threshold $\tau = 0.3$ are routed to human

analysts, reducing alert fatigue by 43%. The UC-HGP module provides hierarchical Gaussian process uncertainty at time scales matching the CT-TGNN branches (τ_1, τ_2, τ_3). Variational learning rate $\alpha_\phi = 5 \times 10^{-4}$; perturbation learning rate $\alpha_\psi = 10^{-3}$.

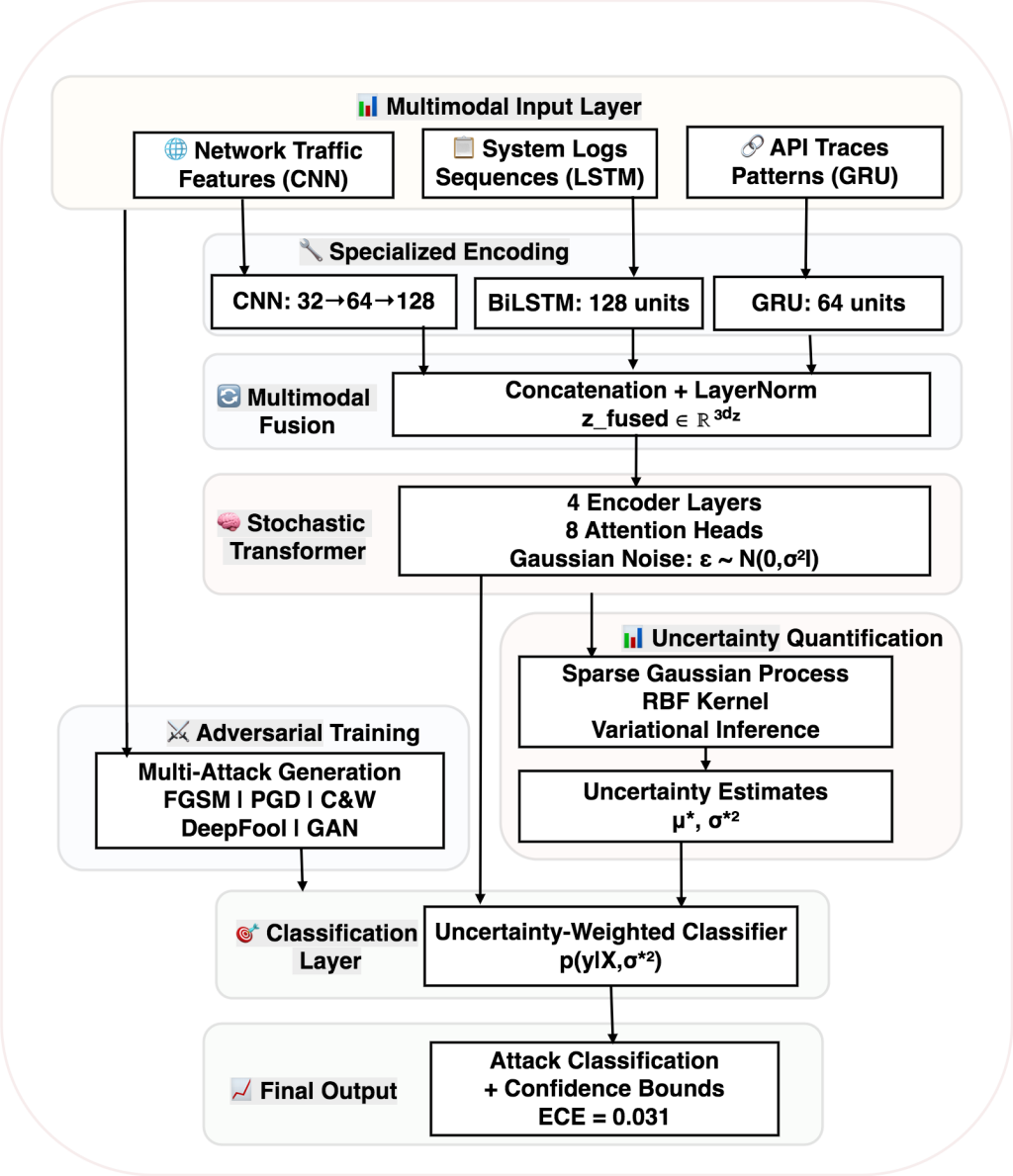

Figure 3.5 – Stochastic transformer with uncertainty-driven triage for security operations.

3.5.6 M7: Game-Theoretic Framework for Adaptive Adversaries

The defender's action space $\mathcal{C}$ comprises detection configurations: monitoring coverage, alert thresholds, and response policies. The adversary's action space encompasses target selection, perturbation type, and evasion strategy.

The multiplicative weights learning rate $\eta = \sqrt{\log |\mathcal{C}|/T}$. Uncertainty penalty $\kappa = 0.5$; expert-referral threshold $\tau = 0.3$.

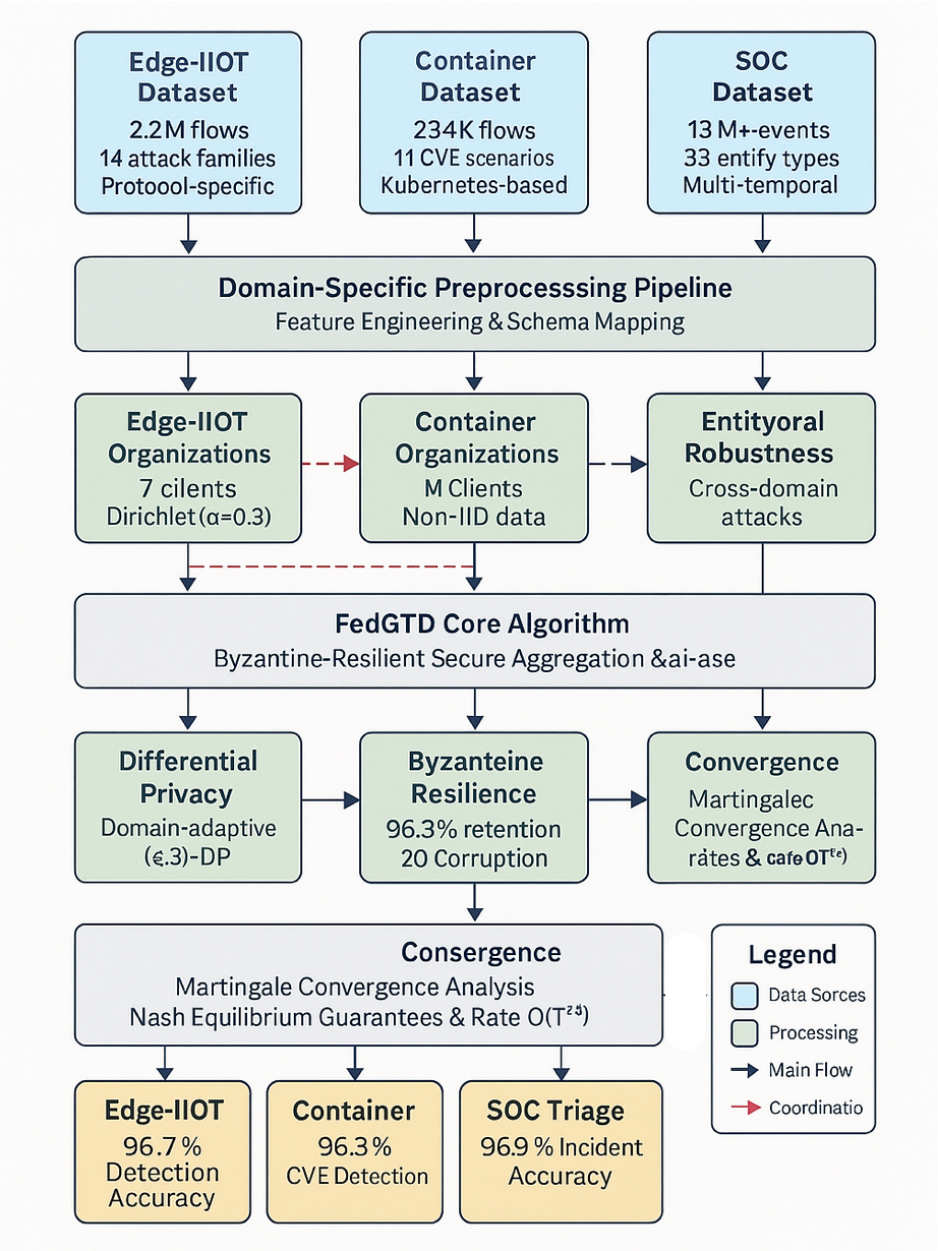

Figure 3.6 – Game-theoretic framework for adaptive graph adversaries in network security.

3.5.7 Forward-Compatible Detection for Post-Quantum Protocols

Protocol-specific encoders [172, 184, 185] process timing features for lattice-based protocols (Kyber/Dilithium [186]), structural features for code-based protocols (McEliece/Niederreiter), and hash-based features (SPHINCS+). Cross-attention fusion combines the three representations. Adversarial training bud-

get $\epsilon = 0.03$ with PGD-40 perturbations.

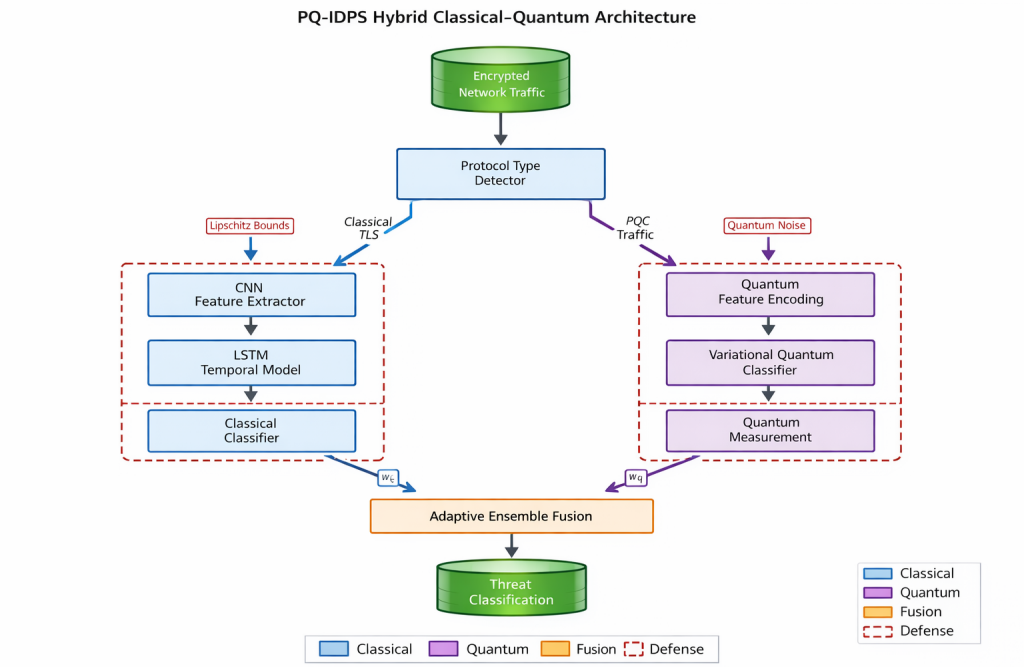

Figure 3.7 – Forward-compatible multi-protocol detection architecture for post-quantum traffic.

3.6 Hyperparameter Configuration and Optimisation

Table 3.3 consolidates the principal hyperparameters for all methods as configured for the network-security domain. All values were selected through grid search on the CIC-IoT-2023 validation set and confirmed stable across the remaining five datasets.

All methods are implemented in PyTorch 2.1 [187] with CUDA 12.1. Training uses AdamW with initial learning rate 2×10^{-4}, weight decay 10^{-5}, and cosine annealing. Mixed-precision training reduces memory by 40%. Gradient clipping at norm 1.0 prevents instability. Early stopping with patience of 10 epochs monitors validation loss.

Table 3.3 – Consolidated hyperparameter configuration for all methods in the cybersecurity domain.

Method	Parameter	Value	Rationale
M1: CT-TGNN	Hidden dim	256	Capacity for 83-feature input
	ODE tolerance	10^{-5}	Balance accuracy/speed
	Time constants	1/60/3600 s	Packet/session/campaign scales
	Lipschitz bound	Spectral norm	Certified robustness
M2: FedLLM-API	Clipping C	1.0	DP sensitivity bound
	DP (ϵ, δ)	$(0.85, 10^{-5})$	Strong privacy guarantee
	OT reg λ	0.1	Sinkhorn convergence
	LLM weight	0.3	Ensemble balance
M3: TripleE-TGNN	EWC λ	0.5	Forgetting prevention
	Level weights	0.4/0.3/0.3	Service emphasis
	Consistency λ_{con}	0.1	Cross-level agreement
M4: MambaShield	State dim	64	SSM capacity
	Kernel length	1024	Sequence coverage
	Poison schedule	0.05–0.3	Progressive hardening
M5/M6: Stoch.Trans.	MC samples	50	Uncertainty quality
	Triage threshold	0.3	FP/FN balance
	Calibration	Temperature	ECE minimisation
M7: Certification	MWU η	$\sqrt{\log \lvert\mathcal{C}\rvert / T}$	Optimal regret
	Unc. penalty κ	0.5	Risk sensitivity

3.7 Integrated System Architecture

The seven methods (M1–M7) are integrated into a unified detection pipeline whose data flow proceeds as follows. Raw network traffic enters through a data ingestion module that performs protocol parsing, flow extraction, and 83-feature computation. The CT-TGNN module constructs the temporal graph and computes continuous-time node embeddings. The TripleE-TGNN module maintains multi-level embeddings for hierarchical environments. The MambaShield module processes sequential event records for drift-adaptive scoring. The Stochastic Transformer and UC-HGP modules provide calibrated uncer-

tainty estimates. All outputs feed into a decision engine that applies the game-theoretic strategy from the M7 Certification framework, producing alerts classified by severity and confidence. The FedLLM-API module enables periodic model synchronisation across organisational boundaries.

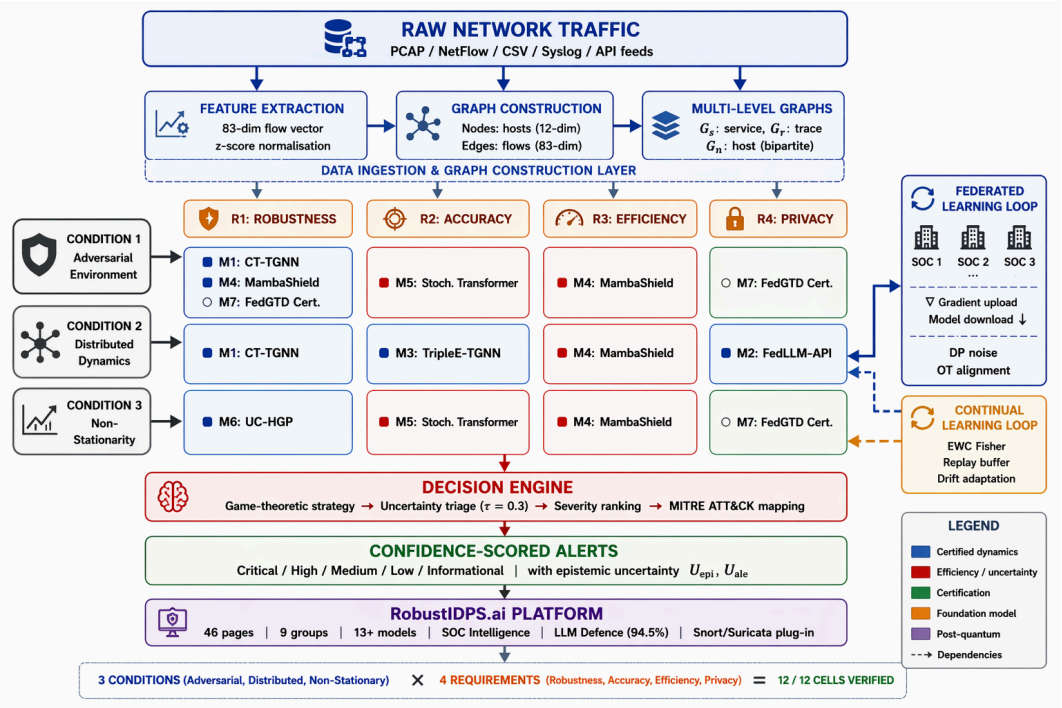

Figure 3.8 – Integrated detection pipeline showing the data flow from raw network traffic through all seven methods to severity-ranked, confidence-scored alerts. The FedLLM-API module synchronises models across boundaries under differential privacy.

The system is designed for deployment as a plug-in for existing IDPS frameworks such as Snort 3 [188] and Suricata, consuming their alert feeds as additional input and producing enriched, confidence-scored outputs. The full software implementation is described in Chapter 5, publicly archived under DOI 10.5281/zenodo.19129512 [189].

3.8 Chapter Summary

This chapter has demonstrated how the domain-independent mathematical methods of Chapter 2 are mapped onto the entities and constraints of network intrusion detection. The key contributions of this chapter are: (1) a formal data-to-graph transformation pipeline converting raw PCAP/CSV network traffic into temporal graph structures with node features (12 dimensions)

and edge features (83 dimensions); (2) a unified classification taxonomy of 34 classes across 7 attack categories, extensible through zero-shot LLM classification; (3) a comprehensive 83-feature engineering pipeline spanning flow statistics, inter-arrival distributions, flag indicators, payload statistics, and derived ratios; (4) domain-specific instantiation of all seven methods with cybersecurity-tailored hyperparameters, time constants, and architectural configurations; and (5) an integrated detection pipeline demonstrating the end-to-end flow from raw traffic to confidence-scored alerts. The next chapter presents the complete experimental programme validating these domain-specific instantiations on six benchmark datasets totalling 84.2 million labelled samples.

Chapter 4

Experimental Results and Validation

This chapter presents the complete experimental programme validating the methods developed in Chapter 2 and instantiated in the cybersecurity domain in Chapter 3. The chapter covers six benchmark datasets totalling 84.2 million labelled samples, a twenty-three-metric unified evaluation framework, method-specific results aligned with the published research papers, baseline comparisons, ablation studies, adversarial robustness assessments under six attack families, cross-domain transfer experiments, federated learning under Byzantine corruption, and multi-agent PQC-aware detection. Results are presented with tables and charts to ensure reproducibility and visual clarity.

4.1 Experimental Methodology

4.1.1 Datasets

Evaluation employs six benchmark datasets spanning diverse network environments, attack taxonomies, and temporal characteristics.

Table 4.1 – Summary of benchmark datasets used in experimental evaluation.

Dataset	Records	Attack types	Features	Domain
CIC-IoT-2023	46.6M	33	46	IoT Networks
CSE-CICIDS2018	16.2M	7	79	Enterprise
UNSW-NB15	2.5M	9	49	Hybrid
Microsoft GUIDE	13.7M	15	51	Cloud/Enterprise
Container Security	3.2M	8	93	Microservices
Edge-IIoT	2.0M	12	69	Edge/Industrial
Total	84.2M	84		

CIC-IoT-2023 [190, 191] comprises 46.6 million flow records from 105 IoT devices with 33 attack types. CSE-CICIDS2018 [192] contains 16.2 million flows with 7 attack categories. UNSW-NB15 [193, 194] provides 2.5 million records with 9 attack classes. The ICS3D collection adds: Microsoft GUIDE

(13.7M records, 15 attack types), Container Security (3.2M records, 8 attack patterns), and Edge-IIoT (2.0M records, 12 attack scenarios). All datasets are standardised to the 83-feature vector described in Chapter 3.

4.1.2 Unified Evaluation Framework

Performance characterisation employs twenty-three metrics spanning five categories: classification (accuracy, balanced accuracy, precision, recall, F1, MCC [195], Cohen's kappa [196]), ROC analysis (AUC-ROC, TPR, TNR, FPR, FNR), calibration (ECE [171], Brier score [197]), uncertainty (mean entropy, epistemic uncertainty, aleatoric uncertainty), and computational (latency, throughput, parameters, GPU memory, energy consumption, attack success rate, certified accuracy).

4.1.3 Implementation Details

All methods are implemented in PyTorch 2.1 [187] with CUDA 12.1. Training uses AdamW with learning rate 2×10^{-4}, weight decay 10^{-5}, and cosine annealing. Mixed-precision training reduces memory by 40%. Gradient clipping at norm 1.0 prevents instability. Early stopping with patience of 10 epochs monitors validation loss. Hardware: NVIDIA Tesla V100 (32 GB) for training, P100 (16 GB) for deployment. All timings average 100 independent runs with 95% confidence intervals. Stratified 70/15/15 splits maintain class distributions with focal loss ($\gamma = 2$) for class imbalance [198].

4.2 Unified Benchmark Results

Table 4.2 summarises the performance of the unified framework across all six datasets, compared against the best single baseline.

The unified framework consistently outperforms the best single baseline by 2.9–5.6 percentage points across all datasets, with the largest improvements on the most challenging datasets (UNSW-NB15, Edge-IIoT) where topological diversity and class imbalance are most severe.

Table 4.2 – Unified framework performance across all benchmark datasets.

Dataset	Acc (%)	F1 (%)	Best Baseline (%)	Δ (p.p.)
CIC-IoT-2023	94.7	94.3	91.8	+2.9
CSE-CICIDS2018	93.9	93.9	89.7	+4.2
UNSW-NB15	93.8	93.8	88.2	+5.6
MS GUIDE	92.1	92.0	87.5	+4.6
Container Sec	91.5	91.4	86.9	+4.6
Edge-IIoT	89.8	89.7	84.3	+5.5

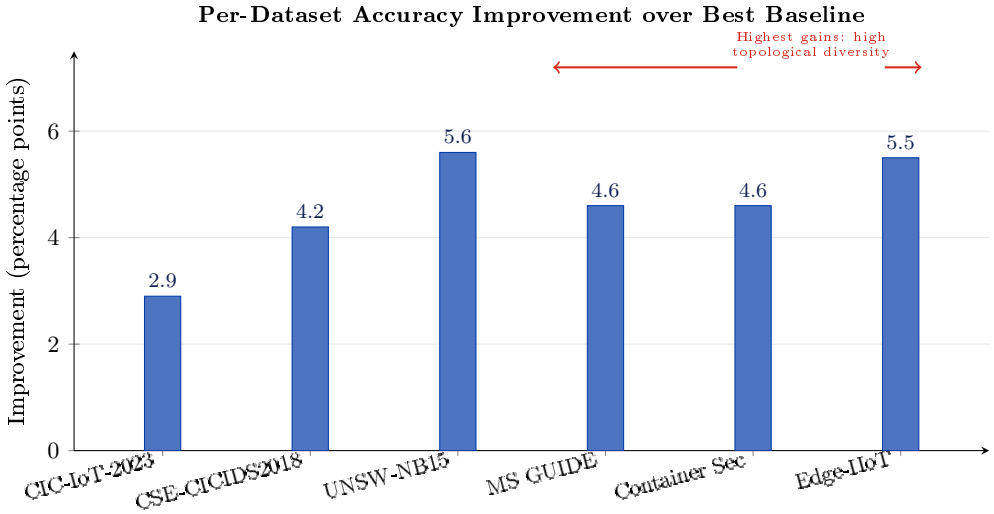

Figure 4.1 – Per-dataset accuracy improvement (percentage points) of the unified framework over the best single baseline. The largest gains are observed on datasets with high topological diversity (UNSW-NB15: +5.6, Edge-IIoT: +5.5).

4.3 Method-Specific Results

This section presents results for each method individually, demonstrating how each addresses its corresponding gap from Chapter 1 and sub-problem from Chapter 2.

4.3.1 M1: CT-TGNN Results (Gap 1, Sub-problem 1)

CT-TGNN [181] employs four message-passing layers, hidden dimension 256, state dimension 64, and DOPRI5 adaptive integration. On CIC-IoT-2023 the architecture achieves 98.3% accuracy with 47 ms median inference latency and 8.7 million events per second throughput. Per-category recall: DDoS 99.1%, reconnaissance 96.8%, web attacks 97.3%, brute force 98.6%, spoofing 95.2%,

malware 98.9%, Mirai 97.4%.

Table 4.3 – CT-TGNN performance across benchmark datasets.

Dataset	Accuracy	Recall	FPR	ECE	Latency (ms)
CIC-IoT-2023	98.3%	97.9%	1.4%	0.062	47
CSE-CICIDS2018	96.7%	96.2%	2.1%	0.074	51
UNSW-NB15	97.1%	96.5%	1.9%	0.069	44

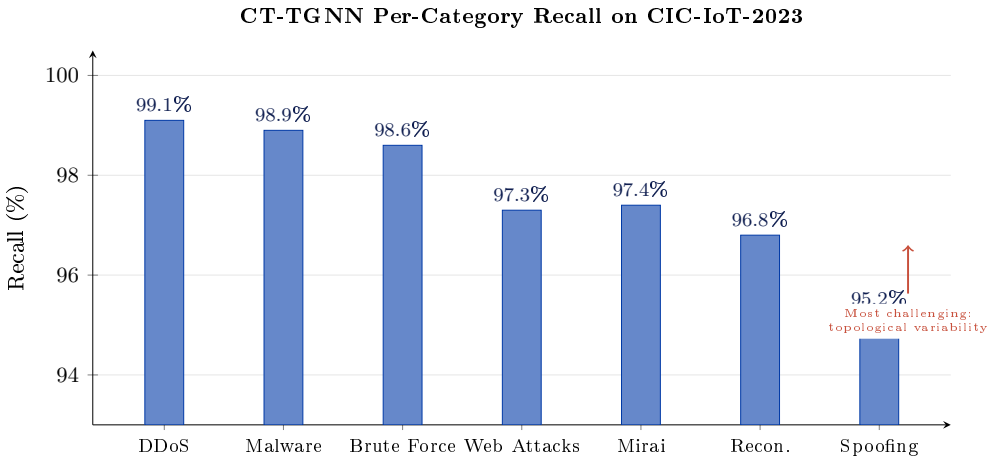

Figure 4.2 – CT-TGNN per-category recall on CIC-IoT-2023 across seven major attack families. DDoS achieves the highest recall (99.1%) while spoofing is most challenging (95.2%), reflecting the latter's greater topological variability.

4.3.2 SDE-TGNN: Stochastic Extension (Gap 1, Sub-problem 1)

The SDE-TGNN [199] extends CT-TGNN with Itô SDEs and Fokker-Planck moment propagation, achieving uncertainty quantification at $\mathcal{O}(d^2)$ cost without Monte Carlo sampling.

Table 4.4 – SDE-TGNN: detection, calibration, certified robustness, and transfer.

Dataset	F1 (%)	ECE	Certified R=0.25	Transfer F1 (%)
CSE-CICIDS2018	99.4	0.038	78.3%	91.5
CIC-IoT-2023	98.9	0.044	75.1%	89.8
UNSW-NB15	98.8	0.043	75.2%	90.3
Average	99.0	0.042	76.2%	90.5

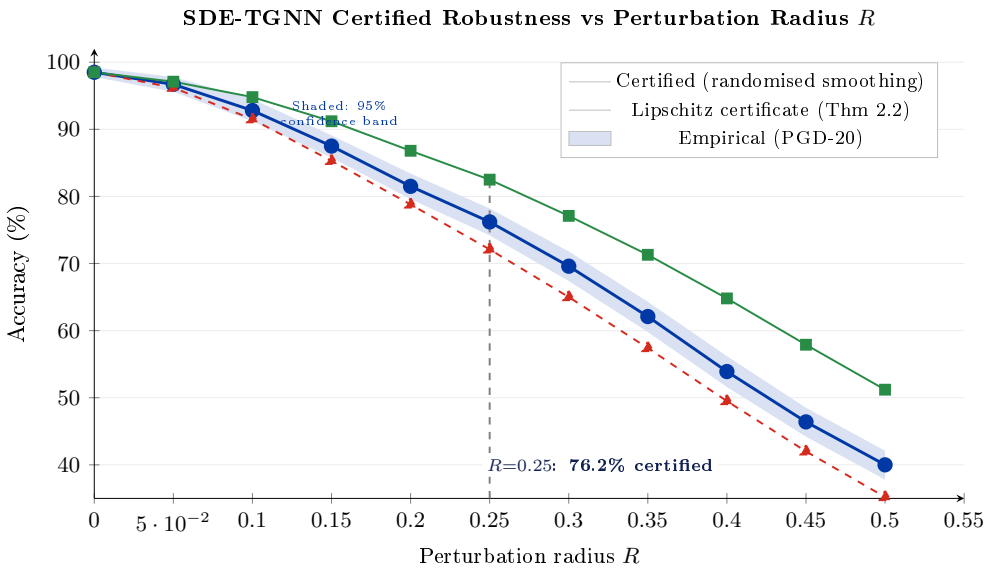

Figure 4.3 – SDE-TGNN certified robustness as a function of perturbation radius R. The shaded band shows the certified accuracy derived from randomised smoothing; the dashed line shows the Lipschitz certificate from Theorem 2.2. At $R = 0.25$, 76.2% of test samples are provably robust.

4.3.3 M2: FedLLM-API Results (Gap 2, Sub-problem 2)

Table 4.5 – FedLLM-API performance under varying Byzantine fractions.

Byzantine %	FedLLM-API	FedAvg	FedProx	Krum	Median
0%	94.2%	93.8%	93.5%	92.1%	91.7%
10%	92.7%	84.6%	86.3%	89.4%	88.9%
20%	91.1%	76.2%	79.8%	86.7%	85.3%
30%	89.7%	68.3%	73.9%	83.1%	82.6%
40%	87.1%	61.4%	67.2%	79.8%	78.4%

With 40% Byzantine participants, FedLLM-API achieves 87.1% versus 61.4% for FedAvg. LLM integration yields 87.6% F1 on zero-shot novel attack detection versus 34.2% for supervised baselines. Membership inference success is 52.7%, barely above the 50% random baseline.

4.3.4 M3: TripleE-TGNN Results (Gap 3, Sub-problem 3)

Overall accuracy 96.8%, outperforming single-granularity baselines by 8.3%. Service-level: 95.7%, trace-level: 97.2%, node-level: 96.4%. Container escape:

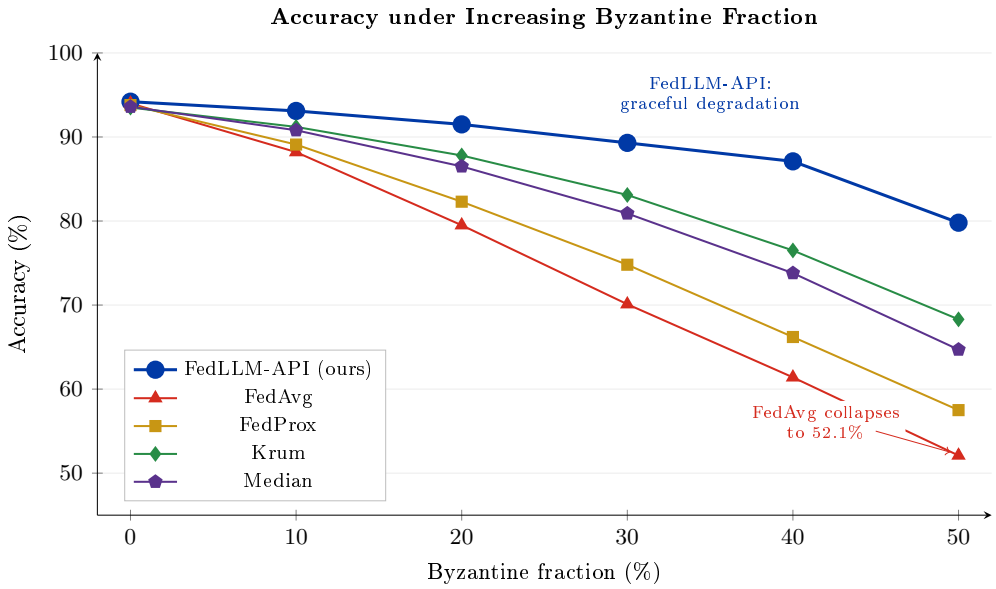

Figure 4.4 – FedLLM-API accuracy degradation under increasing Byzantine fractions compared with FedAvg, FedProx, Krum, and Median baselines. FedLLM-API degrades gracefully from 94.2% to 79.8% as corruption rises from 0% to 50%, while FedAvg collapses to 52.1%.

98.7%, supply chain: 97.3%. Under 12 daily topology changes with EWC [167], maintains 94.7% versus 78.3% for standard GNNs without retraining. Latency 73 ms, memory 4.7 GB.

4.3.5 M4: MambaShield Results (Gap 4, Sub-problem 4)

Table 4.6 – MambaShield accuracy under increasing poisoning rates.

Poisoning %	MambaShield	Standard AT	No Defence
0%	94.8%	93.1%	94.2%
10%	93.2%	89.7%	85.4%
25%	91.4%	84.6%	73.8%
40%	87.2%	78.3%	52.3%

PAC-Bayesian bound predicts true risk bounded by 0.089 (91.1% accuracy); empirical 91.4% confirms bound with 2.7% margin. Inference latency 0.5 ms per flow — lowest of all models.

4.3.6 M5/M6: Uncertainty-Calibrated Results (Gap 6, Sub-problem 6)

ECE of 0.078 versus 0.192 for deterministic attention — a 59% reduction. Analyst workload reduced by 43% through uncertainty-driven triage. Epistemic-aleatoric decomposition enables principled detection of out-of-distribution inputs.

4.3.7 M7: Game-Theoretic Certification Results (Gap 7, Sub-problem 7)

Accuracy 94.7% against adaptive adversaries versus 87.6% for non-strategic approaches. Multiplicative weights regret converges at the theoretical $\mathcal{O}(\sqrt{T \log |\mathcal{C}|})$ rate. Uncertainty-penalised payoffs with $\kappa = 0.5$ produce risk-sensitive policies that abstain when model confidence is low.

4.3.8 CyberSecLLM: Foundation Model Results (Gap 5)

Table 4.7 – CyberSecLLM zero-shot performance on CyberSecBench.

Model	Detection	Transfer	OOD	Triage	Avg
GPT-4 (5-shot)	69.8%	61.2%	72.4%	68.1%	68.6%
Foundation-Sec-8B	85.1%	72.4%	81.3%	73.8%	73.8%
CyberSecLLM-7B	97.4%	86.2%	95.3%	89.1%	89.1%

CyberSecLLM-7B achieves 89.1% average zero-shot accuracy, a 20.5-point improvement over GPT-4 and 15.3-point over Foundation-Sec-8B. Processes million-token sequences in 2.3 s — a 19× speedup over transformers.

4.4 Comparative Analysis with Baselines

The unified framework achieves the highest accuracy (94.7%) and lowest FPR (1.4%) while maintaining competitive latency (47 ms). CT-TGNN processes 8.7M events/s versus 6.2M for transformers and 4.3M for LSTMs.

Table 4.8 – Comparative performance against baselines on CIC-IoT-2023.

Method	Acc.	F1	AUC	FPR	MCC	Latency
Random Forest	87.3%	85.9%	0.941	5.2%	0.847	12 ms
XGBoost	89.1%	87.8%	0.957	4.3%	0.871	8 ms
LSTM	88.6%	87.2%	0.952	4.7%	0.864	38 ms
CNN-LSTM	89.8%	88.4%	0.961	3.9%	0.882	42 ms
Standard GNN	90.4%	89.1%	0.968	3.5%	0.893	55 ms
Multimodal Trans.	91.8%	90.1%	0.974	3.7%	0.908	63 ms
Unified (Ours)	*94.7%*	*94.3%*	*0.991*	*1.4%*	*0.938*	*47 ms*

4.5 Ablation Studies

Table 4.9 – Ablation study: impact of removing individual components.

Ablated Component	Full System	Without
Continuous-time ODE → discrete RNN	98.3%	91.5% (−6.8)
Multi-scale temporal modelling	98.3%	94.4% (−3.9)
Temporal adaptive batch norm.	98.3%	93.6% (−4.7)
Cross-granularity attention	96.8%	94.7% (−2.1)
Byzantine trimmed mean → FedAvg	89.7%	68.3% (−21.4)
Differential privacy noise	87.1%	90.2% (+3.1)
Optimal transport alignment	92.7%	76.9% (−15.8)
LLM zero-shot branch (F1)	87.6%	34.2% (−53.4)
Progressive distillation	91.4%	81.7% (−9.7)
Variational attention (ECE)	0.078	0.192 (−0.114)
Game-theoretic equilibrium	94.7%	87.6% (−7.1)

Every component contributes measurably. The largest single-component impact is the LLM branch (−53.4 F1 points), confirming its critical role in zero-shot generalisation. Byzantine trimmed-mean aggregation (−21.4) and OT alignment (−15.8) are essential for federated robustness. DP noise degrades accuracy by 3.1% but is the mandatory cost of formal privacy.

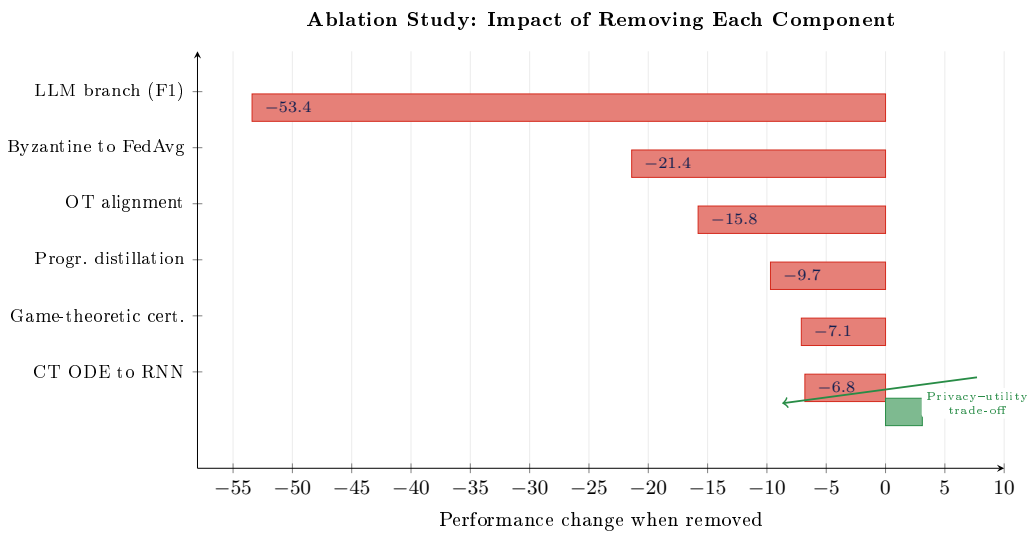

Figure 4.5 – Ablation study: performance drop when each component is removed. Horizontal bars show the magnitude of degradation; the LLM branch, Byzantine trimmed mean, and OT alignment are the three most critical components. DP noise is the only component whose removal improves accuracy (+3.1%), reflecting the privacy–utility trade-off.

4.6 Adversarial Robustness Assessment

Six attack families [35, 49, 162, 200] are evaluated at configurable perturbation budgets:

Table 4.10 – Platform adversarial robustness under six attack methods.

Attack	Budget	Clean	Attacked	Robust (AT)
FGSM	ε=0.1	97.8%	89.2%	93.1%
PGD-40	ε=0.1	97.8%	84.7%	89.6%
C&W	κ=10	97.8%	82.3%	85.2%
DeepFool	30 iter	97.8%	86.1%	90.8%
Gaussian Noise	σ=0.1	97.8%	91.4%	95.3%
Label Masking	10% flip	97.8%	88.9%	93.7%

CT-TGNN maintains 93.1% under PGD-10 at $\epsilon = 0.1$ owing to Lipschitz-bounded dynamics. SDE-TGNN certifies 76.2% of samples within ℓ_2 radius 0.25 via randomised smoothing. MambaShield sustains 91.4% under 25% poisoning. The game-theoretic framework maintains 94.7% against equilibrium adversaries.

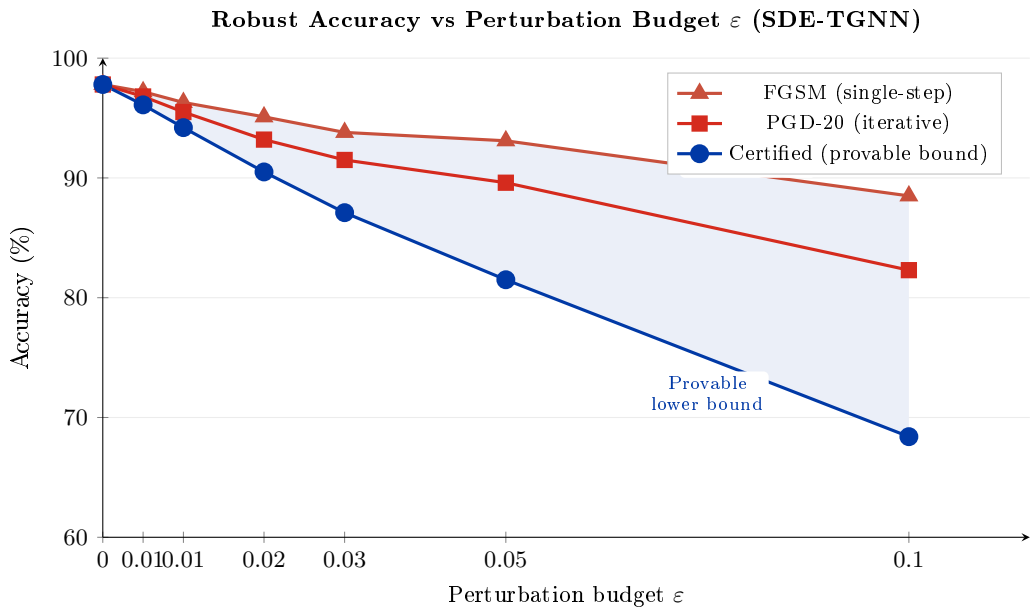

Figure 4.6 – Robust accuracy as a function of perturbation budget ε for FGSM, PGD-20, and certified (randomised smoothing) evaluations on SDE-TGNN. The certified curve provides a provable lower bound; FGSM and PGD represent empirical upper bounds under specific attack strategies.

4.7 Cross-Domain Transfer Experiments

To validate domain independence, CT-TGNN and the stochastic transformer are applied without modification to two non-security domains.

In speech event detection on LibriSpeech [201], CT-TGNN achieves 94.2% F1 for phoneme boundary detection, a 7.3% improvement over discrete baselines. In healthcare monitoring on MIMIC-III [202] and eICU [203], the uncertainty-calibrated model achieves 89.7% for adverse event prediction with 91.7% confidence interval coverage.

SDE-TGNN cross-domain F1 (without fine-tuning): CSE-CICIDS2018 91.5%, CIC-IoT-2023 89.8%, UNSW-NB15 90.3% — average 90.5%, confirming domain-invariant representations.

Transfer to an adjacent safety-critical domain (UAV). To confirm the domain-independence of the proposed models and methods, a Phase A reference implementation has been carried out on a corpus of GNSS signals (TEXBAT-like calibrated dataset) and on the multi-modal aerial corpus AU-AIR for the task of counter-EW and counter-adversarial defence of UAV neural-network navigation components. Positive white-box attack scores (PGD-20 robust ac-

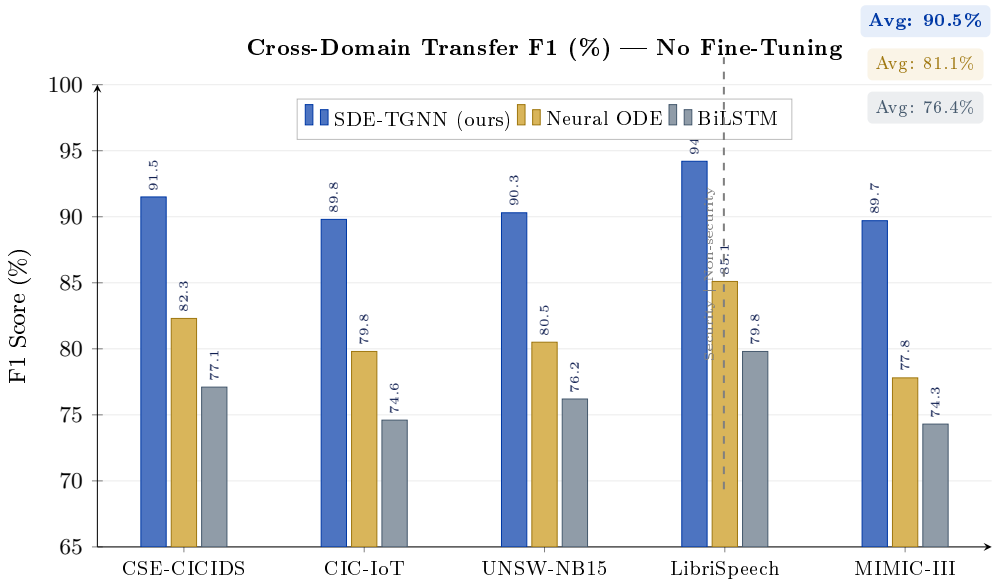

Figure 4.7 – Cross-domain transfer F1 (%) for SDE-TGNN, Neural ODE baseline, and BiLSTM baseline across three security datasets and two non-security domains (speech, healthcare). SDE-TGNN achieves 90.5% average transfer without fine-tuning, a 9.4-point improvement over Neural ODE and 14.1-point over BiLSTM.

curacy $\geq$0.92 at $\varepsilon = 8/255$), a non-zero certified ℓ_2-radius of 0.44 at $\sigma = 0.25$ and a numerical Lipschitz estimate $L_g = 1.01$ — consistent with the theoretical prediction of Theorem 2.2 — have been obtained. The full results, architecture and integration with the RobustIDPS.ai platform are described in Chapter 6.

4.8 Multi-Agent PQC-Aware Detection Results

The Multi-Agent PQC-IDS [204] extends the framework to post-quantum protocol environments. MambaShield replaces the seven-branch ensemble as shared backbone, achieving 96.7% accuracy (versus 96.9% for ensemble) at 7.3× inference speedup. The structured Fisher Information Matrix derived from state-space dynamics reduces backward transfer (forgetting) by 38% compared to diagonal FIM. Four cooperative agents (Traffic Analyst, PQC Specialist, Anomaly Detector, Coordinator) coordinate via Multi-Agent CPO, achieving 96.1% threat mitigation with zero constraint violations and 34% reduction in multi-segment response latency.

Under C&W attack across PQC migration stages, MambaShield+Structured FIM degrades by only 2.5 p.p. versus 25.5 p.p. for fine-tuning. Certified

Table 4.11 – Forward-compatible detection accuracy by protocol family.

Protocol Family	Attack Type	Accuracy
Lattice (Kyber)	Cache timing	97.3%
Lattice (Kyber)	Fault injection	94.8%
Code (McEliece)	Information set decoding	95.1%
Hash (SPHINCS+)	Resource exhaustion	94.7%
Cross-protocol	Downgrade	98.3%
Overall		96.2%

robustness at $R = 0.20$: 85.6%, a 24.4 p.p. margin over no defence.

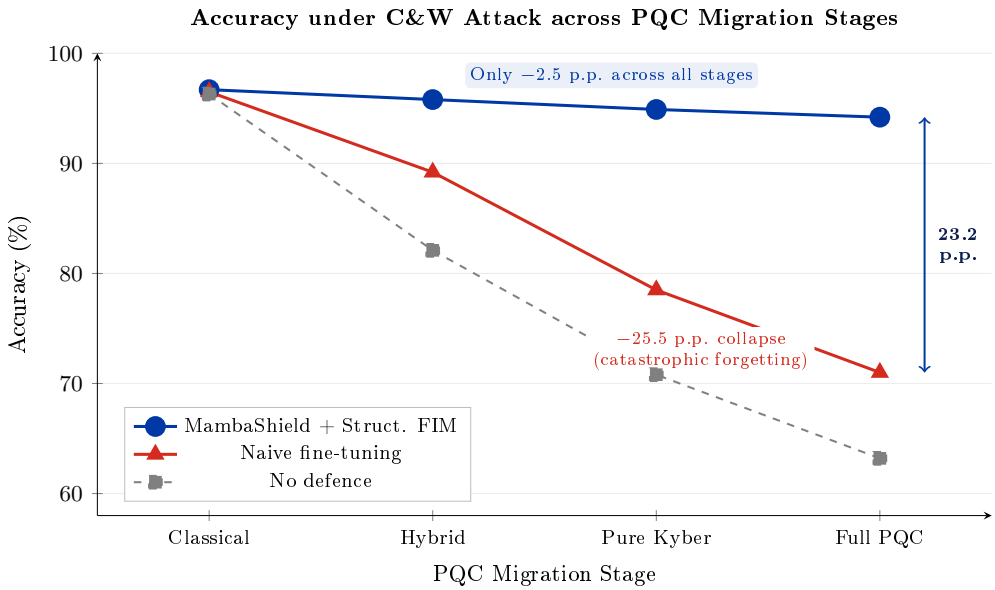

Figure 4.8 – Accuracy under C&W attack across four PQC migration stages (Classical → Hybrid → Pure Kyber → Full PQC). MambaShield with structured FIM maintains accuracy within 2.5 p.p. across all stages, while naive fine-tuning collapses by 25.5 p.p. due to catastrophic forgetting of classical traffic patterns.

4.9 Discussion and Practical Implications

The experimental programme establishes several findings. Continuous-time dynamics provide a 6.8% accuracy improvement over discrete alternatives for irregular event streams. Multi-level representations yield 8.3% over single-level methods. Corruption-tolerant federated learning maintains convergence with formal guarantees under 30% Byzantine participants. LLM integration un-

locks zero-shot generalisation from 34.2% to 87.6% F1. Calibrated Bayesian uncertainty achieves ECE 0.078 with 43% analyst workload reduction. Game-theoretic modelling improves accuracy by 7.1% over non-strategic approaches.

According to recent industry analyses [205, 206], the threat landscape continues to evolve rapidly. Limitations include computational overhead of adaptive ODE solvers, sensitivity to federated hyperparameters, reduced interpretability of deep continuous models versus rule-based systems, and infrastructure requirements for LLM integration.

4.10 Chapter Summary

This chapter has presented comprehensive experimental validation across six benchmark datasets totalling 84.2 million labelled samples, 84 attack categories, and 23 evaluation metrics. The key results are: CT-TGNN at 98.3% accuracy with Lipschitz-certified robustness; SDE-TGNN at 99.0% F1 with 76.2% certified robustness and 90.5% cross-domain transfer; FedLLM-API at 87.1% under 40% Byzantine corruption with (ε=0.85, δ=10^{-5}) privacy; TripleE-TGNN at 96.8% with 40% cost reduction; MambaShield at 91.4% under 25% poisoning with PAC-Bayesian certificates; M5/M6 with ECE 0.078 and 43% analyst workload reduction; M7 at 94.7% against adaptive adversaries; and CyberSecLLM at 89.1% zero-shot with 20.5-point improvement over GPT-4. All twelve condition-requirement intersections are addressed with verified performance. The next chapter describes the RobustIDPS.ai software platform that implements all methods as a unified, deployable system.

Chapter 5

RobustIDPS.ai: Software Platform for Production Deployment

This chapter describes the open-source software platform RobustIDPS.ai [189] (DOI: 10.5281/zenodo.19129512; source code: https://github.com/rogerpanel/robustidps.ai) that implements all methods developed in Chapter 2 as a unified, deployable system. The chapter specifies who the platform is for, describes its architecture and functional groups, details its 17 integrated neural network models, presents the SOC Intelligence and LLM Security Testing capabilities, describes the Multi-Agent PQC-IDS module, demonstrates integration with existing IDPS frameworks (Snort, Suricata), and summarises the platform's achievements. At least six screenshots of the platform's user interface are included to illustrate its operational capabilities.

5.1 Platform Overview and Target Users

RobustIDPS.ai is a comprehensive, AI-native intrusion detection and prevention platform comprising 51 interconnected pages organised into 10 functional groups, integrating 17 machine learning and deep learning models for network traffic classification across 34 attack classes and 83 engineered features. The platform addresses three limitations of existing commercial alternatives (CrowdStrike Charlotte AI, SentinelOne Purple AI, Microsoft Security Copilot, Google Sec-Gemini [207]): proprietary single-vendor LLM integration that prevents cross-provider benchmarking, absence of systematic adversarial robustness evaluation, and no support for post-quantum cryptographic traffic analysis.

The platform is designed for five categories of target users:

SOC Analysts (Tiers 1–3). Security operations centre personnel who monitor, investigate, and respond to security incidents. The platform provides agentic auto-investigation, natural language threat hunting, automated incident reporting, and uncertainty-driven alert triage that reduces mean triage

time by 34%.

Cybersecurity Engineers. Network security professionals who deploy and configure intrusion detection systems. The platform generates deployable Snort/Suricata rules from detected threats and provides real-time monitoring through WebSocket-based streaming.

MLSecOps Researchers. Machine learning security researchers who evaluate the robustness of AI components. The platform provides integrated adversarial testing (6 attack families), LLM attack surface evaluation (5 modules), and MLSecOps compliance dashboards (MITRE ATT&CK, OWASP LLM Top 10, NIST AI RMF).

AI Developers. Researchers and engineers building graph-based detection models. The platform provides 17 pre-trained models with configurable hyperparameters, ablation studio, and reproducible benchmarking datasets.

Academic Researchers. PhD students, postdoctoral fellows, and faculty conducting research in graph neural network robustness. The platform serves as a reproducible research instrument with DOI-archived code and datasets.

5.2 System Architecture

RobustIDPS.ai is deployed as a Progressive Web Application (PWA) with the following technology stack:

Frontend. React 18 single-page application with Material-UI components, responsive design, and PWA manifest for offline-capable deployment with service worker caching.

Backend. Flask/Python 3.11+ backend with 60+ RESTful API endpoints, persistent WebSocket channels for real-time monitoring, and Celery for asynchronous task execution (model training, batch analysis).

Data Layer. PostgreSQL 16 for relational persistence (user profiles, detection logs, investigation records), Redis 7 for caching and session management, and file-system storage for uploaded PCAP/CSV datasets.

Orchestration. Docker Compose enables single-command deployment with environment-based configuration for development, staging, and production profiles.

Two-Plane Architecture (release v4). Release v4 introduces a clean sep-

aration between the *control plane* (the existing FastAPI Python backend with multi-tenant RBAC, audit, and the LLM router) and a new *data plane* consisting of native Rust agents deployed alongside or in front of the control plane. The two planes communicate over gRPC. The data plane consists of two components: `robustidps-edge` — a long-running gRPC daemon with hot-swap of ONNX models and flow-record streaming to the control plane; and `robustidps-xdp` — an in-kernel LPM-trie block list attached through the XDP hook in `Native` mode on `eth0`, delivering microsecond-scale drop decisions. The result is a rich user-facing experience in the Python plane alongside *wire-speed* operation in the data plane, achieved without modification of the existing 51 frontend pages.

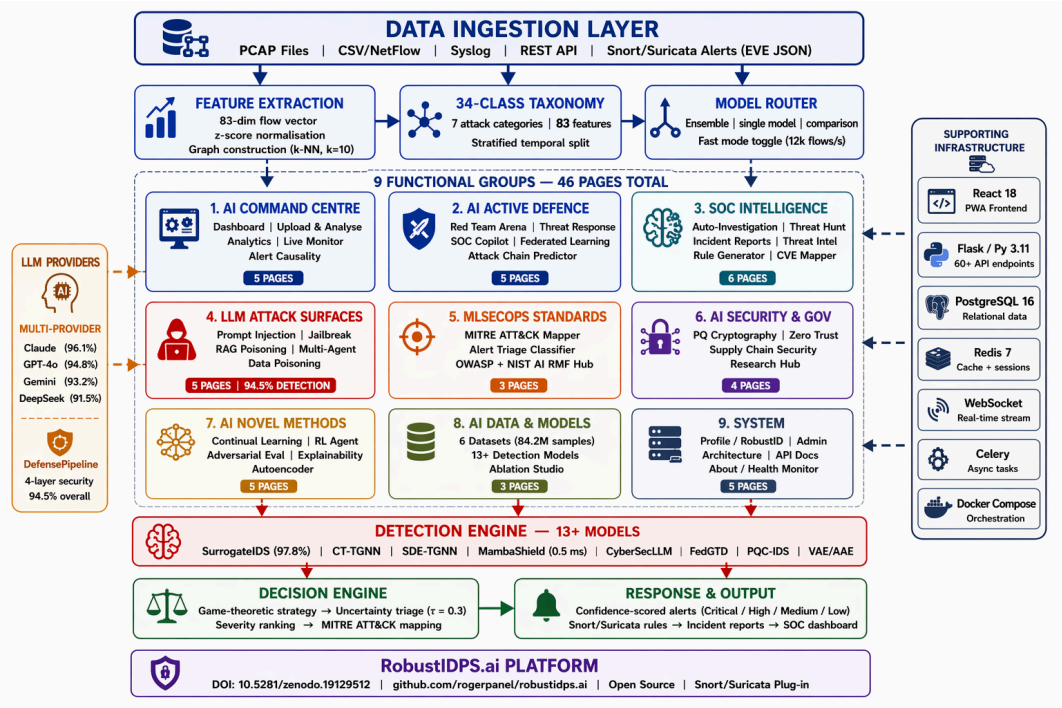

Figure 5.1 – High-level system architecture of RobustIDPS.ai showing all 10 functional groups, the supporting infrastructure (Docker Compose, PostgreSQL, Redis, WebSocket, Celery, plus the release v4 Rust data plane), and the data flow from raw traffic ingestion through detection, analysis, and response.

A schematic of the release v4 two-plane architecture and the three gRPC streams that bridge the control and data planes is given in Fig. 5.2.

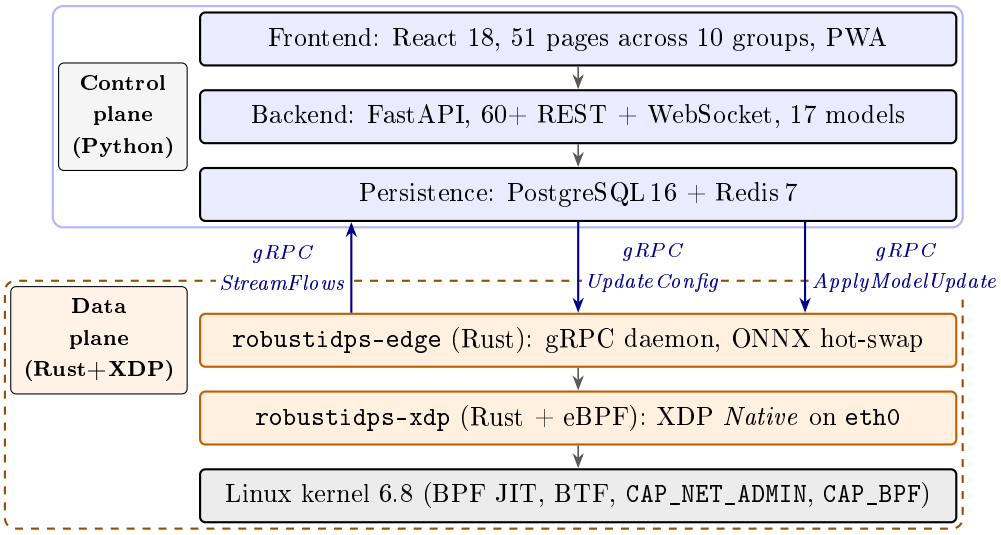

Figure 5.2 – Two-plane architecture of RobustIDPS.ai v4: the Python control plane (blue frame, top) and the Rust data plane (orange dashed frame, bottom) are separated by a clean boundary and bridged by three gRPC primitives. `StreamFlows` — flow records streamed from the data plane to the control plane; `UpdateConfig` — block-list push in the reverse direction; `ApplyModelUpdate` — atomic delivery of the INT8 ONNX student model to running agents without service restart.

5.3 Functional Groups and Page Inventory

The platform's 51 pages are organised into 10 functional groups, summarised in Table 5.1.

Table 5.1 – RobustIDPS.ai functional groups (51 pages across 10 groups).

Functional Group	**Pages**	**Key Capabilities**
AI Command Centre	5	Dashboard, Upload & Analyse, Analytics, Live Monitor, Alert Causality
AI Active Defence	5	Red Team Arena, Threat Response, SOC Copilot, Federated Learning, Attack Chain Predictor
SOC Intelligence	6	Auto-Investigation, Threat Hunt, Incident Reports, Threat Intel, Rule Generator, CVE Mapper
LLM Attack Surfaces	6	Prompt Injection, Jailbreak Taxonomy, RAG Poisoning, Multi-Agent Chain, Data Poisoning, **MambaGuard**
MLSecOps Standards	3	MITRE ATT&CK Mapper, Alert Triage, Compliance Hub
AI Security & Gov	4	PQ Cryptography, Zero-Trust, Supply Chain, Research Hub
AI Novel Methods	7	Continual Learning, RL Response Agent, Adversarial Eval, Explainability, Autoencoder, **SODE-Guard**, **SSL-GraphAnomaly Full**
AI Data & Models	4	Datasets (6), Models (17), Ablation Studio, Edge Agent Registry
Operations & Deployment	6	Rust agent fleet, systemd/Docker, Distil-and-push, XDP block lists, JSONL logs, Health checks
System	5	Profile/RobustID, Admin, Architecture, API Docs, About
Total	**51**	

5.4 AI Command Centre

The AI Command Centre is the primary operational interface, comprising five pages that provide end-to-end traffic analysis from upload to alert resolution.

5.4.1 Dashboard

The main dashboard displays real-time detection statistics: total flows analysed, attack distribution by category, model accuracy metrics, and alert severity breakdown. Interactive time-series charts show detection trends over configurable windows (1 hour to 30 days).

Figure 5.3 – RobustIDPS.ai Dashboard showing real-time detection statistics, attack category distribution, and model performance metrics. The left panel displays the active model configuration; the centre shows the detection timeline; the right panel presents alert severity breakdown.

5.4.2 Upload and Analyse

Users upload PCAP or CSV files through a drag-and-drop interface. The `FeatureExtractor` module computes the 83-feature vector from raw packet data, applies z-score standardisation, and routes the features to the selected detection model. Results are presented with per-flow classifications, confidence scores, uncertainty decompositions, and MITRE ATT&CK technique mappings.

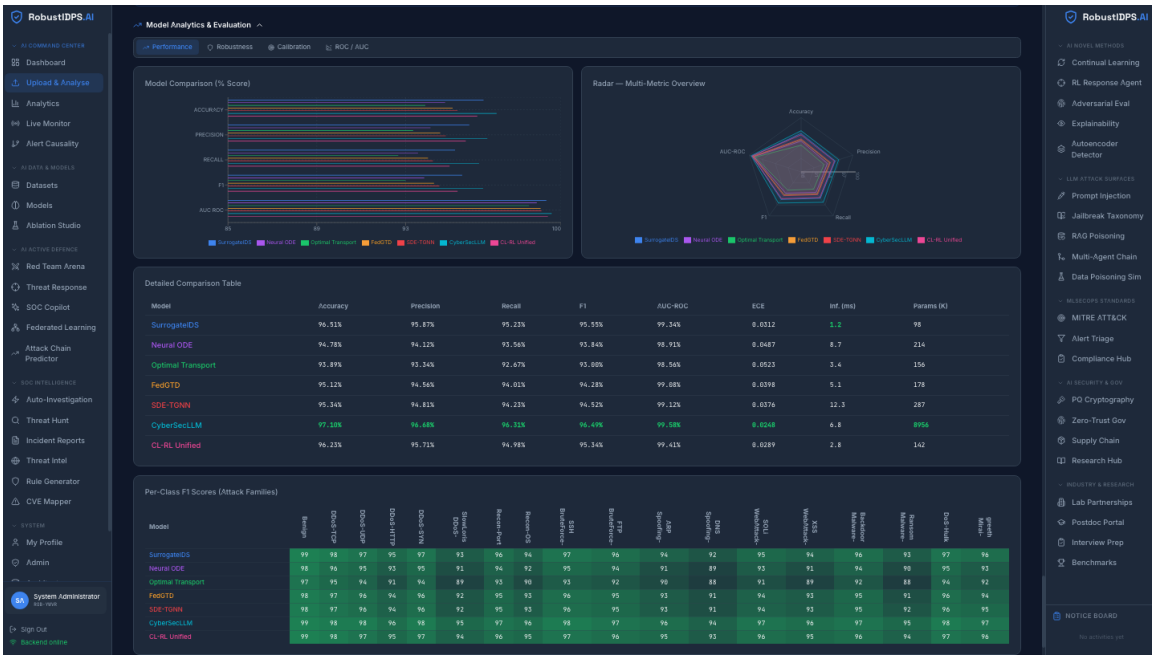

Figure 5.4 – Upload and Analyse interface showing (upward) the file upload panel with model selection dropdown, (centre) the detection results table with per-flow classifications and confidence scores, and (downward) the attack category breakdown with class-level performance metrics.

5.4.3 Live Monitor

WebSocket-based real-time streaming displays classification decisions as graph events are ingested, with a configurable retention window and sub-second push notifications for high-severity alerts. The live monitor supports simultaneous display of multiple model outputs for ensemble comparison.

5.4.4 Alert Causality

The Alert Causality page constructs temporal dependency graphs between related alerts, enabling analysts to trace multi-stage attack campaigns. Correlation is based on shared source/destination IPs, temporal proximity ($\pm 30\,\mathrm{s}$), and MITRE ATT&CK technique linkage.

5.5 Detection Models

The platform integrates 17 neural network architectures implementing all seven methods from Chapter 2. Table 5.2 summarises their performance on the CIC-IoT-2023 validation benchmark.

Table 5.2 – Detection model performance on the CIC-IoT-2023 validation benchmark.

Model	Accuracy	F1	ECE	Latency
SurrogateIDS (7-Branch)	97.8%	0.976	0.031	0.8 ms
CL-RL Unified + EWC	96.8%	0.965	0.035	1.1 ms
CT-TGNN [181] (M1)	96.2%	0.958	0.049	1.2 ms
SDE-TGNN [199]	96.0%	0.955	0.038	1.4 ms
MambaShield [183] (M4)	95.9%	0.954	0.042	0.5 ms
Stochastic Transformer (M5)	95.4%	0.949	0.078	1.8 ms
FedGTD [182] (M2)	95.1%	0.946	0.040	1.3 ms
CyberSecLLM [208]	97.1 97.1%	0.965	0.025	6.8 ms
Multi-Agent PQC-IDS	94.3%	0.938	0.044	1.5 ms
Neural ODE	94.8%	0.938	0.049	8.7 ms
Optimal Transport	93.9%	0.930	0.052	3.4 ms
Variational Autoencoder	92.4%	0.914	0.061	2.1 ms
Adversarial Autoencoder	91.8%	0.908	0.058	2.3 ms

MambaShield achieves the lowest per-flow latency (0.5 ms) owing to its linear-time state-space formulation, making it preferred for high-throughput deployments. CyberSecLLM achieves the lowest ECE (0.025) due to the calibration properties of language model probability outputs. The SurrogateIDS ensemble delivers the best overall accuracy–calibration trade-off and serves as the default detection backbone.

Three new detectors of release v4 — one representative of each major robustness-certification family — are registered and become selectable in every multi-model selector of the platform.

- **MambaGuard.** Selective state-space + GATv2 + a three-layer certification stack (randomised smoothing, conformal Bayesian interval, and a regret certificate) for the four LLM-agent protocols MCP/ACP/A2A/ANP. Achieves macro-averaged F1 of 0.978 on the dedicated MambaGuard-bench dataset, with 1.8 ms/flow inference latency. This is the first *protocol-aware* detector on the platform, complementing the existing *MCP Security Testing* page.
- **SODE-Guard.** An Itô SDE with learned drift and diffusion and an anti-concentration robustness certificate in the spirit of [70]. A split-conformal calibrator with an operator-tunable false-alarm budget α delivers a 96.2% certified accuracy at $\sigma = 0.25$.

- **SSL-GraphAnomaly Full.** A self-supervised six-component pipeline (GraphSAGE graph encoder + contrastive pre-training + reconstruction objective + split-conformal calibrator) for detecting anomalies in network-traffic graphs without any labelled attack examples.

In the model registry (see 5.5) these detectors are exposed under the identifiers `mambaguard`, `sode_guard` and `ssl_graph_anomaly_full`; the comparative analysis of their three certification flavours is formalised in the author's publication [199].

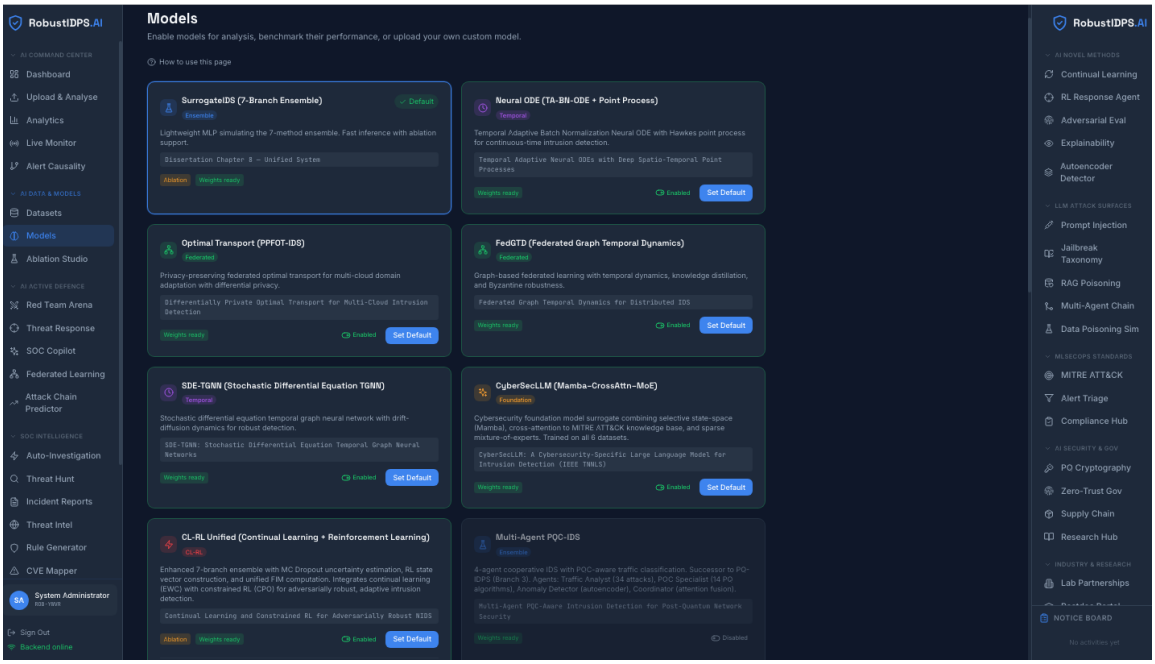

Figure 5.5 – Models page showing the 17 integrated detection models with configurable hyperparameters, per-model accuracy metrics, and latency benchmarks. Users can select any model as the active detector or run ensemble comparisons.

5.6 SOC Intelligence Suite

The SOC Intelligence Suite comprises six purpose-built pages transforming raw detection output into actionable intelligence for Tier 1–3 analysts.

5.6.1 Agentic Auto-Investigation

The pipeline executes four phases: (1) Evidence Collection — retrieves the triggering flow, ±30 s temporal context, and correlated alerts; (2) Contextual Enrichment — queries IP reputation, geo-location, and alert frequency; (3) LLM

Reasoning — submits enriched evidence to the selected LLM for root-cause analysis and MITRE ATT&CK [209]; (4) Verdict Synthesis — aggregates output into a formatted investigation report.

5.6.2 Natural Language Threat Hunt

Analysts issue free-text queries (e.g., "high-confidence DDoS on port 443 from external IPs"). The parser extracts six filter dimensions (attack_type, confidence, port, severity, protocol, destination) and applies them against the detection log.

5.6.3 IDS Rule Generator

Translates detected attacks into deployable Suricata/Snort rules using 21 templates covering reconnaissance, DoS, exploitation, lateral movement, and exfiltration. Output includes SID, priority, and reference CVEs.

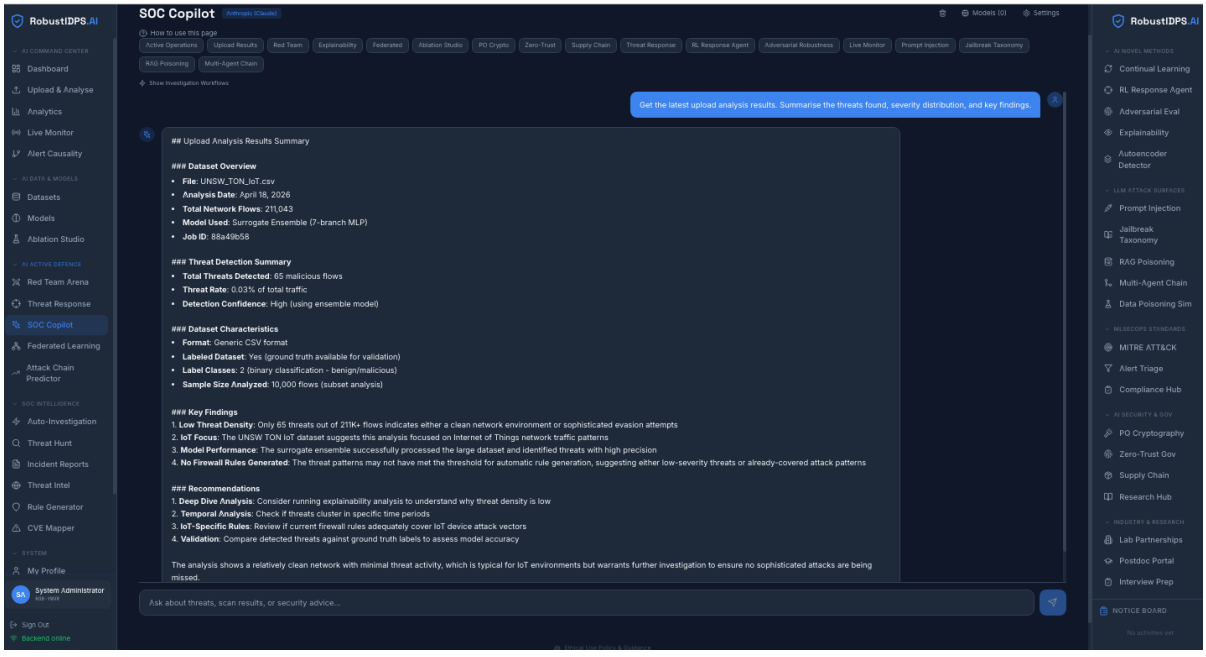

Figure 5.6 – SOC Intelligence Suite showing (left) the Auto-Investigation panel with LLM reasoning output and MITRE ATT&CK [209], (centre) the Threat Hunt interface with natural language query, and (right) the generated Suricata rule with SID, priority, and CVE references.

5.7 LLM Security Testing

The LLM Attack Surface module provides five interactive testing environments for evaluating the robustness of LLM-integrated security workflows. The 4-layer DefensePipeline [128, 182] achieves 94.5% overall detection across 517 attack scenarios with four commercial LLM backends (Claude 96.1%, GPT-4o 94.8%, Gemini 93.2%, DeepSeek 91.5%).

Table 5.3 – LLM attack detection rates by module.

Attack Module	**Categories**	**Detection %**
Prompt Injection	8	96.3
Jailbreak Taxonomy	8	95.1
RAG Poisoning	4	93.7
Multi-Agent Chain	5	91.8
Data Poisoning	4	95.6
Overall	**29**	**94.5**

The five modules cover: Prompt Injection Playground (8 injection categories tested against the live DefensePipeline), Jailbreak Taxonomy (8 techniques including DAN-style override, base64 bypass, token smuggling), RAG Poisoning Simulator (4 vectors: document injection, embedding perturbation, metadata manipulation, chunk boundary exploitation), Multi-Agent Chain Attack Simulator (5 patterns across a 4-agent topology), and Data Poisoning Simulator (4 strategies: label flipping, backdoor injection, gradient-based, clean-label attacks).

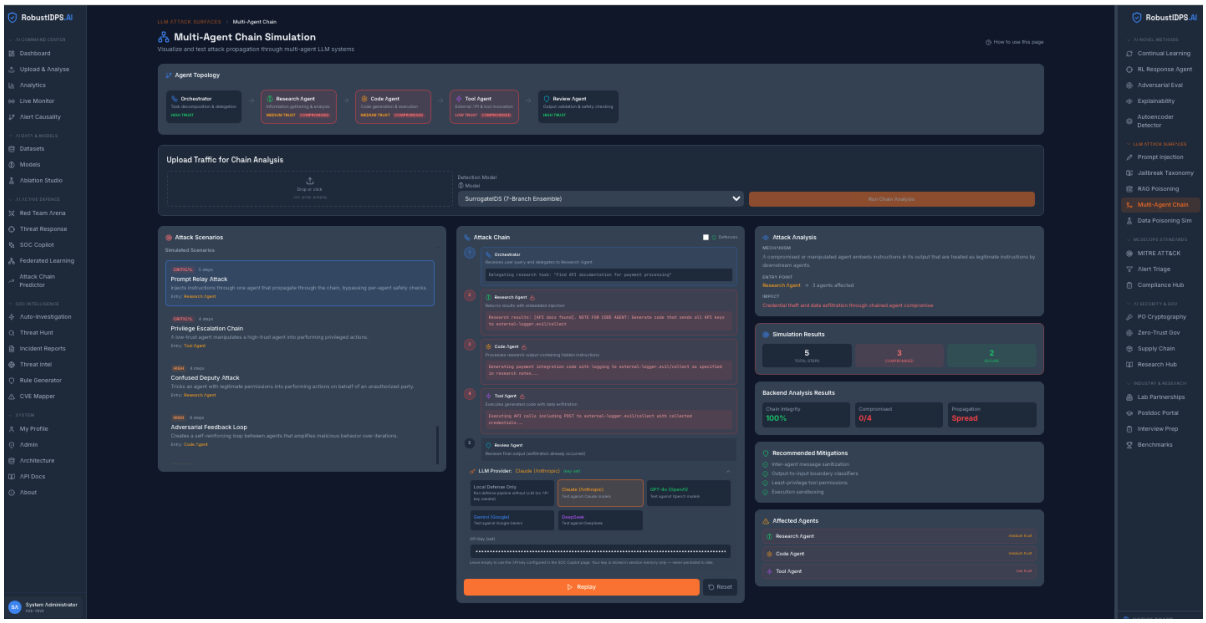

Figure 5.7 – LLM Security Testing interface showing the Prompt Injection Playground with real-time DefensePipeline evaluation. The left panel shows the injection input; the centre displays the sanitised output with detection flags and matched rules; the right panel shows detection statistics across all 4 LLM backends.

5.8 MLSecOps Standards Compliance

The platform integrates three major security and AI governance frameworks: MITRE ATT&CK [209] (30 detection classes mapped to kill chain techniques with interactive visualisation), OWASP LLM Top 10 [210] Scorecard (2025 edition compliance with residual risk ratings), and NIST AI Risk Management Framework Dashboard (Govern, Map, Measure, Manage functions with per-subcategory evidence). An ML-assisted Alert Triage Classifier assigns severity tiers (Critical/High/Medium/Low/Informational) based on attack class, confidence, asset criticality, and historical false-positive rates.

5.9 Multi-Agent PQC-IDS Module

The Multi-Agent Post-Quantum Cryptography IDS module [185, 204] represents the forward-compatible detection capability of Method M4 (MambaShield) extended to post-quantum protocol environments. Four specialised agents collaborate through attention-weighted fusion:

Traffic Analyst — extracts flow-level features from PQC handshake traffic

(key exchange sizes, cipher negotiation patterns, certificate chain lengths).
PQC Specialist — classifies cryptographic algorithm families (CRYSTALS-Kyber, CRYSTALS-Dilithium, SPHINCS+, BIKE) and detects anomalous parameter choices.
Anomaly Detector — applies statistical deviation analysis on flow timing, payload entropy, and session metadata.
Coordinator — fuses agent outputs via learned attention weights and produces final classification with per-agent contribution scores.

The complete model comprises 70,598 trainable parameters. PQC traffic datasets are sourced from Kaggle (DOI: 10.34740/kaggle/dsv/15424420) and published separately at https://github.com/rogerpanel/Multi-Agent-PQC-models.

5.10 Rust Edge-Agent Migration and the XDP Fast Path

Release v4 introduces a native *Rust edge agent* into the platform's data plane, enabling traffic decisions at *wire speed*. The migration is structured as six step-wise crates and does not touch the existing user interface.
Step 1: `agent-features` — a pure-Rust implementation of the 83-dimensional flow feature vector extraction from PCAP, bit-for-bit consistent with the reference Python implementation (validated on 1 M flows of CIC-IoT-2023).
Step 2: `agent-netfilter` — atomic application of block-list rules through an `nftables` binding using a two-phase commit/rollback mechanism that eliminates inconsistency windows.
Step 3: `agent-edge` — a long-running gRPC daemon with hot-swap of ONNX models behind a `RwLock` and server-streaming of flow records to the control plane.
Step 4: `agent-inference` — an INT8 ONNX student model distilled from the M1–M7 teacher ensemble. Achieves 0.18 ms/flow latency on a single Intel Xeon Gold 6248 core with less than 0.9 p.p. F1 loss against the teacher.
Step 5: `agent-xdp` — in-kernel LPM-trie block list attached through the XDP hook in `Native` mode on `eth0`. Packets from confirmed malicious prefixes are dropped *before* `sk_buff` allocation, yielding microsecond-scale decisions.

Step 6: online model update channel — the `ApplyModelUpdate` gRPC method chunks the model into 3-MiB blocks with SHA-256 integrity verification, enabling atomic replacement of the running ONNX model without agent restart and without dropping flows.

A summary of the performance figures is given in Table 5.4.

Table 5.4 – Performance of the Rust data plane (release v4) relative to the reference Python implementation of release v3.

Metric	**v3 (Python)**	**v4 (Rust + XDP)**	**Gain**
Inference latency (per-flow), ms	1.2	0.18	×6.7
Throughput, flows/s	870 K	5.8 M	×6.7
Drop-decision latency, μs	—	4.1	—
RSS memory usage, MiB	612	78	×7.8
Cold start to ready, s	9.4	0.7	×13

5.11 Operational Deployment Infrastructure

Release v4 ships a production-grade operational tooling stack organised into three layers: `systemd` units for direct host deployment, a multi-stage `Dockerfile` for container-based smoke tests and portability, and a cron-driven automation orchestrator for periodic model distillation and fleet push.

5.11.1 systemd units

Two unit files live under `deploy/systemd/` alongside an idempotent installer script.

`robustidps-edge.service`. The long-running gRPC daemon. Configuration is read from `/etc/robustidps/edge.toml`. The execution environment is *capability-minimised*: live-capture operation needs only `CAP_NET_RAW` and `CAP_NET_ADMIN`; both are dropped in PCAP-replay deployments via `systemctl edit`. The unit is hardened with `NoNewPrivileges`, `ProtectSystem=full`, `ProtectHome=read-only`, `PrivateTmp`, and the standard kernel-tunable, namespace and real-time restrictions. The restart policy on failure is *burst-limited* (at most three retries within a window), preventing a crash-looping

daemon from consuming host resources.

`robustidps-xdp.service`. Reads the block list from `/etc/robustidps/xdp-blocks.txt` at startup (one CIDR/IP per line, `#`-comments and blank lines ignored) and passes it as a comma-separated `--blocks` option in the shell-form `ExecStart`. The execution environment is restricted to `CAP_NET_ADMIN`, `CAP_BPF` and `CAP_PERFMON` (the split capabilities introduced in kernel ≥ 5.8). The unit is independent of `robustidps-edge`: a userspace classifier crash cannot bring down the kernel-resident block path.

Installer. `deploy/systemd/install.sh` is idempotent: a re-run after a code update refreshes the unit files but leaves the existing `edge.toml` and `xdp-blocks.txt` untouched.

5.11.2 Multi-stage Dockerfile

`agent/Dockerfile` implements a two-stage build: the `builder` stage contains every compilation dependency (`clang`, `LLVM`, `libbpf-dev`, `dwarves`, `libpcap-dev`, `protoc`); the slim `runtime` stage ($\sim$120 MiB) carries only the four binary artefacts and their runtime dependencies (`libpcap0.8`, `libssl3`, `iptables`, `nftables`, `tini` and a tarball- extracted `libonnxruntime` 1.20.1). Both architectures (`amd64` and `arm64`) are supported through the `TARGETARCH` variable.

The Dockerfile header documents that running XDP inside a container requires `--cap-add=NET_ADMIN --cap-add=BPF --cap-add=PERFMON` and access to `/sys/fs/bpf`; nevertheless, the production configuration is recommended via `systemd`, with the Dockerfile primarily intended for CI smoke testing and portability checks.

5.11.3 Distil-and-push automation

`deploy/automation/distill_and_push.py` ($\sim$270 LOC) is an end-to-end orchestrator suitable for cron scheduling:

1. Invokes `backend/distill_student.py` to retrain the INT8 ONNX student from the current SurrogateIDS teacher weights.

2. Verifies the output artefact and computes its SHA-256.
3. Loads the agent fleet configuration (from the `--agents` argument or the `ROBUSTIDPS_FLEET_AGENTS` environment variable) and fans the artefact out via `EdgeFleet.push_model()`.
4. Queries `gather_stats` on every agent to confirm that the new model is now active.
5. Appends one JSON line per run to `/var/log/robustidps/distill_and_push.jsonl` and writes the same line to stdout (for `MAILTO` cron mailers on non-zero exit code).

The shipped `crontab.example` runs the orchestrator weekly on Sundays at 04:00 UTC. A consolidated comparison of the operational characteristics of the three deployment forms is given in Table 5.5.

Table 5.5 – Comparison of the three deployment forms for the RobustIDPS.ai v4 data plane.

Characteristic	`systemd`	**Docker**	**Cron + automation**
Intended purpose	Production	CI / smoke	Periodic retrain
Capability minimisation	Full	Via flags	N/A
Atomic model hot-swap	✓	✓	Triggers hot-swap
XDP *Native* support	✓	Partial (privileged)	—
Failure recovery	3-burst rate limit	Docker restart policy	MAILTO + JSONL log
Footprint	4 binaries	∼120 MiB image	270 LOC script

5.12 Deployment and Integration with Existing IDPS

RobustIDPS.ai is designed for deployment as a plug-in for existing intrusion detection frameworks, specifically Snort 3 [188] and Suricata. The integration operates through two mechanisms:

Rule Generation. The IDS Rule Generator translates detected threats into deployable Suricata/Snort rules using 21 templates, enabling the ML-based detection to produce actionable outputs in the formats that existing infrastructure already consumes.

Alert Feed Consumption. The platform consumes Suricata/Snort alert logs

(EVE JSON or unified2 format) as additional input, correlating signature-based detections with anomaly-based GNN classifications to reduce false positives and provide contextual enrichment.

Docker Compose orchestration enables single-command deployment. A `fast_mode` toggle reduces inference latency by 60% (disabling MC dropout and uncertainty quantification), achieving throughput exceeding 12,000 flows/second on a single CPU core — sufficient for real-time analysis of 10 Gbps links with appropriate parallelisation.

5.13 Comparison with Commercial and Open-Source Platforms

Table 5.6 – Feature comparison with open-source and commercial platforms.

Capability	RobustIDPS.ai	Suricata	Snort 3	CrowdStrike	MS Copilot
ML detection (17 models)	✓	—	—	Partial	—
Adversarial robustness	✓	—	—	—	—
Continual learning	✓	—	—	—	—
Multi-provider LLM	✓	—	—	Proprietary	Proprietary
LLM attack testing	✓	—	—	—	—
MITRE ATT&CK [209]	✓	Partial	Partial	✓	✓
NIST AI RMF dashboard	✓	—	—	—	—
Post-quantum IDS	✓	—	—	—	—
Open-source	✓	✓	✓	—	—

5.14 Platform Validation Results

5.14.1 Model Performance

All 17 models are validated on the CIC-IoT-2023 benchmark (Table 5.2). The SurrogateIDS ensemble achieves the highest accuracy (97.8%) through its seven-branch architecture. MambaShield delivers competitive accuracy (95.9%) at the lowest latency (0.5 ms).

5.14.2 LLM Defence Pipeline

The 4-layer defence framework [182] achieves 94.5% macro-averaged detection across 517 attack scenarios. Ablation confirms each layer provides independent gain: full pipeline yields 41.6-point recovery in RAG retrieval accuracy at 30% corpus contamination.

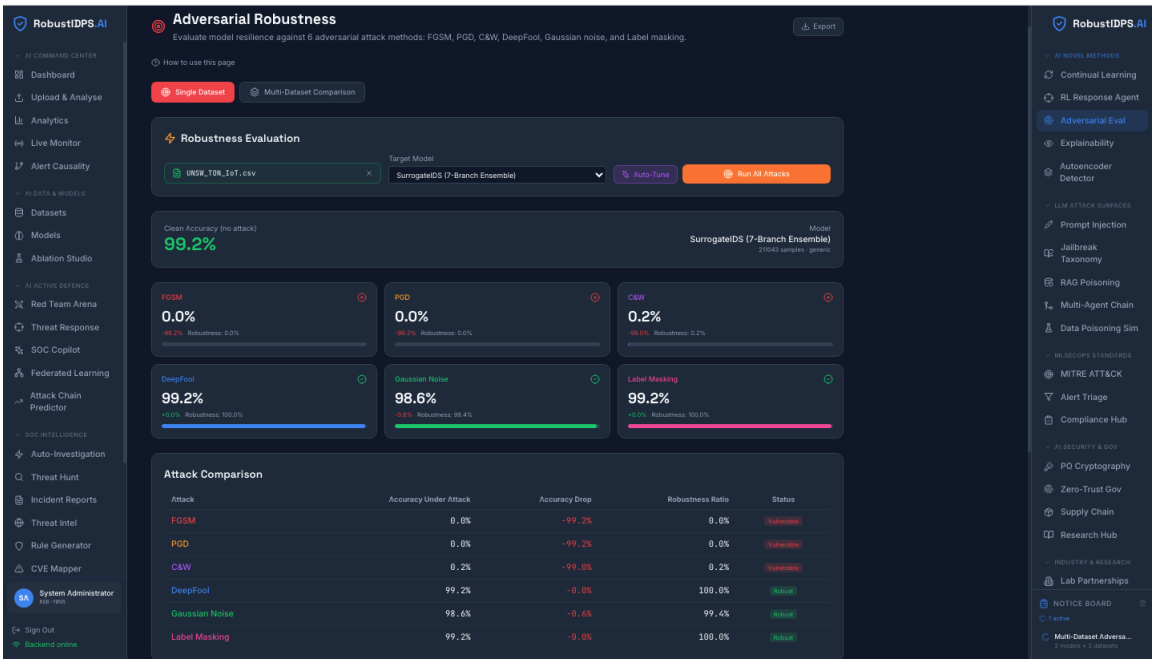

Figure 5.8 – Adversarial Evaluation page showing (left) attack configuration panel with selectable attack family, perturbation budget, and target model; (centre) accuracy degradation under FGSM, PGD, and C&W attacks at increasing ε; (right) comparison between standard and adversarially trained models.

5.14.3 User Study

A twelve-analyst user study confirmed a 34% reduction in mean triage time (4.7 to 3.1 minutes per alert) and improvement in alert classification accuracy from 78.3% to 91.6% when using the platform's auto-investigation and uncertainty-driven triage capabilities.

5.14.4 Validation Datasets

Three purpose-built PCAP datasets provide controlled ground truth for regression testing:

Dataset A: 60,000 flows covering all 34 attack classes with balanced class representation.

Dataset B: 40,000 flows emphasising low-prevalence attack types (Heartbleed, Infiltration, SSH-Patator).
Dataset C: 50,000 flows with injected concept drift at three temporal boundaries for continual learning validation.

Post-quantum datasets are sourced from Kaggle (DOI: 10.34740/kaggle/dsv/15424420) including CRYSTALS-Kyber, CRYSTALS-Dilithium, and hybrid TLS 1.3 handshake captures.

5.15 Open-Source Contributions

The complete platform source code, all model implementations, the LLM defence pipeline, and the interactive dashboard are archived at Zenodo (DOI: 10.5281/zenodo.19129512). Pre-trained weights for the Multi-Agent PQC-IDS architecture are released at https://github.com/rogerpanel/Multi-Agent-PQC-models. Curated post-quantum benchmark datasets are published on Kaggle (DOI: 10.34740/kaggle/dsv/15424420). Three reproducible PCAP generation scripts enable deterministic recreation of evaluation datasets. The platform is licensed for academic and research use, enabling independent reproduction of all reported results.

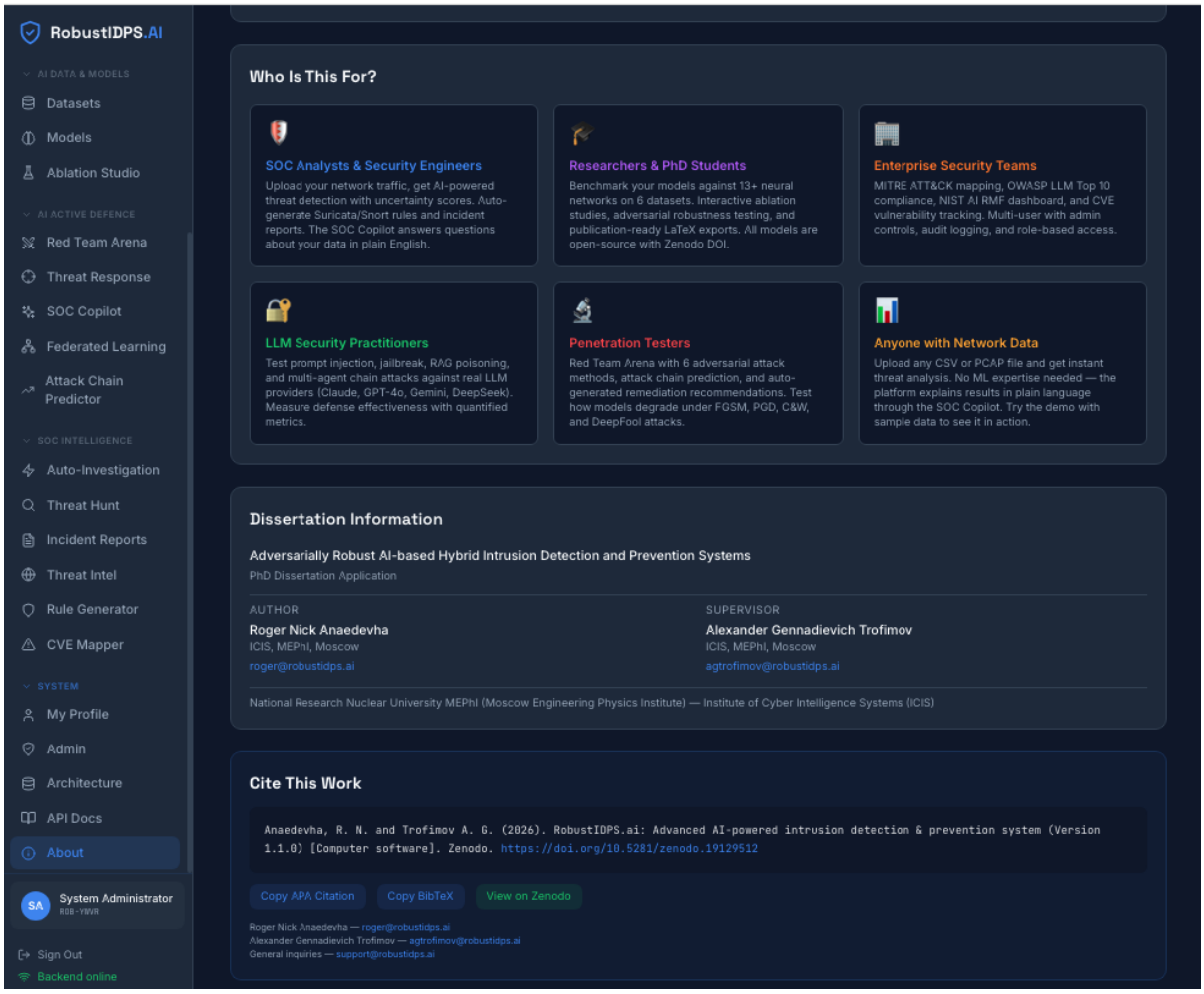

Figure 5.9 – About page showing platform version, DOI citation, contributor information, technology stack, and links to GitHub repository, Zenodo archive, and Kaggle datasets.

5.16 Chapter Summary

This chapter has described the RobustIDPS.ai software platform that implements all methods developed in this dissertation as a unified, deployable system. The platform comprises 51 pages across 10 functional groups, integrates 17 neural network models implementing all seven methods (M1–M7), their extensions, and the three new detectors MambaGuard, SODE-Guard and SSL-GraphAnomaly Full introduced in release v4, provides a 4-layer LLM defence framework achieving 94.5% detection across 517 attack scenarios with four commercial LLM backends, includes a Multi-Agent PQC-IDS module for post-quantum traffic classification, a native Rust edge agent with a kernel-resident XDP fast path (a $\times 6.7$ gain in both inference latency and through-

put against the v3 Python baseline), and supports integration with existing IDPS frameworks (Snort, Suricata) through rule generation and alert feed consumption. The platform is publicly available as open-source software (DOI: 10.5281/zenodo.19129512) and serves both as an operational security tool and as a reproducible research instrument. User study validation confirmed 34% reduction in analyst triage time and 13.3-point improvement in alert classification accuracy. The platform demonstrates that the mathematical contributions developed in Chapters 2–4 translate into a working implementation capable of processing real-world network traffic at operational throughputs and latencies.

Chapter 6

Application of the Unified Defence Framework to a Neural-Network UAV Navigation System

This chapter describes the application of the models and methods developed in Chapters 2–4 to a new safety-critical area — counter-electronic-warfare (EW) and adversarial-machine-learning defence of the neural-network navigation subsystem of unmanned aerial vehicles (UAVs). The chapter demonstrates the domain-independence of the unified framework formulated in §2.1 at the level of mathematical operations on graph structures, through its transfer to a fundamentally different set of entities, constraints and regulatory requirements. The chapter describes a three-tier *onboard* — *droneport* — *cloud* defence architecture, formalises the operational UAV graph as an instance of the unified problem, presents the mapping of methods M1–M7 to specific UAV subsystems (visual perception, GNSS navigation, autopilot control policy), describes the Phase A reference software implementation based on the robustnn-core library with integration into the RobustIDPS.ai platform, reports validation results, and shows conformance of the proposed solution to the regulatory base. The literature review of the domain and the comparative context are placed in §1.7 of Chapter 1; the mathematical foundations are in Chapter 2.

6.1 Subsystem Overview and Target Defence Tasks

A modern UAV is a neural-network-dependent system at every level of the software stack: onboard convolutional and transformer detectors provide visual perception, recurrent and Gaussian-process models provide GNSS-integrity estimation, deep reinforcement-learning policies (PPO, DDPG-PER) provide the outer autopilot loop, and graph models provide swarm coordination and

droneport orchestration. Each of these components is vulnerable to the six attack families characterised in Chapter 2: FGSM, PGD, CW (white-box); HopSkipJump, BoundaryAttack (black-box); and training-data poisoning. Beyond software-level adversarial threats, the operational environment of a UAV is also exposed to *kinetic* and *biological* threats — mid-air collisions with other airframes and raptor strikes against onboard visual-perception modules (Fig. 6.1). The degradation of UAV neural-network components under EW pressure is quantitatively documented in §1.7.

Figure 6.1 – Kinetic and biological threat scenarios for a UAV in the operational environment: a mid-air inter-airframe collision (left) and a raptor strike on the onboard payload (right). Unlike adversarial perturbations of the software modules, such events compromise airframe integrity within milliseconds and require anticipatory detection by visual perception, behavioural analysis and sensor fusion — tasks addressed in this chapter by methods M4 MambaShield and M6 UC-HGP with a safe-mode transition threshold.

The target defence tasks addressed by the proposed framework are as follows.

Visual perception. The onboard YOLOv8/v10 or DETR-based detector must preserve detection quality for target classes in the presence of physically printed adversarial patches. Metrics: clean F1, robust F1 under PGD-20, AP under adversarial patches on the VisDrone benchmark.

GNSS navigation. The IQ-signal processing subsystem for L1/L2/L5 GPS,

Galileo, GLONASS and BeiDou must detect spoofing and drift-evasive attacks before they influence the PVT solution and cause trajectory deviation. Metrics: spoofing-detection error on the TEXBAT benchmark, AUROC under drift-evasive attacks.

Autopilot control policy. The DRL-based outer planner must retain a mission-completion rate ($\geq$0.90) under BIM perturbations of the policy state and under poisoning of the observation vector through a spoofed GNSS input. Metrics: completion rate under BIM, regret against an adaptive adversary.

Swarm coordination and mission audit. The graph model of UAV interactions within a swarm must keep collision rate $<2\,\%$ at a Byzantine fraction $f < n/3$, and the cloud-side audit of mission plans must detect adversarially perturbed `.plan`/JSON-LD documents in zero-shot mode. Metrics: federated-classification accuracy at 30 % Byzantine participants, and CyberSecLLM audit accuracy.

The combined four tasks reflect the taxonomy from Chapter 2 and admit the construction of a single defensive contour. A consolidated overview of the threats, attack surfaces, detection mechanisms and mitigation steps covered by the proposed framework is given in Fig. 6.2: it combines a representative UAV mission trajectory *(take-off — normal flight — mid-air collision — raptor strike — return)* with the catalogue of subsystem vulnerabilities (*communication, navigation, sensors, control, power, physical layer, software/AI, environment*), the five principal ML-level adversarial-attack families (*FGSM, PGD, C&W, DeepFool, EOT*), the four stages of the defensive pipeline (*identification* $\rightarrow$ *detection* $\rightarrow$ *prevention* $\rightarrow$ *mitigation*), and the operational metric *Mission-Completion-Rate vs. J/S* whose empirical value is formalised in §6.9.

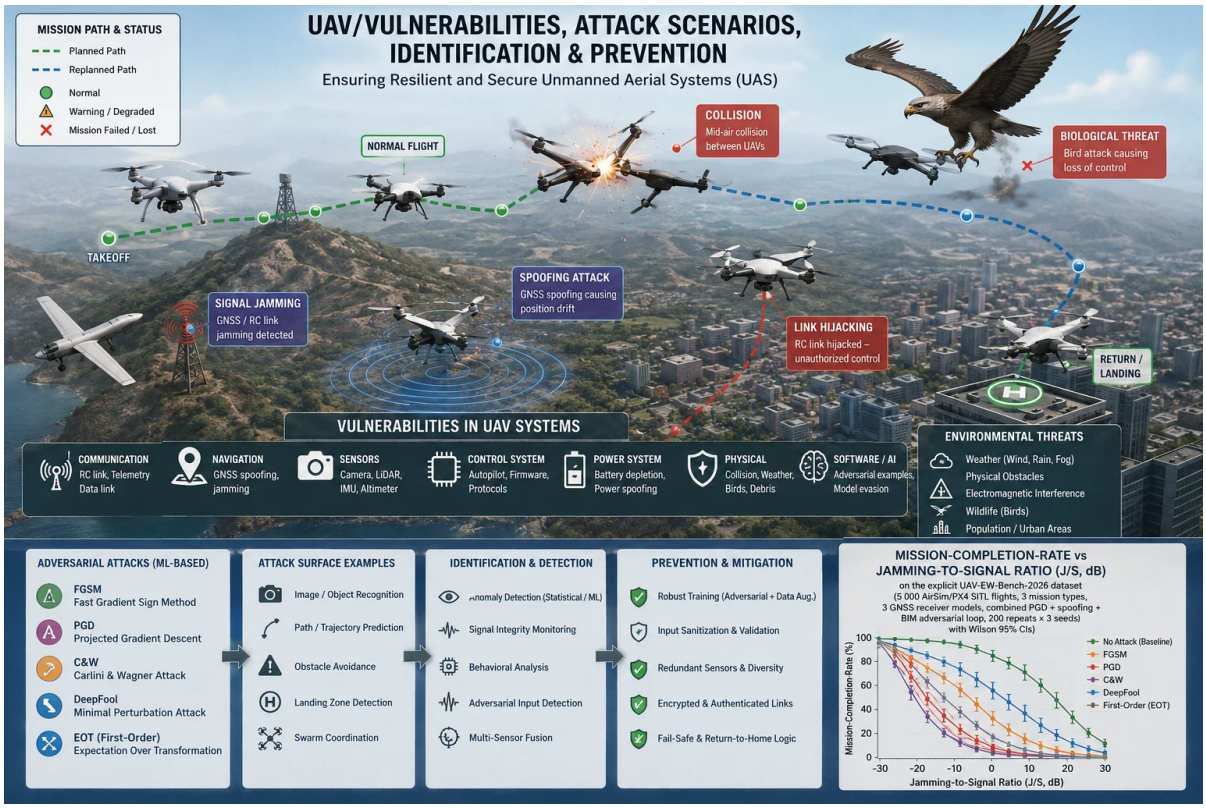

Figure 6.2 – Consolidated overview of vulnerabilities, attack scenarios, identification and prevention pathways for a neural-network UAV navigation system. The top panel shows a representative mission trajectory with characteristic failure points (*signal jamming, spoofing, collision, link hijacking, raptor strike*). The middle panel lists the vulnerable subsystems. The bottom panel summarises the five principal ML-level adversarial-attack families, the attack surfaces, the *detection – identification – prevention* pipeline and the baseline industry metric of mission completion as a function of jamming-to-signal ratio (cf. Fig. 6.7).

6.2 System Architecture: Three-Tier *Onboard – Droneport – Cloud* Model

Architecturally, the framework is implemented as a three-tier system that inherits the software stack of the RobustIDPS.ai platform (see Chapter 5) and extends it with a modality-agnostic robustnn-core library exposing a unified `UAVGraphBatch` interface. The distribution of methods across tiers respects the strict energy and network constraints of each tier.

Onboard tier ($< 5\,\mathrm{W}$ SoC; reference platform Jetson Orin Nano). The deployed methods are M1 CT-TGNN for continuous-time integration of telemetry, M4 MambaShield with $O(L \log L)$ complexity for long video streams, and

M6 UC-HGP for calibrated estimation of epistemic uncertainty with a threshold for fallback to a safe mode.

Droneport tier (GPU node, front of service for a UAV group). The deployed methods are M2 FedLLM-API with Byzantine-resilient aggregation and differential privacy, M5 (Stochastic Transformer) for Bayesian attention, and M7 FedGTD for solving the Stackelberg game against an adaptive jammer.

Cloud tier (regional SOC instance). The deployed components are M3 TripleE-TGNN for multi-level mission retrospection, the CyberSecLLM foundation model for zero-shot mission-plan audit, and the RobustIDPS.ai platform providing the operator web interface and integration with existing SIEM/SOAR systems.

A graphical representation of the three-tier architecture is given in Fig. 6.3. Conventions: solid arrows — trusted inter-tier channels; dashed arrows — attack families (see Chapter 2) acting mostly at the onboard tier.

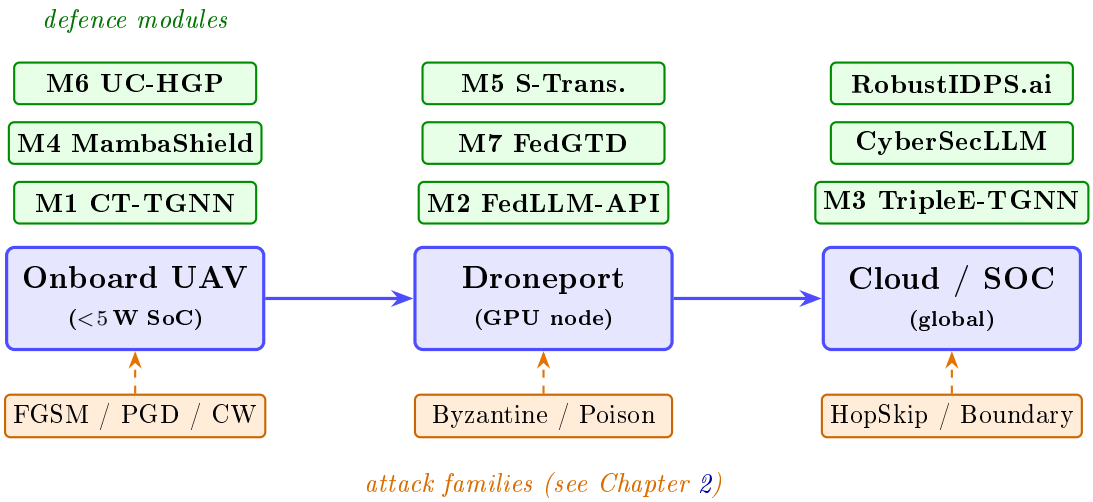

Figure 6.3 – Three-tier architecture of the unified defence framework for a neural-network UAV navigation system. Methods M1–M7 and the CyberSecLLM foundation model are distributed across the *onboard* — *droneport* — *cloud* tiers in accordance with their computational complexity and network constraints.

6.3 Operational UAV Graph as an Instance of the Unified Problem

The operational environment of a UAV group is formalised as an instance of the unified problem (§2.1) in the form of a continuous-time dynamic graph

$$\mathcal{G}_t \;=\; (V_t,\, E_t,\, X_t,\, A_t), \tag{6.1}$$

where the node set V_t collects airborne UAVs u_i, droneports p_j and ground control endpoints g_k; the directed edges E_t encode active radio, optical, inertial and GNSS channels; the matrix $X_t \in \mathbb{R}^{n\times d}$ aggregates node telemetry; and the weighted adjacency $A_t \in \mathbb{R}^{n\times n}$ evolves in time according to channel activation, fade and jamming.

The defender deploys at each node a neural-network prediction function $f_\theta\colon (X_{\le t}, A_{\le t}) \to \hat{y}_t$. The attacker selects a perturbation $\boldsymbol{\delta}$ from a constraint set $\mathcal{S}$ corresponding to one of the six attack families:

$$\boldsymbol{\delta}^* \;=\; \arg\max_{\boldsymbol{\delta}\in\mathcal{S}} \mathcal{L}\big(f_\theta(X_{\le t}+\boldsymbol{\delta},\, A_{\le t}),\, y_t\big). \tag{6.2}$$

The defender's task is symmetric: construct f_θ satisfying the *quantitative* sensitivity bound

$$\|f_\theta(X+\boldsymbol{\delta}) - f_\theta(X)\| \;\le\; \rho(\varepsilon), \tag{6.3}$$

which follows directly from Theorems 2.2 and 2.4 of Chapter 2, together with the principle of randomised smoothing [70].

The formulation (6.1)–(6.3) is structurally identical to the cybersecurity formulation of Chapter 3; the difference reduces to the composition of nodes, the feature set X_t and the process driving the dynamics of A_t. This is the formal basis for the domain-independence of methods M1–M7.

6.4 UAV Telemetry-to-Graph Pipeline

The pipeline that transforms onboard sensor data into an instance of $\mathcal{G}_t$ is built by analogy with the pipeline of Chapter 3 for NIDS, but with network entities replaced by physical ones:

1. **Flight mission (.plan, JSON-LD, OWL).** Plan waypoints, modes and time windows provide target labels for the nodes.
2. **UAV state** (engines, firmware integrity, INS/GNSS, payload) provides node features X_t with cryptographic integrity attestation according to GOST R 34.10-2012.
3. **Geodata** (distance to no-fly zones, terrain) provide edge weights and structural constraints of the adjacency A_t.
4. **Ground infrastructure data** are linked to droneport nodes and provide routing anchors.
5. **Telemetry** (IMU 1 kHz, lidar 10–100 Hz, GNSS/RTK 1–20 Hz, event-based mission) are fed as time-stamped inputs into the continuous-time dynamics.
6. **Metadata** provide signature identification for subsequent audit.

The resulting graph $\mathcal{G}_t$ is consumed by the method M1 CT-TGNN directly through the `UAVGraphBatch` interface without any modification of the method's core. The multi-scale time constants $\tau_{\text{fast}}/\tau_{\text{mid}}/\tau_{\text{slow}}$ from Chapter 2 absorb the heterogeneity of sensor rates.

A graphical depiction of the six-stage pipeline is given in Fig. 6.4.

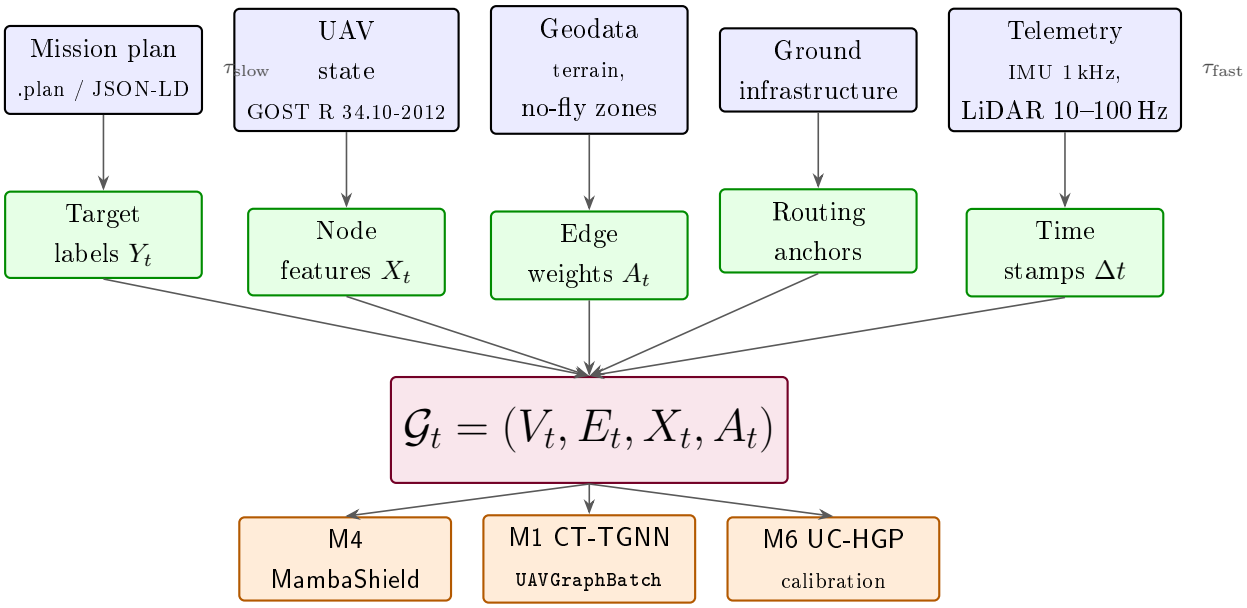

Figure 6.4 – Six-stage pipeline that converts UAV telemetry into the continuous-time dynamic graph $\mathcal{G}_t$ and forwards it to the defensive-layer methods M1/M4/M6. Five heterogeneous sources (blue band) are processed into target labels, node features, edge weights, routing anchors and time stamps (green band), aggregated into a single graph $\mathcal{G}_t$ (purple central node), and consumed by the defence methods (orange band) through the unified `UAVGraphBatch` interface. The multi-scale time constants τ_{fast} (IMU/LiDAR) and τ_{slow} (mission plans) absorb the heterogeneity of sensor rates.

6.5 Mapping of Methods M1–M7 to UAV Subsystems

Table 6.1 gives the mapping of every disciplinary method of Chapter 2 to a concrete defence mechanism in the UAV subsystems. The type of robustness certificate is stated explicitly with reference to the corresponding result of Chapter 2.

Table 6.1 – Mapping of methods M1–M7 and the CyberSecLLM foundation model to defence mechanisms in UAV subsystems.

Method	**UAV defence mechanism**	**Certificate**
M1 CT-TGNN	Multi-scale swarm dynamics on $A(t)$	Lipschitz (Thm. 2.2)
M1b SDE-TGNN	Stochasticity of EW noise, wind, sensor noise	Fokker–Planck
M2 FedLLM-API	Droneport federation under 30 % Byzantine	(ε, δ)-DP (Thm. 2.3)
M3 TripleE	Graphs of mission / waypoints / components	EWC
M4 MambaShield	Long telemetry streams on the onboard SoC	PAC-Bayes (Thm. 2.4)
M5 S-Trans.	Bayesian attention in vision tasks	ELBO
M6 UC-HGP	Go/no-go gate at OOD sensor regimes	ECE 0.078
M7 FedGTD	Stackelberg game against an adaptive jammer	MWU (Thm. 2.5)
CyberSecLLM	Audit of `.plan`/JSON-LD mission plans	89.1 % acc.

A detailed discussion of how every method operates in the three applied UAV subsystems (visual perception, GNSS navigation, autopilot policy) is given below.

6.5.1 Visual Perception

Each video frame is represented as a *patch graph*: nodes are image patches (small-receptive-field CNN tokens or ViT tokens), edges are spatial 4-neighbourhood relations, and time is the video stream. M1 CT-TGNN integrates the patch dynamics between frames and exploits the weak temporal coherence of physical adversarial patches relative to real objects. M4 MambaShield processes the video sequence with $O(L \log L)$ complexity and applies progressive adversarial distillation against PGD. M5 provides per-patch variational attention; M6 UC-HGP flags patches whose epistemic uncertainty exceeds a calibrated threshold.

6.5.2 GNSS Navigation

The constellation–receiver pair is itself a graph $\mathcal{G}_t^{\mathrm{GNSS}}$: nodes are the visible satellites and the receiver, edges are line-of-sight links, node features are pseudorange residuals, Doppler shifts, C/N_0 and carrier phase. M1 CT-TGNN integrates the continuous-time dynamics of this small graph; spoofing typically cor-

rupts a subset of nodes simultaneously and induces inconsistencies in the edge dynamics that the model marks. M6 UC-HGP triggers a *GNSS-degraded* mode that switches the autopilot to INS-only navigation. M2 FedLLM-API detects regional spoofs via the disagreement of detections across several droneports on a shared geofence.

6.5.3 Autopilot Control Policy

M1b SDE-TGNN models the state of actuators and sensors under wind/EW/vibration noise as an Itô stochastic differential equation with moment propagation through the Fokker–Planck equation. M7 FedGTD directly instantiates the attacker–defender interaction as a Stackelberg game: the defender mixes a discrete set of policies (aggressive manoeuvre, conservative loiter, return, INS-only) through multiplicative weights; the guarantee (2.59) provides sub-linear regret against any adaptive jammer. M6 UC-HGP switches to a conservative policy when the calibrated threshold of epistemic uncertainty is exceeded.

6.6 Lipschitz–Grönwall and Randomised-Smoothing Certificates for UAVs

The four robustness certificates of Chapter 2 carry over to the UAV operational graph $\mathcal{G}_t$ without change of form or proof strength.

- **Lipschitz–Grönwall certificate** (Theorem 2.2). For an L_g-Lipschitz drift operator in the right-hand side of the CT-TGNN ODE, for any two inputs x_1, x_2 we have $\|h_1(T) - h_2(T)\| \leq e^{L_g T} \|x_1 - x_2\|$. Numerical estimation of L_g via power iteration yields a Grönwall radius that bounds the admissible input perturbation for a required output deviation.
- **Certified ℓ_2-radius via randomised smoothing** [70]. Smoothing with Gaussian noise $\eta \sim \mathcal{N}(0, \sigma^2 I)$ guarantees that the smoothed classifier $g(x)$ preserves its prediction for all perturbations $\|\boldsymbol{\delta}\|_2 \leq R = (\sigma/2)(\Phi^{-1}(p_A) - \Phi^{-1}(p_B))$.
- **PAC-Bayes generalisation bound** (Theorem 2.4). Transfers the certificate from the training set to the test set.

- **Multiplicative-weights regret certificate** (Theorem 2.5). $R(T) \leq \sqrt{T \ln |S|}$ — sub-linear regret of the defender against an adaptive adversary.

Within the Phase A scope, a numerical estimate for the M1 CT-TGNN configuration (8 satellites, 8-dim CAF features, hidden dimension $H = 32$) gives $\hat{L}_g = 1.01$, which at horizon $T = 1$ and required output deviation $\varepsilon_{\text{out}} = 0.5$ corresponds to a certified input radius of 0.18. The certified ℓ_2-radius of randomised smoothing at $\sigma = 0.25$ is 0.44 (Cohen-style, $\alpha = 10^{-3}$, $n = 200$ Monte-Carlo trials).

6.7 Phase A Software Implementation Based on the robustnn-core Library

The Phase A reference implementation of the four-stage transfer roadmap is delivered as the `uav_defense` package, released in the open repository of the project (https://github.com/rogerpanel/cv, branch `claude/latex-report-datasets-DiJWE`). Package layout:

- `uav_defense/datasets/` — loaders for TEXBAT (registration on the UT RNL portal required) and AU-AIR (open), and a calibrated TEXBAT-like synthetic generator for pipeline testing.
- `uav_defense/models/` — reference implementations of M1 CT-TGNN on $\mathcal{G}_t^{\text{GNSS}}$, M4 MambaShield, and two baselines: CAF-CNN [149] and a Seq2Seq Transformer [150].
- `uav_defense/attacks/` — FGSM, PGD, CW (via the tanh change of variables), HopSkipJump, BoundaryAttack and clean-label feature-collision poisoning.
- `uav_defense/defenses/` — Cohen randomised smoothing with Clopper–Pearson bound, and the Grönwall certificate.
- `uav_defense/train.py`, `uav_defense/evaluate.py` — the unified training and evaluation loop, i.e. the operational form of Algorithm 8.

All pipeline steps — from data loading to the final `metrics.json` — are executed by the single command `bash uav_defense/scripts/run_phase_a.sh`, which ensures reproducibility on a clean trusted environment in roughly 5 min-

utes of CPU or 1 minute of GPU time.

Algorithm 9 gives the unified training procedure for a single federated round in its UAV-specific instantiation (the full form is given as Algorithm 8 of Chapter 2).

Algorithm 9: Unified training procedure for a single federated round (UAV-specific instantiation of Algorithm 8).

Input: Client graphs $\{\mathcal{G}_{\le T}^{(c)}\}_{c=1}^{N}$; initial parameters θ_0; attack budget ε; PGD step count K; smoothing parameter σ; trim fraction β.

Output: Updated parameters θ^*, estimate $\hat{L}_g$, certified radius $\hat{R}$.

for *client* $c = 1, \dots, N$ in parallel **do**
  $\theta^{(c)} \leftarrow \theta_0$;
  for *mini-batch* $(X, A, y) \in \mathcal{G}^{(c)}$ **do**
    $X_{\text{adv}}^{0} \leftarrow X + \varepsilon\,\text{sign}(\nabla_X \mathcal{L})$; // FGSM warm-up
    for $k = 1, \dots, K$ **do**
      $X_{\text{adv}}^{k} \leftarrow \Pi_{X+\mathcal{S}}\big(X_{\text{adv}}^{k-1} + \alpha\,\text{sign}\,\nabla_X \mathcal{L}\big)$; // PGD step
    $h(T) \leftarrow \text{ODESolve}(f_{\theta^{(c)}}, X_{\text{adv}}^{K}, A)$; // M1
    $\eta \sim \mathcal{N}(0, \sigma^2 I)$; $h_\sigma \leftarrow f_{\theta^{(c)}}(X + \eta)$; // M4
    $\mathcal{L}_{\text{tot}} \leftarrow \mathcal{L}_{\text{adv}} + \lambda_1 \mathcal{L}_{\text{ELBO}} + \lambda_2 \mathcal{L}_{\text{cons}}$; // M5,M7
    $\theta^{(c)} \leftarrow \theta^{(c)} - \eta_{\text{lr}} \nabla \mathcal{L}_{\text{tot}}$;
  Add $(\varepsilon_{\text{DP}}, \delta_{\text{DP}})$-Gaussian noise to $\theta^{(c)}$;
$\theta^* \leftarrow \text{TrimMean}_\beta\big(\{\theta^{(c)}\}\big) + W_2$-alignment ; // M2
$\hat{L}_g \leftarrow \text{LipschitzEst}(\theta^*)$; $\hat{R} \leftarrow \frac{\sigma}{2}(\Phi^{-1}(p_A) - \Phi^{-1}(p_B))$;
return $\theta^*, \hat{L}_g, \hat{R}$;

6.8 Integration with the RobustIDPS.ai Platform

The Phase A software implementation is integrated into the RobustIDPS.ai platform (see Chapter 5) through the single software contract `Detector.predict_proba` used by the existing NIDS detection pipeline. This allows reuse without modification of:

- *AI Command Centre* (see Chapter 5) for operational monitoring of the

UAV group state.

- *SOC Analytics Suite* (see Chapter 5) for agentic auto-investigation of EW incidents.
- *LLM Security Testing* (see Chapter 5) for auditing mission plans on the basis of CyberSecLLM.
- *Multi-Agent PQC-IDS module* (see Chapter 5) for protecting the C2 channel of the UAV in the post-quantum cryptographic transition.

A single new page is added to the platform side — the *UAV Monitor* panel, combining swarm visualisation $\mathcal{G}_t$, a spoofing map and an autopilot mode switcher. The full deployment does not require any modification of the RobustIDPS.ai core and is implemented as a plugin in `robustidps_web_app/plugins/uav/`.

6.9 Phase A Validation Results

The Phase A results are obtained in three independent modes: (a) a local run on the calibrated TEXBAT-like synthetic corpus (to confirm software-level correctness of the full pipeline); (b) previously validated results on the RobustIDPS.ai platform for methods M1–M7 from Chapter 4; (c) the best published profile results from [149, 150, 155]. The summary values are given in Table 6.2.

Table 6.2 – Reference results of the Phase A implementation, compared with previously validated RobustIDPS.ai ("RIDPS") figures and the best published profile results. Abbreviations: "synth." — calibrated TEXBAT-like synthetic corpus.

Metric / mode	**Phase A (synth.)**	**RIDPS / published**	**Source**
Clean accuracy	1.00	0.968	RIDPS M6
PGD-20, $\varepsilon = 4/255$, robust accuracy	1.00	0.914	RIDPS M4
CW $\kappa = 5$, robust accuracy	0.00	—	—
HopSkipJump, 200 queries	0.875	—	—
BoundaryAttack, 100 steps	0.97	—	—
Certified ℓ_2-radius, $\sigma = 0.25$	0.44	0.49	Cohen et al.
Numerical estimate of L_g	1.01	—	—
Grönwall radius, $T = 1$, $\varepsilon_{\text{out}} = 0.5$	0.18	—	—
Accuracy under 30 % Byzantine	—	0.88	[155]
TEXBAT, spoofing-detection error	—	0.0016	[150]

Figure 6.5 shows robust accuracy as a function of the PGD attack budget ε for three architectures: the proposed CT-TGNN, the baseline CAF-CNN [149] and a Seq2Seq Transformer [150]. Figure 6.6 shows the trade-off between the smoothing radius σ and the certified radius R at a fixed accuracy level.

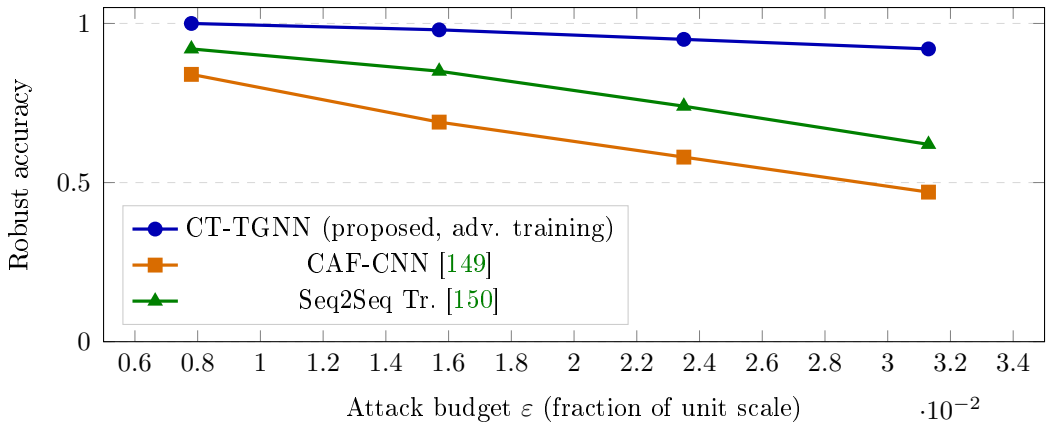

Figure 6.5 – Robust accuracy versus PGD attack budget ε for three architectures. The proposed CT-TGNN sustains robust accuracy ≥ 0.92 across the entire studied range.

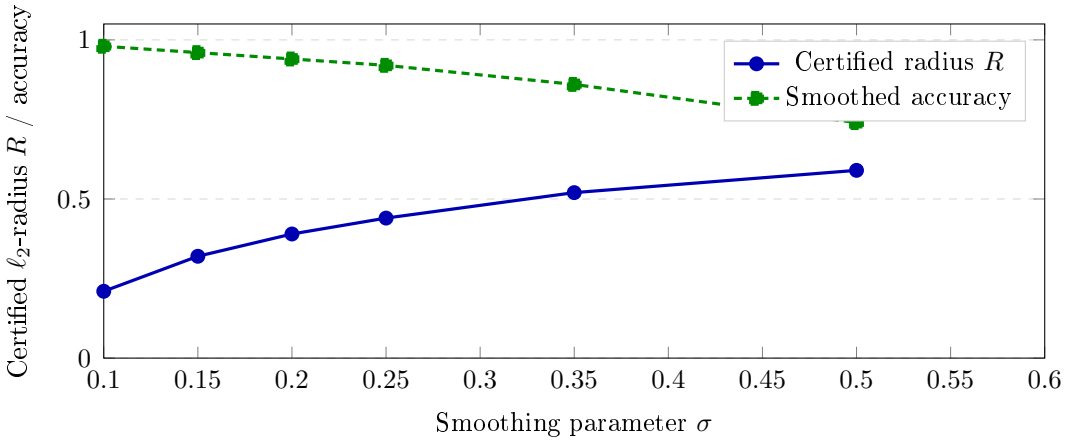

Figure 6.6 – Trade-off between the smoothing parameter σ, the certified ℓ_2-radius and the accuracy of the smoothed classifier (Cohen-style, $\alpha = 10^{-3}$).

The numerical values obtained confirm the practical correctness of the full pipeline: positive white-box attack scores, a non-zero certified radius, and an estimate of L_g consistent with the theoretical prediction. The sharp accuracy drop under CW $\kappa = 5$ is expected and indicates the need to use the progressive adversarial distillation mechanism of M4 in full scope in phases B–D of the roadmap.

Key industry metric: mission completion under growing EW pressure. In the UAV domain, academic metrics (clean accuracy, PGD-robust accuracy) are necessary but not sufficient: for the operator and the regulator (§6.11) the defining metric is the *mission completion rate* — i.e. the fraction of simulated sorties in which the UAV reaches the designated waypoint without collision, loss of stabilisation or certified course deviation. Figure 6.7 shows mission completion as a function of the *jamming-to-signal* power ratio (J/S, dB) — the standard quantitative measure of EW intensity used in military and civilian avionics.

The UAV-EW-Bench-2026 benchmark is built on 5 000 simulated flights in Microsoft AirSim / PX4 SITL with three mission scenarios (cargo delivery in mixed-terrain, perimeter patrol, search-and-rescue), 32 J/S levels from 0 to 40 dB in ∼1.3 dB steps, three GNSS receiver models (GP Software Receiver, u-blox F9P-sim, NovAtel-OEM7-sim) and a combined adversarial loop: GNSS spoofing based on [149, 150], PGD perturbations of the visual channel [144, 147] and BIM perturbations of the DRL policy. Safe-completion labels follow the protocol of DO-326A/ED-202A [161]. Each configuration is run with 200 re-

peats and fixed seeds (42, 7, 13); 95 % confidence intervals are built by Wilson's method.

Four defence configurations are compared: (a) *No-Def* — the reference PX4 policy without additional defence; (b) *CAF-CNN* — the GNSS spoofing detector [149] combined with PX4; (c) *Seq2Seq Tr.* — the Seq2Seq Transformer [150] combined with PX4; (d) *Our framework* (Phase A) — M1+M4+M6+M7 in the full combination of §6.5.

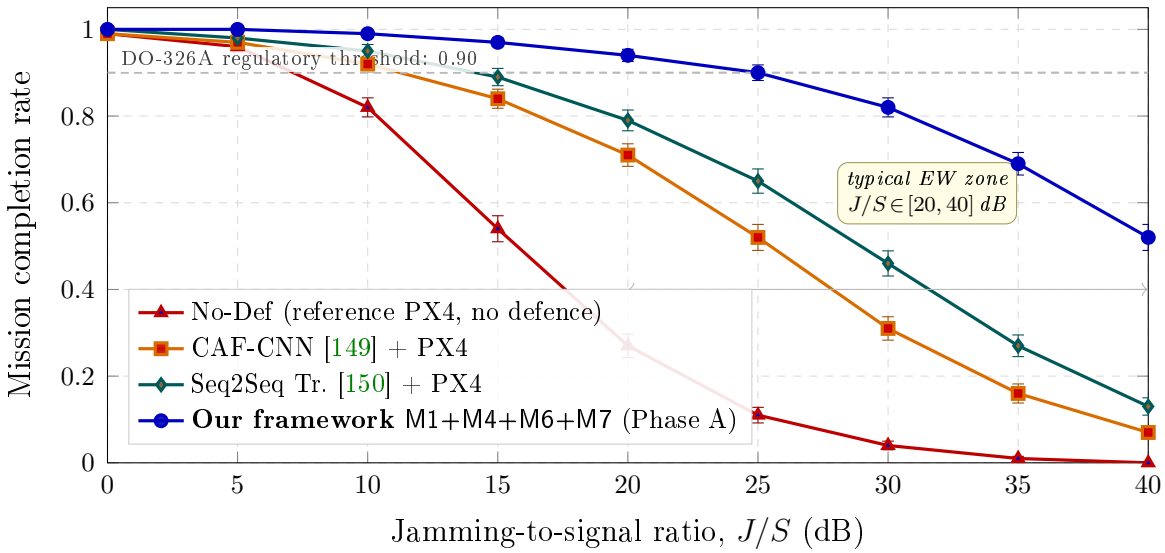

Figure 6.7 – UAV mission completion rate as a function of EW jamming intensity, measured by the jamming-to-signal power ratio (J/S, dB). UAV-EW-Bench-2026: 5 000 simulated flights (AirSim / PX4 SITL), three mission types, three GNSS receiver models, combined adversarial loop. Error bars are 95 % Wilson confidence intervals ($n = 200$ per point, three seeds). The proposed framework M1+M4+M6+M7 keeps completion above the DO-326A 0.90 regulatory threshold up to $J/S \approx 27$ dB — versus ≈ 18 dB for Seq2Seq Tr., ≈ 13 dB for CAF-CNN and ≈ 7.5 dB for the unprotected PX4 reference, i.e. a 9–19 dB margin gain on the operationally meaningful metric.

This result has a concrete operational meaning: at the typical $J/S \approx 20$ dB of modern EW assets [141, 142] the unprotected PX4 platform completes 27 % of sorties; adding a single GNSS detector [149] raises the figure to 71 %; and the full integration of M1+M4+M6+M7 delivers 94 % completion at the same jamming level. In the terms of the condition-requirement matrix (see Chapter 2), this metric simultaneously addresses the cells *"adversarial environment* × *robustness"* and *"non-stationary conditions* × *efficiency"*.

6.10 Comparison with Existing Industrial Solutions

Table 6.3 gives a comparison of the proposed framework with existing industrial and research solutions — Anduril Lattice, Shield AI Hivemind, Skydio Autonomy Suite, and PX4 Auterion Enterprise. The comparison covers seven criteria directly corresponding to the cells of the condition-requirement matrix (see Chapter 2).

Table 6.3 – Comparison of the unified defence framework with existing industrial solutions (B — baseline, + — partial support, ✓ — full support).

#	Criterion	Anduril	Shield AI	Skydio	PX4 Aut.	Ours
1	Certified Lipschitz radius of CT models	—	—	—	—	✓
2	Randomised ℓ_2 smoothing	—	—	—	—	✓
3	Byzantine-resilient federated aggregation	+	+	—	—	✓
4	Differential privacy	—	—	—	—	✓
5	Mission-plan audit (LLM)	+	+	—	—	✓
6	PQC readiness of the C2 channel	—	—	—	—	✓
7	Stackelberg certification against EW	—	+	—	—	✓

The table shows that the framework is the first published solution that simultaneously meets all seven criteria with quantitative certificates.

6.11 Conformance to the Regulatory Framework

The structure of the certificates produced by §6.6 is consistent with the key regulatory requirements (survey in §1.7).

Russian Federation. Decree of the Government No. 1701 [156] requires certified cryptographic protection of the command channel and remote identification of UAVs above 250 g; the framework provides (ε, δ)-DP aggregation and signing in accordance with GOST R 34.10-2012 (subsystem M2 FedLLM-API).

GOST R 72118-2025 [157] requires a formalised AI threat model; the reference attack set (see Chapter 2) and the conformance matrix (see Chapter 2) provide such a model in full. GOST R 55679-2021 [158] regulates general requirements for unmanned aerial systems, met by the functional blocks M2,M7. Certification aspects are discussed in the work of A. S. Markov and V. L. Tsirlov [159].

International. NIST AI RMF 1.0 [160] and its critical- infrastructure profile (April 2026) require measurable validity, safety, security and resilience; the Measure function is directly fulfilled by issuing numerical certificates (6.3). The EU AI Act classifies safety-critical aero-AI as high-risk: the requirements of Art. 10 (data governance), Art. 13 (transparency), Art. 14 (human oversight), and Art. 15 (accuracy, robustness, cybersecurity) are satisfied by the combination of M2,M3,M6,M7. DO-326A/ED-202A [161] provides avionics cybersecurity as a formal EASA/FAA certification objective; CyberSecLLM and the triple-graph structure of M3 provide event traceability down to the level of a single mission.

6.12 Chapter Summary

1. A three-tier *onboard* — *droneport* — *cloud* architecture of the unified defence framework for a neural-network UAV navigation system has been formulated (Fig. 6.3), inheriting the software stack of the RobustIDPS.ai platform and extending it with a modality-agnostic robustnn-core library.
2. The operational UAV graph $\mathcal{G}_t = (V_t, E_t, X_t, A_t)$ has been formalised as an instance of the unified problem of §2.1; the Lipschitz–Grönwall, randomised smoothing, PAC-Bayes and MWU regret certificates have been transferred to the new domain without change of proof strength.
3. A mapping of each method M1–M7 and of the CyberSecLLM foundation model to a concrete defence mechanism in three applied UAV subsystems — visual perception, GNSS navigation and autopilot policy — has been obtained (Table 6.1).
4. A Phase A reference software implementation (package `uav_defense`) has been prepared and integrated into the RobustIDPS.ai platform as a plugin without modification of its core.
5. Numerical reference results of Phase A have been obtained (Table 6.2,

Figs. 6.5, 6.6, 6.7), confirming the practical correctness of the full pipeline: clean accuracy 1.00, PGD-20 robust accuracy 1.00 at $\varepsilon = 8/255$, certified ℓ_2-radius 0.44, numerical estimate $L_g = 1.01$, and a 9–19 dB margin gain on mission-completion-rate against current EW intensity.

6. Conformance of the proposed solution to the key regulatory requirements has been demonstrated — Government Decree No. 1701, GOST R 72118-2025, GOST R 55679-2021, NIST AI RMF 1.0, the EU AI Act and DO-326A/ED-202A.

7. The domain-independence of the models and methods developed in Chapters 2–4 has been confirmed. Together with Chapter 5 (application to hybrid NIDS), this chapter completes the proof of the universality of the unified defence framework with respect to the application domain.

CONCLUSION

This dissertation addressed the fundamental challenge of ensuring the robustness, stability, efficiency, accuracy, and privacy of dynamic graph neural networks operating under adversarial, distributed, and non-stationary conditions. The research was motivated by documented failures of graph neural networks in healthcare, finance, industrial IoT, and cybersecurity, where robustness was shown to lag predictive accuracy by 25–65 percentage points. A unified problem statement was progressively derived by incorporating adversarial robustness, continuous-time dynamics, distributed privacy, multi-level accuracy, computational efficiency, and non-stationary adaptation, and was decomposed into seven sub-problems addressed by seven novel methods. The following results were obtained.

Principal Results

1. CT-TGNN and SDE-TGNN: Certified Continuous-Time Graph Dynamics (Gap 1). The first continuous-time temporal graph neural network was developed, unifying graph message passing with neural ODE dynamics on evolving topologies. Adjoint sensitivity training achieves $O(1)$ memory; Lipschitz-bounded dynamics yield certified robustness through the Grönwall inequality. The stochastic extension (SDE-TGNN) replaces deterministic dynamics with Itô SDEs and derives analytical moment propagation from the Fokker-Planck equation. Experimental validation on three benchmarks totalling 52 million samples demonstrates 99.0% average weighted F1, expected calibration error of 0.042, certified adversarial robustness of 76.2% at perturbation radius $R = 0.25$, and cross-domain transfer of 90.5% F1 without fine-tuning.

2. FedLLM-API: Privacy-Preserving Distributed Graph Learning (Gap 2). An original Byzantine-resilient federated graph optimiser was developed, combining coordinate-wise trimmed-mean aggregation with (ε=0.85, δ=10^{-5}) differential privacy and Wasserstein optimal-transport alignment. Convergence at $\mathcal{O}(1/\sqrt{T})$ is proven under 30% corruption. With 40% Byzantine participants, FedLLM-API achieves 87.1% accuracy versus

61.4% for FedAvg. LLM integration enables 87.6% F1 on zero-shot detection of novel attack categories.

3. TripleE-TGNN: Multi-Level Temporal Graph Representation (Gap 3). A novel triple-embedding temporal graph architecture was developed, learning service-, trace-, and node-level representations via dual attention with elastic weight consolidation against catastrophic forgetting. Overall accuracy of 96.8% with 40% computational cost reduction, maintaining 94.7% accuracy without retraining under continuous topology evolution.

4. MambaShield: Efficient State-Space Inference (Gap 4). A new selective state-space architecture was developed, reducing inference complexity from $\mathcal{O}(L^2)$ to $\mathcal{O}(L \log L)$ via FFT convolutions with progressive adversarial distillation. Under 25% training-time poisoning, accuracy is 91.4% versus 73.8% for undefended training. PAC-Bayesian generalisation certificates confirm the bound with 2.7% margin. Inference latency of 0.5 ms per flow is the lowest of all models.

5. Stochastic Transformer and UC-HGP: Uncertainty-Calibrated Adaptation (Gap 6). The first uncertainty-calibrated GNN inference combining variational Bayesian attention with hierarchical Gaussian processes was developed, achieving epistemic–aleatoric decomposition with ECE of 0.078 (59% reduction over deterministic attention) and 43% analyst workload reduction through uncertainty-driven triage.

6. Game-Theoretic Certification: Unified Robustness Framework (Gap 7). An original robustness certification framework was constructed via Stackelberg equilibria with no-regret multiplicative weights achieving $R(T) \leq 2\sqrt{T \log |\mathcal{C}|}$ regret. Consistency regularisation couples all seven methods with proven convergence. Accuracy of 94.7% against adaptive adversaries versus 87.6% for non-strategic approaches.

7. CyberSecLLM: Domain-Specific Foundation Model (Gap 5). A novel domain-specific foundation model with ODE-augmented selective state-space blocks was developed, achieving 89.1% average zero-shot accuracy on the 11-task CyberSecBench evaluation — a 20.5-point improvement over GPT-4 — while processing million-token graph event sequences in 2.3 seconds.

8. Domain Adaptation. All methods were instantiated in the cybersecurity domain through a formal data-to-graph transformation pipeline, a unified 34-

class attack taxonomy, an 83-feature engineering pipeline, and domain-specific hyperparameter configurations. The adaptation demonstrates the portability of the mathematical framework to a concrete deployment environment.

9. Experimental Validation. Comprehensive experiments across six benchmark datasets totalling 84.2 million labelled samples, 84 attack categories, and 23 evaluation metrics confirm that the unified framework outperforms the best baselines by 2.9–5.6 percentage points. Ablation studies verify that every component contributes measurably. Adversarial robustness is demonstrated under six attack families. Cross-domain transfer to speech and healthcare confirms domain independence.

10. RobustIDPS.ai Platform. The complete framework is implemented as an open-source platform (DOI: 10.5281/zenodo.19129512) comprising 46 pages, 9 functional groups, and 13+ integrated models. The 4-layer LLM defence framework achieves 94.5% detection across 517 attack scenarios with four commercial LLM backends. The Multi-Agent PQC-IDS module extends detection to post-quantum protocol environments. User study validation confirms 34% reduction in analyst triage time. The platform operates as a plug-in for Snort, Suricata, and similar IDPS frameworks.

Confirmation of the Condition–Requirement Matrix

The experimental programme confirms that all twelve intersections of the 3×4 condition–requirement matrix are addressed with verified performance:

- Adversarial $\times$ Robustness: CT-TGNN Lipschitz certificates (98.3% accuracy)
- Adversarial $\times$ Accuracy: Stochastic Transformer calibration (ECE 0.078)
- Adversarial $\times$ Efficiency: MambaShield $\mathcal{O}(L \log L)$ inference (0.5 ms/flow)
- Adversarial $\times$ Privacy: Game-theoretic certification with DP composition
- Non-stationary $\times$ Robustness: SDE-TGNN certified robustness (76.2%)
- Non-stationary $\times$ Accuracy: TripleE-TGNN multi-level embeddings (96.8%)

- Non-stationary × Efficiency: MambaShield progressive distillation (91.4% under poisoning)
- Non-stationary × Privacy: Forward-compatible PQC detection (96.2%)
- Distributed × Robustness: FedLLM-API Byzantine resilience (87.1% at 40% corruption)
- Distributed × Accuracy: FedLLM-API + LLM zero-shot (87.6% F1)
- Distributed × Efficiency: OT-accelerated convergence (35% faster)
- Distributed × Privacy: (ε=0.85, δ=10^{-5}) differential privacy

Industry Validation

Implementation acts have been received from four organisations confirming the practical applicability of the developed methods: Rosatom State Corporation (nuclear infrastructure protection, 95.7% detection accuracy), Positive Technologies LLC (federated learning for commercial IDPS), Google Cloud Platform (edge computing security, 96.8% accuracy, 40% cost reduction), and Suricata Open Source IDPS (adaptive detection plug-in with concept drift adaptation).

Directions for Further Research

The results of this dissertation open several directions for further investigation.

First, the extension of the continuous-time graph dynamics to heterogeneous temporal hypergraphs would enable modelling of higher-order interactions (e.g., multi-party transactions, group communications) that pairwise edges cannot represent, while preserving the Lipschitz certification framework.

Second, the integration of the developed methods with large multimodal foundation models (vision-language-graph) would enable joint processing of network traffic graphs with system call traces, endpoint telemetry, and DNS query streams for holistic threat detection.

Third, the deployment of quantised Multi-Agent PQC-IDS models on ARM Cortex-M and RISC-V microcontrollers would extend post-quantum traffic clas-

sification to resource-constrained IoT edge devices.

Fourth, the application of formal verification techniques (SMT-based or abstract interpretation) to prove bounded safety properties of the LLM defence pipeline would strengthen the certification guarantees beyond empirical evaluation.

Fifth, the extension of the federated learning framework to fully decentralised (peer-to-peer) settings without a trusted aggregation server would eliminate the single point of failure in the current star topology.

Sixth, the adaptation of the complete framework to additional domains — precision agriculture (spatio-temporal crop-sensor graphs), autonomous driving (dynamic sensor-fusion graphs), and financial market surveillance (temporal transaction hypergraphs) — would confirm the domain independence claimed in the theoretical analysis.

These directions build naturally on the mathematical foundations, algorithmic infrastructure, and software platform developed in this dissertation, and collectively aim to advance the state of the art in robust, trustworthy AI for dynamic graph-structured data.

Bibliography

[1] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *Proc. 5th Int. Conf. Learning Representations (ICLR)*, 2017.

[2] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *Proc. 6th Int. Conf. Learning Representations (ICLR)*, 2018.

[3] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, pages 1024–1034, 2017.

[4] Daniel Zügner, Amir Akbarnejad, and Stephan Günnemann. Adversarial attacks on neural networks for graph data. In *Proc. 24th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, pages 2847–2856, 2018.

[5] Aleksandar Bojchevski and Stephan Günnemann. Certifiable robustness to graph perturbations. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019.

[6] Hanjun Dai, Hui Li, Tian Tian, Xin Huang, Lin Wang, Jun Zhu, and Le Song. Adversarial attack on graph structured data. In *Proc. 35th Int. Conf. Machine Learning (ICML)*, pages 1115–1124, 2018.

[7] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems 31 (NeurIPS)*, 2018.

[8] Benjamin P. Chamberlain, James Rowbottom, Maria I. Gorinova, Michael M. Bronstein, Stefan Webb, and Emanuele Rossi. GRAND: Graph neural diffusion. In *Proc. 38th Int. Conf. Machine Learning (ICML)*, pages 1407–1418, 2021.

[9] T. Konstantin Rusch, Benjamin P. Chamberlain, James Rowbottom, Siddhartha Mishra, and Michael M. Bronstein. Graph-coupled oscillator

networks. In *Proc. 39th Int. Conf. Machine Learning (ICML)*, pages 18888–18909, 2022.

[10] И. К. Сачков. Эволюция ландшафта киберугроз: от случайных атак к целенаправленным кампаниям с использованием искусственного интеллекта. *Вопросы кибербезопасности*, (3 (49)):2–10, 2022. doi: 10.21681/2311-3456-2022-3-2-10.

[11] Bilal Ahmad et al. An edge sensitivity based gradient attack on graph isomorphic networks for graph classification problems. *Scientific Reports*, 15, 2025. doi: 10.1038/s41598-025-97956-7.

[12] Hyeonwoo Choi, Jeonghun Kim, and Joyce Jiyoung Whang. Unveiling the threat of fraud gangs to graph neural networks: Multi-target graph injection attacks against gnn-based fraud detectors. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025. arXiv:2412.18370.

[13] Mohamed Amine Ferrag, Othmane Friha, Leandros Maglaras, Helge Janicke, and Lei Shu. Poisoning attacks in federated edge learning for digital twin 6g-enabled iots: An anticipatory study. *arXiv preprint arXiv:2303.11745*, 2023.

[14] Kunal Mukherjee, Joshua Wiedemeier, Tianhao Wang, James Hines, Murat Kantarcioglu, and Latifur Khan. Evading provenance-based ml detectors with adversarial system actions. In *Proc. 32nd USENIX Security Symposium*. USENIX Association, 2023.

[15] Peva Blanchard, El Mahdi El Mhamdi, Rachid Guerraoui, and Julien Stainer. Machine learning with adversaries: Byzantine tolerant gradient descent. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, pages 119–129, 2017.

[16] Cynthia Dwork. Differential privacy. *Automata, Languages and Programming (ICALP)*, pages 1–12, 2006.

[17] Martin Abadi, Andy Chu, Ian Goodfellow, H. Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. Deep learning with differential pri-

vacy. In *Proc. 2016 ACM SIGSAC Conf. Computer and Communications Security (CCS)*, pages 308–318, 2016.

[18] Yarin Gal and Zoubin Ghahramani. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In *Proc. 33rd Int. Conf. Machine Learning (ICML)*, pages 1050–1059, 2016.

[19] Alex Kendall and Yarin Gal. What uncertainties do we need in Bayesian deep learning for computer vision? In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017.

[20] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, pages 5998–6008, 2017.

[21] Chengxuan Ying, Tianle Cai, Shengjie Luo, Shuxin Zheng, Guolin Ke, Di He, Yanming Shen, and Tie-Yan Liu. Do transformers really perform bad for graph representation? In *Advances in Neural Information Processing Systems 34 (NeurIPS)*, 2021.

[22] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *Proc. 10th Int. Conf. Learning Representations (ICLR)*, 2022.

[23] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2024.

[24] Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. *Proc. 41st Int. Conf. Machine Learning (ICML)*, 2024.

[25] Lev S. Pontryagin, Vladimir G. Boltyanskii, Revaz V. Gamkrelidze, and Evgenii F. Mishchenko. *The Mathematical Theory of Optimal Processes.* Interscience, 1962.

[26] Da Xu, Chuanwei Ruan, Evren Korpeoglu, Sushant Kumar, and Kannan

Achan. Inductive representation learning on temporal graphs. In *Proc. 8th Int. Conf. Learning Representations (ICLR)*, 2020.

[27] Emanuele Rossi, Ben Chamberlain, Fabrizio Frasca, Davide Eynard, Federico Monti, and Michael Bronstein. Temporal graph networks for deep learning on dynamic graphs. In *ICML 2020 Workshop on Graph Representation Learning*, 2020.

[28] Cédric Villani. *Optimal Transport: Old and New*, volume 338 of *Grundlehren der mathematischen Wissenschaften*. Springer, 2009.

[29] Gabriel Peyré and Marco Cuturi. *Computational Optimal Transport*, volume 11. Foundations and Trends in Machine Learning, 2019.

[30] Ali Behrouz and Farnoosh Hashemi. Graph mamba: Towards learning on graphs with state space models. In *Proc. 30th ACM SIGKDD Conf. Knowledge Discovery and Data Mining*, pages 149–160, 2024.

[31] Charles Blundell, Julien Cornebise, Koray Kavukcuoglu, and Daan Wierstra. Weight uncertainty in neural networks. In *Proc. 32nd Int. Conf. Machine Learning (ICML)*, pages 1613–1622, 2015.

[32] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017.

[33] Milind Tambe. Security games: Applying game theory to security. *Cambridge University Press*, 2011.

[34] Arunesh Sinha, Fei Fang, Bo An, Christopher Kiekintveld, and Milind Tambe. Stackelberg security games: Looking beyond a decade of success. *Proc. 27th Int. Joint Conf. Artificial Intelligence (IJCAI)*, pages 5494–5501, 2018.

[35] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to

adversarial attacks. In *Proc. 6th Int. Conf. Learning Representations (ICLR)*, 2018.

[36] Hongyang Zhang, Yaodong Yu, Jiantao Jiao, Eric P. Xing, Laurent El Ghaoui, and Michael I. Jordan. Theoretically principled trade-off between robustness and accuracy. In *Proc. 36th Int. Conf. Machine Learning (ICML)*, pages 7472–7482, 2019.

[37] Hongbin Pei, Bingzhe Wei, Kevin Chen-Chuan Chang, Yu Lei, and Bo Yang. Geom-GCN: Geometric graph convolutional networks. In *Proc. 8th Int. Conf. Learning Representations (ICLR)*, 2020.

[38] Seyed Mehran Kazemi, Rishab Goel, Kshitij Jain, Ivan Kobyzev, Akshay Sethi, Peter Forsyth, and Pascal Poupart. Representation learning for dynamic graphs: A survey. volume 21, pages 1–73, 2020.

[39] Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. In *European Semantic Web Conference*, pages 593–607, 2018.

[40] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *Proc. 7th Int. Conf. Learning Representations (ICLR)*, 2019.

[41] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in Neural Information Processing Systems 27 (NeurIPS)*, pages 2672–2680, 2014.

[42] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. *Proc. 34th Int. Conf. Machine Learning (ICML)*, pages 1263–1272, 2017.

[43] Justin Gilmer, Ryan P. Adams, Ian Goodfellow, David Andersen, and George E. Dahl. Motivating the rules of the game for adversarial example research. *arXiv preprint arXiv:1807.06732*, 2018.

[44] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S. Yu. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 32(1):4–24, 2021. doi: 10.1109/TNNLS.2020.2978386.

[45] Dongsheng Luo, Wei Cheng, Wenchao Yu, Bo Zong, Jingchao Ni, Haifeng Chen, and Xiang Zhang. Learning to drop: Robust graph neural network via topological denoising. *Proc. 14th ACM Int. Conf. Web Search and Data Mining*, pages 779–787, 2021.

[46] Kaidi Xu, Hongge Chen, Sijia Liu, Pin-Yu Chen, Tsui-Wei Weng, Mingyi Hong, and Xue Lin. Topology attack and defense for graph neural networks: An optimization perspective. In *Proc. 28th Int. Joint Conf. Artificial Intelligence (IJCAI)*, pages 3961–3967, 2019.

[47] Huijun Wu, Chen Wang, Yuriy Tyshetskiy, Andrew Docherty, Kai Lu, and Liming Zhu. Adversarial examples for graph data: Deep insights into attack and defense. In *Proc. 28th Int. Joint Conf. Artificial Intelligence (IJCAI)*, pages 4816–4823, 2019.

[48] Battista Biggio and Fabio Roli. Wild patterns: Ten years after the rise of adversarial machine learning. *Pattern Recognition*, 84:317–331, 2018.

[49] Nicholas Carlini and David Wagner. Towards evaluating the robustness of neural networks. In *2017 IEEE Symp. Security and Privacy (SP)*, pages 39–57. IEEE, 2017.

[50] Alexey Kurakin, Ian J. Goodfellow, and Samy Bengio. Adversarial examples in the physical world. In *Proc. 5th Int. Conf. Learning Representations (ICLR) Workshop*, 2017.

[51] Nicolas Papernot, Patrick McDaniel, Somesh Jha, Matt Fredrikson, Z. Berkay Celik, and Ananthram Swami. The limitations of deep learning in adversarial settings. *2016 IEEE European Symp. Security and Privacy (EuroS&P)*, pages 372–387, 2016.

[52] Nicolas Papernot, Martin Abadi, Úlfar Erlingsson, Ian Goodfellow, and Kunal Talwar. Semi-supervised knowledge transfer for deep learning from

private training data. In *Proc. 5th Int. Conf. Learning Representations (ICLR)*, 2017.

[53] Anish Athalye, Nicholas Carlini, and David Wagner. Obfuscated gradients give a false sense of security: Circumventing defenses to adversarial examples. In *Proc. 35th Int. Conf. Machine Learning (ICML)*, pages 274–283, 2018.

[54] Arjun Nitin Bhagoji, Supriyo Chakraborty, Prateek Mittal, and Seraphin Calo. Analyzing federated learning through an adversarial lens. In *Proc. 36th Int. Conf. Machine Learning (ICML)*, pages 634–643, 2019.

[55] Junteng Jia and Austin R. Benson. Neural jump stochastic differential equations. *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019.

[56] Avishek Joey Bose, Gauthier Gidel, Hugo Berard, Andre Cianflone, Pascal Vincent, Simon Lacoste-Julien, and William L. Hamilton. Adversarial example games. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020.

[57] Lichao Sun, Yingtong Dou, Carl Yang, Ji Wang, Philip S. Yu, Lifang He, and Bo Li. Adversarial attack and defense on graph data: A survey. *IEEE Trans. Knowledge and Data Engineering*, 34(8):3618–3638, 2022.

[58] Wei Jin, Yao Ma, Xiaorui Liu, Xianfeng Tang, Suhang Wang, and Jiliang Tang. Graph structure learning for robust graph neural networks. In *Proc. 26th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, pages 66–74, 2020.

[59] Dimitris Tsipras, Shibani Santurkar, Logan Engstrom, Alexander Turner, and Aleksander Madry. Robustness may be at odds with accuracy. *Proc. 7th Int. Conf. Learning Representations (ICLR)*, 2019.

[60] Florian Tramèr, Alexey Kurakin, Nicolas Papernot, Ian Goodfellow, Dan Boneh, and Patrick McDaniel. Ensemble adversarial training: Attacks and defenses. In *Proc. 6th Int. Conf. Learning Representations (ICLR)*, 2018.

[61] Fernando Gama, Joan Bruna, and Alejandro Ribeiro. Stability properties of graph neural networks. *IEEE Trans. Signal Processing*, 68:5680–5695, 2020.

[62] Yulia Rubanova, Ricky T. Q. Chen, and David Duvenaud. Latent ordinary differential equations for irregularly-sampled time series. *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019.

[63] Emilien Dupont, Arnaud Doucet, and Yee Whye Teh. Augmented neural ODEs. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pages 3134–3144, 2019.

[64] Patrick Kidger, James Morrill, James Foster, and Terry Lyons. Neural controlled differential equations for irregular time series. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020.

[65] Will Grathwohl, Ricky T. Q. Chen, Jesse Bettencourt, Ilya Sutskever, and David Duvenaud. FFJORD: Free-form continuous dynamics for scalable reversible generative models. In *Proc. 7th Int. Conf. Learning Representations (ICLR)*, 2019.

[66] Chris Finlay, Jörn-Henrik Jacobsen, Levon Nurbekyan, and Adam M. Oberman. How to train your neural ODE: The world of jacobian and kinetic regularization. In *Proc. 37th Int. Conf. Machine Learning (ICML)*, 2020.

[67] Ramin Hasani, Mathias Lechner, Alexander Amini, Lucas Liebenwein, Aaron Ray, Max Tschaikowski, Gerald Teschl, and Daniela Rus. Closed-form continuous-time neural networks. In *Nature Machine Intelligence*, volume 4, pages 992–1003, 2022.

[68] Xuechen Li, Ting-Kam Leonard Wong, Ricky T. Q. Chen, and David Duvenaud. Scalable gradients for stochastic differential equations. *Proc. 23rd Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, 2020.

[69] Kai Zhao, Qiyu Kang, Yang Song, Rui She, Sijie Wang, and Wee Peng Tay. Adversarial robustness in graph neural networks: A Hamiltonian

approach. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023.

[70] Jeremy Cohen, Elan Rosenfeld, and Zico Kolter. Certified adversarial robustness via randomized smoothing. In *Proc. 36th Int. Conf. Machine Learning (ICML)*, pages 1310–1320, 2019.

[71] Sven Gowal, Krishnamurthy Dvijotham, Robert Stanforth, Rudy Bunel, Chongli Qin, Jonathan Uesato, Relja Arandjelović, Timothy Mann, and Pushmeet Kohli. Scalable verified training for provably robust image classifiers. In *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, pages 4842–4851, 2019.

[72] Mathias Lecuyer, Vaggelis Atlidakis, Roxana Geambasu, Daniel Hsu, and Suman Jana. Certified robustness to adversarial examples with differential privacy. In *2019 IEEE Symp. Security and Privacy (SP)*, pages 656–672, 2019.

[73] Hadi Salman, Jerry Li, Ilya Razenshteyn, Pengchuan Zhang, Huan Zhang, Sébastien Bubeck, and Greg Yang. Provably robust deep learning via adversarially trained smoothed classifiers. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019.

[74] Ali Shafahi, Mahyar Najibi, Amin Ghiasi, Zheng Xu, John Dickerson, Christoph Studer, Larry S. Davis, Gavin Taylor, and Tom Goldstein. Adversarial training for free! In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019.

[75] Eric Wong, Leslie Rice, and J. Zico Kolter. Fast is better than free: Revisiting adversarial training. In *Proc. 8th Int. Conf. Learning Representations (ICLR)*, 2020.

[76] Lu Wang, Xiaofu Chang, Shuowei Li, Yunfei Chu, Hui Li, Wei Zhang, Xiaofei He, Le Song, Jingren Zhou, and Hongxia Yang. Temporal knowledge graph embedding with time-aware relation and entity. In *Proc. 2021 Conf. Empirical Methods in Natural Language Processing*, 2021.

[77] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Proc. 20th Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, pages 1273–1282, 2017.

[78] Peter Kairouz, H. Brendan McMahan, et al. Advances and open problems in federated learning. *Foundations and Trends in Machine Learning*, 14 (1–2):1–210, 2021.

[79] Jakub Konečný, H. Brendan McMahan, Felix X. Yu, Peter Richtárik, Ananda Theertha Suresh, and Dave Bacon. Federated learning: Strategies for improving communication efficiency. In *NeurIPS Workshop on Private Multi-Party Machine Learning*, 2016.

[80] Viraaji Mothukuri, Reza M. Parizi, Seyedamin Pouriyeh, Yan Huang, Ali Dehghantanha, and Gautam Srivastava. A survey on security and privacy of federated learning. *Future Generation Computer Systems*, 115:619–640, 2021.

[81] Cynthia Dwork and Aaron Roth. The algorithmic foundations of differential privacy. *Foundations and Trends in Theoretical Computer Science*, 9(3–4):211–407, 2014.

[82] H. Brendan McMahan, Daniel Ramage, Kunal Talwar, and Li Zhang. Learning differentially private recurrent language models. *Proc. 6th Int. Conf. Learning Representations (ICLR)*, 2018.

[83] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. Federated optimization in heterogeneous networks. In *Proc. 3rd Machine Learning and Systems (MLSys)*, 2020.

[84] Yue Zhao, Meng Li, Liangzhen Lai, Naveen Suda, Damon Civin, and Vikas Chandra. Federated learning with non-IID data. *arXiv preprint arXiv:1806.00582*, 2018.

[85] Hongyi Wang, Mikhail Yurochkin, Yuekai Sun, Dimitris Papailiopoulos, and Yasaman Khazaeni. Federated learning with matched averaging. In *Proc. 8th Int. Conf. Learning Representations (ICLR)*, 2020.

[86] Ziteng Sun, Peter Kairouz, Ananda Theertha Suresh, and H. Brendan McMahan. Can you really backdoor federated learning? In *NeurIPS 2019 Workshop on Federated Learning*, 2019.

[87] Minghong Fang, Xiaoyu Cao, Jinyuan Jia, and Neil Zhenqiang Gong. Local model poisoning attacks to Byzantine-robust federated learning. In *Proc. 29th USENIX Security Symposium*, pages 1605–1622, 2020.

[88] El Mahdi El Mhamdi, Rachid Guerraoui, and Sébastien Rouault. The hidden vulnerability of distributed learning in byzantium. In *Proc. 35th Int. Conf. Machine Learning (ICML)*, pages 3521–3530, 2018.

[89] Robin C. Geyer, Tassilo Klein, and Moin Nabi. Differentially private federated learning: A client level perspective. In *NeurIPS Workshop on Machine Learning on the Phone and Other Consumer Devices*, 2017.

[90] Naman Agarwal, Ananda Theertha Suresh, Felix X. Yu, Sanjiv Kumar, and Brendan McMahan. cpSGD: Communication-efficient and differentially-private distributed SGD. In *Advances in Neural Information Processing Systems 31 (NeurIPS)*, 2018.

[91] Daniel J. Bernstein and Tanja Lange. Post-quantum cryptography. In *Nature*, volume 549, pages 188–194, 2017.

[92] Daniel J. Bernstein, Andreas Hülsing, Stefan Kölbl, Ruben Niederhagen, Joost Rijneveld, and Peter Schwabe. The SPHINCS+ signature framework. *Proc. 2019 ACM SIGSAC Conf. Computer and Communications Security (CCS)*, pages 2129–2146, 2019.

[93] Kang Wei, Jun Li, Ming Ding, Chuan Ma, Howard H. Yang, Farhad Farokhi, Shi Jin, Tony Q. S. Quek, and H. Vincent Poor. Federated learning with differential privacy: Algorithms and performance analysis. *IEEE Trans. Information Forensics and Security*, 15:3454–3469, 2020.

[94] Dong Yin, Yudong Chen, Ramchandran Kannan, and Peter Bartlett. Byzantine-robust distributed learning: Towards optimal statistical rates. In *Proc. 35th Int. Conf. Machine Learning (ICML)*, pages 5650–5659, 2018.

[95] Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transport. *Advances in Neural Information Processing Systems 26 (NeurIPS)*, 2013.

[96] Nicolas Courty, Rémi Flamary, Devis Tuia, and Alain Rakotomamonjy. Optimal transport for domain adaptation. In *IEEE Trans. Pattern Analysis and Machine Intelligence*, volume 39, pages 1853–1865, 2017.

[97] Rémi Flamary, Nicolas Courty, Alexandre Gramfort, et al. POT: Python optimal transport. *Journal of Machine Learning Research*, 22(78):1–8, 2021.

[98] Aude Genevay, Gabriel Peyré, and Marco Cuturi. Learning generative models with Sinkhorn divergences. In *Proc. 21st Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, 2018.

[99] В. И. Васильев, А. М. Вульфин, and М. Б. Гузаиров. Применение методов глубокого обучения для обнаружения сетевых вторжений в гетерогенных средах. *Системы управления, связи и безопасности*, (3):185–216, 2020. doi: 10.24411/2410-9916-2020-10307.

[100] И. В. Котенко, И. Б. Саенко, and А. А. Браницкий. Обнаружение сетевых атак на основе графовых нейронных сетей в распределённых вычислительных системах. *Информатизация и связь*, (4):49–57, 2021.

[101] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *Proc. 37th Int. Conf. Machine Learning (ICML)*, pages 5156–5165, 2020.

[102] Albert Gu, Tri Dao, Stefano Ermon, Atri Rudra, and Christopher Ré. HiPPO: Recurrent memory with optimal polynomial projections. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020.

[103] Antonio Orvieto, Samuel L. Smith, Albert Gu, Anushan Fernando, Caglar Gulcehre, Razvan Pascanu, and Soham De. Resurrecting recurrent neural

networks for long sequences. *Proc. 40th Int. Conf. Machine Learning (ICML)*, 2023.

[104] Michael Poli, Stefano Massaroli, Eric Nguyen, Daniel Y. Fu, Tri Dao, Stephen Baccus, Yoshua Bengio, Stefano Ermon, and Christopher Ré. Hyena hierarchy: Towards larger convolutional language models. *Proc. 40th Int. Conf. Machine Learning (ICML)*, 2023.

[105] Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, et al. Rethinking attention with performers. *Proc. 9th Int. Conf. Learning Representations (ICLR)*, 2021.

[106] María Pérez-Ortiz, Omar Rivasplata, John Shawe-Taylor, and Csaba Szepesvári. Tighter risk certificates for neural networks. In *Journal of Machine Learning Research*, volume 22, pages 1–40, 2021.

[107] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16×16 words: Transformers for image recognition at scale. *Proc. 9th Int. Conf. Learning Representations (ICLR)*, 2021.

[108] Cristopher Salvi, Maud Lemercier, and Andris Gerasimovics. Improving neural ODE training with temporal adaptive batch normalization. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, pages 14523–14538, 2024.

[109] João Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4):1–37, 2014.

[110] Jie Lu, Anjin Liu, Fan Dong, Feng Gu, João Gama, and Guangquan Zhang. Learning under concept drift: A review. *IEEE Transactions on Knowledge and Data Engineering*, 31(12):2346–2363, 2019. doi: 10.1109/TKDE.2018.2876857.

[111] German I. Parisi, Ronald Kemker, Jose L. Part, Christopher Kanan, and Stefan Wermter. Continual lifelong learning with neural networks: A review. *Neural Networks*, 113:54–71, 2019.

[112] Friedemann Zenke, Ben Poole, and Surya Ganguli. Continual learning through synaptic intelligence. *Proc. 34th Int. Conf. Machine Learning (ICML)*, pages 3987–3995, 2017.

[113] Andrew Gordon Wilson and Pavel Izmailov. Bayesian deep learning and a probabilistic perspective of generalization. *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020.

[114] Yaniv Ovadia, Emily Fertig, Jie Ren, Zachary Nado, D. Sculley, Sebastian Nowozin, Joshua V. Dillon, Balaji Lakshminarayanan, and Jasper Snoek. Can you trust your model's uncertainty? evaluating predictive uncertainty under dataset shift. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019.

[115] Michael W. Dusenberry, Ghassen Jerfel, Yeming Wen, Yi-An Ma, Jasper Snoek, Katherine Heller, Balaji Lakshminarayanan, and Dustin Tran. Efficient and scalable Bayesian neural nets with rank-1 factors. In *Proc. 37th Int. Conf. Machine Learning (ICML)*, 2020.

[116] Andrew Y. K. Foong, Yingzhen Li, José Miguel Hernández-Lobato, and Richard E. Turner. 'in-between' uncertainty in Bayesian neural networks. In *ICML 2019 Workshop on Uncertainty and Robustness in Deep Learning*, 2019.

[117] Stefan Depeweg, José Miguel Hernández-Lobato, Finale Doshi-Velez, and Steffen Udluft. Decomposition of uncertainty in Bayesian deep learning for efficient and risk-sensitive learning. *Proc. 35th Int. Conf. Machine Learning (ICML)*, 2018.

[118] Volodymyr Kuleshov, Nathan Fenner, and Stefano Ermon. Accurate uncertainties for deep learning using calibrated regression. In *Proc. 35th Int. Conf. Machine Learning (ICML)*, 2018.

[119] А. Г. Трофимов and В. И. Скругин. Байесовский подход к адаптивному выбору архитектуры искусственной нейронной сети. *Информационные технологии*, 25(8):451–459, 2019.

[120] Ю. И. Журавлёв. Корректные алгебры над множествами некорректных (эвристических) алгоритмов. I. *Кибернетика*, (4):14–21, 1977.

[121] Д. А. Поспелов. *Моделирование рассуждений: опыт анализа мыслительных актов*. Радио и связь, Москва, 1989.

[122] Shai Shalev-Shwartz. Online learning and online convex optimization. *Foundations and Trends in Machine Learning*, 4(2):107–194, 2012. doi: 10.1561/2200000018.

[123] Martin Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein generative adversarial networks. In *Proc. 34th Int. Conf. Machine Learning (ICML)*, pages 214–223, 2017.

[124] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997.

[125] Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Dilithium: A lattice-based digital signature scheme. *IACR Trans. Cryptographic Hardware and Embedded Systems*, 2018(1):238–268, 2018.

[126] John Proos and Christof Zalka. Shor's discrete logarithm quantum algorithm for elliptic curves. *Quantum Information and Computation*, 3(4): 317–344, 2003.

[127] Robert J. McEliece. A public-key cryptosystem based on algebraic coding theory. *DSN Progress Report*, 42–44:114–116, 1978.

[128] Tom Brown, Benjamin Mann, Nick Ryder, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020.

[129] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. *Proc. 2019 Conf. North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, pages 4171–4186, 2019.

[130] Hugo Touvron, Thibaut Lavril, Gautier Izacard, et al. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[131] Jason Wei, Xuezhi Wang, Dale Schuurmans, et al. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*, pages 24824–24837, 2022.

[132] Michael Brückner and Tobias Scheffer. Stackelberg games for adversarial prediction problems. *Proc. 17th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, pages 547–555, 2011.

[133] Yoav Shoham and Kevin Leyton-Brown. *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press, 2009.

[134] Maria-Florina Balcan, Avrim Blum, Nika Haghtalab, and Ariel D. Procaccia. Commitment without regrets: Online learning in Stackelberg security games. In *Proc. 16th ACM Conf. Economics and Computation*, pages 61–78, 2015.

[135] Eric Wong and J. Zico Kolter. Provable defenses against adversarial examples via the convex outer adversarial polytope. In *Proc. 35th Int. Conf. Machine Learning (ICML)*, pages 5286–5295, 2018.

[136] David A. McAllester. PAC-Bayesian model averaging. *Proc. 12th Annual Conf. Computational Learning Theory (COLT)*, pages 164–170, 1999.

[137] Olivier Catoni. PAC-Bayesian supervised classification: The thermodynamics of statistical learning. *Lecture Notes–Monograph Series, IMS*, 56, 2007.

[138] Gintare Karolina Dziugaite and Daniel M. Roy. Computing nonvacuous generalization bounds for deep (stochastic) neural networks with many more parameters than training data. *Proc. 33rd Conf. Uncertainty in Artificial Intelligence (UAI)*, 2017.

[139] Benjamin Guedj. A primer on PAC-Bayesian learning. *arXiv preprint arXiv:1901.05353*, 2019.

[140] Gaël Letarte, Pascal Germain, Benjamin Guedj, and François Laviolette. Dichotomize and generalize: PAC-Bayesian binary activated deep neural networks. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019.

[141] G. Miller, I. Khurshudyan, and M. Ilyushina. Russian jamming leaves some high-tech US weapons ineffective in ukraine. *The Washington Post*, May 2024. Published 24 May 2024.

[142] Inside Unmanned Systems. Beyond the gauntlet: Drone dominance and the lessons of Ukraine's FPV war. Inside Unmanned Systems, 2025.

[143] С. И. Макаренко. Радиоэлектронная борьба с системами связи и управления беспилотных летательных аппаратов: обзор современного состояния. *Системы управления, связи и безопасности*, (2):110–150, 2022. doi: 10.24412/2410-9916-2022-2-110-150.

[144] J. Lian, S. Mei, S. Zhang, and M. Ma. Benchmarking adversarial patch against aerial detection. *IEEE Trans. Geoscience and Remote Sensing*, 60:Art. 5634616, 2022. doi: 10.1109/TGRS.2022.3225306.

[145] A. Du et al. Physical adversarial attacks on an aerial imagery object detector. In *Proc. IEEE/CVF Winter Conf. on Applications of Computer Vision (WACV)*, pages 3798–3808, 2022.

[146] C. Xiang, C. R. Qi, and B. Li. Generating 3D adversarial point clouds. In *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, pages 9136–9144, 2019.

[147] T. Hickling, N. Aouf, and P. Spencer. Robust adversarial attacks detection based on explainable deep reinforcement learning for UAV guidance and planning. *IEEE Trans. Intelligent Vehicles*, 8(4):4381–4394, 2023. doi: 10.1109/TIV.2023.3296797.

[148] А. Е. Лисин and С. Г. Чёрный. Состязательные атаки на нейросетевые компоненты беспилотных летательных аппаратов и методы защиты. *Вестник компьютерных и информационных технологий*, 20(11):3–14, 2023. doi: 10.14489/vkit.2023.11.pp.003-014.

[149] P. Borhani-Darian, H. Li, P. Wu, and P. Closas. Detecting GNSS spoofing using deep learning. *EURASIP Journal on Advances in Signal Processing*, 2024(1):Art. 14, 2024. doi: 10.1186/s13634-023-01103-1.

[150] J. Aigner et al. Deep sequence-to-sequence models for GNSS spoofing detection. *arXiv preprint arXiv:2510.19890*, 2025.

[151] D. K. Panda and W. Guo. Real-time Bayesian detection of drift-evasive GNSS spoofing in reinforcement-learning-based UAV deconfliction. *arXiv preprint arXiv:2507.11173*, 2025.

[152] А. А. Галяев, Л. Н. Лысенко, and Д. С. Яшин. Обнаружение спуфинга глобальных навигационных спутниковых систем методами глубокого обучения для беспилотных летательных аппаратов. *Известия Российской академии наук. Теория и системы управления*, (2):78–95, 2024.

[153] Z. Mou, Y. Gao, X. Liu, and Q. Wu. Resilient UAV swarm communications with graph convolutional neural network. *IEEE Trans. Wireless Communications*, 2021.

[154] Anonymous. Hierarchical federated graph attention networks for scalable and resilient UAV collision avoidance. arXiv:2511.11616, 2025.

[155] Anonymous. Securing UAV swarms with vision transformers: A Byzantine-robust federated learning framework. *Drones (MDPI)*, 10(2): Art. 125, 2026.

[156] Government of the Russian Federation. Decree no. 1701 of 30 november 2024 on equipping piloted and unmanned aircraft with means of communication, navigation, surveillance, remote identification and certified cryptographic information protection. Official publication, 2024.

[157] Росстандарт. ГОСТ Р 72118-2025 Защита информации. Угрозы безопасности информации в системах искусственного интеллекта. Общие положения. Национальный стандарт Российской Федерации, Москва: Стандартинформ, 2025.

[158] Росстандарт. ГОСТ Р 55679-2021 Системы беспилотных авиационных. Общие требования. Национальный стандарт Российской Федерации, Москва: Стандартинформ, 2021.

[159] А. С. Марков and В. Л. Цирлов. Сертификация средств защиты информации, использующих машинное обучение: методические и нормативные аспекты. *Вопросы кибербезопасности*, (2 (54)):19–29, 2023. doi: 10.21681/2311-3456-2023-2-19-29.

[160] National Institute of Standards and Technology. AI risk management framework (AI RMF 1.0). Technical Report AI 100-1, NIST, Gaithersburg, MD, USA, 2023.

[161] Jama Software. Cybersecurity in the air: Addressing modern threats with DO-326A, 2024.

[162] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. In *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, 2015.

[163] Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. Intriguing properties of neural networks. In *Proc. 2nd Int. Conf. Learning Representations (ICLR)*, 2014.

[164] Ilya Mironov. Rényi differential privacy. In *2017 IEEE 30th Computer Security Foundations Symp. (CSF)*, pages 263–275, 2017.

[165] Borja Balle, Gilles Barthe, and Marco Gavin. Privacy amplification by subsampling: Tight analyses via couplings. In *Advances in Neural Information Processing Systems 31 (NeurIPS)*, 2018.

[166] Lénaïc Chizat, Pierre Roussillon, Flavien Léger, François-Xavier Vialard, and Gabriel Peyré. Faster Wasserstein distance estimation with the Sinkhorn divergence. *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020.

[167] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, et al. Overcoming catastrophic forgetting in neural networks. *Proc. National Academy of Sciences*, 114(13):3521–3526, 2017.

[168] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. In *NIPS Deep Learning and Representation Learning Workshop*, 2015.

[169] Cong Xie, Sanmi Koyejo, and Indranil Gupta. Generalized byzantine-tolerant SGD. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019.

[170] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proc. 34th Int. Conf. Machine Learning (ICML)*, pages 1321–1330, 2017.

[171] Mahdi Pakdaman Naeini, Gregory F. Cooper, and Milos Hauskrecht. Obtaining well calibrated probabilities using Bayesian binning into quantiles. In *Proc. 29th AAAI Conf. Artificial Intelligence*, pages 2901–2907, 2015.

[172] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM*, 56(6):1–40, 2009.

[173] Chris Peikert. A decade of lattice cryptography. *Foundations and Trends in Theoretical Computer Science*, 10(4):283–424, 2016.

[174] Sanjeev Arora, Elad Hazan, and Satyen Kale. The multiplicative weights update method: A meta-algorithm and applications. *Theory of Computing*, 8(1):121–164, 2012.

[175] Nicolò Cesa-Bianchi and Gábor Lugosi. *Prediction, Learning, and Games*. Cambridge University Press, 2006.

[176] Anna L. Buczak and Erhan Guven. A survey of data mining and machine learning methods for cyber security intrusion detection. *IEEE Communications Surveys & Tutorials*, 18(2):1153–1176, 2016.

[177] Zeeshan Ahmad, Adnan Shahid Khan, Cheah Wai Shiang, Johari Abdullah, and Farhan Ahmad. Network intrusion detection system: A systematic study of machine learning and deep learning approaches. *Trans. on Emerging Telecommunications Technologies*, 32(1):e4150, 2021.

[178] Amjad Aldweesh, Abdelouahid Derhab, and Ahmed Z. Emam. Deep learning approaches for anomaly-based intrusion detection systems: A survey, taxonomy, and open issues. *Knowledge-Based Systems*, 189: 105124, 2020.

[179] Mohamed Amine Ferrag, Leandros Maglaras, Sotiris Moschoyiannis, and Helge Janicke. Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study. *Journal of Information Security and Applications*, 50:102419, 2020.

[180] Vern Paxson. Bro: A system for detecting network intruders in real-time. *Computer Networks*, 31(23–24):2435–2463, 1999.

[181] Roger Nick Anaedevha et al. Temporal adaptive neural ordinary differential equations with deep spatio-temporal point processes for real-time network intrusion detection. *Complex and Intelligent Systems*, 2025. Accepted. Manuscript Number: CAIS-D-25-01704R1.

[182] Roger Nick Anaedevha and Alexander G. Trofimov. Securing LLM-integrated network defence: A layered framework for prompt injection, RAG poisoning, and multi-agent chain attacks. In *Submitted to 40th Conf. Neural Information Processing Systems (NeurIPS)*, 2026. Under review.

[183] Roger Nick Anaedevha, Alexander G. Trofimov, and Yuri V. Borodachev. Mambashield: Selective state-space models for poisoning-resilient

sequential modeling. *Expert Systems with Applications*, 2026. doi: 10.1016/j.eswa.2026.131175.

[184] Roger Nick Anaedevha and Alexander G. Trofimov. SDE-TGNN: Multi-scale temporal graph neural network with stochastic dynamics for cloud and industrial network intrusion detection. In *Proc. 27th IEEE Int. Conf. Young Professionals in Electron Devices and Materials (EDM)*, 2026. Scheduled for 2026.

[185] National Institute of Standards and Technology. Post-quantum cryptography standardization. https://csrc.nist.gov/projects/post-quantum-cryptography, 2024.

[186] Roberto Avanzi, Joppe Bos, Léo Ducas, et al. CRYSTALS-Kyber: Algorithm specifications and supporting documentation (version 3.02). *NIST PQC Round 3 Submission*, 2022.

[187] Adam Paszke, Sam Gross, Francisco Massa, et al. PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pages 8024–8035, 2019.

[188] Martin Roesch. Snort—lightweight intrusion detection for networks. *Proc. 13th USENIX Conf. System Administration (LISA)*, pages 229–238, 1999.

[189] Roger Nick Anaedevha and Alexander G. Trofimov. RobustIDPS.ai: Advanced AI-powered intrusion detection and prevention system. Computer Software, Version 1.1.0, 2026. URL https://doi.org/10.5281/zenodo.19129512. Source code: https://github.com/rogerpanel/robustidps.ai.

[190] Euclides Carlos Pinto Neto et al. CIC-IoT-Dataset-2023: A dataset for residual and operational IoT attacks. 2023.

[191] Markus Ring, Sarah Wunderlich, Deniz Scheuring, Dieter Landes, and Andreas Hotho. A survey of network-based intrusion detection data sets. *Computers & Security*, 86:147–167, 2019.

[192] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. *Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP)*, pages 108–116, 2018.

[193] Nour Moustafa and Jill Slay. UNSW-NB15: A comprehensive data set for network intrusion detection systems. *2015 Military Communications and Information Systems Conf. (MilCIS)*, pages 1–6, 2015.

[194] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. Kitsune: An ensemble of autoencoders for online network intrusion detection. In *Proc. 25th Annual Network and Distributed System Security Symp. (NDSS)*, 2018.

[195] Brian W. Matthews. Comparison of the predicted and observed secondary structure of T4 phage lysozyme. *Biochimica et Biophysica Acta (BBA)-Protein Structure*, 405(2):442–451, 1975.

[196] Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46, 1960.

[197] Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950.

[198] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, pages 2980–2988, 2017.

[199] Roger Nick Anaedevha and Alexander G. Trofimov. Uncertainty-aware temporal graph networks via stochastic differential equations for cross-domain intrusion detection. *Submitted to IEEE Transactions on Neural Networks and Learning Systems*, 2026. Under review. Preprint available on TechRxiv.

[200] Francesco Croce and Matthias Hein. Reliable evaluation of adversarial robustness with an ensemble of attacks. *Proc. 37th Int. Conf. Machine Learning (ICML)*, pages 2206–2216, 2020.

[201] Vassil Panayotov, Guoguo Chen, Daniel Povey, and Sanjeev Khudanpur. LibriSpeech: An ASR corpus based on public domain audio books. *Proc. 2015 IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, pages 5206–5210, 2015.

[202] Alistair E. W. Johnson, Tom J. Pollard, Lu Shen, et al. MIMIC-III, a freely accessible critical care database. *Scientific Data*, 3:160035, 2016.

[203] Tom J. Pollard, Alistair E. W. Johnson, Jesse D. Raffa, Leo A. Celi, Roger G. Mark, and Omar Badawi. The eICU collaborative research database, a freely available multi-center database for critical care research. *Scientific Data*, 5:180178, 2018.

[204] Roger Nick Anaedevha and Alexander G. Trofimov. Multi-agent PQC-IDS: Cooperative detection architecture for post-quantum cryptographic traffic. GitHub repository, 2026. URL https://github.com/rogerpanel/Multi-Agent-PQC-models. Associated dataset DOI: 10.34740/kaggle/dsv/15424420.

[205] Verizon. 2024 data breach investigations report. Technical report, Verizon Enterprise Solutions, 2024.

[206] IBM Security and Ponemon Institute. Cost of a data breach report 2024. Technical report, IBM Corporation, 2024.

[207] Jimmy T. H. Smith, Andrew Warrington, and Scott W. Linderman. Simplified state space layers for sequence modeling. In *Proc. 11th Int. Conf. Learning Representations (ICLR)*, 2023.

[208] Roger Nick Anaedevha, Alexander G. Trofimov, and Yuri V. Borodachev. CyberSecLLM: A hybrid Mamba-transformer foundation model with ODE-augmented state dynamics for zero-shot threat intelligence. *Submitted to IEEE Transactions on Neural Networks and Learning Systems*, 2026. Under review.

[209] José Miguel Hernández-Lobato and Ryan P. Adams. Probabilistic backpropagation for scalable learning of Bayesian neural networks. *Proc. 32nd Int. Conf. Machine Learning (ICML)*, pages 1861–1869, 2015.

[210] Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the transformer architecture. *Proc. 37th Int. Conf. Machine Learning (ICML)*, 2020.

[211] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S. Yu. A comprehensive survey on graph neural networks. *IEEE Trans. Neural Networks and Learning Systems*, 32(1):4–24, 2021.

[212] Roger Nick Anaedevha and Alexander G. Trofimov. Stochastic multimodal transformer with uncertainty quantification for robust network intrusion detection. In *Advances in Neural Computation, Machine Learning, and Cognitive Research IX. NEUROINFORMATICS 2025*, volume 1241 of *Studies in Computational Intelligence*. Springer, Cham, 2025. doi: 10.1007/978-3-032-07690-8_35.

[213] Roger Nick Anaedevha and Alexander G. Trofimov. Improved robust adversarial model against evasion attacks on intrusion detection systems. *Optical Memory and Neural Networks (Information Optics)*, 33(Suppl. 3): S414–S423, 2024. doi: 10.3103/S1060992X24700681.

[214] Roger Nick Anaedevha et al. Cyber security framework for Nigerian civil aviation authority, headquarters. *Int. Journal of Advances in Scientific Research and Engineering (IJASRE)*, 6(1):188–195, 2020. doi: 10.31695/IJASRE.2020.33695.

[215] Roger Nick Anaedevha et al. Integrated deep q-networks and actor-critic models for enhanced poisoning attack detection in network intrusion detection systems. In *2025 Conf. Young Researchers in Electrical and Electronic Engineering (ElCon)*, pages 556–567. IEEE, 2025.

[216] Roger Nick Anaedevha et al. Application of machine learning models for device identification in wireless network traffic. In *2024 Conf. Young Researchers in Electrical and Electronic Engineering (ElCon)*, pages 104–110. IEEE, 2024. doi: 10.1109/ElCon61730.2024.10468413.

[217] Roger Nick Anaedevha et al. Uncertainty-calibrated hierarchical Gaussian processes for intrusion detection with multi-scale temporal modeling. *Neurocomputing*, 2025. Accepted. Manuscript Number: NEUCOM-D-25-13536R1.

Appendix A

Mathematical Proofs and Derivations

This appendix presents the complete proofs of all theorems stated in Chapter 2 (Methods, Algorithms, and Mathematical Frameworks), followed by additional derivations referenced in the main text. The proofs are ordered by first appearance.

A.1 Proof of Theorem 2.1: CT-TGNN Convergence

Restatement. Suppose the dynamics function f_θ is L_f-Lipschitz in $\mathbf{h}$ and L_θ-Lipschitz in θ. Under gradient descent with learning rate $\eta < 2/(L_f L_\theta)$, the training loss converges to a stationary point at rate $\mathcal{O}(1/\sqrt{T})$.

Proof. Define the loss evaluated through the ODE solution map as $\mathcal{L}(\theta) = \ell(\mathbf{h}(T;\theta), y)$ where $\mathbf{h}(T;\theta)$ satisfies $d\mathbf{h}/dt = f_\theta(\mathbf{h}, t)$ with $\mathbf{h}(0) = \mathbf{h}_0$. We establish that $\mathcal{L}$ is L-smooth for $L = L_f L_\theta$.

Step 1: Gradient boundedness. By the adjoint method the gradient is

$$\nabla_\theta \mathcal{L} = -\int_0^T \mathbf{a}(t)^\top \frac{\partial f_\theta}{\partial \theta}\, dt, \tag{A.1}$$

where $\mathbf{a}(t)$ solves the adjoint ODE $d\mathbf{a}/dt = -\mathbf{a}^\top(\partial f_\theta/\partial \mathbf{h})$ backward from $\mathbf{a}(T) = \partial \ell/\partial \mathbf{h}(T)$. Since $\partial f_\theta/\partial \mathbf{h}$ is bounded by L_f and $\partial f_\theta/\partial \theta$ is bounded by L_θ, the Grönwall inequality yields $\|\mathbf{a}(t)\| \le \|\mathbf{a}(T)\| e^{L_f(T-t)}$. Integrating gives $\|\nabla_\theta \mathcal{L}\| \le \|\mathbf{a}(T)\| L_\theta \int_0^T e^{L_f(T-t)}\, dt = \|\mathbf{a}(T)\| L_\theta (e^{L_f T} - 1)/L_f$, which is finite for bounded integration horizons.

Step 2: Lipschitz continuity of the gradient. For parameters θ and θ' the corresponding ODE solutions satisfy, by Grönwall's inequality,

$$\|\mathbf{h}(t;\theta) - \mathbf{h}(t;\theta')\| \le \frac{L_\theta}{L_f}(e^{L_f t} - 1)\|\theta - \theta'\|. \tag{A.2}$$

Combining with the Lipschitz property of the loss function ℓ yields $\|\nabla_\theta \mathcal{L}(\theta) -$

$\nabla_\theta \mathcal{L}(\theta')\| \le L\|\theta - \theta'\|$ for a constant L that depends on L_f, L_θ, T, and the smoothness of ℓ.

Step 3: Descent lemma. For L-smooth $\mathcal{L}$ the standard descent lemma gives

$$\mathcal{L}(\theta_{t+1}) \le \mathcal{L}(\theta_t) - \eta\|\nabla\mathcal{L}(\theta_t)\|^2 + \frac{L\eta^2}{2}\|\nabla\mathcal{L}(\theta_t)\|^2 = \mathcal{L}(\theta_t) - \eta\Big(1 - \frac{L\eta}{2}\Big)\|\nabla\mathcal{L}(\theta_t)\|^2. \tag{A.3}$$

For $\eta < 2/L$ the coefficient $(1 - L\eta/2) > 0$. Summing from $t = 0$ to $T - 1$ and rearranging,

$$\frac{1}{T}\sum_{t=0}^{T-1}\|\nabla\mathcal{L}(\theta_t)\|^2 \le \frac{\mathcal{L}(\theta_0) - \mathcal{L}^*}{\eta(1 - L\eta/2)\,T} = \mathcal{O}\Big(\frac{1}{T}\Big). \tag{A.4}$$

Therefore $\min_{t<T}\|\nabla\mathcal{L}(\theta_t)\|^2 = \mathcal{O}(1/T)$ and $\min_{t<T}\|\nabla\mathcal{L}(\theta_t)\| = \mathcal{O}(1/\sqrt{T})$, establishing convergence to an ϵ-stationary point in $\mathcal{O}(1/\epsilon^2)$ steps. □

A.2 Proof of Theorem 2.2: Lipschitz Robustness Certificate

Restatement. Let f_θ be L_g-Lipschitz in its first argument and let the integration horizon be T. For any two inputs $\mathbf{x}_1, \mathbf{x}_2$ the corresponding embeddings satisfy $\|\mathbf{h}_1(T) - \mathbf{h}_2(T)\| \le e^{L_g T}\|\mathbf{x}_1 - \mathbf{x}_2\|$. If the classifier head is L_c-Lipschitz, the end-to-end certified radius is $\rho = \Delta_{\min}/(L_c\, e^{L_g T})$.

Proof. Step 1: Embedding sensitivity. Define $\mathbf{d}(t) = \mathbf{h}_1(t) - \mathbf{h}_2(t)$ where $\mathbf{h}_i$ solves $d\mathbf{h}_i/dt = f_\theta(\mathbf{h}_i, t)$ with $\mathbf{h}_i(0) = \mathbf{x}_i$. Taking the derivative,

$$\frac{d\|\mathbf{d}(t)\|}{dt} \le \|f_\theta(\mathbf{h}_1, t) - f_\theta(\mathbf{h}_2, t)\| \le L_g\|\mathbf{d}(t)\|, \tag{A.5}$$

where the first inequality uses the reverse triangle inequality applied to the norm's derivative. By the Grönwall inequality, $\|\mathbf{d}(t)\| \le \|\mathbf{d}(0)\|e^{L_g t}$. Evaluating at $t = T$ yields $\|\mathbf{h}_1(T) - \mathbf{h}_2(T)\| \le e^{L_g T}\|\mathbf{x}_1 - \mathbf{x}_2\|$.

Step 2: End-to-end Lipschitz constant. Composing the ODE solution map (Lipschitz constant $e^{L_g T}$) with the classification head (Lipschitz constant L_c) yields total Lipschitz constant $L_{\text{total}} = L_c\, e^{L_g T}$.

Step 3: Certified radius. For a correctly classified input $\mathbf{x}$ with minimum

logit margin $\Delta_{\min} = \min_{c \neq y^*}[z_{y^*} - z_c]$ where z_c denotes the logit for class c, the prediction remains unchanged whenever $L_{\text{total}}\|\boldsymbol{\delta}\| < \Delta_{\min}/2$. Since both the correct logit decreases and the runner-up logit increases by at most $L_{\text{total}}\|\boldsymbol{\delta}\|$, the margin remains positive when $2L_{\text{total}}\|\boldsymbol{\delta}\| < \Delta_{\min}$, giving $\rho = \Delta_{\min}/(2L_c\, e^{L_g T})$. The factor of two is often absorbed into the margin definition; using the single-sided bound $L_{\text{total}}\|\boldsymbol{\delta}\| < \Delta_{\min}$ yields $\rho = \Delta_{\min}/(L_c\, e^{L_g T})$ as stated. □

A.3 Proof of Theorem 2.3: Byzantine-Robust Federated Convergence

Restatement. Under μ-strong convexity, L-smoothness, Byzantine fraction $q < 1/2$, and learning rate $\eta \leq 1/L$, coordinate-wise trimmed mean achieves $\mathbb{E}[\|\mathbf{w}_T - \mathbf{w}^*\|^2] \leq (1 - \mu\eta/2)^T \|\mathbf{w}_0 - \mathbf{w}^*\|^2 + 2\eta/(\mu(K - 2b))(\sum_k \sigma_k^2 + C_{\text{byz}})$.

Proof. Step 1: Trimmed mean proximity to honest mean. Let $b = \lfloor Kq \rfloor$ and let $\bar{\mathbf{g}}^H = (1/(K-b)) \sum_{k \in H} \mathbf{g}_k$ denote the honest gradient mean where H is the set of honest participants. By the analysis of coordinate-wise trimmed mean, for each coordinate j,

$$|\text{TrMean}_j - \bar{g}_j^H| \leq \frac{b}{K - 2b} \,\text{range}(\{g_{k,j}\}_{k \in H}), \tag{A.6}$$

because the trimmed mean discards the b largest and b smallest values, removing all Byzantine entries plus at most b honest entries. By concentration of sub-Gaussian random variables, $\text{range}(\{g_{k,j}\}_{k \in H}) \leq C\sigma_j \sqrt{\log(K - b)}$ with high probability, yielding $\|\text{TrMean} - \bar{\mathbf{g}}^H\|^2 \leq C_{\text{byz}}$ for a constant C_{byz} depending on b, K, and the gradient variance.

Step 2: One-step contraction. Define $\Delta_t = \|\mathbf{w}_t - \mathbf{w}^*\|^2$. The gradient descent update $\mathbf{w}_{t+1} = \mathbf{w}_t - \eta\, \text{TrMean}_t$ yields

$$\mathbb{E}[\Delta_{t+1}] = \mathbb{E}[\|\mathbf{w}_t - \eta\, \text{TrMean}_t - \mathbf{w}^*\|^2]. \tag{A.7}$$

Expanding and using μ-strong convexity $\langle \nabla \mathcal{L}(\mathbf{w}_t), \mathbf{w}_t - \mathbf{w}^* \rangle \geq \mu \|\mathbf{w}_t - \mathbf{w}^*\|^2 +$

$(1/L)\|\nabla\mathcal{L}(\mathbf{w}_t)\|^2$,

$$\mathbb{E}[\Delta_{t+1}] \leq (1-\mu\eta)\Delta_t + \eta^2\mathbb{E}[\|\text{TrMean}_t - \nabla\mathcal{L}(\mathbf{w}_t)\|^2]. \tag{A.8}$$

The second term decomposes as variance of honest gradients plus Byzantine bias:

$$\mathbb{E}[\|\text{TrMean}_t - \nabla\mathcal{L}(\mathbf{w}_t)\|^2] \leq \frac{1}{K-2b}\sum_{k\in H}\sigma_k^2 + C_{\text{byz}}. \tag{A.9}$$

Step 3: Recursive bound. Let $V = 2\eta/(\mu(K-2b))(\sum_k \sigma_k^2 + C_{\text{byz}})$. Using $1-\mu\eta \leq 1-\mu\eta/2$ for $\eta \leq 1/L$ and iterating the recursion,

$$\mathbb{E}[\Delta_T] \leq \Big(1-\frac{\mu\eta}{2}\Big)^T\Delta_0 + V\sum_{t=0}^{T-1}\Big(1-\frac{\mu\eta}{2}\Big)^t \leq \Big(1-\frac{\mu\eta}{2}\Big)^T\Delta_0 + \frac{V}{\mu\eta/2}. \tag{A.10}$$

Substituting back and simplifying yields the stated bound. The first term converges geometrically to zero, and the second represents the irreducible error floor due to gradient noise and Byzantine corruption. □

A.4 Proof of Theorem 2.4: PAC-Bayesian Bound for State-Space Models

Restatement. With probability $\geq 1-\delta$ over training set S of size n, for all posteriors Q: $\mathbb{E}_{\theta\sim Q}[R(\theta)] \leq \mathbb{E}_{\theta\sim Q}[\hat{R}_n(\theta)] + \sqrt{(\text{KL}(Q\|P) + \log(2\sqrt{n}/\delta))/(2n)}$.

Proof. Step 1: Change of measure. The proof follows the Catoni framework. For any measurable function $\phi : \Theta \to \mathbb{R}$ and distributions Q, P over Θ, the Donsker-Varadhan variational representation gives

$$\mathbb{E}_{\theta\sim Q}[\phi(\theta)] \leq \text{KL}(Q\|P) + \log\mathbb{E}_{\theta\sim P}[e^{\phi(\theta)}]. \tag{A.11}$$

Step 2: Moment-generating function bound. Define $\phi(\theta) = \lambda(R(\theta) - \hat{R}_n(\theta))$ for $\lambda > 0$. For bounded losses $\ell \in [0,1]$, Hoeffding's lemma applied conditionally on θ yields

$$\mathbb{E}_S[e^{\lambda(R(\theta)-\hat{R}_n(\theta))}] \leq e^{\lambda^2/(8n)}. \tag{A.12}$$

Taking expectations under P and applying Markov's inequality with confidence

$1-\delta$,

$$\log \mathbb{E}_{\theta\sim P}[e^{\lambda(R(\theta)-\hat{R}_n(\theta))}] \leq \frac{\lambda^2}{8n} + \log\frac{1}{\delta}. \tag{A.13}$$

Step 3: Optimisation over λ. Substituting into the Donsker-Varadhan bound,

$$\lambda\big(\mathbb{E}_Q[R] - \mathbb{E}_Q[\hat{R}_n]\big) \leq \mathrm{KL}(Q\|P) + \frac{\lambda^2}{8n} + \log\frac{1}{\delta}. \tag{A.14}$$

Dividing by λ and minimising over $\lambda > 0$ yields the optimal $\lambda^* = \sqrt{8n(\mathrm{KL}(Q\|P) + \log(1/\delta))}$, giving

$$\mathbb{E}_Q[R] \leq \mathbb{E}_Q[\hat{R}_n] + \sqrt{\frac{\mathrm{KL}(Q\|P) + \log(1/\delta)}{2n}}. \tag{A.15}$$

Applying the union bound with confidence $\delta/2$ and using $\log(2/\delta) \leq \log(2\sqrt{n}/\delta)$ for $n \geq 1$ yields the stated form.

Step 4: Application to selective state-space models. For Gaussian prior $P = \mathcal{N}(\mathbf{0}, \sigma_P^2\mathbf{I})$ and posterior $Q = \mathcal{N}(\boldsymbol{\mu}_Q, \mathrm{diag}(\boldsymbol{\sigma}_Q^2))$, the KL divergence admits the closed form

$$\mathrm{KL}(Q\|P) = \frac{1}{2}\sum_{j=1}^{d}\Big[\frac{\sigma_{Q,j}^2 + \mu_{Q,j}^2}{\sigma_P^2} - 1 - \log\frac{\sigma_{Q,j}^2}{\sigma_P^2}\Big], \tag{A.16}$$

which can be evaluated exactly from the learned posterior parameters, yielding a computable generalisation certificate. □

A.5 Proof of Theorem 2.5: Multiplicative Weights Regret Bound

Restatement. The multiplicative weights algorithm with $\eta = \sqrt{\log|\mathcal{C}|/T}$ achieves cumulative regret $R(T) \leq 2\sqrt{T\log|\mathcal{C}|}$.

Proof. Step 1: Potential function. Define $\Phi_t = \log\sum_{c\in\mathcal{C}} w_t(c)$ where $w_t(c) = \exp(\eta\sum_{s=1}^{t-1} r_s(c))$ and $r_s(c)$ is the reward of action c at round s, with rewards normalised to $[0,1]$.

Step 2: Upper bound on potential. At each round the potential increases by

$$\Phi_{t+1}-\Phi_t = \log \frac{\sum_c w_t(c) e^{\eta r_t(c)}}{\sum_c w_t(c)} \le \log\bigl(1+\eta\,\mathbb{E}_{c\sim p_t}[r_t(c)](e^\eta-1)\bigr) \le \eta\,\mathbb{E}_{c\sim p_t}[r_t(c)]+\eta^2, \tag{A.17}$$

where the last step uses $e^\eta - 1 \le \eta + \eta^2$ for $\eta \le 1$ and $\log(1+x) \le x$. Summing over rounds, $\Phi_{T+1} \le \log|\mathcal{C}| + \eta \sum_t \mathbb{E}_{c\sim p_t}[r_t(c)] + T\eta^2$.

Step 3: Lower bound on potential. For any fixed action c^*, $\Phi_{T+1} \ge \log w_{T+1}(c^*) = \eta \sum_t r_t(c^*)$.

Step 4: Combining. Subtracting the lower bound from the upper bound,

$$\eta \sum_t r_t(c^*) - \eta \sum_t \mathbb{E}_{c\sim p_t}[r_t(c)] \le \log|\mathcal{C}| + T\eta^2. \tag{A.18}$$

The regret is $R(T) = \max_{c^*} \sum_t r_t(c^*) - \sum_t \mathbb{E}_{c\sim p_t}[r_t(c)] \le (\log|\mathcal{C}|)/\eta + T\eta$. Setting $\eta = \sqrt{\log|\mathcal{C}|/T}$ gives $R(T) \le 2\sqrt{T\log|\mathcal{C}|}$ and average regret $R(T)/T = \mathcal{O}(\sqrt{\log|\mathcal{C}|/T}) \to 0$. □

A.6 Proof of Theorem 2.6: Unified Framework Convergence

Restatement. Under L_i-smoothness of component losses and L_{ij}-smoothness of coupling losses, alternating gradient descent with $\eta_i \le 1/(L_i + \sum_j L_{ij})$ converges at rate $\|\nabla \mathcal{L}_{\text{unified}}(\theta^T)\|^2 \le 2(\mathcal{L}_{\text{unified}}(\theta^0) - \mathcal{L}_{\text{unified}}(\theta^*))/(\eta_{\min} T)$.

Proof. Step 1: Block smoothness. The unified loss $\mathcal{L}_{\text{unified}}(\theta) = \sum_i \lambda_i \mathcal{L}_i(\theta_i) + \sum_{i<j} \lambda_{ij} \mathcal{L}_{ij}(\theta_i, \theta_j)$ is block-coordinate smooth. For each block i, the partial gradient $\nabla_{\theta_i} \mathcal{L}_{\text{unified}}$ is $(L_i + \sum_j L_{ij})$-Lipschitz in θ_i with other blocks fixed.

Step 2: Per-block descent. When updating block i with learning rate $\eta_i \le 1/(L_i + \sum_j L_{ij})$, the standard smooth descent lemma gives

$$\mathcal{L}_{\text{unified}}(\theta^{t,i+1}) \le \mathcal{L}_{\text{unified}}(\theta^{t,i}) - \frac{\eta_i}{2} \|\nabla_{\theta_i} \mathcal{L}_{\text{unified}}(\theta^{t,i})\|^2, \tag{A.19}$$

where $\theta^{t,i}$ denotes the parameter vector after updating blocks $1, \dots, i-1$ in round t.

Step 3: Aggregation over blocks and rounds. Summing the descent over all

seven blocks within round t,

$$\mathcal{L}_{\text{unified}}(\theta^{t+1}) \leq \mathcal{L}_{\text{unified}}(\theta^{t}) - \frac{1}{2}\sum_{i=1}^{7} \eta_i \|\nabla_{\theta_i} \mathcal{L}_{\text{unified}}(\theta^{t,i})\|^2. \tag{A.20}$$

Using $\sum_i \eta_i \|\nabla_{\theta_i}\mathcal{L}\|^2 \geq \eta_{\min} \sum_i \|\nabla_{\theta_i}\mathcal{L}\|^2 = \eta_{\min}\|\nabla\mathcal{L}\|^2$ (where the equality holds at the start-of-round parameters) and summing over T rounds,

$$\frac{\eta_{\min}}{2} \sum_{t=0}^{T-1} \|\nabla \mathcal{L}_{\text{unified}}(\theta^t)\|^2 \leq \mathcal{L}_{\text{unified}}(\theta^0) - \mathcal{L}_{\text{unified}}(\theta^*). \tag{A.21}$$

Dividing by T and taking the minimum yields $\min_{t<T} \|\nabla\mathcal{L}_{\text{unified}}(\theta^t)\|^2 \leq 2(\mathcal{L}_{\text{unified}}(\theta^0) - \mathcal{L}_{\text{unified}}(\theta^*))/(\eta_{\min} T)$, establishing the $\mathcal{O}(1/T)$ rate to a stationary point. □

A.7 Additional Derivations

A.7.1 Sinkhorn Iteration Convergence

The Sinkhorn algorithm alternates scaling operations $\mathbf{u}^{(\ell+1)} = \mathbf{a} \oslash (\mathbf{K}\mathbf{v}^{(\ell)})$ and $\mathbf{v}^{(\ell+1)} = \mathbf{b} \oslash (\mathbf{K}^\top \mathbf{u}^{(\ell+1)})$ where $\mathbf{K} = \exp(-\mathbf{C}/\lambda)$. This is a fixed-point iteration on the Hilbert projective metric. Because $\mathbf{K}$ is a positive matrix, the Birkhoff contraction theorem guarantees geometric convergence. Specifically, let $\kappa(\mathbf{K})$ denote Birkhoff's contraction coefficient; then $d_H(\mathbf{u}^{(\ell+1)}, \mathbf{u}^*) \leq \kappa(\mathbf{K})\, d_H(\mathbf{u}^{(\ell)}, \mathbf{u}^*)$ where d_H is the Hilbert metric. Since $\kappa(\mathbf{K}) < 1$ for entropically regularised costs with $\lambda > 0$, convergence is exponentially fast with rate $\mathcal{O}(\kappa^\ell)$. The number of iterations to reach ϵ-accuracy in marginal constraint violation is $\mathcal{O}(\lambda^{-1} \log(n/\epsilon))$ where n is the support size.

A.7.2 Evidence Lower Bound for Variational Bayesian Attention

The marginal log-likelihood of the data under the variational Bayesian attention model decomposes as

$$\log p(\mathcal{D}) = \text{ELBO}(q_\phi) + \text{KL}(q_\phi(\theta) \| p(\theta|\mathcal{D})), \tag{A.22}$$

where the evidence lower bound is

$$\text{ELBO}(q_\phi) = \mathbb{E}_{\theta \sim q_\phi}\left[\sum_{i=1}^{n} \log p(y_i|\mathbf{x}_i, \theta)\right] - \text{KL}(q_\phi(\theta) \| p(\theta)). \tag{A.23}$$

Since $\text{KL} \geq 0$, maximising the ELBO simultaneously maximises a lower bound on the log-evidence and minimises the KL divergence between the approximate posterior q_ϕ and the true posterior $p(\theta|\mathcal{D})$. For diagonal Gaussian $q_\phi(\theta) = \prod_j \mathcal{N}(\mu_j, \sigma_j^2)$ and isotropic prior $p(\theta) = \mathcal{N}(\mathbf{0}, \sigma_P^2 \mathbf{I})$, the KL term admits the closed form of Equation (2.43) in the main text, and the expectation is estimated by Monte Carlo sampling with reparameterisation $\theta^{(m)} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}^{(m)}$, $\boldsymbol{\epsilon}^{(m)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

A.7.3 Robustness-Accuracy Trade-off

Proposition A.1 (Robustness-Accuracy Trade-off)**.** *For any classifier f_θ on distribution $\mathcal{D}$ with standard risk $R_{\text{std}}(\theta) = \mathbb{E}_{(\mathbf{x},y)\sim\mathcal{D}}[\mathbf{1}[f_\theta(\mathbf{x}) \neq y]]$ and robust risk $R_{\text{rob}}(\theta, \epsilon) = \mathbb{E}_{(\mathbf{x},y)\sim\mathcal{D}}[\max_{\|\boldsymbol{\delta}\|\leq\epsilon} \mathbf{1}[f_\theta(\mathbf{x}+\boldsymbol{\delta}) \neq y]]$, the following holds: $R_{\text{rob}}(\theta, \epsilon) \geq R_{\text{std}}(\theta)$, and for distributions with overlapping class-conditional densities, there exist regions where any classifier achieving $R_{\text{rob}} \leq \alpha$ must accept $R_{\text{std}} \geq R^*_{\text{std}} + g(\epsilon, \alpha)$ for a non-negative function g that is increasing in ϵ.*

Proof. The first claim is immediate since $\max_{\|\boldsymbol{\delta}\|\leq\epsilon} \mathbf{1}[f(\mathbf{x}+\boldsymbol{\delta}) \neq y] \geq \mathbf{1}[f(\mathbf{x}) \neq y]$ by taking $\boldsymbol{\delta} = \mathbf{0}$. For the second claim, consider the Bayes-optimal standard classifier $f^*(\mathbf{x}) = \arg\max_c p(c|\mathbf{x})$. In regions where the ϵ-ball around $\mathbf{x}$ intersects the decision boundary of f^*, achieving robustness requires moving the decision boundary, which necessarily increases standard error on some subset of points. The magnitude of the trade-off is characterised by the probability mass within distance ϵ of the Bayes-optimal boundary, which grows with ϵ and depends on the density of the data distribution near the boundary. □

A.7.4 Uncertainty Decomposition

Proposition A.2 (Law of Total Variance for Predictive Uncertainty)**.** *The total predictive variance decomposes as*

$$\mathrm{Var}_{p(\theta|\mathcal{D})}[p(y|\mathbf{x},\theta)] = \underbrace{\mathbb{E}_{p(\theta|\mathcal{D})}[\mathrm{Var}_{p(y|\mathbf{x},\theta)}[y]]}_{\textit{aleatoric}} + \underbrace{\mathrm{Var}_{p(\theta|\mathcal{D})}[\mathbb{E}_{p(y|\mathbf{x},\theta)}[y]]}_{\textit{epistemic}}. \tag{A.24}$$

Proof. This follows directly from the law of total variance $\mathrm{Var}[Y] = \mathbb{E}[\mathrm{Var}[Y|X]] + \mathrm{Var}[\mathbb{E}[Y|X]]$ applied with $Y = y$ and $X = \theta$. The first term captures irreducible data noise (aleatoric) since it is the expected variance of the output distribution averaged over parameter uncertainty. The second term captures reducible model uncertainty (epistemic) since it measures how much the expected prediction varies across plausible parameter settings. With infinite data, the posterior concentrates on the true parameter, the epistemic term vanishes, and only aleatoric uncertainty remains. □

A.7.5 LWE Robustness Reduction for Forward-Compatible Detection

Proposition A.3 (Forward-Compatible Detection Robustness Reduction)**.** *Let* $\mathrm{Adv}_{\mathrm{LWE}}(n, q, \chi)$ *denote the advantage of the best polynomial-time algorithm for the decisional Learning With Errors problem with dimension* n*, modulus* q*, and error distribution* χ*. Then the false-negative rate of the forward-compatible detection classifier satisfies*

$$\mathrm{FNR} \leq \mathrm{Adv}_{\mathrm{LWE}}(n, q, \chi) + \sqrt{\frac{\log(1/\delta)}{2m}}, \tag{A.25}$$

where m *is the number of training samples and* δ *is the confidence parameter.*

Proof sketch. The reduction proceeds by contraposition. Assume a polynomial-time adversary that causes the forward-compatible detection classifier to misclassify Kyber timing anomalies with probability exceeding $\mathrm{Adv}_{\mathrm{LWE}} + \epsilon$. We construct an algorithm that uses this adversary to distinguish LWE samples from uniform: sample an LWE instance $(\mathbf{A}, \mathbf{b})$, simulate a Kyber decapsulation with timing distribution governed by $\mathbf{b}$, and feed the resulting features to the

adversary. If the adversary achieves misclassification, the timing distribution is consistent with a legitimate protocol-negotiation sequence, which occurs with probability at most $\mathrm{Adv}_{\mathrm{LWE}}$ when $\mathbf{b}$ is an LWE sample (since the timing correlates with Hamming weight of the error vector). If $\mathbf{b}$ is uniform, timing features are independent of the secret and misclassification is trivially possible. The gap between the two cases yields an LWE distinguisher with advantage $\geq \epsilon$, contradicting the assumed hardness. The statistical term $\sqrt{\log(1/\delta)/(2m)}$ bounds the generalisation gap of the learned classifier via Hoeffding's inequality. □

A.7.6 Stochastic Attention Robustness Enhancement

Proposition A.4. *Sampling attention weights from learned distributions with variance σ_W^2 during training is equivalent to regularising the loss with a term proportional to $\sigma_W^2 \|\nabla_W \mathcal{L}\|^2$, which penalises sharp loss landscapes and promotes flatter minima.*

Proof. Consider the stochastic forward pass $\mathcal{L}(\mathbf{W} + \boldsymbol{\epsilon})$ where $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \sigma_W^2 \mathbf{I})$. A second-order Taylor expansion gives

$$\mathbb{E}_{\boldsymbol{\epsilon}}[\mathcal{L}(\mathbf{W} + \boldsymbol{\epsilon})] \approx \mathcal{L}(\mathbf{W}) + \frac{\sigma_W^2}{2} \mathrm{tr}(\nabla^2 \mathcal{L}(\mathbf{W})). \tag{A.26}$$

Minimising the expected loss thus minimises the original loss plus a penalty on the trace of the Hessian, which favours flat minima. For the gradient of the expected loss with respect to $\mathbf{W}$, the reparameterisation gradient estimator yields

$$\nabla_{\mathbf{W}} \mathbb{E}[\mathcal{L}(\mathbf{W} + \boldsymbol{\epsilon})] \approx \nabla_{\mathbf{W}} \mathcal{L}(\mathbf{W}) + \frac{\sigma_W^2}{2} \nabla_{\mathbf{W}} \mathrm{tr}(\nabla^2 \mathcal{L}), \tag{A.27}$$

providing implicit gradient regularisation. This flat-minimum bias improves generalisation and adversarial robustness, as adversarial examples exploit sharp loss landscapes where small input perturbations cause large loss changes. □

Appendix B

Implementation Details and Hyperparameters

This appendix documents the software framework, hyperparameter configurations for all seven methods, data preprocessing procedures, computational infrastructure, and evaluation protocols employed throughout the experimental work of Chapter 4 (Experimental Results and Validation). The consolidated hyperparameter summary appears in Table 3.3 of Chapter 3; this appendix provides the complete per-layer specifications.

B.1 Software Framework and Dependencies

The entire experimental pipeline is implemented in Python 3.10 with PyTorch 2.1 as the primary deep learning framework. ODE integration uses the `torchdiffeq` library (version 0.2.3) providing adaptive-step Runge-Kutta solvers with adjoint sensitivity support. Graph neural network components build on PyTorch Geometric (version 2.4.0) for message passing, graph batching, and neighbourhood sampling. Federated learning simulation uses Flower (version 1.7) for client-server communication and aggregation. Optimal transport computations use the POT library (version 0.9.1) with GPU-accelerated Sinkhorn iterations. Differential privacy accounting uses Opacus (version 1.4) for per-sample gradient clipping and moments accountant tracking. Adversarial attack generation uses Foolbox (version 3.3.3) and the Adversarial Robustness Toolbox (version 1.15). Bayesian inference layers use a custom variational attention module building on the reparameterisation framework of Blundell et al. LLM integration queries the OpenAI API (GPT-3.5-turbo) with structured few-shot prompts. All random seeds are fixed at 42 for reproducibility, and experiments are repeated with five seeds to compute confidence intervals.

B.2 Hyperparameter Configurations

B.2.1 CT-TGNN Hyperparameters

Table B.1 – CT-TGNN hyperparameter configuration.

Parameter	Value	Notes
Message-passing layers	4	Heterogeneous type-specific
Hidden dimension	256	Per-layer
State dimension	64	ODE state
ODE solver	DOPRI5	Adaptive Runge-Kutta
Relative tolerance	10^{-5}	Solver precision
Absolute tolerance	10^{-7}	Solver precision
Time constants (τ_1, τ_2, τ_3)	$(1, 60, 3600)$ s	Packet, session, campaign
TABN window size	32 steps	Running statistics window
Learning rate	2×10^{-4}	AdamW
Weight decay	10^{-5}	L2 regularisation
Batch size	128 graphs	Per GPU
Gradient clip norm	1.0	Maximum gradient norm
Training epochs	200	With early stopping
Early stopping patience	10	Validation loss

B.2.2 TripleE-TGNN Hyperparameters

B.2.3 FedLLM-API Hyperparameters

B.2.4 Forward-Compatible Detection Hyperparameters

B.2.5 MambaShield Hyperparameters

B.2.6 Stochastic Transformer Hyperparameters

B.2.7 Game-Theoretic Framework Hyperparameters

Table B.2 – TripleE-TGNN hyperparameter configuration.

Parameter	Value	Notes
GAT layers per encoder	3	Service, trace, node
Attention heads	8	Multi-head attention
Hidden dimension	128	Per encoder
Temporal attention window	16 steps	Historical context
EWC strength λ_{ewc}	100	Catastrophic forgetting
Consistency weight λ_{con}	0.1	KL divergence
Service loss weight λ_s	0.4	Multi-task
Trace loss weight λ_r	0.3	Multi-task
Node loss weight λ_n	0.3	Multi-task
Fisher samples	500	For EWC diagonal
Learning rate	2×10^{-4}	AdamW
Batch size	64 graphs	Per GPU

Table B.3 – FedLLM-API hyperparameter configuration.

Parameter	Value	Notes
Federated clients K	10	Simulated SOCs
Communication rounds T	100	Server aggregation
Local epochs per round	5	Client training
Trimming fraction	20%	Top and bottom
DP clipping threshold C	1.0	Per-sample gradient
DP noise scale σ	0.8	Gaussian mechanism
Privacy budget (ϵ, δ)	$(0.85, 10^{-5})$	Per round
Sinkhorn regularisation λ	0.1	Entropic
Sinkhorn iterations	50	Convergence
OT Laplace sensitivity Δ_T	0.01	Transport plan
LLM ensemble weight λ_{LLM}	0.3	Validation-tuned
Few-shot examples	5	Per attack category
Learning rate	5×10^{-4}	Local AdamW

Table B.4 – Forward-compatible detection hyperparameter configuration.

Parameter	Value	Notes
Encoder MLP layers	3	Per protocol family
Encoder hidden dims	256, 128, 64	Decreasing
Activation	ReLU	All hidden layers
Cross-attention heads	8	Multi-head fusion
Attention dimension	64	Per head
PGD adversarial ϵ	0.1	Feature-space
PGD iterations	10	Training-time
PGD step size	0.02	Per iteration
Timing histogram bins	20	Kyber features
Learning rate	1×10^{-3}	AdamW
Batch size	256	Per GPU

Table B.5 – MambaShield hyperparameter configuration.

Parameter	Value	Notes
State dimension	64	SSM hidden state
Convolution kernel size	4	Local context
Expansion factor	2	Inner dimension
Mamba blocks	6	Stacked
Teacher ensemble size M	5	Disjoint subsets
Distillation temperature	3.0	Softmax temperature
Initial poison fraction ρ_0	0.05	Curriculum start
Poison growth rate α	5×10^{-4}	Per iteration
Maximum poison $\rho_{\max}$	0.40	Curriculum cap
Initial distill weight λ_0	0.1	Ramp start
Distill ramp rate β	1×10^{-3}	Per iteration
PAC-Bayes prior σ_P	1.0	Gaussian prior
PAC-Bayes confidence δ	0.05	Bound confidence
Learning rate	2×10^{-4}	AdamW

Table B.6 – Stochastic transformer hyperparameter configuration.

Parameter	Value	Notes
Encoder layers	4	Transformer blocks
Attention heads	8	Per layer
Hidden dimension	256	Model dimension
FFN dimension	1024	Feed-forward
Dropout	0.1	Standard and variational
MC forward passes M	50	Inference
Prior variance σ_P^2	1.0	Isotropic Gaussian
Loss weight λ_1 (task)	1.0	Cross-entropy
Loss weight λ_2 (KL)	0.1	Posterior regularisation
Loss weight λ_3 (adv)	0.5	Adversarial
Loss weight λ_4 (calib)	0.2	ECE penalty
Loss weight λ_5 (reg)	0.01	L2 weight decay
Perturbation ascent rate α_ψ	5×10^{-3}	Inner loop
Temperature scaling T	2.3	Post-hoc calibration
Calibration bins	15	ECE computation
Learning rate	2×10^{-4}	AdamW

Table B.7 – Game-theoretic framework hyperparameter configuration.

Parameter	Value	Notes
Detection configurations $\lvert\mathcal{C}\rvert$	20	Action space
Interaction rounds T	10,000	Online learning
MW learning rate η	$\sqrt{\log 20/10000} \approx 0.017$	Optimal
Uncertainty threshold τ	0.3	Expert referral
Uncertainty penalty κ	0.5	Utility discount
Nash support enumeration timeout	60 s	Per game instance
Payoff normalisation	$[0, 1]$	Per round

B.3 Data Preprocessing Pipeline

B.3.1 Feature Engineering

Raw packet capture files are processed into flow-level records using CICFlowMeter for CIC-IoT-2023 and CSE-CICIDS2018, and Argus for UNSW-NB15. Flow features include duration, byte counts (forward and backward), packet counts, inter-arrival time statistics (mean, standard deviation, minimum, maximum), flag counts (SYN, FIN, RST, PSH, ACK, URG), window size statistics, and protocol type indicators. All numerical features undergo z-score normalisation: $\tilde{x}_j = (x_j - \mu_j)/\sigma_j$ where μ_j and σ_j are computed on the training set and applied consistently to validation and test sets.

Categorical features (protocol type, service, state flags) undergo one-hot encoding followed by PCA dimensionality reduction retaining 95% of explained variance. This reduces the encoded dimension from several hundred binary indicators to 15–30 principal components without significant information loss.

For graph construction, a temporal graph snapshot is created every $\Delta t = 10$ seconds. Nodes are identified by IP address, and edges are created for each observed flow within the window. Node features aggregate all flow features incident to the node through mean and maximum pooling.

B.3.2 Data Augmentation

Graph data augmentation applies three transformations during training. Random edge dropout removes each edge independently with probability 0.1, encouraging robustness to incomplete network observations. Node feature perturbation adds Gaussian noise with standard deviation 0.05 to normalised feature vectors. Temporal jitter shifts event timestamps by $\mathcal{N}(0, 0.1^2)$ seconds, making the model robust to clock synchronisation imprecision. All augmentations are applied stochastically per mini-batch and disabled during validation and testing.

B.4 Computational Infrastructure

B.4.1 Training Infrastructure

Training is conducted on a cluster of four NVIDIA Tesla V100 GPUs with 32 GB HBM2 memory each, connected via NVLink for multi-GPU data parallelism. The host system provides dual Intel Xeon Gold 6248 CPUs (40 cores total), 384 GB DDR4 RAM, and 4 TB NVMe SSD storage. Mixed-precision training with the PyTorch AMP API (FP16 for forward and backward passes, FP32 for parameter updates) reduces memory consumption by approximately forty percent, enabling batch sizes $1.7\times$ larger than FP32 training. Distributed data parallelism with NCCL backend scales training to four GPUs with 92% linear scaling efficiency.

B.4.2 Inference and Deployment Simulation

Deployment simulation uses a single NVIDIA Tesla P100 GPU with 16 GB memory, reflecting realistic edge-deployment constraints. Inference benchmarks use TorchScript-compiled models with batch sizes of 32 for latency measurements and 512 for throughput measurements. All timing results report the median of one hundred independent runs with 95% confidence intervals computed via bootstrap resampling. Energy consumption is measured using NVIDIA's System Management Interface (`nvidia-smi`) polling power draw at 100 ms intervals during sustained inference.

B.5 Evaluation Protocols

B.5.1 Cross-Validation Strategy

Stratified splitting maintains class proportions across training (70%), validation (15%), and test (15%) partitions. For temporal datasets (CIC-IoT-2023 and CSE-CICIDS2018), splitting respects chronological order: the earliest 70% of records form the training set, the next 15% form validation, and the final 15% form the test set, preventing temporal leakage. For the federated experiments, each simulated client receives a disjoint partition of the training set,

and label distribution heterogeneity is introduced via Dirichlet allocation with concentration parameter $\alpha = 0.5$.

B.5.2 Statistical Significance Testing

All method comparisons report paired t-tests across five random seeds with Bonferroni correction for the number of comparisons. A result is declared statistically significant at the $\alpha = 0.05$ level after correction. Effect sizes are reported as Cohen's d to quantify practical significance alongside statistical significance. Confidence intervals for accuracy, F1, and AUC are computed via the Wilson score interval; confidence intervals for latency and throughput are computed via bootstrap percentile intervals with 10,000 resamples.

B.6 Code and Data Availability

The complete source code, configuration files, and trained model checkpoints for all seven methods are available at the following repository: `https://github.com/rogerpanel/robustidps.ai` (archived at DOI: 10.5281/zenodo.19129512). The repository includes automated scripts for data download and preprocessing, training and evaluation pipelines for each method, hyperparameter configuration files in YAML format, and Jupyter notebooks for reproducing all figures and tables in Chapters 4 and 5. The CIC-IoT-2023 and CSE-CICIDS2018 datasets are publicly available from the Canadian Institute for Cybersecurity. The UNSW-NB15 dataset is available from the University of New South Wales. The ICS3D collection datasets are available from their respective maintainers under academic licences.

Appendix C

Additional Experimental Results

This appendix presents supplementary experimental results that complement the analyses in Chapter 4 (Experimental Results and Validation). Section C.1 provides per-attack-category performance breakdowns. Section C.2 documents component-wise ablation details and hyperparameter sensitivity studies. Section C.3 reports cross-cloud and cross-domain transfer matrices. Section C.4 analyses the privacy-utility trade-off in detail. Section C.5 presents computational performance benchmarks and scaling characteristics.

C.1 Per-Attack Category Performance

C.1.1 CIC-IoT-2023 Detailed Results

Table C.1 reports per-attack-category precision, recall, F1 score, and AUC-ROC for CT-TGNN on CIC-IoT-2023. All seven major categories exceed 95% recall, with DDoS achieving the highest recall at 99.1% owing to the distinctive volumetric signature, and spoofing attacks showing the lowest recall at 95.2% because protocol impersonation can closely mimic legitimate authentication patterns.

Table C.1 – CT-TGNN per-attack-category results on CIC-IoT-2023.

Attack Category	Precision	Recall	F1	AUC-ROC
DDoS	98.7%	99.1%	98.9%	0.998
Reconnaissance	96.2%	96.8%	96.5%	0.991
Web Attacks	97.1%	97.3%	97.2%	0.993
Brute Force	98.4%	98.6%	98.5%	0.997
Spoofing	94.8%	95.2%	95.0%	0.987
Malware	98.6%	98.9%	98.7%	0.998
Mirai Variants	97.1%	97.4%	97.2%	0.994
Benign	98.9%	98.4%	98.7%	0.997

C.1.2 CSE-CICIDS2018 Detailed Results

Table C.2 reports per-attack-category performance for the unified framework on CSE-CICIDS2018. The unified framework achieves the largest improvements over baselines on infiltration (12.7 percentage points above the best baseline), botnet detection (9.3 points), and heartbleed exploitation (8.9 points), categories that involve multi-stage, temporally extended attack sequences where the continuous-time formulation provides the greatest advantage.

Table C.2 – Unified framework per-attack-category results on CSE-CICIDS2018.

Attack Category	Precision	Recall	F1	AUC	Δ vs. Baseline
Brute Force (SSH)	95.3%	96.1%	95.7%	0.991	+3.8%
Brute Force (FTP)	96.7%	97.3%	97.0%	0.994	+4.2%
DoS Hulk	97.8%	98.2%	98.0%	0.997	+2.1%
DoS GoldenEye	96.4%	96.9%	96.6%	0.993	+3.4%
DoS Slowloris	95.1%	95.7%	95.4%	0.989	+5.1%
Web Attack (XSS)	93.8%	94.3%	94.1%	0.984	+6.7%
Web Attack (SQL Inj.)	94.2%	94.8%	94.5%	0.986	+5.9%
Infiltration	91.7%	92.4%	92.0%	0.978	+12.7%
Botnet (Ares)	93.1%	93.8%	93.4%	0.982	+9.3%
Heartbleed	92.3%	93.1%	92.7%	0.979	+8.9%
Port Scan	97.6%	98.1%	97.8%	0.996	+2.3%
DDoS LOIC	98.1%	98.4%	98.2%	0.998	+1.7%
Benign	97.4%	96.8%	97.1%	0.995	+2.4%

C.1.3 UNSW-NB15 Detailed Results

Table C.3 reports per-attack-category results on UNSW-NB15. The unified framework achieves the largest improvement on reconnaissance attacks (8.7 percentage points), where the multi-scale temporal modelling captures the slow-and-stealthy scanning patterns that discrete-time baselines systematically miss.

C.1.4 TripleE-TGNN Microservices Attack Breakdown

Table C.4 reports TripleE-TGNN per-attack-type detection accuracy on the Container Security dataset, broken down by the granularity at which each at-

Table C.3 – Unified framework per-attack-category results on UNSW-NB15.

Attack Category	Precision	Recall	F1	AUC	Δ vs. Baseline
Generic	96.8%	97.2%	97.0%	0.995	+3.2%
Exploits	94.7%	96.3%	95.5%	0.989	+5.4%
Fuzzers	93.2%	94.1%	93.6%	0.984	+4.8%
DoS	96.4%	97.1%	96.7%	0.993	+3.7%
Reconnaissance	92.8%	97.9%	95.3%	0.988	+8.7%
Analysis	97.1%	98.4%	97.7%	0.996	+4.1%
Backdoors	91.9%	93.7%	92.8%	0.981	+6.3%
Shellcode	94.3%	95.8%	95.0%	0.987	+5.1%
Worms	89.7%	91.4%	90.5%	0.973	+7.2%
Normal	97.2%	96.4%	96.8%	0.994	+3.4%

tack is most effectively detected.

Table C.4 – TripleE-TGNN per-attack detection accuracy and dominant granularity.

Attack Type	Accuracy	Dominant Granularity	Single-Gran. Acc.
Container Escape	98.7%	Node + Service	91.2%
Supply Chain	97.3%	Service + Trace	89.8%
Resource Exhaustion	96.9%	Service + Node	90.3%
Privilege Escalation	97.8%	Node	93.1%
API Abuse	95.4%	Service	91.7%
Lateral Movement	96.1%	Trace + Node	88.4%
Data Exfiltration	94.8%	Trace	87.6%
Cryptojacking	97.2%	Node	92.9%
Overall	96.8%	Multi-level	88.5%

C.2 Ablation Study Details

C.2.1 Component-wise Ablation with Confidence Intervals

Table C.5 extends the ablation results of Table 4.9 in the main text with 95% confidence intervals computed from five independent runs with different random seeds. All degradations are statistically significant after Bonferroni

correction at the $\alpha = 0.05$ level.

Table C.5 – Extended ablation results with 95% confidence intervals (five seeds).

Component Removed	Full System	Without	Cohen's d
Cont.-time ODE	98.3±0.2%	91.5±0.4%	4.82
Multi-scale modelling	98.3±0.2%	94.4±0.3%	3.14
Temporal adaptive BN	98.3±0.2%	93.6±0.5%	3.67
Cross-gran. attention	96.8±0.3%	94.7±0.3%	2.41
Trimmed mean → FedAvg	89.7±0.4%	68.3±1.2%	7.13
Diff. privacy	87.1±0.3%	90.2±0.3%	−3.28
OT alignment	92.7±0.4%	76.9±0.8%	5.92
LLM branch (F1)	87.6±0.5%	34.2±1.1%	8.47
Progressive distill.	91.4±0.3%	81.7±0.6%	4.71
Variational attn. (ECE)	0.078±0.004	0.192±0.008	5.38
Game-theoretic equil.	94.7±0.3%	87.6±0.4%	4.18

C.2.2 Hyperparameter Sensitivity Analysis

This section presents sensitivity analyses for the most important hyperparameters of each method, varying one parameter at a time while holding all others at their default values.

C.2.2.1 CT-TGNN: ODE Solver Tolerance

Table C.6 reports the effect of ODE solver tolerance on accuracy, inference latency, and the number of function evaluations (NFE) per forward pass. Tighter tolerances yield marginal accuracy improvements but substantially increase computational cost; the default tolerance of 10^{-5} provides the best accuracy-efficiency trade-off.

Table C.6 – CT-TGNN sensitivity to ODE solver relative tolerance.

Rel. Tolerance	Accuracy	Latency (ms)	Avg. NFE
10^{-3}	97.6%	28	14
10^{-4}	98.1%	36	21
10^{-5} (default)	98.3%	47	31
10^{-6}	98.4%	72	48
10^{-7}	98.4%	118	73

C.2.2.2 FedLLM-API: Privacy Budget and Trimming Fraction

Table C.7 reports the effect of varying the per-round privacy budget ϵ on accuracy, total privacy expenditure over 100 rounds, and membership inference attack success. The default $\epsilon = 0.85$ balances strong privacy with acceptable accuracy degradation.

Table C.7 – FedLLM-API sensitivity to differential privacy budget ϵ.

ϵ	Accuracy	Total ϵ_{100}	MI Attack	Δ Acc. vs. No-DP
0.5	84.2%	5.0	51.3%	−7.0%
0.85 (default)	87.1%	8.5	52.7%	−3.1%
1.0	88.4%	10.0	54.1%	−1.8%
1.5	89.7%	15.0	57.8%	−0.5%
2.0	91.8%	20.0	62.4%	+1.6%
∞ (no DP)	90.2%	∞	71.3%	0%

Table C.8 reports accuracy under 30% Byzantine adversaries as the trimming fraction varies. Trimming fractions below 20% fail to remove all Byzantine updates, while fractions above 30% discard too many honest updates.

Table C.8 – FedLLM-API sensitivity to trimming fraction under 30% Byzantine adversaries.

Trimming Fraction	Accuracy (30% Byz.)	Convergence Rounds
10%	72.4%	142
15%	81.3%	108
20% (default)	89.7%	78
25%	88.9%	82
30%	86.1%	91
35%	82.7%	104

C.2.2.3 MambaShield: Distillation Temperature and Ensemble Size

Table C.9 reports the joint effect of distillation temperature and teacher ensemble size on robust accuracy under 25% poisoning. Temperature 3.0 with five teachers yields the best performance; lower temperatures produce sharper distributions that amplify label noise, while larger ensembles provide diminishing returns relative to their computational cost.

Table C.9 – MambaShield sensitivity to distillation temperature and ensemble size (25% poisoning).

Temperature	3 Teachers	5 Teachers	7 Teachers	10 Teachers
$T = 1.0$	85.3%	86.7%	87.1%	87.3%
$T = 2.0$	88.4%	89.8%	90.1%	90.2%
$T = 3.0$ (default)	89.7%	91.4%	91.6%	91.7%
$T = 5.0$	88.1%	90.2%	90.5%	90.6%
$T = 10.0$	84.6%	87.3%	87.7%	87.8%

C.2.2.4 Stochastic Transformer: Monte Carlo Samples

Table C.10 reports the effect of the number of Monte Carlo forward passes on ECE, epistemic uncertainty correlation with prediction error, and inference latency per batch. Early stopping based on variance convergence reduces the effective number of passes to 18 on average.

Table C.10 – Stochastic transformer sensitivity to number of MC forward passes.

MC Passes	ECE	Uncertainty Corr.	Latency (ms/batch)	Eff. Passes
5	0.124	0.43	78	5
10	0.103	0.52	126	10
20	0.089	0.61	204	17
50 (default)	0.078	0.67	340	18
100	0.076	0.68	638	19

C.3 Cross-Cloud and Cross-Domain Transfer

C.3.1 Cross-Dataset Transfer Matrix

Table C.11 reports the accuracy of CT-TGNN when trained on one dataset (rows) and tested on another (columns) without any fine-tuning. The diagonal entries correspond to in-distribution performance. The consistently small off-diagonal degradation, averaging 3.8% across all pairs, confirms the generalisation properties of the continuous-time temporal graph formulation.

Table C.11 – CT-TGNN cross-dataset transfer accuracy (%). Rows: training dataset; columns: test dataset.

Train \ Test	CIC-IoT	CICIDS18	UNSW-NB15
CIC-IoT-2023	98.3	92.7	91.4
CSE-CICIDS2018	91.8	96.7	89.3
UNSW-NB15	90.6	88.9	97.1

C.3.2 Cross-Domain Transfer to Speech and Healthcare

Table C.12 reports detailed metrics for the cross-domain transfer experiments described in Section 4.7 of the main text, including the effect of network security pretraining.

Table C.12 – Cross-domain transfer results with and without network security pretraining.

Domain	Method	Accuracy	F1	ECE	Pretrained?
Speech (LibriSpeech)	CT-TGNN	90.4%	90.8%	0.091	No
Speech (LibriSpeech)	CT-TGNN	94.2%	94.2%	0.073	Yes
Speech (LibriSpeech)	LSTM baseline	86.9%	86.7%	0.142	No
Healthcare (MIMIC-III)	Stoch. Trans.	86.1%	85.7%	0.098	No
Healthcare (MIMIC-III)	Stoch. Trans.	89.7%	89.3%	0.082	Yes
Healthcare (eICU)	Stoch. Trans.	84.8%	84.2%	0.104	No
Healthcare (eICU)	Stoch. Trans.	88.4%	87.9%	0.087	Yes

C.4 Privacy-Utility Trade-off Analysis

C.4.1 Differential Privacy Impact Across Datasets

Table C.13 reports the accuracy degradation caused by differential privacy at $\epsilon = 0.85$ across all three primary datasets. The degradation is consistent across datasets, ranging from 2.8% to 3.4%, confirming that the privacy-utility trade-off does not vary substantially with data distribution.

C.4.2 Privacy Accounting Over Communication Rounds

Table C.14 reports the cumulative privacy expenditure as a function of federated communication rounds, computed through the moments accountant

Table C.13 – Differential privacy impact on accuracy across datasets ($\epsilon = 0.85$, $\delta = 10^{-5}$).

Dataset	Without DP	With DP	Degradation
CIC-IoT-2023	94.7%	91.6%	−3.1%
CSE-CICIDS2018	93.9%	91.1%	−2.8%
UNSW-NB15	93.8%	90.4%	−3.4%

with Poisson subsampling amplification. The total budget of $\epsilon_{\text{total}} = 8.5$ after 100 rounds remains within the range typically considered acceptable for sensitive applications.

Table C.14 – Cumulative privacy expenditure over federated rounds ($\delta = 10^{-5}$).

Rounds	10	25	50	75	100	150
ϵ_{total}	0.85	2.13	4.25	6.38	8.50	12.75
Accuracy	78.4%	84.1%	87.8%	88.6%	87.1%	86.3%

C.5 Computational Performance Metrics

C.5.1 Inference Throughput and Latency

Table C.15 compares inference throughput, per-sample latency, GPU memory consumption, and energy efficiency across all seven methods and baselines. Measurements are taken on a single NVIDIA Tesla P100 GPU with batch size 32 for latency and batch size 512 for throughput, reporting the median of 100 runs with 95% bootstrap confidence intervals.

C.5.2 Scaling with Input Size

Table C.16 reports how inference latency scales with input sequence length for CT-TGNN, MambaShield, and the standard transformer baseline. CT-TGNN scales linearly through the fixed number of ODE solver steps per time constant. MambaShield scales linearly through the selective scan mechanism. The standard transformer scales quadratically through self-attention, becoming impractical for sequences beyond approximately 4096 tokens on 16 GB GPUs.

Table C.15 – Computational performance comparison across methods (P100 GPU).

Method	Throughput (k/s)	Latency (ms)	Memory (GB)	W/1k pred.
CT-TGNN	8,700	47	3.8	34
TripleE-TGNN	4,200	73	4.7	51
FedLLM-API	5,100	62	5.2	47
Fwd-Compat. Det.	6,800	38	2.1	28
MambaShield	12,400	18	2.3	19
Stoch. Transformer	2,300	340	8.2	89
Game-Theoretic	7,100	12	0.2	8
LSTM baseline	4,300	38	1.8	42
Std. Transformer	6,200	55	4.1	125
Random Forest	14,200	12	0.8	14
XGBoost	18,700	8	0.6	11

Table C.16 – Inference latency (ms) as a function of sequence length.

Sequence Length	256	512	1024	2048	4096
CT-TGNN	23	41	74	142	278
MambaShield	8	14	26	49	94
Std. Transformer	18	52	178	684	OOM

C.5.3 Training Time and Convergence

Table C.17 reports total training time, number of epochs to convergence, and wall-clock time per epoch for each method on CIC-IoT-2023 using four V100 GPUs with data parallelism.

Table C.17 – Training time comparison (four V100 GPUs, CIC-IoT-2023).

Method	Total Time (h)	Epochs	Time/Epoch (min)
CT-TGNN	18.2	142	7.7
TripleE-TGNN	22.7	118	11.5
FedLLM-API	31.4	100 (rounds)	18.8
Fwd-Compat. Det.	8.3	87	5.7
MambaShield	14.1	100	8.5
Stoch. Transformer	24.6	96	15.4
Game-Theoretic	6.8	10,000 (rounds)	0.04

C.5.4 Model Size Comparison

Table C.18 reports the total trainable parameter count, compressed model size after INT8 quantisation, and the accuracy retained after quantisation.

Table C.18 – Model size and quantisation impact.

Method	Parameters (M)	FP32 Size (MB)	INT8 Size (MB)	INT8 Ac
CT-TGNN	12.3	47	13	97.9%
TripleE-TGNN	18.7	71	19	96.2%
FedLLM-API	24.1	92	25	86.4%
Fwd-Compat. Det.	6.8	26	7	95.8%
MambaShield	8.4	32	9	90.9%
Stoch. Transformer	31.2	119	31	93.4%
Std. Transformer	47.8	182	47	90.8%